CPE334 Software Engineering — Deliverable 1

NutTiew (นัดติว) — One-to-One Tutoring Appointment System

- **Academic Term:** Semester 1/2026
- **Department & Institution:** Department of Computer Engineering, Faculty of Engineering, King Mongkut's University of Technology Thonburi (KMUTT)
- **Assigned Theme:** Theme 1 — Appointment & Scheduling
- **Repository URL:** https://github.com/Tanaboonnnnn/CPE334-NutTiew
- **Integration Staging Base SHA:** `9d24fca` on branch `sprint1`
- **Verified Code & DB Baseline SHA:** `808b906` on branch `docs/18-deliverable-1-integration` (PR #43)
- **Verified GitHub Actions CI Run:** Run 42783770373 — 3/3 Jobs PASS (`Unit / types / build` , `PostgreSQL 18 / Auth / Slot / Race A` , `Microsoft Edge / Auth / Tutor UI E2E`)
- **Target Release Branch:** `main` (Pending teammate review and approval of PR #43; followed by reviewed `sprint1 -> main` release PR and immutable tag)
- **Course Handout Deadline:** 8 November 2026, 23:59 ICT *(subsequent PR remediation & final submission timing transparently documented below)*
- **Report Compilation Date:** 10 November 2026 (Asia/Bangkok)
- **Total PDF Pages:** 199 pages

Team Members & Identification

Student Full Name	Student ID	GitHub Username	Project Role & Responsibility
Tanboon Teawsawat	*(Supplied Privately via LEB2 handoff)*	@Tanaboonnnnn	Requirements Discovery Lead (Q01–Q32), Whole-Product SRS & SDS Architecture, Three-Sprint Plan, Project #6 Kanban, Peer Reviewer
Chxtamos	*(Supplied Privately via LEB2 handoff)*	@Chxtamos	Express 5 REST API & PostgreSQL Schema/Trigger Implementation, Concurrency Lock Ordering, React 19 UI, Automated Tests, PDF Lead

> **Privacy Notice on Student Identification:** In compliance with student privacy protection and academic repository standards, full KMUTT Student ID numbers are withheld from public Git commit histories and supplied privately through the official university course submission portal (LEB2).

Executive Summary & Document Navigation Guide

NutTiew (นัดติว) is a one-to-one tutoring appointment and scheduling web application developed under CPE334 Software Engineering. This document is a **fully self-contained Deliverable 1 submission** packaging **Parts A through F** verbatim from approved canonical repository contracts and verified Sprint 1 test runs, gradable completely offline without opening the repository.

Actual PDF Page Ranges & Traceability Navigation

Document Part / Section	Primary Content & Scope	Relevant Requirement / Contract IDs	PDF Pages
Cover, Team & Navigation	Metadata, team roles, confidentiality note, actual PDF navigation index	D1 §4.3	p. 1–2

Document Part / Section	Primary Content & Scope	Relevant Requirement / Contract IDs	PDF Pages
Part A: Whole-Product SRS	Complete problem domain, actors, Sprints 1–3 requirements, rules & Q01–Q32	`FR-001..036`, `BR-001..035`, `NFR-001..016`, `Q01–Q32`	p. 3–32
Part B: Feature Inventory & Traceability	Whole-product feature catalog and canonical `AC-S1-01..35` / `VC/TEST-S1-01..39` mapping	`F-S1-01..05`, `F-S2-01..05`, `F-S3-01..02`, `AC-S1-01..35`	p. 33–48
Part C: Whole-Product System SDS	System architecture, Express 5 + React 19 + PostgreSQL DDL, Mermaid diagrams, future Lab 5 deployment design	C4 diagrams, `btree_gist`, `slot_guard_s1`, `booking_guard_s1`	p. 49–66
Part D: Feasible Three-Sprint Plan	Milestones, feature dependency order, two-student capacity, risks & Sprint 1 DoD boundary	Deliverable D1/D2/Final checkpoints	p. 67–73
Part E: Sprint 1 Feature Contracts & STSs	Complete embedded Part E Feature SDS, UI Spec, UX Amendments, API Spec, Data Spec, Transaction Spec, STS & DoD	All 6 Part E components (30 full contract docs)	p. 74–183
— **E.1 F-S1-01 Authentication & Sessions**	SDS, UI Spec, API Contract, Data Contract, STS & Gate	`AUTH-01..23`, `OD-005-A..E`	p. 74–87
— **E.2 F-S1-02 Tutor Slot Availability**	SDS, UI Spec, Tutor Calendar UX Decision, API Spec, Data Spec, STS	`SLOT-01..17`, Race A	p. 87–111
— **E.3 Shared Booking Data Foundation**	Foundation SDS, Shared Data Spec, Shared DB STS & Trigger Guards	`BDF-01..28`, `booking_guard_s1`	p. 111–126
— **E.4 F-S1-03 Customer Availability**	SDS, UI Spec, API Contract, Data Contract, STS	`AVA-01..19`	p. 126–143
— **E.5 F-S1-04 Exclusive Booking & Races**	SDS, UI Spec, API Spec, Data Spec, Transaction Lock/Retry Contract, STS	`BOOK-01..33`, Race B, Race C, `BDF-23`	p. 143–164
— **E.6 F-S1-05 Booking Retrieval & Tutor View**	SDS, UI Spec, API Spec, Data Contract, STS	`RET-01..17`, `INV-RET-01..05`	p. 164–183
Part F: Sprint 1 Verification & Release Evidence	199 distinct tests, `AC-S1-01..35` & `VC/TEST-S1-01..39` matrices, Race A/B/C proofs, verbatim excerpts, TDD history, 8 journey screenshots, peer reviews, contributions, AI use & honest DoD	199/199 tests PASS; release/LEB2 pending	p. 184–199

Part A: Whole-Product Software Requirements Specification (SRS)

0. Document control

Field	Value
Product	CPE334 NutTiew, Theme 1: Appointment and Scheduling (tutoring context)
Course	CPE334, Introduction to Software Engineering in the Age of AI Agents, semester 1/2026
Repository	Tanaboonnnnn/CPE334-NutTiew
Artifact / version	`docs/srs.md`, v0.1.0, 3 November 2026
Status	**DRAFT FOR PEER REVIEW; NOT INSTRUCTOR-APPROVED**
Requirements engineer / PR author	@Tanaboonnnnn
Required independent reviewer	@Chxtamos (review request and approval are separate actions)
Coverage	Whole-product software requirements for Sprints 1, 2 and 3
Primary readers	Stakeholder/instructor, peer reviewer, requirements/design engineers, test authors and release reviewers
Baseline	Confirmed stakeholder decisions Q01-Q32, reviewed Sprint 1 `docs/requirements.md`, available instructor handouts
Instructor-source status	`group_work_specs_v1.pdf` was examined; explicitly **Draft for instructor review**, issued 6 October 2026. The final `group_work_specs.docx`, amendments and Theme 1 eligibility are **UNVERIFIED**; formal sign-off is blocked (OD-011/012).
Implementation and verification status	This artifact specifies planned behavior and planned validation. It does not certify code, executed tests, a deployed release or passed acceptance tests.
Delivery boundary	Deliverable 1 **Part A only**. Parts B-F and Labs 5-7 are separate deliverables.

0.1 Reading rules and status vocabulary

- **Confirmed (Qxx):** a stakeholder decision recorded in the agreed Q01-Q32 baseline. Its product behavior is binding unless a new decision is explicitly approved.
- **Instructor-draft requirement (GW/D1):** a rule contained in the instructor-format handouts actually examined. Treat it as the available course baseline, subject to comparison with the final Master Specification.
- **Engineering interpretation:** a precise, reviewable reading of a confirmed outcome, not a new stakeholder policy (for example, transaction-safe evaluation of Q05).
- **Proposed:** a team suggestion or target that cannot be presented as approved until its owner decides.
- **Open / pending approval:** the source does not settle a policy or the necessary independent confirmation is absent.
- **Verified:** a test or source check actually performed with recorded evidence. No application verification is claimed here.

The statement `MUST` denotes a confirmed stakeholder rule or an applicable instructor-draft minimum, with its provenance in the requirement catalog. `SHOULD (PROPOSED)` is not an acceptance condition until approved. New numeric limits, schema fields, API responses, or service policies must not be disguised as existing requirements.

0.2 Navigation

1. Executive overview
2. Stakeholders, actors and glossary
3. Product scope and sprint boundaries
4. System context and user journeys

5. Functional requirements
6. Business rules
7. Non-functional requirements
8. Authorization and privacy
9. Logical domain and lifecycle
10. Time, conflicts and concurrency
11. Exception and boundary scenarios
12. Source and requirement traceability
13. Open decisions and approval gates
14. Exclusions and future compatibility
15. Verification planning
16. Risks, dependencies and change control
17. Review checklist and sign-off
18. References

0.3 Document history

Version	Date	Change	Review / approval
0.1.0	2026-11-03	Initial whole-product SRS based on Q01-Q32, the Sprint 1 baseline and identified instructor handouts	Draft; independent review and instructor Master verification outstanding

1. Executive overview

1.1 Problem, objective and value

Learners need a dependable way to reserve one-to-one lessons with a Tutor at times the Tutor has actually offered. Tutors need control of their own availability and a trustworthy view of reservations. Without a single authoritative reservation workflow, a learner may see stale availability, two learners may claim the same lesson, or one learner may inadvertently reserve overlapping lessons with different Tutors. Cancellation, rescheduling and schedule changes add further ways for the displayed calendar to disagree with the saved records.

NutTiew will let a Tutor publish individual sixty-minute offers and let a Customer reserve an eligible offer. The complete product will support tightly governed cancellation and rescheduling, private Tutor-cancellation notifications, controlled withdrawal of unused offers and a Tutor-only appointment summary. A successful operation must leave coherent persisted records; a failed operation must not change the booking/schedule outcome by accident.

Business value is a clear reservation trail and fewer schedule conflicts, with the Tutor and Customer seeing only records their roles are permitted to access. Payment collection, lesson delivery and attendance certification are not part of the confirmed service.

1.2 System boundary and principal risks

The software boundary covers authenticated web interactions, server-side role/ownership and business-rule checks, and persistent storage of Slots, Bookings, history and authorized notifications. A browser's display is never authoritative for a reservation; accepted mutations are determined by the server against current committed data.

Principal risks are simultaneous conflicting writes, ambiguous scheduling policies, disclosure of another learner's booking/cancellation reason, incorrect timezone or cutoff interpretation, loss of history on rescheduling/withdrawal, and treating proposed engineering limits as stakeholder approvals. The final instructor Master and team Theme allocation are also unverified (Section 13).

1.3 Agreed product model

One Booking reserves **one exclusive 60-minute Slot** for **one Customer and one Tutor**. The Tutor is the Staff actor; the Customer is the learner. Slots are explicitly published one at a time. Same-Tutor currently offered Slots do not overlap, and one Customer's active reservations do not overlap across Tutors. Adjacent time intervals are permitted by the conflict rules. Appointments that have ended remain `Booked` unless legitimately cancelled; their `Past` label is a time-derived presentation, not evidence that a lesson was completed.

2. Stakeholders, actors and glossary

2.1 Roles and boundaries

Actor / stakeholder	Goal and responsibility	Permitted product operations	Trust, visibility and restrictions
Customer (Learner)	Find and reserve one-to-one tutoring; maintain their own bookings	Authenticated availability, create Booking, view own Booking, cancel and reschedule their own eligible Booking, read own Tutor-cancellation notification	Cannot publish/manage Tutor Slots, view another Customer's private records, see Tutor-only summary, or cancel/reschedule another Customer's Booking
Tutor (Staff)	Offer time, serve assigned Bookings, maintain own timetable	Publish individual Slots, view own authorized schedule/Bookings, cancel assigned Booking within policy, withdraw eligible own Slot, read own Appointment Summary	No global management of another Tutor's schedule; no Customer-originated rescheduling; no arbitrary Customer private data
Anonymous visitor	Reach the entry/login experience	Access a login interface; authenticate with valid credentials	Cannot read protected identity, private availability/schedule/Booking/notification data, or mutate Slots/Bookings. Public availability is not promised by the baseline.
Application / server	Apply business policy and own data consistency	Authenticate session, authorize requests, sample server time, arbitrate conflicts, persist atomic outcomes and return role-filtered views	Client IDs, clocks, role flags and stale UI state cannot override server rules
Instructor / product owner	Confirm permitted theme, assess business baseline and approve material changes	Review the SRS, decide unresolved policy and course-source conflicts	Final Master and Theme assignment require dated evidence
Team reviewer	Independently check requirements, tests and change history	Review the Issue-linked PR and approve/request changes	Review request, review submission, approval and merge are distinct recorded states

Tutor = Staff is a role equivalence in this product. It does not introduce a separate administrator role. If a Tutor also wants to book a lesson as a learner, an explicit account/role policy would be required; staff privileges alone do not grant Customer-only actions.

2.2 Glossary

Term	Definition
Availability Slot / Slot	One Tutor-owned individual offer with a specified start and end exactly 60 minutes apart (Q03-Q04)
Currently offered Slot	A published, not-withdrawn offer; it may be unoccupied or occupied
Bookable Slot	Currently offered, unoccupied, valid future Slot satisfying the new-Booking cutoff and all eligibility checks at the authoritative decision time
Withdrawn Slot	Retained historical Slot that is no longer offered; cannot itself be reactivated (Q32)

Term	Definition
Appointment	Scheduled one-to-one Customer/Tutor interval associated with a Booking and Slot; not proof of attendance
Booking	Persistent reservation with stable identity, owner, assigned Slot/Tutor and business state
Active Booking	A `Booked` reservation that has not been cancelled; passage of the session end does not automatically release it
Cancelled Booking	Booking transitioned to `Cancelled` by an authorized cancellation workflow; retained according to unresolved data-governance policy
Reschedule	Authorized atomic reassignment of an existing Booking to a different eligible Slot while preserving Booking ID and success history
Reschedule history	Record of a **successful** change containing old/new Tutor, Slot/time and change timestamp (Q15)
In-app notification	Private persisted message for a Customer whose Booking was cancelled by their Tutor (Q21-Q25)
Appointment Summary	Tutor-only view of the authorized Tutor's Upcoming Booked, Past Booked and Cancelled counts and underlying lists (Q31)
Half-open interval	`[start, end)`; includes the start, excludes the end, permitting back-to-back adjacency without overlap
Business timezone	`Asia/Bangkok` (UTC+07:00), uniform for scheduling and display; the server clock is authoritative
T_elig	Server-derived eligibility instant sampled in the final guarded transaction check after any contention wait; adopted as the precise Sprint 1 review interpretation of Q05, subject to Feature SDS approval
FR / BR / NFR	Functional requirement / business rule / non-functional requirement
AC / STS / VAL	Acceptance criterion / software test specification / **planned** validation scenario in this SRS

3. Product scope and sprint boundaries

3.1 Confirmed whole-product capabilities

The agreed product includes Tutor-owned 60-minute slot publishing; Customer browsing and exclusive booking; persisted, role-limited Booking views; Customer cancellation with reopening; Customer-initiated one-time atomic rescheduling (including across Tutors); Tutor cancellation with mandatory reason and Slot withdrawal; private in-app notice with durable read access; controlled withdrawal/replacement of unoccupied future offers; and an authorized Tutor summary. Q01-Q32 are the behavioral baseline, not thirty-two optional enhancements.

3.2 Instructor minimum and technical constraints

The examined Group Work Specification v1 §§5-6 and Theme 1 example require a meaningful, tested UI → API/application → persistent-database journey, real login/logout/current identity in Sprint 1, Customer and Staff roles, backend authorization, protected Customer/Staff data, persistent Booking, past/invalid/occupied rejection and concurrent single-Slot exclusivity. The instructor materials call for versioned engineering contracts, automated tests, reviewed GitHub PRs, and later Docker/GCP/CI deployment; the Sprint 1 working software and evidence are **separate Issues** from this SRS.

3.3 Sprint allocation is a planning proposal

Sprint	Proposed delivery responsibility	Requirement status
Sprint 1, Deliverable 1	Login/logout/current identity, role/ownership enforcement, manual sixty-minute Tutor Slots, own Tutor schedule, availability browsing, exclusive Booking, own Customer/Tutor reads, persistence, invalid/too-soon/overlap rejection, three concurrency invariants, Bangkok time, Booked/Past correctness	Scope aligns with reviewed `docs/requirements.md` and the available Theme 1 minimum
Sprint 2, proposed	Customer cancellation/reopening; one-time Customer reschedule across Tutors, atomic history; Tutor cancellation/reason/withdrawal; authorized unoccupied Slot withdrawal; private cancellation notifications and history	Q07-Q27 and Q32 are confirmed **product rules**; placing these in Sprint 2 is a **proposal**, not an additional stakeholder decision

Sprint	Proposed delivery responsibility	Requirement status
Sprint 3, proposed	Tutor-only summary with three categories, counts and lists; remaining approved refinements, integration and full regression	Q31 is a confirmed product rule; timing and scope allocation require review in Issue #12

Part D's detailed three-sprint plan belongs to **Issue #12**. This table assigns responsibility so SRS verification is understandable without writing that separate plan. A later-sprint requirement is not a claim that its UI, API or code already exists.

3.4 Explicit exclusions, assumptions and unresolved policy

Payments, pricing, self-registration, email/SMS/push reminders, group sessions, lesson platform/location, external calendar synchronization, attendance verification, automatic recurring Slots, and a global administrator are **not approved product capabilities**. They are excluded from the currently agreed baseline; adding them requires a decision and change review, not implementation by inference.

Working hours, permitted days, maximum look-ahead, allowed start-minute grid, exact availability presentation, retention/deletion policy, in-progress summary classification, session/token mechanics, retry response behavior and proposed quality thresholds are **open** (Section 13). In particular, no confirmed working-hours restriction does not establish a promise of twenty-four-hour service.

4. System context and user journeys

For each journey below, `S1/S2/S3` is the **proposed verification sprint**, not proof of delivery. On any rejected mutation, previously committed records remain authoritative. API paths, database schema, error codes and UI field layouts belong to the separate design/contracts.

4.1 J-01: Tutor publishes one Slot (S1)

Preconditions: authenticated Tutor; proposed start is valid/future and does not intersect a currently offered Slot belonging to the same Tutor. **Flow:** Tutor selects one Bangkok start; server derives/checks a sixty-minute interval, confirms ownership and current-offer non-overlap, then persists one offer. **Alternatives/failures:** invalid, past, wrong-role or conflicting proposals are refused; simultaneous overlapping proposals must not both commit. **Postcondition:** the Tutor can retrieve the new unoccupied Slot on their own schedule. **References:** FR-005..007, BR-002..004/006/035; VAL-003..005.

4.2 J-02: Customer finds an eligible Slot (S1)

Preconditions: authenticated Customer and one or more current offers. **Flow:** server returns permitted Tutor/time details and correct eligibility as of the lookup; Customer selects an offer. **Alternatives/failures:** occupied, too-soon and past Slots cannot be presented as guaranteed bookable; no third-party Booking or cancellation reason is disclosed. Whether ineligible offers are hidden or visibly marked is an open UI decision. **Postcondition:** a possible selection, **not** a reservation. **References:** FR-008/009, BR-005/028/029; VAL-006/036.

4.3 J-03: Customer books an exclusive session (S1)

Preconditions: authenticated Customer; unoccupied currently offered Slot; no overlapping Active Booking for that Customer; at least 60 minutes before start at authoritative eligibility check. **Flow:** server revalidates all rules under transaction/concurrency control, commits one `Booked` record with trusted Customer and Slot's Tutor, then returns success. **Alternatives/failures:** stale selection, too-soon request, duplicate claim, same-Customer overlap or persistence failure produces no conflicting/partial committed claim. **Postcondition:** Slot is occupied and the Booking is retrievable by its owner and authorized Tutor. **References:** FR-010..013/016/018, BR-001/005/028/034; VAL-007..011/014/030.

4.4 J-04: Customer retrieves their Booking (S1)

Preconditions: Customer owns a committed Booking. **Flow:** Customer requests own list/detail; server authenticates and filters by ownership, returning saved Tutor/Slot/interval and correct business state. **Alternatives/failures:** a known foreign Booking ID yields no private payload; expired session cannot recover protected content. **Postcondition:** no state change; elapsed appointment may display `Past` while retaining `Booked`. **References:** FR-014/016/017, BR-002/030; VAL-012/013.

4.5 J-05: Tutor inspects their schedule and Bookings (S1)

Preconditions: authenticated Tutor. **Flow:** server retrieves only Slots and associated Bookings on the Tutor's authorized schedule. **Alternatives/failures:** another Tutor's ID or manipulated filters cannot expand access; no global Customer data retrieval. **Postcondition:** read-only, persisted schedule/Booking view. **References:** FR-007/015, BR-002/033; VAL-002/012.

4.6 J-06: Customer cancels own Booking (S2)

Preconditions: authenticated owning Customer; active Booking starts at least 30 minutes from authoritative request decision. **Flow:** cancel the Booking as `Cancelled` and reopen its original Slot as one consistent persisted change. **Alternatives/failures:** nonowner, already cancelled or less-than-30-minute attempt is rejected with unchanged records. **Postcondition:** the reopened offer may still be **unbookable** under the independent 60-minute new-Booking rule. **References:** FR-020/021, BR-007/008; VAL-015/016.

4.7 J-07: Customer reschedules once (S2)

Preconditions: owning Customer; current `Booked` appointment and desired unoccupied Slot; original and destination each start at least 60 minutes after the authoritative decision; no earlier successful reschedule. **Flow:** atomically move the same Booking ID to the destination (possibly a different Tutor), reopen the old Slot, record old/new Tutor/Slot/interval and change time, and increment successful-reschedule count once. **Alternatives/failures:** late original/destination, occupied destination, unauthorized owner, quota exhausted, conflict or persistence failure leaves original Booking, old Slot, destination and history unchanged; failed attempt does **not** consume allowance. **Postcondition:** one live reservation with retained identity and audit history. **References:** FR-022..027, BR-009..016; VAL-017..020.

4.8 J-08: Tutor cancels their assigned Booking (S2)

Preconditions: authenticated currently assigned Tutor, active Booking at least 60 minutes away, meaningful nonblank reason. **Flow:** server atomically cancels the Booking, withdraws its Slot instead of reopening it, preserves the reason for the owning Customer and creates their private in-app notification. **Alternatives/failures:** wrong Tutor, too-late request, blank/meaningless reason or failed persistence gives no partial cancellation/withdrawal/notification. **Postcondition:** Booking is `Cancelled`, original Slot withdrawn and the Customer informed through the application. **References:** FR-028..031, BR-017..023/026; VAL-021..023.

4.9 J-09: Customer reads and rereads a private notice (S2)

Preconditions: successful Tutor cancellation created a notice addressed to Customer. **Flow:** Customer reads notice and reason, marks it read if supported, and later opens it again; cancelled Booking detail also shows the authorized reason. **Alternatives/failures:** different Customer, Tutor without applicable authorization or anonymous request gets no notice/reason. **Postcondition:** read state must not erase the message, and no timer automatically expires it. A separate retention/deletion policy remains open. **References:** FR-031/032, BR-021..025; VAL-023/024.

4.10 J-10: Tutor withdraws an unoccupied future Slot (S2)

Preconditions: authenticated owning Tutor; unoccupied future currently offered Slot. **Flow:** Tutor withdraws that Slot, preserves it as history, then may independently create a **new Slot ID** at a permitted time, even if it overlaps the withdrawn interval. **Alternatives/failures:** direct edit/delete/withdraw while an Active Booking exists is forbidden; withdrawn Slot cannot be reactivated under the same identity. **Postcondition:** old Slot remains historical and no longer blocks current-offer overlap checks. **References:** FR-033/034, BR-026/027/032; VAL-025..027.

4.11 J-11: Tutor views Appointment Summary (S3)

Preconditions: authenticated Tutor with authorized scheduling records. **Flow:** server provides counts and corresponding lists for Upcoming Booked, Past Booked and Cancelled, restricted to that Tutor's authorized scope; summary rows may lead to permitted details. **Alternatives/failures:** Customer/anonymous/foreign Tutor access is denied; an appointment in progress has no approved bucket until OD-006 is resolved. **Postcondition:** report read changes no Booking business state and does not imply attendance. **References:** FR-035/036, BR-030/031; VAL-028/029.

5. Functional requirements

Every `FR-xxx` is a stable **whole-product** identifier. `FR-S1-01..19` remain unchanged in the reviewed Sprint 1 source; the explicit crosswalk is in §12.4. `M` means Must for the stated **product baseline**, not an assertion that a later-sprint feature is due in Sprint 1. Sprint allocation is proposed in §3.3. Each `VAL-xxx` identifies a **planned** way to inspect the requirement, never a passed test. Every mutating FR implicitly requires the backend identity, ownership, timing and transaction rules in §§6, 8 and 10.

5.1 Identity and access

ID	Actor and testable requirement, including safe failure	Source	Priority / sprint	Planned proof; dependency
FR-001	**Customer/Tutor:** With existing valid credentials, authenticate and establish only that principal's effective role; reject invalid credentials without authenticated access.	GW §5.2; Sprint 1 baseline	M / S1	VAL-001; identity contract
FR-002	**Authenticated Customer/Tutor:** Return the current principal's own identity and role without accepting a client-supplied impersonation; anonymous requests reveal no protected identity.	GW §5.2	M / S1	VAL-001/002; FR-001
FR-003	**Authenticated Customer/Tutor:** Log out and render the prior session/access unusable for protected operations; the exact invalidation/expiry method must be decided in the auth contract.	GW §5.2	M / S1	VAL-001/002; OD-005
FR-004	**Application/server:** Authenticate and enforce role and record ownership on every protected Slot, Booking, notification and summary action, including crafted requests and URL/ID access; reject unauthorized access without private payload.	GW §5.2/Theme 1; Q02	M / S1, extended S2/S3	VAL-002/036; FR-001/002

5.2 Tutor availability, discovery and new Booking

ID	Actor and testable requirement, including safe failure	Source	Priority / sprint	Planned proof; dependency
FR-005	**Tutor:** Given authorization to their own schedule and a valid future start, publish exactly one explicit sixty-minute Slot; persist it under the authenticated Tutor, without recurrence or auto-generated bulk offers.	Q02-Q04; Theme 1	M / S1	VAL-003; FR-004, OD-001..003
FR-006	**Server on Tutor creation:** Reject malformed/past Slots and overlaps between currently offered Slots of the same Tutor; permit adjacent intervals and independent same-time offers by another Tutor when otherwise valid; leave existing offers unchanged on rejection.	Q03/Q06/Q29; Theme 1	M / S1	VAL-004/005; FR-005
FR-007	**Tutor:** Retrieve own offered Slots and authorized occupied/unoccupied schedule view from committed data; reject another Tutor's protected schedule lookup.	Q02; Theme 1	M / S1	VAL-002/012; FR-004/005
FR-008	**Customer:** Browse Tutor/Slot/time information sufficient to choose a specific offer, without disclosure of another Customer's private reservation; the concrete field allowlist remains an API-contract choice.	Q01/Q02; Theme 1	M / S1	VAL-006/036; FR-004/005
FR-009	**Server/availability view:** Mark an offer bookable only when current, unoccupied, eligible and not too close to its start; an earlier view must not guarantee later acceptance. Hide-versus-disabled presentation is unresolved.	Q05/Q06; Theme 1	M / S1	VAL-006/009; OD-004
FR-010	**Customer:** Given an eligible Slot and no overlapping Active Booking, request one exclusive reservation for their authenticated identity; commit one `Booked` Booking referencing the actual Slot and Tutor before acknowledging success.	Q01/Q05; Theme 1	M / S1	VAL-007; FR-001/004/005

ID	Actor and testable requirement, including safe failure	Source	Priority / sprint	Planned proof; dependency
FR-011	**Server on Booking:** Revalidate offered/unoccupied state, user ownership, cutoff and overlaps immediately before a guarded Booking write; reject stale, invalid, past, too-soon or occupied choices with no partial reservation.	Q05/Q06/Q28; Theme 1; engineering interpretation of T_elig	M / S1	VAL-008/009/014; FR-010
FR-012	**Server on competing Bookings:** Under simultaneous attempts by different Customers for the same exclusive Slot, allow at most one committed Active Booking and never return two successful claims.	Q01; Theme 1	M / S1	VAL-010; FR-010/011
FR-013	**Server on Customer Booking:** Reject the same Customer's overlapping Active reservation, including requests for two different Tutors; accept adjacency under this specific rule, with at most one conflicting success under concurrency.	Q28	M / S1	VAL-004/011; FR-010/011
FR-014	**Customer:** Retrieve a persisted list and detail of **own** Bookings with Booking identity, Tutor/Slot/interval and business status; foreign IDs must not disclose Booking data.	Q02/Q30; Theme 1	M / S1	VAL-012/036; FR-004/010
FR-015	**Tutor:** Retrieve Bookings attached to the Tutor's authorized schedule only; requesting another Tutor's records must not disclose them.	Q02; Theme 1	M / S1	VAL-002/012; FR-004/010
FR-016	**Application:** Authorized reopened/refresh/restart views must reflect successfully committed Slots/Bookings and correct ownership; a failed transaction must not produce a persisted or reported phantom Booking.	GW §5.1; Theme 1	M / S1	VAL-007/014; FR-005/010
FR-017	**Application views:** Display `Booked` accurately and, when a Booking's end is past, optionally derive `Past`; never assert `Completed`, attendance or auto-release solely from elapsed time.	Q30	M / S1	VAL-013; FR-014/015
FR-018	**Application/server:** Return an observable, non-success rejection for forbidden, invalid, stale, conflicting or failed mutations, without leaking protected records or leaving partial state. Specific HTTP status/payload formats remain contract decisions.	Q02/Q05/Q06/Q28; Theme 1	M / S1, extended S2	VAL-002/014/030; OD-005/013
FR-019	**System:** Support the Tutor offer → Customer browse → committed Booking → authorized retrieval journey through a real UI, application/API and durable database, surviving refresh and application restart.	GW §5.1; D1 §2	M / S1	VAL-007/031; engineering contracts outside Issue #9

5.3 Customer cancellation and rescheduling

ID	Actor and testable requirement, including safe failure	Source	Priority / sprint	Planned proof; dependency
FR-020	**Owning Customer:** Cancel their current Booking when the Appointment starts **at least 30 minutes** after the server's authoritative decision; at the exact boundary allow, below it deny; foreign or nonactive Booking cannot be cancelled.	Q07	M / proposed S2	VAL-015; FR-004/014
FR-021	**Server on successful Customer cancellation:** Change Booking to `Cancelled` and reopen its **same** original Slot atomically; keep it unbookable for any later new request failing the independent Q05 cutoff.	Q08/Q05	M / proposed S2	VAL-015/016; FR-020
FR-022	**Owning Customer:** Initiate rescheduling of an existing `Booked` Booking; reject nonowners and disallowed Booking state.	Q09	M / proposed S2	VAL-017; FR-004/014
FR-023	**Server on reschedule:** Require both original Appointment start **and destination Slot start** to be at least 60 minutes away at authoritative decision; reject either too-late side including the just-below-boundary case.	Q10/Q11	M / proposed S2	VAL-018; FR-022

ID	Actor and testable requirement, including safe failure	Source	Priority / sprint	Planned proof; dependency
FR-024	**Owning Customer:** Select a valid destination with a different Tutor if desired; do not restrict rescheduling to the original Tutor, while still enforcing destination occupancy and same-Customer overlap.	Q12/Q28	M / proposed S2	VAL-019; FR-023
FR-025	**Server on reschedule:** Atomically update Booking assignment, occupy destination, release/reopen original Slot and record history; any validation, conflict or persistence failure leaves all related records at their pre-attempt state.	Q13/Q14	M / proposed S2	VAL-020; FR-022..024
FR-026	**Server:** Preserve the Booking ID across a successful reschedule and append a successful-change record with former/new Tutor, Slot/interval and timestamp; rejected attempts do not create successful history.	Q15	M / proposed S2	VAL-020; FR-025
FR-027	**Server:** Permit **at most one successful** reschedule per Booking; a failed attempt leaves the allowance available, while a second successful-change request is rejected.	Q16	M / proposed S2	VAL-017/020; FR-025/026

5.4 Tutor cancellation, Slot withdrawal and notice

ID	Actor and testable requirement, including safe failure	Source	Priority / sprint	Planned proof; dependency
FR-028	**Currently assigned Tutor:** Cancel an Active Booking belonging to the Tutor's authorized schedule only when its start is at least 60 minutes away; exact boundary passes and wrong-Tutor/too-late attempts fail.	Q17/Q18	M / proposed S2	VAL-021; FR-004/015
FR-029	**Tutor/server:** Require a meaningful nonblank reason before accepting Tutor cancellation; reject absent, whitespace-only or semantically empty reason without changing Booking or Slot. Length/format details need approved design.	Q20	M / proposed S2	VAL-021; FR-028, OD-014
FR-030	**Server on successful Tutor cancellation:** Mark Booking `Cancelled` and **withdraw** its original Slot instead of reopening it; preserve trace/history and ensure no partial outcome.	Q19	M / proposed S2	VAL-022; FR-028/029
FR-031	**Server:** On successful Tutor cancellation create one private in-app notice for that Booking's owning Customer, including the authorized reason; the reason also appears in that Customer's cancelled Booking details, never in third-party views.	Q21-Q23	M / proposed S2	VAL-023/036; FR-028..030
FR-032	**Owning Customer:** Retrieve and reread Tutor-cancellation notices after marking them read; absence of automatic time-based expiry must hold, while deletion/retention governance remains undecided.	Q24/Q25	M / proposed S2	VAL-024; FR-031, OD-008
FR-033	**Server/Tutor:** Forbid direct edit, deletion or withdrawal of a Slot while it carries an Active Booking; require the separate authorized cancellation workflow first.	Q26	M / proposed S2	VAL-025; FR-004/028
FR-034	**Owning Tutor:** Withdraw an **unoccupied future** currently offered Slot while retaining its history; permit later creation of a distinct new Slot with a new ID that may overlap the withdrawn interval; reject reopening/reactivating the withdrawn historical Slot.	Q27/Q32	M / proposed S2	VAL-026/027; FR-005/033

5.5 Tutor-only reporting

ID	Actor and testable requirement, including safe failure	Source	Priority / sprint	Planned proof; dependency
FR-035	**Tutor:** Retrieve an Appointment Summary with **counts and corresponding lists** for Upcoming Booked, Past Booked and Cancelled on the Tutor's authorized schedule, calculated from stored Booking state and server time; in-progress classification remains OD-006.	Q31/Q30	M / proposed S3	VAL-028/029; FR-015/017/028
FR-036	**Server:** Deny Customers, anonymous callers and unrelated Tutors access to the Tutor-only summary and its linked Booking details; counts/lists must share the same visibility and classification criteria.	Q31/Q02	M / proposed S3	VAL-029/036; FR-004/035

6. Business rules

Each rule below is a normative **business outcome** sourced to a Q decision or an explicitly identified instructor minimum. The `VAL` column gives a planned check. Rule numbers follow **Q01-Q32** deliberately, making omitted stakeholder decisions detectable; BR-033..035 capture additional instructor/security and integrity constraints. These IDs are **new product IDs**, not replacements for the existing `BR-S1-*` IDs.

ID	Normative rule and boundary	Authority	Verification sprint / anchor
BR-001	One Slot reserves one exclusive **one-to-one** Customer/Tutor session; at most one Active Booking may claim it.	Q01	S1 / VAL-007/010
BR-002	Tutor = Staff and may manage or inspect **their own** authorized schedule; Customer = learner and may inspect only own private Booking data.	Q02	S1 / VAL-002/012
BR-003	Every Availability Slot has a duration of **exactly 60 minutes**.	Q03	S1 / VAL-003/004
BR-004	Tutor creates **individual explicit** Slots; no automatic recurrence or bulk Slot generation is authorized.	Q04	S1 / VAL-003
BR-005	For a new Booking, `SlotStart - authoritative server eligibility time >= 60 minutes`; equality is eligible, even if later commit succeeds after the check. A 59:59 remaining interval fails.	Q05; reviewed T_elig interpretation	S1 / VAL-008/009
BR-006	Two **currently offered** Slots of the **same Tutor** cannot overlap. Adjacent half-open intervals are permitted; different Tutors may independently offer the same time.	Q06	S1 / VAL-004/005
BR-007	Only the owning Customer may cancel an Active Booking at least **30 minutes** before Appointment start, including equality.	Q07	proposed S2 / VAL-015
BR-008	Successful Customer cancellation reopens the **original** Slot, but a future new Booking still independently requires BR-005.	Q08	proposed S2 / VAL-016
BR-009	Only the Booking-owning Customer may initiate its reschedule.	Q09	proposed S2 / VAL-017
BR-010	Reschedule's **original** Appointment must start at least 60 minutes after the authoritative decision; equality passes.	Q10	proposed S2 / VAL-018
BR-011	Reschedule's **destination** Slot must start at least 60 minutes after the authoritative decision; equality passes.	Q11	proposed S2 / VAL-018
BR-012	A valid Customer reschedule may move to another Tutor; original-Tutor-only restrictions are forbidden.	Q12	proposed S2 / VAL-019
BR-013	Reschedule is **all-or-nothing** across assignment, both Slot states and successful history. Failure leaves the previous Booking intact.	Q13	proposed S2 / VAL-020
BR-014	Successful reschedule reopens the original Slot, subject to independent new-Booking eligibility.	Q14	proposed S2 / VAL-020/016
BR-015	Successful reschedule keeps the **same Booking ID** and records old/new Tutor, Slot/time and change timestamp.	Q15	proposed S2 / VAL-020
BR-016	At most **one successful** reschedule per Booking; failures do **not** use that allowance.	Q16	proposed S2 / VAL-017/020
BR-017	Only the **currently authorized Tutor assigned to the Booking** may cancel it as Tutor.	Q17	proposed S2 / VAL-021
BR-018	Tutor cancellation requires Appointment start at least **60 minutes** after the authoritative decision, equality included.	Q18	proposed S2 / VAL-021
BR-019	Successful Tutor cancellation **withdraws** the original Slot; it must not reopen it.	Q19	proposed S2 / VAL-022

ID	Normative rule and boundary	Authority	Verification sprint / anchor
BR-020	Tutor cancellation requires a **meaningful nonblank reason**; an empty/whitespace-only input is invalid. Further textual-validation details require approval.	Q20	proposed S2 / VAL-021
BR-021	Each successful Tutor cancellation creates an in-app notice for the **affected owning Customer only**.	Q21	proposed S2 / VAL-023
BR-022	Tutor-cancellation reason is visible only to the **owning Customer**, not to other Customers or unrelated Tutors in accessible results.	Q22	proposed S2 / VAL-023/036
BR-023	That Customer sees the cancellation reason in both the **in-app notice** and **cancelled Booking detail**.	Q23	proposed S2 / VAL-023
BR-024	Reading a notification does not make it inaccessible; it remains available to reread.	Q24	proposed S2 / VAL-024
BR-025	Notifications do **not** expire automatically with elapsed time. This does not settle user/admin deletion, retention or privacy governance.	Q25	proposed S2 / VAL-024
BR-026	Tutor cannot directly edit, delete or withdraw a Slot with an **Active Booking**; the authorized Booking cancellation must occur first.	Q26	proposed S2 / VAL-025
BR-027	Tutor may withdraw an **own, unoccupied future Slot** and create a **separate new** Slot while preserving withdrawal history.	Q27	proposed S2 / VAL-026
BR-028	One Customer cannot have overlapping **Active Bookings** across any Tutors; **adjacent** appointments do not overlap.	Q28	S1 / VAL-004/011
BR-029	`Asia/Bangkok` (UTC+07:00) is the business timezone; authoritative time comes from the server, not a browser or user-supplied timestamp.	Q29	S1 / VAL-008/013
BR-030	Booking business states are `Booked` and `Cancelled`; `Past` is **derived from time**, not a third state or evidence of `Completed`/attendance.	Q30	S1 (Booked/Past), S2 (Cancelled) / VAL-013/015
BR-031	Only authorized Tutor gets Appointment Summary **counts and lists** for Upcoming Booked, Past Booked and Cancelled. Customer gets no Tutor-only summary.	Q31	proposed S3 / VAL-028/029
BR-032	Historical **withdrawn** Slots do not block a new separate overlapping offer; they are **not reactivated under the same historical Slot ID**.	Q32	proposed S2 / VAL-027
BR-033	Protected actions require server-enforced authenticated role/ownership checks; anonymous, foreign-record and cross-role operations cannot gain private data or mutation rights.	GW §5.2; Theme 1	S1 and extensions / VAL-001/002/036
BR-034	Accepted Booking/Slot mutations commit an internally consistent outcome; rejected validations, contention and persistence failures cannot leave partially claimed Slots, duplicate active reservations or false success. The precise locking/constraint scheme is not selected here.	GW §5.1/Theme 1; Q01/Q06/Q28 outcome	S1 and extensions / VAL-005/010/011/014/020
BR-035	Invalid or nonfuture Slot proposals and attempts to book past, occupied or ineligible offers are rejected without changing committed records; rules beyond confirmed time policies cannot be fabricated.	GW Theme 1; Q03/Q05/Q29	S1 / VAL-004/006/014

Interval rule: `[startA, endA)` overlaps `[startB, endB)` exactly when `startA < endB && startB < endA`. If A ends at 14:00 and B begins at 14:00, then `startB < endA` is false. Consequently, the appointment intervals are adjacent rather than conflicting. This comparison governs BR-006 and BR-028; other validity and cutoff rules still apply.

Rule interactions that need separate checks: A Slot reopened 30 minutes before a session by Customer cancellation is offered again under BR-008, yet BR-005 prevents a new Booking inside its 60-minute cutoff. Tutor cancellation takes the opposite path: BR-019 withdraws the Slot and BR-032 allows a later **separate** overlapping offer. A successful reschedule releases only its original Slot, claims the destination and preserves its Booking identity in **one** transaction; a failed reschedule consumes neither the destination nor the one-success allowance.

7. Non-functional requirements

The following targets separate **required outcomes** from **proposed quantification**. Where a metric is unapproved, the measurement describes a candidate acceptance gate, not a claim that the system already meets it. A quality attribute without an invented availability or performance promise can still have a concrete correctness inspection.

ID	Attribute and requirement / measured target	Source and approval status	Planned check; sprint / approval owner
NFR-001	**Authorization:** every enumerated anonymous, forbidden-role, foreign-Customer and foreign-Tutor protected API path must reject or filter without private payload disclosure. Measure zero unauthorized record disclosure in seeded negative cases.	**Required outcome**, GW §5.2/Q02	VAL-002/036; S1-S3; backend/peer review
NFR-002	**Credential security:** with locally managed passwords, store adaptive salted hashes, not plaintext; tracked repository content contains no real credentials. Verify using test DB inspection and secret/config review.	**Required**, GW §5.2	VAL-034; S1; auth contract
NFR-003	**Durability:** an accepted Slot/Booking and its authorized view must persist after browser refresh and application-process restart against the same data store; no in-memory-only persistence.	**Required**, GW §5.1/D1 §2	VAL-007/031; S1, regression S2/S3
NFR-004	**Concurrent conflict integrity:** all three real-DB races must preserve BR-001/006/028, regardless of interleaving; a 20-attempt same-Slot stress burst and a 10-attempt same-Customer burst are **proposed test sizes**, not approved business capacity limits.	Outcome **required**; run sizes **PROPOSED**, baseline NFR-S1-04/05	VAL-005/010/011; S1; size via OD-010
NFR-005	**Atomic consistency:** accepted cancellation/reschedule/Booking/withdrawal changes all required records together; deliberate persistence failures leave the prior committed state and no false confirmation.	**Required outcome**, Q13/Q19 and Theme 1 integrity	VAL-014/020/022; S1/S2
NFR-006	**Time correctness:** cutoff checks use server-authoritative instants; accept exact 60:00 or 30:00 boundaries as applicable, reject just-below; Bangkok calendar/hour remains stable for a non-Bangkok browser.	**Required**, Q05/Q07/Q10/Q11/Q18/Q29	VAL-008/009/013/015/018/021; S1/S2
NFR-007	**Data minimization:** protected Booking, reason, notification and report responses do not expose third-party identity, cancellation reason, credentials or arbitrary private schedule fields. Review response bodies and negative payloads, not UI controls only.	**Required**, Q02/Q21-Q23/Q31, GW §5.2	VAL-023/029/036; S1-S3
NFR-008	**Retry safety:** repeated/uncertain-response requests cannot violate exclusivity, successful reschedule quota or atomic history. Whether retries return the original success response and any token format/retention are open.	Invariant **required**; protocol **OPEN**	VAL-030/020; S1/S2; OD-013
NFR-009	**Keyboard usability (PROPOSED):** complete login, availability, Booking and own-detail flows by keyboard with visible focus on primary controls; record manual checklist, browser and failures.	**PROPOSED**, baseline NFR-S1-09, not stakeholder-approved	VAL-033; candidate S1/S3; OD-010, @Tanaboonnnnn/@Chxtamos
NFR-010	**Local response time (PROPOSED):** p95 availability and own-Booking-detail requests at or below **2.0 s**, each measured over at least **50** warm-up-excluded requests with documented hardware, fixture size and database.	**PROPOSED**, baseline NFR-S1-10, not instructor-approved	VAL-032; candidate S1/S3; OD-010, team review
NFR-011	**Change traceability:** each implemented requirement must identify source Q/GW/D1, accountable feature/Issue, reviewed contract, planned verification and release evidence; changes to Q rules need recorded stakeholder approval.	**Required process**, D1 Parts A-E / AGENTS.md	VAL-031, document/PR inspection; S1-S3

ID	Attribute and requirement / measured target	Source and approval status	Planned check; sprint / approval owner
NFR-012	**Reproducibility:** engineering evidence must identify code version, environment/setup, DB/seed preparation, test commands and real pass/fail/skip results; mere presence of tests is not a pass.	**Required process**, GW §5.3, D1	VAL-031; S1-S3
NFR-013	**Deployment readiness:** the selected architecture must support container packaging and durable GCP deployment, with build/tests on relevant PRs and repeatable accepted release from `main` after Lab 5. No cloud success is asserted at D1.	**Required later**, GW §§5.1/5.3; Lab 5	VAL-035; Lab 5 and later; separate Issues
NFR-014	**Notice durability:** a Tutor-cancellation notice remains retrievable after being read and is not removed solely by age; additional retention/deletion rights await owner decision.	**Required**, Q24/Q25	VAL-024; proposed S2, OD-008
NFR-015	**Safe diagnostics:** response errors expose no credentials, session tokens, protected reason or internal stack traces; operational logs must avoid secrets and preserve enough non-sensitive context to investigate failed mutations. Exact logging fields/retention are engineering decisions.	Security outcome **required** by GW §5.2; logging structure **PROPOSED**	VAL-034/036; S1-S3; OD-005/008
NFR-016	**Reporting correctness:** for every defined Tutor Summary category, count must equal its corresponding accessible row set under the same time/state predicates; no completion is inferred from elapsed time. Treatment of in-progress sessions remains open.	**Required**, Q30/Q31	VAL-028/029; proposed S3, OD-006/009

Approval rule: Do not copy `NFR-009` or `NFR-010` into a feature's final acceptance contract as a mandatory threshold until the product decision owner and peer reviewer approve it, including environment and workload. No uptime SLA, password complexity number, request-rate limit, backup frequency, pagination size or retention period has been approved by Q01-Q32.

8. Authorization and privacy

Server authorization is authoritative. The role in an authenticated principal, the actual Booking owner and the Tutor assigned to the relevant Slot/Booking determine access. URL parameters, browser state, hidden buttons or a submitted `customerId` / `tutorId` are not an authorization grant.

Legend: **Own** means only records owned/assigned under the stated condition; **Deny** means protected access is forbidden; **N/A** means no product-level action is assigned to that role. Login at the anonymous boundary requires valid credentials, not a public session. S2/S3 actions are **planned**, not present in Sprint 1.

Operation / protected view	Anonymous	Customer	Tutor (Staff)	Planned sprint
Open login / submit valid credentials	Login only	Login	Login	S1
View current authenticated identity	Deny	Own	Own	S1
Logout current session	N/A	Own	Own	S1
Create explicit Slot	Deny	Deny	**Own schedule**	S1
Inspect offered/occupied Tutor schedule	Deny	Deny	**Own schedule**	S1
Discover customer-facing availability	No public policy approved	Authenticated offers, privacy-filtered	N/A as Customer workflow	S1
Create new Booking	Deny	**Own identity**, eligible Slot	Deny under Tutor authority	S1
Retrieve Customer Booking list/detail	Deny	**Owned Bookings only**	No global Customer query	S1

Operation / protected view	Anonymous	Customer	Tutor (Staff)	Planned sprint
Retrieve Tutor Booking schedule	Deny	Deny	**Currently authorized Tutor schedule only**	S1
Cancel a Customer Booking	Deny	**Owning Customer**, Q07 cutoff	Deny via Customer route	S2
Reschedule Booking	Deny	**Owning Customer**, quota and both cutoffs	Deny	S2
Cancel a Booking as Tutor	Deny	Deny	**Currently assigned Tutor**, cutoff + reason	S2
Direct edit/delete/withdraw occupied Slot	Deny	Deny	**Deny while Active Booking exists**	S2
Withdraw unoccupied future offered Slot	Deny	Deny	**Own Slot**	S2
Read Tutor-cancellation notification	Deny	**Notice addressee only**	Deny	S2
Read Tutor-cancellation reason in notice/detail	Deny	**Owning Customer only**	No read authority inferred from Tutor role	S2
View Appointment Summary counts/lists	Deny	Deny	**Own authorized Tutor summary only**	S3
Read another Customer's or another Tutor's restricted records	Deny	Deny	Deny	All

8.1 Protected response and mutation boundaries

- **Customer availability:** sufficient non-sensitive Tutor label/identifier and Slot start/end to make a selection; do not include another Customer's Booking, contact information, cancellation reason or restricted schedule data. Exact field names, availability visibility and sorting are pending OD-004.
- **Customer's own Booking:** stable Booking ID, relevant Tutor and current appointment interval, business status, plus authorized history and reason only when applicable. Do not expose another learner's Booking through direct IDs, searches, pagination or error responses.
- **Tutor's schedule:** current Tutor's permitted Slot/Booking facts. The precise Customer identity representation shown to a Tutor is an unapproved minimum-field decision; it must be justified and bounded in the API/UI contracts. Reschedule-history visibility for prior Tutors, if any, is unresolved and cannot create global access.
- **Cancellation notice:** only the targeted Customer sees its contents; the reason is present in that Customer's cancelled Booking detail. Read state is separate from deletion or automatic expiry. No consent is implied to broadcast reasons to other Tutors or Customers.
- **Write authorization:** Booking owner comes from the authenticated Customer, Tutor ownership from the actual Slot and currently assigned Booking, and cancellation/reschedule eligibility from server state. Any failed authentication or ownership check must precede protected detail disclosure.
- **Error safety:** choose precise 401/403/404 behavior, conflict codes and error fields in separate feature API contracts; whatever the choice, rejected access must disclose no protected information. Never use a success response for an uncommitted write.

9. Logical domain and lifecycle

The following is a **logical requirements model**, not a relational schema, migration design or API DTO. Detailed entity fields, keys, constraints, indexing, locking and storage implementation are assigned to Issue #11 and subsequent Feature SDS.

Concept	Required logical information and relationships	Invariants / lifecycle
Authenticated user	Stable identity, effective role Customer or Tutor; session lifecycle	Identity is server-trusted; a principal cannot gain another role through submitted IDs
Tutor schedule	Association of one Tutor with their own Slots and associated authorized Bookings	Cross-Tutor reads/writes restricted; current offer list differs from withdrawal history
Availability Slot	Stable Slot identity, Tutor owner, scheduled start/end, currently offered or withdrawn historical meaning, occupancy relationship	Exactly 60 minutes; one Tutor per Slot; currently offered same-Tutor Slots cannot overlap; withdrawn ID not reactivated
Booking	Stable Booking identity, owning Customer, **current** Slot/Tutor assignment, scheduled interval and `Booked` or `Cancelled` status	At most one Active Booking per Slot; one Customer cannot hold overlapping Active Bookings; identity survives reschedule
Successful reschedule history	One record per successful change, including Booking ID, old/new Tutor and Slot/interval, server change timestamp	At most one successful entry per Booking under Q16; a rejected attempt does not produce a successful entry
Cancellation reason	Meaningful nonblank reason supplied on Tutor cancellation and associated with its Booking	Owning Customer sees it in notice and cancelled Booking detail; unauthorized callers never see it
In-app notification	Owner Customer, related Tutor-cancelled Booking, private message/reason and read-state meaning	Remains accessible when read; no automatic age expiry; deletion/retention rules open
Historical withdrawn Slot	Former Slot identity, Tutor, original interval and withdrawn status/history	Not a current offer; does not block new separate overlapping offers; never reactivated under same Slot ID
Tutor Appointment Summary	Authorized Tutor scope and derived categories: Upcoming Booked, Past Booked, Cancelled	Counts equal corresponding lists; no global Customer view; in-progress category pending

9.1 Booking state and Slot lifecycle

The only agreed **Booking business states** are `Booked` and `Cancelled` (Q30).

1. **New Booking success:** eligible Slot becomes occupied and a Booking is created in `Booked`. Failure creates no Booking or occupied change.
2. **Customer cancellation:** `Booked → Cancelled` for an authorized owner at the 30-minute cutoff; original Slot becomes a currently offered unoccupied Slot (Q07-Q08).
3. **Customer reschedule:** Booking **remains** `Booked` and retains its ID; the assignment moves atomically to the destination; original Slot reopens and successful history is appended (Q09-Q16).
4. **Tutor cancellation:** `Booked → Cancelled` for the currently assigned authorized Tutor at the 60-minute cutoff, requiring reason; original Slot is **withdrawn**, and the private notification is created (Q17-Q25).
5. **Tutor withdrawal without Booking:** offered unoccupied future Slot becomes historical withdrawn; a **new Slot**, with a distinct ID, may subsequently be offered (Q26-Q27/Q32).
6. **Elapsed time:** past the appointment **end**, the display may derive `Past`; `Booked` remains `Booked`. There is no `Completed`, `Attended`, `No-show`, auto-cancel or implied auto-release transition.

An active Booking is one still in `Booked`, **even if the lesson's end is past**. This protects historical consistency and prevents inappropriate automatic release. State transition details at time boundaries and unsupported repeated cancellation attempts require explicit negative validation. Which cancellation actor performed a `Cancelled` transition and the historical detail available to each role must be represented without assuming an additional business state.

9.2 Cardinality and conservation obligations

- One Slot has one owning Tutor and is attached to **at most one Active Booking** at a given time; one Customer may have multiple Bookings only when none of the Active intervals overlap.
- A successful reschedule conserves **one Booking ID** while moving its *current* appointment link. Old/new links remain in history rather than becoming two live Bookings.

- Customer cancellation preserves the Slot as a potentially available offer. Tutor cancellation preserves the Slot **as withdrawn history**. These outcomes are intentionally different.
- Failed cancellation, reschedule, creation or Booking cannot leave part of the change committed, including premature notification creation, wrong successful-reschedule count or incorrect occupancy.
- Reports are projections of the authoritative logical entities; a report count cannot independently redefine Booking state, ownership or elapsed-time semantics.

10. Time, conflicts and concurrency

10.1 One timezone and explicit inequalities

All user-visible appointment times use `Asia/Bangkok` (UTC+07:00). The server controls the clock used for acceptance checks. Persisted instants and transport representation are engineering choices, but must preserve this business meaning independent of the client's timezone, locale and clock skew.

Let `S` be the relevant appointment or Slot start instant and `T` the authoritative server instant at the applicable guarded decision point. Equality at each confirmed cutoff **passes**, if all other rules pass.

Action	Confirmed eligibility formula	Example for 15:00 start (Bangkok)	Primary source
New Booking	`S - T_elig >= 60 min`	`14:00:00` eligible; `14:00:01` too late	Q05; BR-005
Customer cancellation	`S - T >= 30 min`	`14:30:00` eligible; `14:30:01` too late	Q07; BR-007
Reschedule old Appointment	`OldStart - T >= 60 min`	Old `15:00`; `14:00:00` eligible	Q10; BR-010
Reschedule destination	`NewStart - T >= 60 min`	Destination `15:00`; `14:00:01` too late	Q11; BR-011
Tutor cancellation	`S - T >= 60 min`	`14:00:00` eligible; `14:00:01` too late	Q18; BR-018
Tutor Slot creation	Start must be valid and in future; **no Q05 new-Booking cutoff imposed**	Future valid start; further scheduling grid/hours/horizon open	Q03/Q04; Theme 1; OD-001..003

New-Booking precision (`T_elig`): The reviewed Sprint 1 baseline defines a single server-authoritative time sampled **inside the transaction after contention waits and immediately before final guarded validation/write**. An HTTP request for a 15:00 Slot may arrive at 13:59:59 yet wait until `T_elig=14:00:01`; it must be denied. If `T_elig=14:00:00`, a later commit timestamp alone does not invalidate the previously valid guarded check. This is an **engineering interpretation** for review, not permission to weaken Q05 or ignore other checks. The Feature SDS must confirm the actual transaction/time-control design and tests (AC-S1-35, VC/TEST-S1-39).

For **later** cancellation/rescheduling cutoffs, Q07/Q10/Q11/Q18 settle the numerical boundary and server-clock authority through Q29; the exact transactional sampling order, lock behavior and retry handling must still be set in their Feature SDS. Using the same final-guard pattern as `T_elig` is a **proposal** for consistent implementation, not an independently confirmed stakeholder Q decision.

10.2 Half-open overlap and Current versus Historical

Two intervals intersect iff `a.start < b.end && b.start < a.end`. Exact adjacency (`a.end = b.start`) is allowed under BR-006 and BR-028. Partial overlap by even one positive-duration instant is prohibited for same-Tutor currently offered Slots and the same Customer's Active Bookings.

A **withdrawn historical** Slot must be excluded from the set of current offers checked against new Slot creation. This does **not** exempt a current occupied offer from same-Tutor overlap rules, and it does not mean an old historical Slot may be reactivated. Different Tutors' offered intervals may coincide; **one Customer** still cannot reserve overlapping intervals across them.

10.3 Three mandatory race proofs

Hazard	Simultaneous operation	Required final committed state	Planned contract/test
Race A: Tutor current offers	Same Tutor creates two partially overlapping 60-minute offered Slots	At most one of the conflicting offers remains current; loser has no partial Slot. If different Tutors create same time, both can be valid.	Slot contract, VAL-005; baseline AC-S1-13 / VC-S1-37
Race B: exclusive Slot	Two distinct Customers attempt the same eligible Slot	At most one Active Booking occupies that Slot. No second success or orphan occupancy.	Booking contract, VAL-010; baseline AC-S1-25 / VC-S1-25
Race C: Customer across Tutors	One Customer simultaneously requests two distinct overlapping offers from different Tutors	At most one conflicting Active Booking commits for that Customer, despite each Slot being individually free.	Booking contract, VAL-011; baseline AC-S1-26 / VC-S1-26

For all three, controlled concurrency tests must start requests under actual overlap/interleaving and inspect **committed database state**, not only returned HTTP codes or disabled frontend controls. The System/Feature SDS must choose an isolation/locking/constraint strategy that enforces these outcomes for every write path. No particular SQL lock, ORM, idempotency key or isolation level is approved by this SRS.

10.4 Atomic change, stale reads and uncertain responses

- A stale availability view cannot reserve or grandfather an occupied/withdrawn/too-soon Slot. The server checks current state at the final write guard, and a rejected request changes no Booking or Slot relationship.
- A duplicate click, network retry or unknown response after a possible commit must preserve the invariants. Returning an existing success versus a conflict is an **unresolved transport/idempotency decision** (OD-013).
- A failed Customer cancellation leaves `Booked` and occupancy unchanged; a successful cancellation reopens the same Slot. A failed Tutor cancellation must not leave the Slot withdrawn or a false private notice.
- A failed reschedule leaves the original Booking assignment and its Slot occupied, destination unoccupied, no success history and the one-success quota unconsumed. This also holds when destination is taken concurrently.
- Under a lost database write or rollback, no client may be told a reservation/change succeeded unless the corresponding complete state committed. Exact error vocabulary and reconciliation behavior belong to the later design contracts.

11. Exception and boundary scenarios

This is a requirements-level outcome matrix. `VAL` cases are **planned**, and actual fixtures, HTTP status codes and test files are owned by later STS/implementation Issues.

Scenario	Trigger / precondition	Observable required outcome	Governing rules; planned check
Invalid Slot start/duration	Missing, malformed, past or non-sixty-minute proposed offer	No invalid Slot, no changed existing offer	BR-003/035; VAL-003/004
Same Tutor overlap	Existing 13:00-14:00 current offer; proposed 13:30-14:30	Reject proposed Slot; original preserved	BR-006; VAL-004
Slot adjacency	Existing 13:00-14:00; proposed 14:00-15:00	Adjacency itself does not reject otherwise-valid offer	BR-006; VAL-004
Cross-Tutor offers	Tutors A/B each create 13:00-14:00	Both current offers allowed if each independently valid	BR-006; VAL-004
Unauthorized Slot write	Customer or Tutor B targets Tutor A's schedule	No change to Tutor A's offers; no protected disclosure	BR-002/033; VAL-002
Anonymous Booking	Caller lacks session	No Booking or protected data	BR-033; VAL-002
Foreign Customer Booking	Customer B guesses A's Booking ID	No private details or related notification/reason exposed	BR-002/022/033; VAL-002/036

Scenario	Trigger / precondition	Observable required outcome	Governing rules; planned check
Foreign Tutor schedule	Tutor B guesses Tutor A's schedule/Booking ID	No foreign schedule or private Booking disclosed	BR-002/033; VAL-002
Too-soon new Booking	Slot at 15:00; `T_elig=14:00:01`	Reject without occupation	BR-005; VAL-008
Exact new-Booking boundary	Slot at 15:00; `T_elig=14:00:00`	Lead-time rule passes, subject to all other guards	BR-005; VAL-008
Cutoff crossed while waiting	HTTP arrives 13:59:59; final `T_elig=14:00:01`	Reject, regardless of early HTTP arrival	BR-005/034; VAL-009
Occupied Slot/stale list	Availability was earlier displayed, but now occupied/withdrawn	No duplicate or false Booking; view reconciles	BR-001/034; VAL-006/014
Same Customer across Tutors	Active 13:00-14:00, candidate 13:30-14:30	Reject overlapping Booking	BR-028; VAL-011
Adjacent Customer appointment	Active ends 14:00; proposed begins 14:00	Overlap rule permits; independent lead-time applies	BR-028/005; VAL-004/011
Race A	Concurrent same-Tutor overlapping Slot creation	At most one conflicting offer committed	BR-006/034; VAL-005
Race B	Two Customers, one Slot concurrently	At most one Active claim committed	BR-001/034; VAL-010
Race C	One Customer, overlapping Tutor Slots concurrently	No two overlapping Active bookings commit	BR-028/034; VAL-011
Fault during Booking persistence	Failure before transaction completion	No phantom success, partial Booking or inconsistent occupation	BR-034; VAL-014
Customer cancellation at cutoff	At `start - 30:00` versus just after	Equality eligible; just-after denied, no partial state	BR-007; VAL-015
Reopened Slot inside booking cutoff	Customer cancels 30 minutes before start	Original Slot reopened, but new Booking at fewer than 60 minutes rejected	BR-008/005; VAL-016
Reschedule wrong owner / quota	Foreign Customer or already one success	Reject, old Booking and history unchanged	BR-009/016; VAL-017
Reschedule original/destination cutoffs	Old or destination starts at `T + 59:59`	Reject even if other start qualifies; exact `T + 60:00` eligible	BR-010/011; VAL-018
Reschedule across Tutors	Valid new Tutor offered Slot	Accept; same Booking ID, new Tutor/Slot, old Slot reopened	BR-012..015; VAL-019/020
Reschedule failure/rollback	Destination occupied, transaction fault, collision or late boundary	No partial move, lost reservation, false history or consumed quota	BR-013/016; VAL-020
Tutor cancellation cutoff/ownership	Wrong assigned Tutor, `start - 59:59`, or exact 60:00	Wrong/late denied; exact boundary eligible with valid reason	BR-017/018; VAL-021
Missing Tutor cancellation reason	Empty, whitespace-only or meaninglessly empty reason	No cancellation/withdrawal/notice	BR-020; VAL-021
Tutor cancellation state	Valid cancellation reason and cutoff	Booking Cancelled, original Slot withdrawn **not reopened**, private notice created	BR-019/021; VAL-022/023
Notification privacy/re-read	Wrong Customer reads; owner reads twice	Wrong reader denied; owner sees reason and may reopen read notice	BR-021..025; VAL-023/024
Direct occupied Slot change	Tutor edits/deletes/withdraws own Slot with Active Booking	Deny direct change until authorized Booking cancellation	BR-026; VAL-025

Scenario	Trigger / precondition	Observable required outcome	Governing rules; planned check
Historical withdrawn overlap	Tutor withdraws unoccupied future 13:00-14:00 then creates new 13:00-14:00 offer	New distinct ID allowed; old history preserved; old ID not reactivated	BR-027/032; VAL-026/027
Summary and protected counts	Tutor requests own categories; Customer or foreign Tutor requests same	Authorized counts equal lists; unauthorized gets no report	BR-030/031; VAL-028/029
Browser outside Bangkok	UTC/other-zone device displays Slot and sends operation	Same intended Bangkok hour and cutoff evaluation; server time wins	BR-029; VAL-013
Booking ended	Now after appointment end; no cancellation	`Past` may appear, `Booked` persists; no Completed/attendance or auto-release	BR-030; VAL-013

12. Source and requirement traceability

12.1 Provenance and audit rules

The tables below are **self-contained Part A coverage evidence**; they are **not** the separate Deliverable 1 Part B feature inventory or `docs/traceability.md` assigned to Issue #10.

- **SRC-Q:** stakeholder discovery decisions Q01-Q32 as recorded in the already reviewed/merged Sprint 1 requirements source; each row below is marked **Confirmed stakeholder baseline**, not independently instructor-approved.
- **SRC-GW-v1:** `group_work_specs_v1.pdf`, issued 6 October 2026, explicitly **Draft for instructor review**. Requirements drawn from it may need delta review against the missing final Master.
- **SRC-D1-v1:** `project_deliverable_1_v1.pdf`, due 8 November 2026; inspected from the supplied project archive.
- **SRC-S1:** `docs/requirements.md` as present on the `sprint1` baseline after merged PR #7; retains the original FR/BR/NFR/AC/VC naming.
- **SRC-AGENT:** repository root `AGENTS.md` merged through PR #20; this governs workflow, not new product behavior.

Coverage criterion: every Q decision has a product rule, an appropriate FR or NFR, a planned verification and proposed delivery sprint. An unapproved additional policy is listed in Section 13, not silently inserted into a Q row. At the requirements-writing stage, `VAL` is a test **plan**, not executed validation.

12.2 Stakeholder Q01-Q32 decision coverage audit

Every decision in this table has approval state **Confirmed in stakeholder discovery baseline**. The current document, final instructor source and engineering contract have **not** yet received their own approval. Some decisions were confirmed for the **product** but are scheduled in a **proposed** future sprint.

Q	Confirmed meaning (condensed without changing boundary)	Product FR / BR / applicable NFR	Target verification sprint	Planned evidence
Q01	Exclusive one-Customer/one-Tutor session	FR-010/012; BR-001; NFR-004	S1	VAL-007/010
Q02	Tutor = Staff, Customer = learner, schedule/Booking ownership	FR-004/007/014/015; BR-002; NFR-001	S1	VAL-002/012
Q03	Slot lasts exactly 60 minutes	FR-005/006; BR-003	S1	VAL-003/004
Q04	Tutor creates Slots individually, without bulk/recurrence	FR-005; BR-004	S1	VAL-003
Q05	New Booking at least 60 minutes ahead; equality eligible	FR-009/011; BR-005; NFR-006	S1	VAL-008/009
Q06	Same-Tutor current offered Slot non-overlap, adjacency valid	FR-006; BR-006; NFR-004	S1	VAL-004/005

Q	Confirmed meaning (condensed without changing boundary)	Product FR / BR / applicable NFR	Target verification sprint	Planned evidence
Q07	Only owner Customer cancels at least 30 minutes ahead	FR-020; BR-007; NFR-006	proposed S2	VAL-015
Q08	Customer cancellation reopens original Slot, Q05 still applies	FR-021; BR-008	proposed S2	VAL-016
Q09	Only owning Customer initiates reschedule	FR-022; BR-009	proposed S2	VAL-017
Q10	Original appointment at least 60 minutes ahead to reschedule	FR-023; BR-010; NFR-006	proposed S2	VAL-018
Q11	Destination Slot at least 60 minutes ahead	FR-023; BR-011; NFR-006	proposed S2	VAL-018
Q12	Cross-Tutor reschedule allowed	FR-024; BR-012	proposed S2	VAL-019
Q13	Reschedule atomic/all-or-nothing	FR-025; BR-013; NFR-005	proposed S2	VAL-020
Q14	Reschedule reopens old Slot	FR-025; BR-014	proposed S2	VAL-020
Q15	Same Booking ID; successful old/new Tutor, Slot/time, timestamp history	FR-026; BR-015	proposed S2	VAL-020
Q16	One successful reschedule max; failures do not consume quota	FR-027; BR-016; NFR-008	proposed S2	VAL-017/020
Q17	Only currently assigned authorized Tutor cancels	FR-028; BR-017; NFR-001	proposed S2	VAL-021
Q18	Tutor cancellation at least 60 minutes ahead	FR-028; BR-018; NFR-006	proposed S2	VAL-021
Q19	Tutor cancellation withdraws original Slot, does not reopen	FR-030; BR-019; NFR-005	proposed S2	VAL-022
Q20	Tutor cancellation reason meaningful and nonblank	FR-029; BR-020	proposed S2	VAL-021
Q21	Tutor cancellation creates in-app notice only for owning Customer	FR-031; BR-021; NFR-007	proposed S2	VAL-023
Q22	Reason visible only to affected owning Customer	FR-031; BR-022; NFR-007	proposed S2	VAL-023/036
Q23	Reason in both private notice and cancelled Booking detail	FR-031; BR-023; NFR-007	proposed S2	VAL-023
Q24	Read notice remains accessible	FR-032; BR-024; NFR-014	proposed S2	VAL-024
Q25	No automatic time-based notice expiry; other retention open	FR-032; BR-025; NFR-014	proposed S2	VAL-024
Q26	Active Booking blocks direct Slot edit/delete/withdrawal	FR-033; BR-026	proposed S2	VAL-025
Q27	Tutor withdraws own unoccupied future Slot, recreates separate Slot; history retained	FR-034; BR-027	proposed S2	VAL-026
Q28	One Customer's Active Bookings never overlap across Tutors; adjacency allowed	FR-013; BR-028; NFR-004	S1	VAL-004/011
Q29	Bangkok timezone, server-authoritative clock	FR-011/017; BR-029; NFR-006	S1	VAL-008/013
Q30	States Booked/Cancelled; Past derived and not Completed	FR-017/035; BR-030; NFR-016	S1 (Booked/Past), S2 (Cancelled), S3 (report)	VAL-013/015/028
Q31	Tutor-only summary with Upcoming Booked/Past Booked/Cancelled counts and lists	FR-035/036; BR-031; NFR-016	proposed S3	VAL-028/029
Q32	Withdrawn history does not block separate overlap or permit same-ID reactivation	FR-034; BR-032	proposed S2	VAL-027

Audit result at drafting: Q01-Q32 are each accounted for in a distinct row, with an authorized meaning and planned validation. No Q decision is excluded from the whole-product baseline. Formal source approval and code/test completion remain separate gates.

12.3 Available instructor requirements to SRS mapping

Instructor-draft source	Requirement actually stated in examined handout	Whole-product mapping	Earliest planned verification	Source status
GW v1 §5.1	Working UI → application/API → persistent DB, meaningful domain transaction, restart durability	FR-016/019; NFR-003; BR-034	S1: VAL-007/031	**Draft; final master not checked**
GW v1 §5.2	Real login, logout, current identity, session lifecycle, two roles	FR-001..004; NFR-002	S1: VAL-001/034	**Draft; final master not checked**
GW v1 §5.2 / Theme 1	Customer own Booking only, Tutor authorized schedule, backend role checks	FR-004/007/014/015; BR-002/033; NFR-001/007	S1: VAL-002/012/036	**Draft; final master not checked**
GW v1 §6 Theme 1	Staff creates Slot, Customer views/Books/retrieves; reject occupied/past/invalid; competing single-Slot claims	FR-005..016; BR-001/035; NFR-004/005	S1: VAL-003..014	**Draft; eligibility not checked**
GW v1 §6 Theme 1	Complete product cancellation/rescheduling, schedule administration, appointment reporting	FR-020..036; BR-007..027/031/032	proposed S2/S3: VAL-015..029	**Draft; combined with Q decisions**
GW v1 §5.3	Reviewed GitHub workflow, automated PR build/test and later Docker/GCP delivery	NFR-011/012/013	S1 contract/review VAL-031; Lab 5 VAL-035	**Draft; separate engineering Issues**
D1 v1 §§3.1-3.4	Whole-product SRS with FR/BR/NFR, measurable verification and 3-sprint allocation	Document §§0-18; NFR-011/012	Part A document inspection; later Parts B-D	**Version examined; master cross-check pending**

No line in the available handouts prescribes a tutoring price, service modality, weekday hours, maximum scheduling horizon, minute increment, performance p95 or automatic attendance state. Adding such a constraint would require an explicit source or approved decision.

12.4 Sprint 1 baseline ID crosswalk and consistency

The original `docs/requirements.md` is a **Sprint 1 document** and retains all its IDs. The product-level IDs in this SRS are new identifiers and do not rewrite or renumber the source. The old file's header documents its initial draft stage; GitHub PR #7 is now **merged**, as verified from PR history. The single approved surgical correction in Issue #9 updates its §16.1 AC range to `AC-S1-01..35`; `AC-S1-35` itself already exists in that file and maps to `VC/TEST-S1-39`.

FR crosswalk: the meanings of `FR-S1-01..19` correspond **in order** to `FR-001..019`. Specifically:

Sprint 1 IDs	Whole-product IDs	Requirement group
FR-S1-01..04	FR-001..004	Login, identity, logout, backend authorization
FR-S1-05..07	FR-005..007	Tutor Slot creation, validity/overlap, own schedule
FR-S1-08..09	FR-008..009	Customer availability and bookability
FR-S1-10..13	FR-010..013	Booking, cutoff, exclusive claim, Customer overlap
FR-S1-14..17	FR-014..017	Private reads, persistence and Booked/Past
FR-S1-18..19	FR-018..019	Failure handling and full-stack vertical slice

BR crosswalk: the Sprint 1 source sometimes separates one Q rule into several narrower guards. These remain distinct IDs in that document.

Baseline BR-S1	Product BR	Baseline BR-S1	Product BR
01	BR-001	12	BR-029
02	BR-003	13	BR-005/034 (T_elig interpretation)
03	BR-004	14	BR-030
04	BR-002	15	BR-002
05	BR-006	16	BR-002
06	BR-006	17	BR-033
07	BR-005	18	BR-034
08	BR-001	19	BR-001/028/034
09	BR-028	20	BR-002/033
10	BR-028	21	BR-005/034
11	BR-035	22	BR-030

NFR crosswalk: the first number below is the original Sprint 1 suffix; proposed versus required status must carry forward unchanged.

Baseline NFR-S1	Product NFR	Baseline NFR-S1	Product NFR
01	NFR-001/007	07	NFR-012
02	NFR-002	08	NFR-007/015
03	NFR-003	09	NFR-009 (**PROPOSED**)
04	NFR-004 (stress size proposed)	10	NFR-010 (**PROPOSED**)
05	NFR-004 (stress size proposed)	11	NFR-005
06	NFR-006	12	NFR-011

Existing AC and validation anchors (not duplicated as a Part E STS):

Existing Sprint 1 evidence anchor	Product verification and meaning
AC-S1-01..07; VC/TEST-S1-01..07	VAL-001/002; login, identity, logout, authorization
AC-S1-08..14; VC/TEST-S1-08..14, 37	VAL-003..005/013; Slot duration, ownership, overlap and Race A
AC-S1-15..18; VC/TEST-S1-13/15/22	VAL-006/013; availability, privacy, stale view
AC-S1-19..24; VC/TEST-S1-16..24	VAL-007/008/011; Booking, lead-time and overlap
AC-S1-25; VC/TEST-S1-25	VAL-010; Race B
AC-S1-26; VC/TEST-S1-26	VAL-011; Race C
AC-S1-27..29; VC/TEST-S1-22/27/33/38	VAL-014/030; rollback, conflict, duplicates
AC-S1-30..34; VC/TEST-S1-28..32	VAL-012/013; owned reads, Tutor visibility and Past
AC-S1-35; VC/TEST-S1-39	**VAL-009; booking request crossing cutoff while waiting**
VC/TEST-S1-34..36	VAL-031/032/033/034; planned reproducibility, security, UX and performance checks

All `AC-S1-01..35` and `VC-S1-01..39` / `TEST-S1-01..39` remain owned by the original baseline; references here are cross-references, not redefinitions or claims that test code exists.

13. Open decisions and approval gates

This register is the **explicit home for uncertainty**. A proposed choice below is not approved simply because the SRS names it. Existing confirmed Q01-Q32 behavior must remain intact while decisions are obtained. `OD` IDs are SRS decision-register identifiers, not requirements or new stakeholder Q numbers.

ID	Question and existing evidence	Affected scope / reasonable options	Decision owner and required stage	Status / risk
OD-001	Which Tutor working days, opening hours, holidays and calendar boundaries apply? Q03 fixes duration, not operating hours; baseline TBD-S1-01 is open.	FR-005/006, BR-035; explicitly approve a restriction or document why none is imposed.	@Tanaboonnnnn proposes; @Chxtamos reviews **before Slot Feature SDS**; stakeholder approval for any new business constraint	**OPEN.** An invented workweek could reject valid requests.
OD-002	Is there a maximum advance scheduling/booking horizon? Q05 provides only a **minimum** lead time; baseline TBD-S1-02 is open.	FR-005/009/011; choose approved upper bound or explicitly decide none, without assuming 7/14/30 days.	Stakeholder decision documented by team and reviewed **before Slot/Booking contracts**	**OPEN.** Inconsistent form and server validation risk.
OD-003	What start-minute increments are permitted? Exactly 60-minute duration does not imply starts at :00 or :30; baseline TBD-S1-03.	FR-005/006; approve a grid or leave all otherwise-valid minute instants available as a documented decision.	Stakeholder/team + peer **before Slot SDS/STS**	**OPEN.** Hidden UI grid could alter business scope.
OD-004	Should Customer availability show only bookable Slots or all offers with unavailable markers? Which Tutor labels, sort/filter/page and sensitive fields? Baseline TBD-S1-04 and PR #7 review.	FR-008/009, FR-014/015, NFR-007. Bookable-only is **proposed**, not approved; either presentation must avoid false availability and private data.	Team API/UI contract, @Chxtamos **before S1 feature implementation**	**OPEN PROPOSAL.** UX/privacy ambiguity.
OD-005	What are session/token invalidation, expiry, refresh and replay safeguards, safe error schemas and exact transactional/time-control mechanisms? Baseline TBD-S1-05.	FR-001..004/011/018, NFR-001/002/015; evaluate technologies in System/Feature SDS without changing the business outcomes.	Engineering design by @Tanaboonnnnn; peer review **before code**	**OPEN DESIGN.** Security and cutoff implementation risk.
OD-006	Where does an appointment **in progress** fall in Upcoming Booked versus Past Booked? Q30 says Past is derived; the S1 glossary derives it after **end**. Q31 names only the three summary categories.	FR-035, BR-030/031, NFR-016; options include a separately disclosed in-progress subgroup or a specifically approved treatment, without inventing a third Booking **state**.	Stakeholder clarifies; peer review **before S3 Summary SDS/STS**	**OPEN.** Counts could silently omit or misclassify a current lesson.
OD-007	May the same withdrawn historical Slot be reactivated? **No**: Q32 explicitly prohibits reactivation. Only whether any special restoration request needs a **new separately approved** workflow remains undecided.	FR-034, BR-032. Normal design uses **new Slot ID**; no override is authorized by current baseline.	Q32 already governs; any proposed exception needs **new stakeholder approval before scope change**	**REACTIVATION PROHIBITED; hypothetical exception UNSPECIFIED.** Do not treat prohibition as an open permission.
OD-008	How long may private reasons/notifications/history be retained or deleted, and by whom? Q24/Q25 require rereadability and no automatic time expiry, but do not resolve user deletion or formal retention governance.	FR-031/032, NFR-007/014/015; no auto-expiry, pending deletion/export/access policy.	Stakeholder/data owner and security reviewer **before notice design/sign-off**	**OPEN.** Privacy and durability tradeoff.

ID	Question and existing evidence	Affected scope / reasonable options	Decision owner and required stage	Status / risk
OD-009	Which filtering, ordering, pagination, category counting snapshots and historical associations apply to Tutor Summary? Q31 gives categories/counts/lists, not query shape or page size.	FR-035/036, NFR-016, OD-006; approve lists and counts against same filters and visibility.	Team + @Chxtamos **before S3 Summary SDS/STS**	**OPEN DESIGN.** Count/list disagreement.
OD-010	Are keyboard-only flow and p95 <= 2.0 s acceptable product NFR targets? What machines, fixtures, concurrency sample size and exact workloads are approved? PR #7 keeps thresholds proposed.	NFR-004 run sizes, NFR-009/010 targets; accept, revise or defer explicit criteria with recorded methodology.	@Tanaboonnnnn and @Chxtamos, stakeholder/instructor if required **before affected STS approval**	**OPEN PROPOSAL.** No measured performance/UX result yet.
OD-011	Is the examined `group_work_specs_v1.pdf` the latest final Master? The available copy states **Draft for instructor review, issued 6 Oct 2026**; D1 references `group_work_specs.docx` as master. Searches in the locally approved repository and connected Drive did not locate a newer authoritative master.	All source-derived GW/Theme 1 entries. Obtain current instructor-issued `group_work_specs.docx` or dated official amendment; compare every Part A/Theme rule, record version/date and differences.	Instructor/TA source; @Tanaboonnnnn verifies and @Chxtamos reviews **before formal SRS/Deliverable sign-off**	**UNVERIFIED; FORMAL SIGN-OFF BLOCKED.** PR may proceed as a labeled draft.
OD-012	Is Theme 1 officially assigned to this two-student team? GW v1 §6 allocates themes from students' final ID digits; no dated assignment/ID evidence is present in reviewed materials.	Entire tutoring context. Obtain instructor/TA theme confirmation or verifiable eligibility record.	Instructor/TA; @Tanaboonnnnn supplies; @Chxtamos checks **before formal sign-off**	**UNVERIFIED; FORMAL SIGN-OFF BLOCKED.** Do not claim eligibility proved.
OD-013	What result, idempotency key and replay window should repeated Booking/reschedule/cancel submissions use after network uncertainty? Q invariants constrain outcomes but not response protocol.	FR-018/025/027, NFR-008; choose recoverable reconciliation and duplicate policies with real-DB validation.	Engineering + @Chxtamos **before related Feature SDS/STS**	**OPEN DESIGN.** Duplicate effects and misleading response risk.
OD-014	What exactly qualifies as a `meaningful` Tutor cancellation reason beyond nonblank? Q20 rules out empty/meaningless text but sets no byte count or prohibited vocabulary.	FR-029, BR-020; propose a reviewable validation policy; do not invent a word/minimum-length requirement as confirmed.	Stakeholder/team review **before S2 Tutor cancellation SDS/STS**	**OPEN DETAIL.** Overly weak or arbitrary rejection risk.
OD-015	Which historical Booking/reschedule fields may a **previous Tutor** see after cross-Tutor reassignment? Q12/Q15 require history; Q02 restricts protected Tutor scope.	FR-015/024/026, NFR-007; evaluate authorized history without granting foreign schedule lookup.	Stakeholder/team privacy and peer **before S2 reschedule SDS/STS**	**OPEN PRIVACY DESIGN.** History must exist without broadening unrelated Tutor access.

13.1 Source-verification record and required delta procedure

Item checked	Evidence actually available	Finding	Approval consequence
Group Work Specification	`group_work_specs_v1.pdf` in user-supplied `group_project_spec_files.zip`; 15 pages; title page states "Semester 1/2026; Draft for instructor review; Issued: 6 October 2026"	Contains GW §§5.1-5.3 and Theme 1 §6, including full-stack, authentication, team workflow and appointment example	**Draft verified as draft; final master unverified**
Deliverable 1 instructions	`project_deliverable_1_v1.pdf` in same archive; 8 pages; due 8 Nov 2026 at 23:59	Part A requires whole-product FR/BR/NFR and reviewable assumptions; Parts B-D cover product, E-F Sprint 1	Applied as available D1 instruction; check against later amendments

Item checked	Evidence actually available	Finding	Approval consequence
Current repo baseline	`docs/requirements.md` on remote `sprint1`, merged PR #7; root `AGENTS.md`, merged PR #20	Stable S1 identifiers and Q01-Q32 applicability, peer review/dispositions and repo rules are available	Source verified for this document, **not** instructor-approved by that fact
Final master file	Instructor's referenced `group_work_specs.docx` and dated amendments	Not present in the examined supplied archive, current repo or relevant connected Drive search results	**Cannot compute a final-vs-draft difference**; OD-011 blocks sign-off
Formal Theme 1 allocation	Student ID digit/official theme decision or instructor/TA confirmation	Not present in the examined evidence	**Cannot certify eligibility**; OD-012 blocks sign-off

When the final master becomes available, record **filename, issue date, version, source URL/location, reviewer, comparison date, changed requirements and affected FR/BR/NFR/AC/VAL IDs**. If a clause contradicts Q01-Q32, record the conflict and ask the correct decision owner before amending the baseline. Do not overwrite confirmed stakeholder rules silently.

14. Exclusions and future compatibility

The project uses three distinct categories, with different approval consequences:

Category	Example	Consequence
Not approved for this product	Payments, lesson mode/video link, group tutoring, course catalogs, email/SMS reminders, external calendar, attendance, new admin role	No requirement or implementation follows from the current baseline; stakeholder and instructor scope approval would be needed to add one.
Agreed for product, deferred from Sprint 1	Customer cancellation/reopening (Q07-Q08), reschedule/history (Q09-Q16), Tutor cancellation/reason/notice/withdrawal (Q17-Q27/Q32), Tutor-only summary (Q31)	Must be covered in the whole-product SRS now, but remains **proposed S2/S3 allocation** with future reviewed contracts and code.
Engineering solution not yet selected	API paths/schema, transaction lock strategy, session/token design, presentation/filtering, idempotency, logging details, measured UX/performance targets	Must preserve business outcomes; propose and approve in SDS/STS, with no invented stakeholder policy.

Forward compatibility from S1: the Booking model must later support `Cancelled` without inventing `Completed`; reschedule must retain the Booking ID/history and update Tutor association; withdrawn Slots must remain historical without blocking current offers; reasons/notifications need ownership boundaries; Tutor summary must use the same authoritative state and time semantics. These constraints are requirements for future design, not authorization to implement S2 features in S1.

15. Verification planning

Each `VAL-xxx` is an **unexecuted requirements-level verification plan**, not an executable STS or a report of passing tests. The test author must later specify fixtures, exact inputs, controlled server clock, synchronized race barriers where relevant, database assertions, expected API/UI behavior, and concrete result/version. Below, `negative` means a forbidden or failed case; `boundary` means exact equality and just below/above as appropriate.

15.1 Planned validation catalog

Anchor	Scope and planned method	Required observable evidence	Linked requirements	Target
VAL-001	API/UI: valid and invalid Customer/Tutor login, identity query, logout and old-session reuse	Correct own role, bad credentials denied, no usable access after logout	FR-001/002/003; BR-033; NFR-001	S1

Anchor	Scope and planned method	Required observable evidence	Linked requirements	Target
VAL-002	API integration: anonymous, forged role, foreign Slot/Booking IDs and wrong Tutor requests	All protected actions denied/filtered, payload contains no foreign private data	FR-004/007/014/015/018; BR-002/033; NFR-001	S1 and regression
VAL-003	UI/API/DB: Tutor creates one future offer; invalid duration/automatic recurrence attempt	Exactly one owned 60-minute offer saved; no automatic repeating Slots	FR-005; BR-003/004; NFR-003	S1
VAL-004	Boundary/API/DB: past/invalid, same-Tutor overlap, adjacency, different-Tutor same-time, Customer adjacency	Invalid/overlapping offer denied; adjacency/cross-Tutor allowed when other checks pass	FR-006/013; BR-003/006/028/035	S1
VAL-005	Real-DB synchronized Race A: concurrent same-Tutor overlapping offer creation	At most one conflicting current Slot committed; losing attempt no partial Slot	FR-006; BR-006/034; NFR-004	S1
VAL-006	Availability API/UI: eligible, past, occupied, too-soon and stale offers	No false selectable booking; no private third-party data; server rechecks stale result	FR-008/009; BR-005/035; NFR-007	S1
VAL-007	E2E/API/DB: authorized offer → availability → Booking → reload/restart → own retrieval	Exactly one Booked reservation saved and readable from persistent DB by authorized actors	FR-010/016/019; BR-001; NFR-003	S1
VAL-008	Controlled backend clock: exact 60:00 vs 59:59 cutoff and wrong browser timezone	Exact boundary eligible, below rejected; correct Bangkok scheduling meaning	FR-011; BR-005/029; NFR-006	S1
VAL-009	Controlled transaction waiting at Q05 cutoff, including prior HTTP arrival	Reject when final guarded `T_elig` is too late; no false success or occupied Slot	FR-009/011; BR-005/034; NFR-006	S1
VAL-010	Real-DB synchronized Race B: two Customers, one Slot	At most one committed Active Booking and one occupied Slot	FR-012; BR-001/034; NFR-004	S1
VAL-011	API/DB sequential plus synchronized Race C: same Customer, overlapping Tutors	No overlapping Active Bookings committed; adjacency allowed by overlap rule	FR-013; BR-028/034; NFR-004	S1
VAL-012	API/UI: own Customer list/detail; Tutor schedule; foreign IDs, refresh	Authorized records visible and persisted, foreign data absent	FR-007/014/015; BR-002; NFR-001/007	S1
VAL-013	Time/UI/API: past end, non-Bangkok device and Booking status check	Past display may be derived; status remains Booked; no Completed or time shift	FR-017; BR-029/030; NFR-006	S1, report regression S3
VAL-014	DB fault injection and stale selection/invalid input for Slot or Booking	No partial claim, false confirmation or mutation after rejection	FR-011/016/018; BR-034/035; NFR-005	S1
VAL-015	S2 API/DB: owning/nonowning Customer cancel; exact 30:00 and just below	Valid owner cancels to Cancelled with Slot re-open; invalid leaves original untouched	FR-020/021; BR-007; NFR-006	proposed S2
VAL-016	S2 boundary/DB: newly reopened original Slot inside new-Booking cutoff	Slot exists as offered/unoccupied but new Booking under 60 minutes denied	FR-021; BR-005/008/014	proposed S2
VAL-017	S2 API/DB: foreign Customer reschedule, one previous success, unsuccessful attempt	Foreign/second-success rejected; failed attempt leaves quota available	FR-022/027; BR-009/016; NFR-008	proposed S2
VAL-018	S2 controlled clock: old and destination starts at exactly 60:00 vs just below	Both boundaries enforced independently; no partial move	FR-023; BR-010/011; NFR-006	proposed S2
VAL-019	S2 API/DB: valid cross-Tutor destination	New Tutor assignment allowed, independent conflicts still enforced	FR-024; BR-012/028	proposed S2

Anchor	Scope and planned method	Required observable evidence	Linked requirements	Target
VAL-020	S2 real-DB: successful and rejected reschedule, destination collision, forced rollback	Same Booking ID; old/new history on success only; old Slot reopened, destination claimed, failed attempt restores prior state/quota	FR-025/026/027; BR-013..016; NFR-005/008	proposed S2
VAL-021	S2 API: current versus wrong Tutor; exact 60:00, 59:59; blank/meaningless reason	Only valid owner/cutoff/reason permits action; otherwise no state changes	FR-028/029; BR-017/018/020; NFR-006	proposed S2
VAL-022	S2 API/DB: Tutor cancellation with committed Booking/Slot relation	Booking Cancelled; original Slot withdrawn, not reoffered; no partial result on fault	FR-030; BR-019; NFR-005	proposed S2
VAL-023	S2 API/UI: successful Tutor cancellation; owning vs foreign Customer, reason fields	Exactly intended owner gets notice and reason in both notice and Booking detail; no leak	FR-031; BR-021..023; NFR-007	proposed S2
VAL-024	S2 persistence/UI: mark notice read, reenter later, simulate age passage	Same owner can reread; no automatic time expiry; deletion policy remains unresolved	FR-032; BR-024/025; NFR-014	proposed S2
VAL-025	S2 API/DB: attempt direct edit/delete/withdraw with Active Booking	All such direct operations denied without modifying occupied Slot	FR-033; BR-026	proposed S2
VAL-026	S2 API/DB: own unoccupied future withdraw and separate new Slot creation	Old historical record retained; new distinct ID accepted if otherwise valid	FR-034; BR-027	proposed S2
VAL-027	S2 API/DB: new offer overlapping withdrawn history and attempted historical reactivation	New offer not blocked by withdrawn Slot; original ID not reactivated	FR-034; BR-032	proposed S2
VAL-028	S3 API/UI/DB: known Booked-upcoming, Booked-past and Cancelled fixtures	Counts and accessible lists match exact categories and state/time without Completed	FR-035; BR-030/031; NFR-016	proposed S3
VAL-029	S3 API: Customer/anonymous/foreign Tutor summary; in-progress fixture	No foreign summary; in-progress category follows **approved future decision**, not guessed default	FR-035/036; BR-031; NFR-001/016	proposed S3
VAL-030	API/DB: repeated/uncertain Booking or reschedule calls	Invariants/quota/history unchanged by unsafe duplicates; concrete retry response to be specified	FR-018/025/027; BR-001/034; NFR-008	S1/S2
VAL-031	Documentation/release evidence review and reproducible local start	Documented SHA, setup, persistent data checks, actual command outputs, peer-reviewed PR chain	FR-019; NFR-003/011/012	S1-S3
VAL-032	Candidate p95 benchmark over 50+ warm-up-excluded GETs per operation with recorded environment	Record raw data/p95 and compare only after threshold approval; **no execution here**	NFR-010	proposed S1/S3
VAL-033	Candidate manual keyboard walk-through with recorded browser/focus checklist	Record accessibility findings; compare only after target approval	NFR-009	proposed S1/S3
VAL-034	Security review of stored test passwords, secrets/configuration and error/log traces	No plaintext secrets and no protected fields; approved hashing method demonstrated when implemented	NFR-002/015; FR-001	S1 and regression
VAL-035	Lab 5+ pipeline/packaging/deploy check, linked release/version	Docker/GCP persistence, required PR build/test gate and repeatable main release with actual outputs	NFR-013	Lab 5+, outside Issue #9
VAL-036	Negative API payload inspection for availability, Booking, notification, summary and error paths	No foreign owner data, cancellation reason or restricted Tutor records in serialized results	FR-004/008/014/015/031/036; BR-022/033; NFR-001/007/015	S1-S3

15.2 Coverage and test-readiness boundaries

At the **requirements** stage, VAL-001..036 give a planned validation path for each FR-001..036, BR-001..035 and NFR-001..016. Crosswalk §12.4 preserves the planned Sprint 1 anchors `AC-S1-01..35` and `VC/TEST-S1-01..39`. The true implementation later needs tests that fail for the right reason before code (TDD), seeded role fixtures, a controlled server clock, a real transactional database for the three races, injected rollback failures and exact recorded outcomes. Final Test Specification IDs, test source files and pass/fail counts belong to Parts E/F, not this SRS.

Validation is not approval: a structural ID check validates document consistency, a teammate review evaluates the requirements, a formal stakeholder decision authorizes disputed behavior, and an executed test validates implemented behavior. These evidence types must not be substituted for one another.

16. Risks, dependencies and change control

Risk / dependency	Exposure	Control and required evidence	Owner / gate
Final instructor specification and Theme unverified	Wrong scope or unapproved team topic	Locate dated final Master + Theme decision; record exact version and differences; revise via reviewed PR	Instructor/TA, @Tanaboonnnnn; OD-011/012 before formal sign-off
New working-hours/horizon/start grid invented in design	Unapproved rejection of valid scheduling	Keep OD-001..003 open, seek specific policy approval, update FR/BR/AC/VAL before implementation	Product decision owner; Slot/Booking contracts
Concurrent Slot/Booking collisions	Double offers, double claims, cross-Tutor overlap	Synchronized real-DB Race A/B/C tests and reviewable transaction guarantees	Engineering + @Chxtamos; S1 contracts
Cutoff evaluated at client or early request time	Incorrect exact-boundary outcomes	Preserve Q05 and reviewed `T_elig` interpretation, test contention crossing; define S2 mutation decision points	Feature contract reviewers
Partial reschedule/cancellation/notification writes	Lost reservation, wrong Slot state or private false notice	Atomic transaction and fault-injection evidence, exact history/quota assertions	S2 Feature SDS/STS
Privacy and reschedule history	Other Customer/Tutor sees reason, past association or summary	Role/record matrix plus full response allowlist review and negative API tests; OD-015	Security/data owner + peer
Summary in-progress classification	Incorrect count/list or dropped current session	Decide OD-006 and OD-009 before S3 contract; compare counts to underlying authorized row sets	Stakeholder and peer
Unapproved NFR target silently treated as pass gate	Unsupported performance or UX claims	Record approval status and environment/method before adopting; preserve measured results	OD-010; team/stakeholder
Feature-order dependency	Later sprints assume unavailable S1 identity/transaction model	S1 auth/Slots/Booking first; S2 cancel/reschedule/notice/withdraw; S3 reports/hardening; revalidate after review	Issue #12 planning, separate implementation Issues
Scope or document drift	Q decisions contradicted, undocumented changes enter app	Versioned SRS and explicit change record with source/decision owner, affected IDs, planned tests and peer review	All future PRs

Change-control procedure: A proposal changing Q01-Q32 must identify the decision ID, present its source and impact, obtain explicit stakeholder approval, then revise affected product FR/BR/NFR, open decisions, acceptance/validation mappings and the later design/test contracts in reviewable commits. An instructor-final discrepancy requires a recorded source comparison and decision owner. A formatting or technical design choice must not silently relax a confirmed cutoff, role boundary, one-to-one exclusivity, transaction integrity or notification privacy rule.

17. Review checklist and sign-off

These checkboxes are review tasks, **not claims of completion**. Human reviewer should inspect the entire document against the linked evidence.

17.1 Reviewer checklist for @Chxtamos

- [ ] Whole-product scope is distinct from the Sprint 1-only source; S2/S3 capabilities are present without inventing implementation.
- [ ] Every Q01-Q32 appears once in §12.2 and the mapped FR/BR describes the same approved decision, with no unsupported extra business policy.
- [ ] All FR-001..036, BR-001..035 and NFR-001..016 have unique meanings, sources, proposed target sprints and usable validation anchors.
- [ ] Theme 1/Deliverable 1 instructor minimum is accurately identified as an **examined draft** where appropriate, not falsely claimed final.
- [ ] Exactly sixty-minute manual offers, Q05 equality, Q07 30-minute equality, Q10/Q11/Q18 60-minute equalities and Bangkok server time agree across catalogs and scenarios.
- [ ] Same-Tutor current offers, exclusive Slot and same-Customer cross-Tutor races are three **separate** committed-DB obligations; adjacency and withdrawn-history exceptions are correct.
- [ ] Customer cancellation reopens old Slot, but Tutor cancellation withdraws old Slot; failed reschedule preserves old Booking and quota; one success preserves Booking ID/history.
- [ ] Privacy matrix denies foreign Customer/Tutor reads, notice/reason leaks and Customer access to the Tutor-only summary.
- [ ] Booked/Cancelled are the only Booking business states; Past is derived; no Completed/attendance assumption appears.
- [ ] Open policies OD-001..015, especially final Master/Theme and proposed NFR performance/UX targets, are not written as approvals.
- [ ] Existing Sprint 1 `FR-S1`, `BR-S1`, `NFR-S1`, `AC-S1` and `VC/TEST-S1` identifiers are not redefined; §16.1 in `requirements.md` includes `AC-S1-35`.
- [ ] Full-document Markdown, relative links, table consistency, ID references, source accuracy, scope and Git file allowlist are checked.
- [ ] No application code, Feature SDS, test implementation or release/deployment claims are part of Issue #9.
- [ ] A real GitHub **Review Request** was issued to @Chxtamos, and their subsequent review is separately recorded before any merge.

17.2 Sign-off record

Review gate	Responsible party	Current evidence and status
Author's requirements drafting/structural validation	@Tanaboonnnnn	Document submitted as draft for Issue #9; structural/semantic review results belong in PR verification report
Independent peer review	@Chxtamos	**PENDING**; no approval recorded in this SRS
Material open policy/NFR decisions	Stakeholder/product owner and team	**PENDING** OD-001..006, OD-008..010, OD-013..015
Final instructor Master version and Theme 1 eligibility	Instructor/TA, verified by team	**UNVERIFIED** OD-011/012; formal approval **BLOCKED**
Whole-product SRS approval	Authorized decision owner following independent review	**NOT APPROVED**
Application implementation, automated tests, release and deployment	Separate engineering Issues	**NOT VERIFIED BY THIS DOCUMENT**

The SRS can be reviewed and improved through an Issue-linked PR while these external gates are open. Peer approval alone must not be represented as the instructor's final requirements approval. No merge is authorized by this document.

18. References

Only materials actually inspected or explicitly provided as the agreed baseline are listed as evidence. Titles and versions are reproduced from sources, without fabricated approval dates or inaccessible references.

Key	Source and location	Purpose / limitation
SRC-Q, SRC-S1	Sprint 1 Requirements Specification, `docs/requirements.md` on sprint1; stakeholder decisions Q01-Q32 and numbered Sprint 1 catalog	Verified repository baseline from merged PR #7. This is **not** the whole-product SRS or proof of final instructor approval.
SRC-GW-v1	`group_work_specs_v1.pdf`, CPE 334 Group Work Specification, **Draft for instructor review**, issued **6 October 2026**, 15 pages; user-supplied `group_project_spec_files.zip`	Actual available theme/technical/course handout. Its final `group_work_specs.docx` replacement was **not verified**.
SRC-D1-v1	`project_deliverable_1_v1.pdf`, CPE 334 Project Deliverable 1, due 8 November 2026 at 23:59, 8 pages; same supplied archive	Whole-product Part A requirements and downstream Parts B-F boundaries.
SRC-AGENT	Repository AGENTS.md, incorporated by PR #20	Repository workflow, scope, verification and no-self-merge obligations, **not new stakeholder rules**.
SRC-ISSUE	Issue #9: D1-01 Part A Whole-product SRS	Assigned deliverable, allowed files and acceptance criteria.
SRC-REVIEW	PR #7 review history	Peer feedback and final correction request that §16.1 include `AC-S1-35`; independently reviewed S1 baseline.
Structural reference only	SE-Project-God-of-Merchandise-Store	Repository root listing was accessible, but no SRS file was available to inspect there. **No Store business rule, quotation or claimed SRS content is used**.

End of v0.1.0 draft. This document can support independent engineering review. Formal sign-off remains subject to the final instructor Master/Theme confirmation, resolution of material open decisions and an actual reviewer decision.

Part B: Feature Inventory & Requirements Traceability Matrix

B.1 Feature Inventory (docs/features.md)

Draft for teammate review. SRS is authoritative. Sprint 2/3 allocations are proposed, not approved.

Stable ID	Feature and required outcome	Actor	Sprint	Dependency	Linked FR/BR/NFR	Planned verification
F-S1-01	Customer/Tutor can log in, retrieve trusted identity, and log out; backend rejects unauthorized roles and records	Customer/Tutor	S1	none	FR-001, FR-002, FR-003, FR-004, BR-002, BR-033, NFR-001, NFR-002	VAL-001/002 (planned)
F-S1-02	Tutor creates and retrieves own 60-minute Slots; concurrent overlap is rejected and adjacent offers allowed	Tutor	S1	F-S1-01	FR-005, FR-006, FR-007, BR-003, BR-004, BR-006, BR-035	VAL-003/004/005 (planned)
F-S1-03	Customer discovers eligible Tutor/time offers without third-party Booking data	Customer	S1	F-S1-02	FR-008, FR-009	VAL-006 (planned)
F-S1-04	Customer books one eligible Slot persistently; concurrent same-Slot claims and overlapping Customer bookings are rejected	Customer	S1	F-S1-01/02/03	FR-010, FR-011, FR-012, FR-013, BR-001, BR-005, BR-028, NFR-008 (S1 Booking retry safety)	VAL-007..011/014/030 (planned)
F-S1-05	Customer reads only own persisted Bookings and Tutor reads only own schedule after refresh; Past is not Completed	Customer/Tutor	S1	F-S1-01/04	FR-014, FR-015, FR-016, FR-017, BR-030	VAL-012/013 (planned)

Stable ID	Feature and required outcome	Actor	Sprint	Dependency	Linked FR/BR/NFR	Planned verification
F-S2-01	Customer cancels own eligible Booking and atomically reopens original Slot (proposed)	Customer	S2 proposed	F-S1-04	FR-020, FR-021, BR-007, BR-008	VAL-015/016 (planned)
F-S2-02	Customer reschedules at most once, even across Tutors, retaining ID and atomic history (proposed)	Customer	S2 proposed	F-S1-04/05	FR-022, FR-023, FR-024, FR-025, FR-026, FR-027, BR-009, BR-010, BR-011, BR-012, BR-013, BR-014, BR-015, BR-016, NFR-008	VAL-017..020 (planned)
F-S2-03	Authorized Tutor cancels with a reason and atomically withdraws the Slot (proposed)	Tutor	S2 proposed	F-S1-02/04	FR-028, FR-029, FR-030, BR-017, BR-018, BR-019, BR-020	VAL-021/022 (planned)
F-S2-04	Owning Customer privately reads and rereads durable cancellation notice (proposed)	Customer	S2 proposed	F-S2-03	FR-031, FR-032, BR-021, BR-022, BR-023, BR-024, BR-025, NFR-014	VAL-023/024 (planned)
F-S2-05	Tutor withdraws unoccupied future Slot, preserving history and allowing distinct replacement (proposed)	Tutor	S2 proposed	F-S1-02	FR-033, FR-034, BR-026, BR-027, BR-032	VAL-025..027 (planned)
F-S3-01	Tutor views authorized counts and matching lists for Upcoming, Past and Cancelled (proposed)	Tutor	S3 proposed	F-S1-05	FR-035, FR-036, BR-031, NFR-016	VAL-028/029 (planned)

Stable ID	Feature and required outcome	Actor	Sprint	Dependency	Linked FR/BR/NFR	Planned verification
SYS-01	System preserves atomic/durable changes, authorization, reproducible evidence and a proposed deployment design	System	S1-S3/Lab5+	all	FR-018, FR-019, BR-029, BR-034, NFR-003, NFR-004, NFR-005, NFR-006, NFR-007, NFR-009, NFR-010, NFR-011, NFR-012, NFR-013, NFR-015	VAL-005/006/010/011/012/014/020/022/023/029/030..036 (planned across S1/S2/S3)

Baseline and invariants

F-S1-01..05 retain the five accepted Sprint 1 identities (Auth, Tutor Slots, Availability, Booking, Protected Read) from requirements.md. SYS-01 owns cross-cutting engineering obligations, while all features still enforce role and record authorization. NFR-004 is shared by F-S1-02 (Race A) and F-S1-04 (Races B/C), while NFR-007 privacy applies to F-S1-03/F-S1-05 in S1, F-S2-04 in proposed S2, and F-S3-01 in proposed S3. These shared applications do not create duplicate primary owners; see traceability.md.

Race A: F-S1-02 / Q06 / FR-006 / BR-006/034 / VAL-005 — same-Tutor overlapping offered Slots.

Race B: F-S1-04 / Q01 / FR-012 / BR-001/034 / VAL-010 — two Customers, one Slot.

Race C: F-S1-04 / Q28 / FR-013 / BR-028/034 / VAL-011 — one Customer, overlapping Tutors.

All three require actual committed database checks. S2 cancellation, rescheduling, withdrawal and notices and S3 summary are planned only. See traceability.md for coverage and SRS §13 for OD-001..015.

Acceptance and review

Each feature must preserve its listed requirement outcomes and planned VAL evidence; Feature SDS/STS must be peer reviewed before implementation. Owner @Chxtamos; independent reviewer @Tanaboonnnnn. Issue #10 requires a PR into sprint1.

B.2 Requirement Traceability Matrix (docs/traceability.md)

DRAFT. Source: SRS, features, Sprint 1 requirements. All validation is PLANNED, NOT EXECUTED.

Product requirement	Complete SRS intent (linked source)	Accountable feature / responsibility	Target sprint	Planned validation
FR-001	Customer/Tutor: With existing valid credentials, authenticate and establish only that principal's effective role; reject invalid credentials without authenticated access. (full FR-001)	F-S1-01	S1	VAL-001 (planned)
FR-002	Authenticated Customer/Tutor: Return the current principal's own identity and role without accepting a client-supplied impersonation; anonymous requests reveal no protected identity. (full FR-002)	F-S1-01	S1	VAL-001, VAL-002 (planned)

Product requirement	Complete SRS intent (linked source)	Accountable feature / responsibility	Target sprint	Planned validation
FR-003	Authenticated Customer/Tutor: Log out and render the prior session/access unusable for protected operations; the exact invalidation/expiry method must be decided in the auth contract. (full FR-003)	F-S1-01	S1	VAL-001, VAL-002 (planned)
FR-004	Application/server: Authenticate and enforce role and record ownership on every protected Slot, Booking, notification and summary action, including crafted requests and URL/ID access; reject unauthorized access without private payload. (full FR-004)	F-S1-01	S1 / S2 proposed / S3 proposed	VAL-002, VAL-036 (planned)
FR-005	Tutor: Given authorization to their own schedule and a valid future start, publish exactly one explicit sixty-minute Slot; persist it under the authenticated Tutor, without recurrence or auto-generated bulk offers. (full FR-005)	F-S1-02	S1	VAL-003 (planned)
FR-006	Server on Tutor creation: Reject malformed/past Slots and overlaps between currently offered Slots of the same Tutor; permit adjacent intervals and independent same-time offers by another Tutor when otherwise valid; leave existing offers unchanged on rejection. (full FR-006)	F-S1-02	S1	VAL-004, VAL-005 (planned)
FR-007	Tutor: Retrieve own offered Slots and authorized occupied/unoccupied schedule view from committed data; reject another Tutor's protected schedule lookup. (full FR-007)	F-S1-02	S1	VAL-002, VAL-012 (planned)
FR-008	Customer: Browse Tutor/Slot/time information sufficient to choose a specific offer, without disclosure of another Customer's private reservation; the concrete field allowlist remains an API-contract choice. (full FR-008)	F-S1-03	S1	VAL-006, VAL-036 (planned)
FR-009	Server/availability view: Mark an offer bookable only when current, unoccupied, eligible and not too close to its start; an earlier view must not guarantee later acceptance. Hide-versus-disabled presentation is unresolved. (full FR-009)	F-S1-03	S1	VAL-006, VAL-009 (planned)
FR-010	Customer: Given an eligible Slot and no overlapping Active Booking, request one exclusive reservation for their authenticated identity; commit one `Booked` Booking referencing the actual Slot and Tutor before acknowledging success. (full FR-010)	F-S1-04	S1	VAL-007 (planned)
FR-011	Server on Booking: Revalidate offered/unoccupied state, user ownership, cutoff and overlaps immediately before a guarded Booking write; reject stale, invalid, past, too-soon or occupied choices with no partial reservation. (full FR-011)	F-S1-04	S1	VAL-008, VAL-009, VAL-014 (planned)
FR-012	Server on competing Bookings: Under simultaneous attempts by different Customers for the same exclusive Slot, allow at most one committed Active Booking and never return two successful claims. (full FR-012)	F-S1-04	S1	VAL-010 (planned)
FR-013	Server on Customer Booking: Reject the same Customer's overlapping Active reservation, including requests for two different Tutors; accept adjacency under this specific rule, with at most one conflicting success under concurrency. (full FR-013)	F-S1-04	S1	VAL-004, VAL-011 (planned)
FR-014	Customer: Retrieve a persisted list and detail of own Bookings with Booking identity, Tutor/Slot/interval and business status; foreign IDs must not disclose Booking data. (full FR-014)	F-S1-05	S1	VAL-012, VAL-036 (planned)
FR-015	Tutor: Retrieve Bookings attached to the Tutor's authorized schedule only; requesting another Tutor's records must not disclose them. (full FR-015)	F-S1-05	S1	VAL-002, VAL-012 (planned)

Product requirement	Complete SRS intent (linked source)	Accountable feature / responsibility	Target sprint	Planned validation
FR-016	Application: Authorized reopened/refresh/restart views must reflect successfully committed Slots/Bookings and correct ownership; a failed transaction must not produce a persisted or reported phantom Booking. (full FR-016)	F-S1-05	S1	VAL-007, VAL-014 (planned)
FR-017	Application views: Display `Booked` accurately and, when a Booking's end is past, optionally derive `Past`; never assert `Completed`, attendance or auto-release solely from elapsed time. (full FR-017)	F-S1-05	S1	VAL-013 (planned)
FR-018	Application/server: Return an observable, non-success rejection for forbidden, invalid, stale, conflicting or failed mutations, without leaking protected records or leaving partial state. Specific HTTP status/payload formats remain contract decisions. (full FR-018)	SYS-01	S1-S3/Lab5+	VAL-002, VAL-014, VAL-030 (planned)
FR-019	System: Support the Tutor offer → Customer browse → committed Booking → authorized retrieval journey through a real UI, application/API and durable database, surviving refresh and application restart. (full FR-019)	SYS-01	S1-S3/Lab5+	VAL-007, VAL-031 (planned)
FR-020	Owning Customer: Cancel their current Booking when the Appointment starts at least 30 minutes after the server's authoritative decision; at the exact boundary allow, below it deny; foreign or nonactive Booking cannot be cancelled. (full FR-020)	F-S2-01	S2 proposed	VAL-015 (planned)
FR-021	Server on successful Customer cancellation: Change Booking to `Cancelled` and reopen its same original Slot atomically; keep it unbookable for any later new request failing the independent Q05 cutoff. (full FR-021)	F-S2-01	S2 proposed	VAL-015, VAL-016 (planned)
FR-022	Owning Customer: Initiate rescheduling of an existing `Booked` Booking; reject nonowners and disallowed Booking state. (full FR-022)	F-S2-02	S2 proposed	VAL-017 (planned)
FR-023	Server on reschedule: Require both original Appointment start and destination Slot start to be at least 60 minutes away at authoritative decision; reject either too-late side including the just-below-boundary case. (full FR-023)	F-S2-02	S2 proposed	VAL-018 (planned)
FR-024	Owning Customer: Select a valid destination with a different Tutor if desired; do not restrict rescheduling to the original Tutor, while still enforcing destination occupancy and same-Customer overlap. (full FR-024)	F-S2-02	S2 proposed	VAL-019 (planned)
FR-025	Server on reschedule: Atomically update Booking assignment, occupy destination, release/reopen original Slot and record history; any validation, conflict or persistence failure leaves all related records at their pre-attempt state. (full FR-025)	F-S2-02	S2 proposed	VAL-020 (planned)
FR-026	Server: Preserve the Booking ID across a successful reschedule and append a successful-change record with former/new Tutor, Slot/interval and timestamp; rejected attempts do not create successful history. (full FR-026)	F-S2-02	S2 proposed	VAL-020 (planned)
FR-027	Server: Permit at most one successful reschedule per Booking; a failed attempt leaves the allowance available, while a second successful-change request is rejected. (full FR-027)	F-S2-02	S2 proposed	VAL-017, VAL-020 (planned)
FR-028	Currently assigned Tutor: Cancel an Active Booking belonging to the Tutor's authorized schedule only when its start is at least 60 minutes away; exact boundary passes and wrong-Tutor/too-late attempts fail. (full FR-028)	F-S2-03	S2 proposed	VAL-021 (planned)

Product requirement	Complete SRS intent (linked source)	Accountable feature / responsibility	Target sprint	Planned validation
FR-029	Tutor/server: Require a meaningful nonblank reason before accepting Tutor cancellation; reject absent, whitespace-only or semantically empty reason without changing Booking or Slot. Length/format details need approved design. (full FR-029)	F-S2-03	S2 proposed	VAL-021 (planned)
FR-030	Server on successful Tutor cancellation: Mark Booking `Cancelled` and withdraw its original Slot instead of reopening it; preserve trace/history and ensure no partial outcome. (full FR-030)	F-S2-03	S2 proposed	VAL-022 (planned)
FR-031	Server: On successful Tutor cancellation create one private in-app notice for that Booking's owning Customer, including the authorized reason; the reason also appears in that Customer's cancelled Booking details, never in third-party views. (full FR-031)	F-S2-04	S2 proposed	VAL-023, VAL-036 (planned)
FR-032	Owning Customer: Retrieve and reread Tutor-cancellation notices after marking them read; absence of automatic time-based expiry must hold, while deletion/retention governance remains undecided. (full FR-032)	F-S2-04	S2 proposed	VAL-024 (planned)
FR-033	Server/Tutor: Forbid direct edit, deletion or withdrawal of a Slot while it carries an Active Booking; require the separate authorized cancellation workflow first. (full FR-033)	F-S2-05	S2 proposed	VAL-025 (planned)
FR-034	Owning Tutor: Withdraw an unoccupied future currently offered Slot while retaining its history; permit later creation of a distinct new Slot with a new ID that may overlap the withdrawn interval; reject reopening/reactivating the withdrawn historical Slot. (full FR-034)	F-S2-05	S2 proposed	VAL-026, VAL-027 (planned)
FR-035	Tutor: Retrieve an Appointment Summary with counts and corresponding lists for Upcoming Booked, Past Booked and Cancelled on the Tutor's authorized schedule, calculated from stored Booking state and server time; in-progress classification remains OD-006. (full FR-035)	F-S3-01	S3 proposed	VAL-028, VAL-029 (planned)
FR-036	Server: Deny Customers, anonymous callers and unrelated Tutors access to the Tutor-only summary and its linked Booking details; counts/lists must share the same visibility and classification criteria. (full FR-036)	F-S3-01	S3 proposed	VAL-029, VAL-036 (planned)
BR-001	One Slot reserves one exclusive one-to-one Customer/Tutor session; at most one Active Booking may claim it. (full BR-001)	F-S1-04	S1	VAL-007, VAL-010 (planned)
BR-002	Tutor = Staff and may manage or inspect their own authorized schedule; Customer = learner and may inspect only own private Booking data. (full BR-002)	F-S1-01	S1	VAL-002, VAL-012 (planned)
BR-003	Every Availability Slot has a duration of exactly 60 minutes. (full BR-003)	F-S1-02	S1	VAL-003, VAL-004 (planned)
BR-004	Tutor creates individual explicit Slots; no automatic recurrence or bulk Slot generation is authorized. (full BR-004)	F-S1-02	S1	VAL-003 (planned)
BR-005	For a new Booking, `SlotStart - authoritative server eligibility time >= 60 minutes`; equality is eligible, even if later commit succeeds after the check. A 59:59 remaining interval fails. (full BR-005)	F-S1-04	S1	VAL-008, VAL-009 (planned)
BR-006	Two currently offered Slots of the same Tutor cannot overlap. Adjacent half-open intervals are permitted; different Tutors may independently offer the same time. (full BR-006)	F-S1-02	S1	VAL-004, VAL-005 (planned)

Product requirement	Complete SRS intent (linked source)	Accountable feature / responsibility	Target sprint	Planned validation
BR-007	Only the owning Customer may cancel an Active Booking at least 30 minutes before Appointment start, including equality. (full BR-007)	F-S2-01	S2 proposed	VAL-015 (planned)
BR-008	Successful Customer cancellation reopens the original Slot, but a future new Booking still independently requires BR-005. (full BR-008)	F-S2-01	S2 proposed	VAL-016 (planned)
BR-009	Only the Booking-owning Customer may initiate its reschedule. (full BR-009)	F-S2-02	S2 proposed	VAL-017 (planned)
BR-010	Reschedule's original Appointment must start at least 60 minutes after the authoritative decision; equality passes. (full BR-010)	F-S2-02	S2 proposed	VAL-018 (planned)
BR-011	Reschedule's destination Slot must start at least 60 minutes after the authoritative decision; equality passes. (full BR-011)	F-S2-02	S2 proposed	VAL-018 (planned)
BR-012	A valid Customer reschedule may move to another Tutor; original-Tutor-only restrictions are forbidden. (full BR-012)	F-S2-02	S2 proposed	VAL-019 (planned)
BR-013	Reschedule is all-or-nothing across assignment, both Slot states and successful history. Failure leaves the previous Booking intact. (full BR-013)	F-S2-02	S2 proposed	VAL-020 (planned)
BR-014	Successful reschedule reopens the original Slot, subject to independent new-Booking eligibility. (full BR-014)	F-S2-02	S2 proposed	VAL-016, VAL-020 (planned)
BR-015	Successful reschedule keeps the same Booking ID and records old/new Tutor, Slot/time and change timestamp. (full BR-015)	F-S2-02	S2 proposed	VAL-020 (planned)
BR-016	At most one successful reschedule per Booking; failures do not use that allowance. (full BR-016)	F-S2-02	S2 proposed	VAL-017, VAL-020 (planned)
BR-017	Only the currently authorized Tutor assigned to the Booking may cancel it as Tutor. (full BR-017)	F-S2-03	S2 proposed	VAL-021 (planned)
BR-018	Tutor cancellation requires Appointment start at least 60 minutes after the authoritative decision, equality included. (full BR-018)	F-S2-03	S2 proposed	VAL-021 (planned)
BR-019	Successful Tutor cancellation withdraws the original Slot; it must not reopen it. (full BR-019)	F-S2-03	S2 proposed	VAL-022 (planned)
BR-020	Tutor cancellation requires a meaningful nonblank reason; an empty/whitespace-only input is invalid. Further textual-validation details require approval. (full BR-020)	F-S2-03	S2 proposed	VAL-021 (planned)
BR-021	Each successful Tutor cancellation creates an in-app notice for the affected owning Customer only. (full BR-021)	F-S2-04	S2 proposed	VAL-023 (planned)
BR-022	Tutor-cancellation reason is visible only to the owning Customer, not to other Customers or unrelated Tutors in accessible results. (full BR-022)	F-S2-04	S2 proposed	VAL-023, VAL-036 (planned)
BR-023	That Customer sees the cancellation reason in both the in-app notice and cancelled Booking detail. (full BR-023)	F-S2-04	S2 proposed	VAL-023 (planned)
BR-024	Reading a notification does not make it inaccessible; it remains available to reread. (full BR-024)	F-S2-04	S2 proposed	VAL-024 (planned)
BR-025	Notifications do not expire automatically with elapsed time. This does not settle user/admin deletion, retention or privacy governance. (full BR-025)	F-S2-04	S2 proposed	VAL-024 (planned)

Product requirement	Complete SRS intent (linked source)	Accountable feature / responsibility	Target sprint	Planned validation
BR-026	Tutor cannot directly edit, delete or withdraw a Slot with an Active Booking; the authorized Booking cancellation must occur first. (full BR-026)	F-S2-05	S2 proposed	VAL-025 (planned)
BR-027	Tutor may withdraw an own, unoccupied future Slot and create a separate new Slot while preserving withdrawal history. (full BR-027)	F-S2-05	S2 proposed	VAL-026 (planned)
BR-028	One Customer cannot have overlapping Active Bookings across any Tutors; adjacent appointments do not overlap. (full BR-028)	F-S1-04	S1	VAL-004, VAL-011 (planned)
BR-029	`Asia/Bangkok` (UTC+07:00) is the business timezone; authoritative time comes from the server, not a browser or user-supplied timestamp. (full BR-029)	SYS-01	S1-S3/Lab5+	VAL-008, VAL-013 (planned)
BR-030	Booking business states are `Booked` and `Cancelled`; `Past` is derived from time, not a third state or evidence of `Completed`/attendance. (full BR-030)	F-S1-05 (shared: F-S2-01/03, F-S3-01)	S1 / S2 proposed / S3 proposed	VAL-013, VAL-015 (planned)
BR-031	Only authorized Tutor gets Appointment Summary counts and lists for Upcoming Booked, Past Booked and Cancelled. Customer gets no Tutor-only summary. (full BR-031)	F-S3-01	S3 proposed	VAL-028, VAL-029 (planned)
BR-032	Historical withdrawn Slots do not block a new separate overlapping offer; they are not reactivated under the same historical Slot ID. (full BR-032)	F-S2-05	S2 proposed	VAL-027 (planned)
BR-033	Protected actions require server-enforced authenticated role/ownership checks; anonymous, foreign-record and cross-role operations cannot gain private data or mutation rights. (full BR-033)	F-S1-01	S1 / S2 proposed / S3 proposed	VAL-001, VAL-002, VAL-036 (planned)
BR-034	Accepted Booking/Slot mutations commit an internally consistent outcome; rejected validations, contention and persistence failures cannot leave partially claimed Slots, duplicate active reservations or false success. The precise locking/constraint scheme is not selected here. (full BR-034)	SYS-01 (shared: F-S1-02/04, F-S2-02/03)	S1 / S2 proposed	VAL-005, VAL-010, VAL-011, VAL-014, VAL-020, VAL-022 (planned)
BR-035	Invalid or nonfuture Slot proposals and attempts to book past, occupied or ineligible offers are rejected without changing committed records; rules beyond confirmed time policies cannot be fabricated. (full BR-035)	F-S1-02	S1	VAL-004, VAL-006, VAL-014 (planned)
NFR-001	Authorization: every enumerated anonymous, forbidden-role, foreign-Customer and foreign-Tutor protected API path must reject or filter without private payload disclosure. Measure zero unauthorized record disclosure in seeded negative cases. (full NFR-001)	F-S1-01 (auth foundation; role/ownership regression for each new S2/S3 endpoint)	S1 / S2 proposed / S3 proposed	VAL-002, VAL-036 (planned; repeat authorization regression as endpoints expand)
NFR-002	Credential security: with locally managed passwords, store adaptive salted hashes, not plaintext; tracked repository content contains no real credentials. Verify using test DB inspection and secret/config review. (full NFR-002)	F-S1-01	S1	VAL-034 (planned)
NFR-003	Durability: an accepted Slot/Booking and its authorized view must persist after browser refresh and application-process restart against the same data store; no in-memory-only persistence. (full NFR-003)	SYS-01	S1 / S2 proposed / S3 proposed	VAL-007, VAL-031 (planned)

Product requirement	Complete SRS intent (linked source)	Accountable feature / responsibility	Target sprint	Planned validation
NFR-004	Concurrent conflict integrity: all three real-DB races must preserve BR-001/006/028, regardless of interleaving; a 20-attempt same-Slot stress burst and a 10-attempt same-Customer burst are proposed test sizes, not approved business capacity limits. (full NFR-004)	SYS-01 (shared: F-S1-02, F-S1-04)	S1	VAL-005, VAL-010, VAL-011 (planned)
NFR-005	Atomic consistency: accepted cancellation/reschedule/Booking/withdrawal changes all required records together; deliberate persistence failures leave the prior committed state and no false confirmation. (full NFR-005)	SYS-01	S1 / S2 proposed / S3 proposed	VAL-014, VAL-020, VAL-022 (planned)
NFR-006	Time correctness: cutoff checks use server-authoritative instants; accept exact 60:00 or 30:00 boundaries as applicable, reject just-below; Bangkok calendar/hour remains stable for a non-Bangkok browser. (full NFR-006)	SYS-01	S1 / S2 proposed / S3 proposed	VAL-008, VAL-009, VAL-013, VAL-015, VAL-018, VAL-021 (planned)
NFR-007	Data minimization: protected Booking, reason, notification and report responses do not expose third-party identity, cancellation reason, credentials or arbitrary private schedule fields. Review response bodies and negative payloads, not UI controls only. (full NFR-007)	SYS-01 (shared: F-S1-03/05, F-S2-04, F-S3-01)	S1 / S2 proposed / S3 proposed	VAL-006, VAL-012, VAL-023, VAL-029, VAL-036 (planned)
NFR-008	Retry safety: repeated/uncertain-response requests cannot violate exclusivity, successful reschedule quota or atomic history. Whether retries return the original success response and any token format/retention are open. (full NFR-008)	F-S1-04 (S1 Booking retry; AC-S1-29 / VC/TEST-S1-38); F-S2-02 (S2 proposed Reschedule retry)	S1 / S2 proposed	VAL-030 (S1), VAL-020 (S2 proposed; planned)
NFR-009	Keyboard usability (PROPOSED): complete login, availability, Booking and own-detail flows by keyboard with visible focus on primary controls; record manual checklist, browser and failures. (full NFR-009)	SYS-01	S1-S3/Lab5+	VAL-033 (planned)
NFR-010	Local response time (PROPOSED): p95 availability and own-Booking-detail requests at or below 2.0 s, each measured over at least 50 warm-up-excluded requests with documented hardware, fixture size and database. (full NFR-010)	SYS-01	S1-S3/Lab5+	VAL-032 (planned)
NFR-011	Change traceability: each implemented requirement must identify source Q/GW/D1, accountable feature/Issue, reviewed contract, planned verification and release evidence; changes to Q rules need recorded stakeholder approval. (full NFR-011)	SYS-01	S1-S3/Lab5+	VAL-031 (planned)
NFR-012	Reproducibility: engineering evidence must identify code version, environment/setup, DB/seed preparation, test commands and real pass/fail/skip results; mere presence of tests is not a pass. (full NFR-012)	SYS-01	S1-S3/Lab5+	VAL-031 (planned)
NFR-013	Deployment readiness: the selected architecture must support container packaging and durable GCP deployment, with build/tests on relevant PRs and repeatable accepted release from `main` after Lab 5. No cloud success is asserted at D1. (full NFR-013)	SYS-01	S1-S3/Lab5+	VAL-035 (planned)
NFR-014	Notice durability: a Tutor-cancellation notice remains retrievable after being read and is not removed solely by age; additional retention/deletion rights await owner decision. (full NFR-014)	F-S2-04	S2 proposed	VAL-024 (planned)
NFR-015	Safe diagnostics: response errors expose no credentials, session tokens, protected reason or internal stack traces; operational logs must avoid secrets and preserve enough non-sensitive context to investigate failed mutations. Exact logging fields/retention are engineering decisions. (full NFR-015)	SYS-01	S1 / S2 proposed / S3 proposed	VAL-034, VAL-036 (planned)

Product requirement	Complete SRS intent (linked source)	Accountable feature / responsibility	Target sprint	Planned validation
NFR-016	Reporting correctness: for every defined Tutor Summary category, count must equal its corresponding accessible row set under the same time/state predicates; no completion is inferred from elapsed time. Treatment of in-progress sessions remains open. (full NFR-016)	F-S3-01	S3 proposed	VAL-028, VAL-029 (planned)

Q01–Q32 accountability

Confirmed meanings and full Q-to-FR/BR/VAL crosswalk remain in SRS §12.2.

Decision	Owner	Sprint
Q01	F-S1-04	S1
Q02	F-S1-01	S1
Q03	F-S1-02	S1
Q04	F-S1-02	S1
Q05	F-S1-04	S1
Q06	F-S1-02	S1
Q07	F-S2-01	S2 proposed
Q08	F-S2-01	S2 proposed
Q09	F-S2-02	S2 proposed
Q10	F-S2-02	S2 proposed
Q11	F-S2-02	S2 proposed
Q12	F-S2-02	S2 proposed
Q13	F-S2-02	S2 proposed
Q14	F-S2-02	S2 proposed
Q15	F-S2-02	S2 proposed
Q16	F-S2-02	S2 proposed
Q17	F-S2-03	S2 proposed
Q18	F-S2-03	S2 proposed
Q19	F-S2-03	S2 proposed
Q20	F-S2-03	S2 proposed
Q21	F-S2-04	S2 proposed
Q22	F-S2-04	S2 proposed
Q23	F-S2-04	S2 proposed
Q24	F-S2-04	S2 proposed
Q25	F-S2-04	S2 proposed
Q26	F-S2-05	S2 proposed
Q27	F-S2-05	S2 proposed
Q28	F-S1-04	S1

Decision	Owner	Sprint
Q29	SYS-01	S1-S3/Lab5+
Q30	F-S1-05 (Booked/Past); F-S2-01 and F-S2-03 (Cancelled); F-S3-01 (summary/report)	S1 / S2 proposed / S3 proposed
Q31	F-S3-01	S3 proposed
Q32	F-S2-05	S2 proposed

Sprint 1 acceptance and test mapping

All 35 AC and all 39 VC/TEST identifiers below are **planned evidence anchors, not executed results**. IDs, scenarios and expected outcomes are taken from requirements.md §9 and §13; this document only assigns feature ownership. Shared entries do not dilute primary accountability.

Acceptance criteria: AC-S1-01..35

AC ID	Primary owner	Other impacted owner	Baseline scenario and expected result	Planned VC reference
AC-S1-01	F-S1-01	—	Existing valid Customer credentials — Authenticated principal is Customer; user sees correct own identity/role	VC-01/03 (planned)
AC-S1-02	F-S1-01	—	Existing valid Tutor credentials — Authenticated principal is Tutor; own-schedule operations are available	VC-01/03 (planned)
AC-S1-03	F-S1-01	—	Invalid password or unknown credentials — Access denied; no protected identity/session returned	VC-02 (planned)
AC-S1-04	F-S1-01	—	Authenticated user — Protected action rejected according to documented session lifecycle	VC-04 (planned)
AC-S1-05	F-S1-01	—	Unauthenticated browser or caller — No operation granted and no private payload disclosed	VC-05 (planned)
AC-S1-06	F-S1-01	—	Authenticated Customer — Backend denies request; Tutor schedule unchanged	VC-06 (planned)
AC-S1-07	F-S1-01	F-S1-05	Tutor A and B have different schedules — No protected records or writes under B's identity	VC-07/31 (planned)
AC-S1-08	F-S1-02	—	Own authorized Tutor schedule; valid future Bangkok start — Exactly one 60-minute offered Slot is persisted and available on schedule refresh	VC-08/14 (planned)
AC-S1-09	F-S1-02	—	Same Tutor offers 13:00–14:00 — New Slot rejected, existing Slot unchanged	VC-09 (planned)
AC-S1-10	F-S1-02	—	Same Tutor offers 13:00–14:00 — Adjacent Slot allowed by non-overlap rule, subject to independent validity	VC-10 (planned)
AC-S1-11	F-S1-02	—	Tutor A offers 13:00–14:00 — Different Tutor offer permitted	VC-11 (planned)

AC ID	Primary owner	Other impacted owner	Baseline scenario and expected result	Planned VC reference
AC-S1-12	F-S1-02	—	Invalid or past date/start — No invalid Slot persisted, no collateral mutation	VC-12 (planned)
AC-S1-13	F-S1-02	—	Same Tutor proposes two overlapping Slot writes concurrently — Database never contains two overlapping currently offered Slots for that Tutor	VC-37 (planned)
AC-S1-14	F-S1-02	F-S1-03	Browser timezone is not Bangkok — Same intended Bangkok appointment date and hour displayed	VC-13 (planned)
AC-S1-15	F-S1-03	—	Customer logged in; eligible offered Slot exists — Selectable Tutor and Bangkok appointment times shown without private other-Customer details	VC-15 (planned)
AC-S1-16	F-S1-03	—	Offered Slot occupied or inside Q05 cutoff — Slot is not selectable as bookable and never reveals its Customer; proposed default: show only currently bookable offers, pending the reviewed UI contract in TBD-S1-04	VC-15/22 (planned)
AC-S1-17	F-S1-03	—	Displayed offer becomes ineligible before submission — Backend rechecks and rejects; UI does not report a successful Booking	VC-22 (planned)
AC-S1-18	F-S1-03	—	Device locale is another timezone — Bangkok time meaning does not drift with device timezone	VC-13/15 (planned)
AC-S1-19	F-S1-04	—	Eligible unoccupied Slot, Customer logged in without overlapping Booking — One Booked record is committed with correct Customer, Tutor and Slot identifiers	VC-16 (planned)
AC-S1-20	F-S1-04	—	Slot begins 15:00; backend `T_elig` is exactly 14:00:00 — Exact 60-minute cutoff accepted if all other checks pass and transaction subsequently commits	VC-17 (planned)
AC-S1-21	F-S1-04	—	Slot begins 15:00; backend `T_elig` is 14:00:01 — Less-than-60-minute cutoff rejected, no Booking or occupation	VC-18 (planned)
AC-S1-22	F-S1-04	—	Customer A owns active Booking on Slot X — Second request rejected, first Booking preserved	VC-19 (planned)
AC-S1-23	F-S1-04	—	Customer holds Tutor A 13:00–14:00 — Reject overlapping appointment even across Tutors	VC-20 (planned)
AC-S1-24	F-S1-04	—	Customer holds an interval ending 14:00 — Adjacency itself not blocked; other checks still apply	VC-21 (planned)
AC-S1-25	F-S1-04	—	Two Customers simultaneously request one eligible Slot — At most one committed Active Booking; no false second success	VC-25 (planned)
AC-S1-26	F-S1-04	—	Same Customer concurrently requests distinct overlapping Tutor Slots — No two overlapping Active Bookings committed for Customer	VC-26 (planned)
AC-S1-27	F-S1-04	F-S1-02	Previously eligible Slot becomes invalid/occupied or persistence fails — No phantom success, partial Booking or inconsistent occupied state	VC-22/27/33 (planned)

AC ID	Primary owner	Other impacted owner	Baseline scenario and expected result	Planned VC reference
AC-S1-28	F-S1-04	F-S1-01	Customer alters request body to impersonate other identity/Tutor — Owner derived from trusted session; Tutor from actual Slot, not supplied roles	VC-24 (planned)
AC-S1-29	F-S1-04	—	Customer repeats request while first outcome might be unknown — Exclusivity and Customer-overlap invariants hold; exact retry response remains contract TBD	VC-38 (planned)
AC-S1-30	F-S1-05	—	Customer already has a committed Booking — Correct Booking identity, Tutor, Bangkok interval and Booked status	VC-28 (planned)
AC-S1-31	F-S1-05	—	Customer A has Booking; Customer B knows its ID — No protected Booking information disclosed	VC-29 (planned)
AC-S1-32	F-S1-05	—	Tutor owns current Booking schedule — Authorized Booking visible on own schedule only	VC-30 (planned)
AC-S1-33	F-S1-05	—	Tutor B knows Tutor A schedule ID — No protected foreign Tutor Booking disclosed	VC-31 (planned)
AC-S1-34	F-S1-05	—	Booked 13:00–14:00 appointment now ended — Booking still Booked; may show Past; no Completed/attendance assertion	VC-32 (planned)
AC-S1-35	F-S1-04	—	Slot starts 15:00; Booking HTTP request reaches server at 13:59:59 but waits for contention until final transactional `T_elig` is 14:00:01 — Reject as less than 60 minutes remaining, preserve Slot and all previously committed Bookings, and do not report success merely because HTTP arrival preceded the cutoff	VC-39 (planned)

Verification cases: VC/TEST-S1-01..39

VC / TEST ID	Primary owner	Other impacted owner	Baseline scenario and expected result	Status
VC-S1-01/TEST-S1-01	F-S1-01	—	Existing Customer and Tutor demo credentials → each logs in — Correct own identity/role; no cross-role grant	Planned, not run
VC-S1-02/TEST-S1-02	F-S1-01	—	Invalid password → login attempt — Authentication refused; no valid session/identity exposed	Planned, not run
VC-S1-03/TEST-S1-03	F-S1-01	—	Logged-in user → current identity query — Authenticated user's own ID and role, not arbitrary submitted identity	Planned, not run
VC-S1-04/TEST-S1-04	F-S1-01	—	Logged-in user → logout → reuse prior access — Protected access rejected according to chosen session lifecycle	Planned, not run
VC-S1-05/TEST-S1-05	F-S1-01	—	Anonymous caller → private Booking / create Slot / book — All protected actions denied without protected data	Planned, not run
VC-S1-06/TEST-S1-06	F-S1-01	—	Customer → attempts Tutor-only create action — Rejected by backend, even when request is crafted manually	Planned, not run

VC / TEST ID	Primary owner	Other impacted owner	Baseline scenario and expected result	Status
VC-S1-07/TEST-S1-07	F-S1-01	—	Tutor A → attempts to create/view private schedule of Tutor B — Access rejected or B's records absent; A's own schedule unaffected	Planned, not run
VC-S1-08/TEST-S1-08	F-S1-02	—	Authorized Tutor; future date → create one Slot — Exactly one offered 60-minute Slot persisted under that Tutor	Planned, not run
VC-S1-09/TEST-S1-09	F-S1-02	—	Tutor's offered Slot 13:00–14:00 → create 13:30–14:30 — Reject overlap; old Slot unchanged, no duplicate offer	Planned, not run
VC-S1-10/TEST-S1-10	F-S1-02	—	Tutor's offered Slot 13:00–14:00 → create 14:00–15:00 — Overlap rule permits adjacency; other validity checks remain	Planned, not run
VC-S1-11/TEST-S1-11	F-S1-02	—	Tutor A offers 13:00–14:00 → Tutor B creates same time — Allowed as distinct Tutor offer if valid	Planned, not run
VC-S1-12/TEST-S1-12	F-S1-02	—	Proposed Slot starts in the past/invalid → Tutor submits — Reject with no stored Slot or collateral mutation	Planned, not run
VC-S1-13/TEST-S1-13	F-S1-02	F-S1-03	Time shown by browser in another timezone → create/check Bangkok Slot — Tutor and Customer see same intended Bangkok appointment time	Planned, not run
VC-S1-14/TEST-S1-14	F-S1-02	—	Tutor owns offered Slot → opens schedule after refresh — Exact Slot and correct offered/occupied state still visible	Planned, not run
VC-S1-15/TEST-S1-15	F-S1-03	—	Customer logged in; eligible future offered Slot → browse — Tutor and time identifiable; no other Customer's private details exposed	Planned, not run
VC-S1-16/TEST-S1-16	F-S1-04	—	Eligible unoccupied Slot → Customer requests Booking — One persisted Booking, state Booked, own Customer and selected Tutor/Slot	Planned, not run
VC-S1-17/TEST-S1-17	F-S1-04	—	Slot at 15:00, backend `T_elig` is 14:00:00 — Exactly 60-minute Q05 lead time passes if otherwise valid; later commit does not retroactively invalidate a passing checked write	Planned, not run
VC-S1-18/TEST-S1-18	F-S1-04	—	Slot at 15:00, backend `T_elig` is 14:00:01 — Q05 lead time fails; no Booking/occupation	Planned, not run
VC-S1-19/TEST-S1-19	F-S1-04	—	Offered Slot already occupied → another Customer books it — Reject; previous Booking preserved and only one active claim remains	Planned, not run
VC-S1-20/TEST-S1-20	F-S1-04	—	Customer holds 13:00–14:00 Tutor A → books 13:30–14:30 Tutor B — Reject cross-Tutor Customer overlap with no second Booking	Planned, not run
VC-S1-21/TEST-S1-21	F-S1-04	—	Customer holds 13:00–14:00 → requests 14:00–15:00 — Customer-overlap check permits adjacent interval, subject to lead time	Planned, not run
VC-S1-22/TEST-S1-22	F-S1-04	F-S1-03	Selected Slot was eligible at browse but is taken/too soon by submission — Backend rejects stale selection; no false success	Planned, not run

VC / TEST ID	Primary owner	Other impacted owner	Baseline scenario and expected result	Status
VC-S1-23/TEST-S1-23	F-S1-04	F-S1-01	Invalid/missing/past Slot or unauthorized actor → booking attempt — Reject without claimed Slot or private data disclosure	Planned, not run
VC-S1-24/TEST-S1-24	F-S1-04	F-S1-01	Customer alters client-supplied user/Tutor ownership IDs — Persist Booking only for authenticated Customer and actual Slot Tutor, or reject	Planned, not run
VC-S1-25/TEST-S1-25	F-S1-04	—	Two distinct Customers submit concurrently for same eligible exclusive Slot — At most one committed Active Booking; other gets non-success; DB invariant holds	Planned, not run
VC-S1-26/TEST-S1-26	F-S1-04	—	One Customer submits concurrently for distinct overlapping eligible Slots of two Tutors — No two overlapping Active Bookings commit; no partial losing claim	Planned, not run
VC-S1-27/TEST-S1-27	F-S1-04	—	Persistence failure injected before Booking transaction finishes — No false success, no committed Booking or partial occupied Slot	Planned, not run
VC-S1-28/TEST-S1-28	F-S1-05	—	Customer creates successful Booking → refresh/restart app with same DB — Own Booking and identity/time/state retrievable unchanged	Planned, not run
VC-S1-29/TEST-S1-29	F-S1-05	—	Customer B knows Customer A's Booking ID → tries detail/list — Private Booking content not revealed	Planned, not run
VC-S1-30/TEST-S1-30	F-S1-05	—	Tutor A → inspect own schedule with Booking — Correct assigned Booking visible without exposing another Tutor's protected records	Planned, not run
VC-S1-31/TEST-S1-31	F-S1-05	—	Tutor B knows Tutor A's schedule/Booking identifiers → read attempt — Access denied/filtered; Tutor A records remain private	Planned, not run
VC-S1-32/TEST-S1-32	F-S1-05	—	Booking interval fully elapsed, no cancellation → open Booking — Business state remains Booked; may show Past; never Completed or assumed attended	Planned, not run
VC-S1-33/TEST-S1-33	F-S1-04	—	User submits failed/invalid request → reopen respective views — No new phantom Booking, no changed Slot occupancy from failed attempt	Planned, not run
VC-S1-34/TEST-S1-34	SYS-01	F-S1-01/02/03/04/05	Real app boot and seed setup → complete J-S1-01 then J-S1-02 through UI and DB — Full verified workflow, repeatable commands and actual test results recorded later	Planned, not run
VC-S1-35/TEST-S1-35	SYS-01	F-S1-01	Test invalid access and inspect tracked env/config — No plaintext stored passwords/secrets and no private-data leaks	Planned, not run
VC-S1-36/TEST-S1-36	SYS-01	F-S1-03/05	Benchmark local availability/details and keyboard-only walk-through — Record measurements/checklist; compare with proposed, not approved, UX/performance targets	Planned, not run
VC-S1-37/TEST-S1-37	F-S1-02	—	Two requests concurrently create overlapping future offered Slots for same Tutor — At most one overlapping offer commits; no partial side effects	Planned, not run
VC-S1-38/TEST-S1-38	F-S1-04	—	Customer duplicates Booking submission while first outcome is in flight or unknown — At most one Active Booking for a Slot; Customer overlap invariant preserved, response semantics defined later	Planned, not run

VC / TEST ID	Primary owner	Other impacted owner	Baseline scenario and expected result	Status
VC-S1-39/TEST-S1-39	F-S1-04	—	For 15:00 Slot, request arrives at 13:59:59 but transaction waits until `T_elig = 14:00:01` before final guarded write — Refuse too-late Booking with no partial occupancy, despite pre-cutoff arrival; use controlled server clock/transaction synchronization, not client timestamps	Planned, not run

Three separate real-database race obligations

Race	Owner	AC and VC/TEST evidence	Must prove
A: same Tutor creates overlapping offered Slots	F-S1-02	AC-S1-13; VC/TEST-S1-37	At most one conflicting current offer commits
B: two Customers compete for one exclusive Slot	F-S1-04	AC-S1-25; VC/TEST-S1-25	At most one active claim commits, no false success
C: one Customer books overlapping Slots with different Tutors	F-S1-04	AC-S1-26; VC/TEST-S1-26	No overlapping active Bookings commit for that Customer

Changes to future-sprint validation or policy proposals require the SRS §13 approval gates. There is no claim that any of the above tests has been run.

Audit and approval gates

FR: 36/36 assigned; BR: 35/35 assigned; NFR: 16/16 assigned; Q: 32/32 assigned. No silent orphans. SYS-01 owns cross-cutting requirements while relevant features must still satisfy them.

S2/S3 assignments are proposals pending team approval in Issue #12. OD-001..015 remain open or conditional per SRS §13; OD-011/012 instructor Master/Theme unverified, so final sign-off is blocked. Proposed NFR-009/010 are not mandatory thresholds until approved.

Issue #10 PR into sprint1 requires @Tanaboonnnnn peer review; this document neither proves test passes nor grants stakeholder approval.

Part C: Whole-Product System Software Design Specification (SDS)

0. Document control and design authority

Field	Value
Deliverable	CPE334 Deliverable 1, Part C; GitHub Issue #11
Artifact	docs/system-sds.md, design draft v0.1.0, 3 November 2026
Product	One-to-one Tutoring Appointment System (Theme 1: Appointment & Scheduling)
Coverage	Whole-product architecture for Sprints 1–3; detailed implementation contracts remain feature-scoped
Author / required peer reviewer	@Tanaboonnnnn / @Chxtamos
Status	**PROPOSED ENGINEERING DESIGN — pending substantive peer review; NOT instructor-approved or implemented**
Intended integration	Issue branch -> reviewed PR into sprint1 -> later separately reviewed sprint1 -> main
Repo state inspected	Remote sprint1 at ba577db79ccee6c0f0861b761fb361bbb252b027 (3 Nov 2026); tracked files are AGENTS.md, README.md and docs/*.md; no application scaffold, package manifest, Prisma schema or CI was present
Source gates	Final instructor Master and official team Theme 1 allocation remain unverified (SRS OD-011/OD-012); do not infer formal sign-off

Order of authority. An instructor-issued *final* Master/change notice outranks the available draft handout. Until verified, use the reviewed stakeholder decisions Q01–Q32, whole-product SRS, Sprint 1 requirements, feature inventory, traceability, Issue #11 and root AGENTS.md. The documents describe different levels: business rules are normative outcomes; technology choices in this SDS are **engineering proposals awaiting peer approval**, not new stakeholder rules. Preserve all existing IDs and report conflicts before modifying them.

Boundary. This file selects a feasible architecture and explains verifiable invariants. It does **not** add app code, create a working API, run database tests, deploy to GCP, or finalize individual endpoint/field validation contracts. Part E Feature SDS/STS must be versioned and peer-reviewed **before** each corresponding feature's code. Sprint 2/3 placement remains proposed until Issue #12 is accepted.

1. Design objectives and decisions

The Sprint 1 minimum is a working **browser UI -> authenticated API/service -> durable relational database** slice: login/logout/current user, Tutor publishing individual 60-minute offered Slots, Customer availability discovery and exclusive Booking, authorized Customer/Tutor reads, and the three concurrency invariants. Records survive refresh and restart. Sprint 2 adds cancellation, one-time cross-Tutor rescheduling, withdrawn Slot history and durable private notices; Sprint 3 adds a Tutor-only summary. Payments, attendance confirmation, recurring/bulk Slots, automatic SLA or notifications by email/SMS are **not authorized** by the product baseline.

1.1 Engineering decision register

Decision	Selected design / reason	Trade-off and current approval
SD-01	A TypeScript modular monolith, not microservices: a single Express API exposes bounded identity, scheduling, Booking and future notice/report modules.	Smaller operational and review burden for two students; **proposed for peer approval**.
SD-02	React 19 + Vite 8, client-side navigation, accessible shared form/list/status components.	No SSR complexity for the accepted workflows; **proposed**.

Decision	Selected design / reason	Trade-off and current approval
SD-03	Node.js 24 LTS + npm 11 workspaces with one root lockfile; Express 5, TypeScript 5.9.	Matches the inspected development runtime and supports reproducible builds; **proposed**.
SD-04	PostgreSQL 18, Prisma 7 with PostgreSQL adapter, versioned SQL migrations; native PostgreSQL range/exclusion constraints supplement ORM checks.	Relational integrity is essential for races A/B/C; raw SQL constraints require review and DB-specific tests; **proposed**.
SD-05	Keep one immutable 60-minute time interval per Slot. Derive Tutor schedule from owned Slots and Bookings, not a separate writable schedule table.	Avoids drift and unnecessary entity state; a Tutor schedule is a **logical read projection**.
SD-06	Derive occupancy from an active Booking; do not independently toggle a fragile Slot.isBooked flag. Maintain Slot lifecycle Offered/Withdrawn for later history.	Simplifies atomic cancellation/reschedule and protects historical records; **proposed**.
SD-07	PostgreSQL unique/exclusion constraints + SERIALIZABLE transactions + deterministic row locks + bounded retries for genuine transaction aborts.	More careful migrations and error handling; selected to prove all three real-DB races.
SD-08	Use DB clock_timestamp() *after lock contention* at the final guarded eligibility decision, and UTC instants in storage/transport with Asia/Bangkok presentation.	Prevents the Q05 request-arrival-time bug; clock/rounding tests are mandatory.
SD-09	Local Argon2id password hashes and opaque, server-revocable, cookie-based sessions; roles and record ownership checked by API/service.	Simpler reliable logout than stateless unrevocable tokens; cookie and CSRF safeguards are mandatory.
SD-10	A single-origin production web/API deployment (API can serve built web assets), durable managed PostgreSQL and managed secrets on GCP later.	Avoids cross-origin cookie/CORS complexity; **planning only** for Lab 5.
SD-11	Structured JSON errors and response allowlists; generated request IDs, no secrets, stack traces or third-party record details.	Feature SDS specifies exact endpoints/DTOs, not this whole-system overview.
SD-12	No invented Tutor work hours, look-ahead maximum, start-minute grid, private data visibility or retry-key retention policy.	Unresolved stakeholder choices remain gates OD-001..015, not implicit defaults.

All SD-01..12 require review in Issue #11's PR. These selections are *decisions proposed for approval*, not evidence of running packages, applied SQL or instructor authorization.

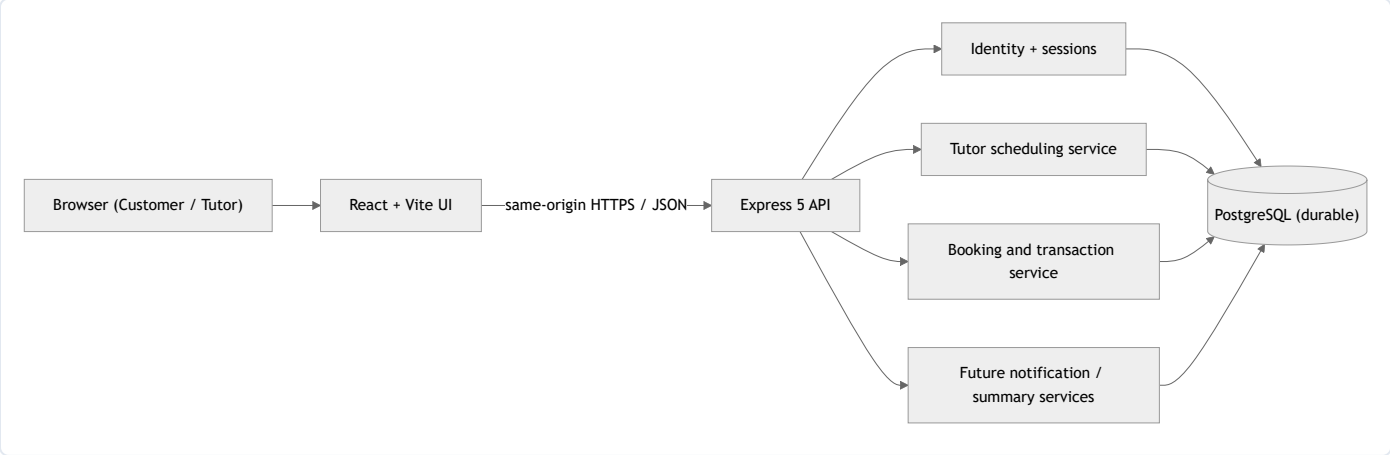
1.2 Technology baseline (design pins, not installed app dependencies)

Layer / tool	Version target	Basis / implementation note
Node.js / npm	Node 24.x LTS; npm 11.x	Actual inspected machine: v24.14.0 / 11.18.0; choose Node 24 rather than introducing new Node 26 to this project.
Frontend	React 19.3.1, Vite 8.3.5, React Router 7.x	Candidate current registry versions checked 3 Nov; Router exact patch determined when installed.
API and validation	Express 5.3.0, TypeScript 5.9.3, Zod 4.6.7	End-to-end TypeScript, validated boundaries; deliberately choose mature TS 5.9 rather than a new major.
DB/ORM	PostgreSQL 18.x; prisma, @prisma/client and @prisma/adapter-pg **7.10.1** plus pg	Prisma 7 requires its supported driver-adapter/config path; maintain SQL migrations for exclusions/partial indexes.
Credential/session	argon2 0.45.2; Node crypto secure random and SHA-256 for session-token digests	Password hashes use Argon2id with distinct salts; token digests are **not** password hashes.
Tests	Vitest 5.1.0, Supertest 7.3.1, Playwright 1.64.1; real PostgreSQL	Versions were read from npm registry; no project test suite currently exists.
Local tooling	Git, Docker, PostgreSQL service/container, optional psql	Docker CLI detected (29.9.2); psql CLI was **not found on PATH**. No DB container or running DB was verified.

Version discipline: Record these as *proposed baseline pins*. During the scoped Part E implementation, check real peer-dependency compatibility, resolve patches using npm install, commit package-lock.json, use npm ci thereafter, and update this

table only through an explained reviewed change. Do not claim that a package is installed merely because npm registry reports it. The PostgreSQL 18 target must also be checked against the GCP managed offering and required extensions before Lab 5.

2. System architecture and component boundaries



- **UI:** renders navigation and the permitted controls, converts UTC instants for display in Asia/Bangkok, preserves form state on errors. UI checks are usability feedback **not authorization**.
- **Transport/API boundary:** authenticates the session, validates and normalizes input, imposes request/body limits, assigns a correlation ID, calls one application use case, and serializes an explicit data allowlist.
- **Domain/services:** keep time and Booking rules, role/record ownership, mutation transaction boundaries and feature-specific outcomes in one place. A controller must not implement a second inconsistent cutoff check.
- **Persistence:** Prisma Client manages typed access, migration provenance and transactions; reviewed parameterized SQL handles PostgreSQL row locking, constraints and DB-time sampling where required. No in-memory array acts as the source of truth.
- **Future capabilities:** Customer cancellation/reschedule and Tutor withdrawal/notice reuse the Booking service and Slot/Booking constraints. Tutor summary derives count and list from the same authorized query predicates; no independent source of state.

Boundaries: anonymous browser -> session-checked API is the primary trust boundary; API -> database uses a least-privileged account. A client-provided customerId, tutorId, user role, Booking owner, current status or cutoff timestamp is **never trusted**. Public Customer availability is a deliberately narrow projection, not the Tutor's restricted full schedule. API and web may be separately built in development but share one origin through the Vite proxy; production can serve prebuilt web assets through the API image.

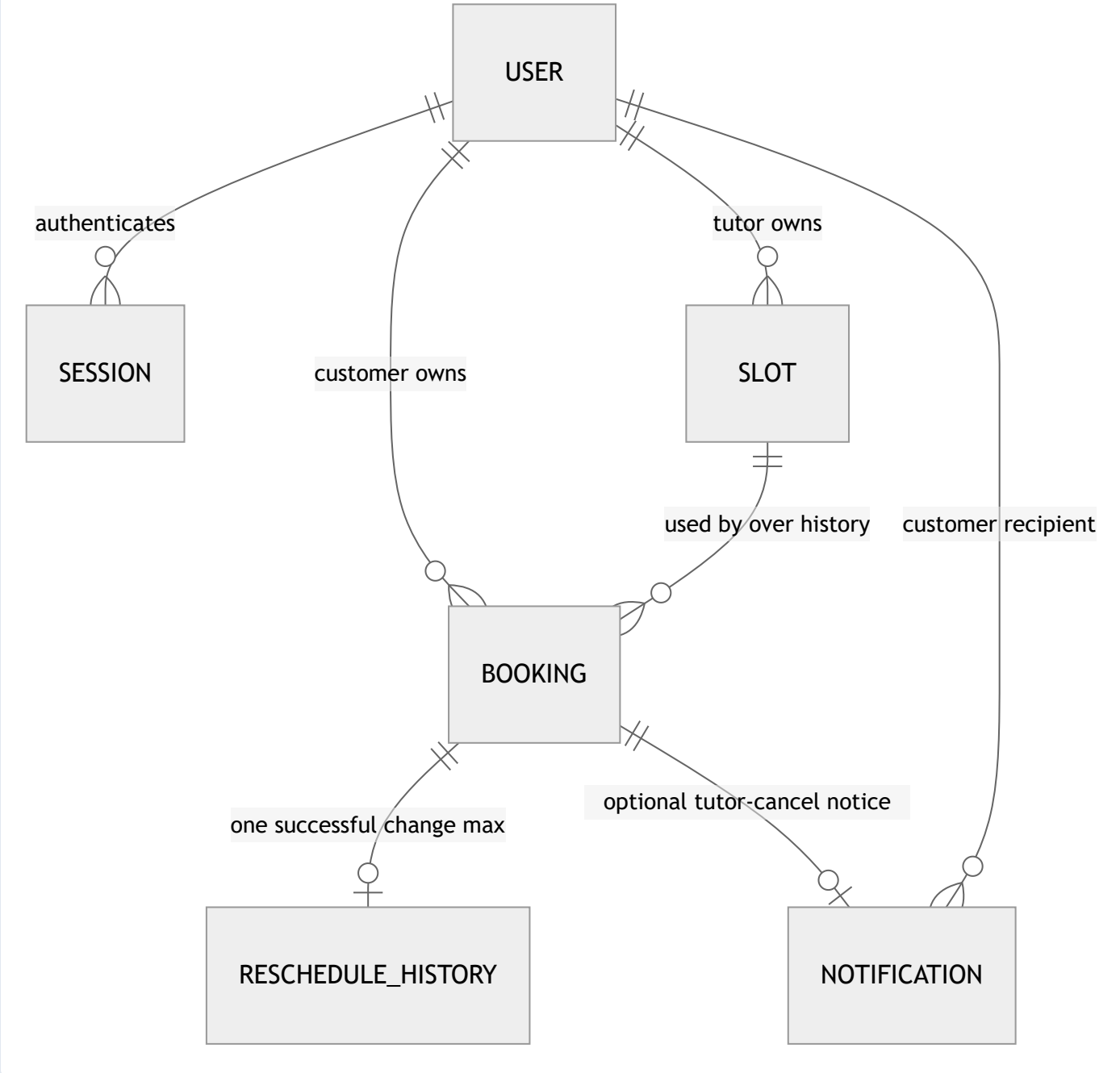
Suggested future repository layout (none of these folders are created by Issue #11):

```
package.json                # private npm workspaces, pinned Node/npm; scripts
package-lock.json           # committed after first install
apps/web/                   # React/Vite, routes, page components, API client
apps/api/src/               # Express controllers, auth, services, DB adapter
apps/api/prisma/            # Prisma schema, versioned migrations, seed
apps/api/prisma.config.ts   # Prisma 7 connection/migration/seed configuration
apps/api/test/              # API, DB-integrity and concurrency tests
docs/                      # product SRS, traceability, System SDS, Feature SDS/STS
```

No separate payment, background-queue, cache, notification-provider or microservice infrastructure is required by Sprint 1.

3. Relational domain, ownership and evolution

3.1 Entity relationship view



A **Tutor Schedule** is the filtered projection of one Tutor's Slots and the currently assigned Bookings. It is **not** a second mutable appointment record. This keeps Q02 ownership, Q06 overlap and Q31 counts attached to authoritative relations.

3.2 Proposed entities and keys

Entity (stage)	Essential columns / relationships	Validation and persistence invariant
User (S1)	id UUID PK; email with normalized unique index; passwordHash; role Customer/Tutor; isActive; timestamps	Exactly one effective role per account; active account for authentication; role/ownership verified server-side, never from a client body. No self-registration is in scope.

Entity (stage)	Essential columns / relationships	Validation and persistence invariant
Session (S1)	id UUID PK; userId FK -> User; unique tokenDigest; createdAt; expiresAt; revokedAt (nullable); optional revocationReason for audit (proposal)	Store digest, not raw cookie token. Logout revokes **presented Session only**; proposed credential reset/compromise handling revokes **all** relevant User sessions. No protected request may use a revoked/expired token or inactive User (OD-005).
Slot (S1 fields, S2 lifecycle)	id UUID PK; tutorId FK -> User; startsAt and endsAt TIMESTAMPTZ; lifecycle Offered or Withdrawn; createdAt; withdrawnAt nullable	Interval exactly 60 minutes; immutable Tutor/time/ID; offered Slots of one Tutor never overlap; withdrawn Slot never reactivated under the same ID.
Booking (S1 fields, S2 extension)	id UUID PK; customerId FK -> User; currentSlotId FK -> Slot; startsAt/endsAt current interval snapshot; status Booked or Cancelled; cancellationActor and tutorCancellationReason nullable (S2); timestamps	One active Booking per Slot; one Customer's active windows never overlap; interval copied from locked Slot, never accepted from a client; current Tutor derives from Slot.tutorId.
RescheduleHistory (S2 only)	id UUID PK; bookingId FK -> Booking **UNIQUE**; old/new Slot IDs and Tutor IDs; old/new intervals; changedAt	Successful change appends exactly one immutable record while retaining Booking ID. Rejected change appends none and uses no quota.
Notification (S2 only)	id UUID PK; recipientCustomerId FK -> User; bookingId FK -> Booking; kind TutorCancelled; createdAt; readAt nullable	One relevant private notice per successful Tutor cancellation, persisted in the same transaction; readAt never deletes the notice. Reason may be safely projected from owning Booking rather than duplicated.

- Slot -> Booking is **one-to-many historically**, but **zero-or-one active**. Cancelled Bookings keep their Slot history and no longer occupy it. Slot Offered means **published**, not necessarily free: occupancy is calculated from Booking status Booked.
- Past is a derived display classification for Booked appointments whose **end** is earlier than the evaluation instant; it is **not** a Booking status, and the application must never invent Completed or attendance.
- A Customer cancellation sets Booking Cancelled and leaves its original Slot Offered; another new Booking must still pass Q05 independently. A Tutor cancellation sets Booking Cancelled and original Slot Withdrawn **in one transaction**. Direct withdrawal while an active Booking exists is forbidden.
- On reschedule (S2), retain Booking.id, move currentSlotId and current interval, write history atomically, free the original offered Slot and claim the destination. Rescheduling to a different Tutor is valid when all rules pass. Prior-Tutor historical visibility is **not** assumed (OD-015).
- Optional reporting fields, Tutor profile, teaching subject, room, payment, price or attendance are **not** mandatory and must not be added on invented stakeholder authority.
- FKs ensure stable ownership references; User role cannot be inferred from an arbitrary FK alone, so service authorization additionally validates role/active status. The database user is not granted app-user impersonation rights.

3.3 Native PostgreSQL constraints (migration intent, not executed SQL)

Prisma schema alone cannot express the required conditional interval/exclusion guarantees in the chosen design. Part E must keep the following *reviewed migration logic* in a versioned SQL migration, test applying it against a fresh PostgreSQL 18 database, and ensure Prisma-generated migrations **do not drop** these constraints. Actual column names/types must match the implemented schema before applying the snippets.

```
-- Requires btree_gist to compare UUID equality inside GiST exclusions.
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE "Slot"
  ADD CONSTRAINT slot_exactly_one_hour
  CHECK ("endsAt" = "startsAt" + INTERVAL '60 minutes');

ALTER TABLE "Slot"
  ADD CONSTRAINT offered_slot_no_overlap_per_tutor
  EXCLUDE USING gist (
    "tutorId" WITH =,
    tstzrange("startsAt", "endsAt", '[)') WITH &&
  )
  WHERE ("lifecycle" = 'Offered');

-- Both cancelled and rescheduled history may refer to a Slot,
-- but at most one current Booked record may occupy it.
CREATE UNIQUE INDEX booking_one_active_per_slot
  ON "Booking" ("currentSlotId")
  WHERE "status" = 'Booked';

ALTER TABLE "Booking"
  ADD CONSTRAINT booked_customer_no_overlap
  EXCLUDE USING gist (
    "customerId" WITH =,
    tstzrange("startsAt", "endsAt", '[)') WITH &&
  )
  WHERE ("status" = 'Booked');

-- Protect the duplicated current appointment interval from diverging
-- from the immutable selected Slot, including reschedule updates.
ALTER TABLE "Slot"
  ADD CONSTRAINT slot_id_window_unique
  UNIQUE ("id", "startsAt", "endsAt");

ALTER TABLE "Booking"
  ADD CONSTRAINT booking_current_slot_window_fk
  FOREIGN KEY ("currentSlotId", "startsAt", "endsAt")
  REFERENCES "Slot" ("id", "startsAt", "endsAt")
  ON UPDATE RESTRICT;
```

Migration preconditions: all identifiers not-null where needed; UUID columns really use UUID; timestamps use TIMESTAMPTZ; enums/columns have the exact quoted values above or their implemented equivalents. Add explicit FK/indexes for customerId, tutorId, currentSlotId and history/notice references, and check nonblank required values in the appropriate feature contract. The database exclusion predicates are **only** for currently Offered Slots and **only** for Booked Bookings: past *Booked* still counts as active, whereas withdrawn historical Slots do not block new distinct offers (Q32). Adjacent half-open intervals never conflict.

The preceding constraints directly enforce **same-Tutor offered overlap** (Race A), **single active owner per Slot** (Race B) and **same-Customer active time overlap** (Race C). They do **not** by themselves enforce the cross-table lifecycle invariant:

I-L (Q26/Q27/Q32, FR-033/034, BR-026/032/034): For every committed Booking whose status is Booked, the referenced Slot **must** have lifecycle Offered. Conversely, a committed Withdrawn Slot **must never** have any active Booked Booking. This holds across all application instances, including a direct SQL mutation that bypasses an application service guard. A historical Withdrawn Slot cannot transition back to Offered under the same ID.

Required database enforcement contract (to implement and prove in the Part E migrations, not executable SQL supplied here):

1. Install a PostgreSQL **immediate BEFORE INSERT/UPDATE row trigger on Booking** for changes affecting status, currentSlotId or the interval. When the resulting Booking is Booked, lock the referenced Slot row with SELECT ... FOR UPDATE (within the writer's transaction), then reject unless lifecycle is Offered, the Slot is still eligible, and the immutable Slot interval matches the Booking. For **new Booking INSERT**, this guard also samples the DB-authoritative Q05 clock *after* required locks, as clarified in §§4 and 7.3. The existing unique/exclusion constraints remain in force. The runtime service must acquire the §7.1 locks **before** changing Booking; the trigger is a final DB guard even when a service is bypassed.
2. Install a PostgreSQL **immediate BEFORE UPDATE lifecycle trigger on Slot**. On Offered -> Withdrawn, the updating transaction already holds the Slot row lock; reject if any Booked Booking references it. On Withdrawn -> Offered, reject

unconditionally (Q32). Do not permit arbitrary changes to Slot ID/Tutor/interval or DELETE of an occupied Slot. For Tutor cancellation, change the Booking to Cancelled **before** withdrawing its Slot within one atomic transaction, after obtaining all locks in §7.1 order.

3. Both trigger paths must use the same serialized Slot row for the target Slot; a competing Booking/withdrawal writer must wait and then re-read lifecycle/occupancy on a fresh transaction view or abort/retry on Serializable serialization failure. Triggers must throw a non-success error so the entire transaction rolls back, not silently repair partial state. The database account used by the API must not have privilege to disable triggers. An unexpected direct SQL writer may deadlock against the documented service lock order; that attempt must abort/retry, **never commit an invalid lifecycle**.
4. **Implementation gate:** Part E must validate actual PostgreSQL trigger firing order, locking, MVCC visibility and Serializable failure behavior with a disposable real DB. If the proposed immediate trigger/locking protocol cannot demonstrate I-L at commit under all tested interleavings, revise the implementation contract (e.g., additional DB serialization/constraint strategy) **before** accepting the feature. Documentation alone does not prove the triggers safe. Do not defer enforcement as optional S2 work: the Booking guard belongs to S1 and the withdrawal guard must exist before S2 withdrawal is enabled.

Traceable proof: S1 F-S1-04/STS tests direct SQL attempts to insert a Booked Booking against a Withdrawn Slot once the lifecycle exists; S2 F-S2-05 and F-S2-03/STS test direct withdrawal of an occupied Slot and simultaneous Booking-versus-withdrawal; SYS-01 checks the resulting committed Slot/Booking rows, error response and rollback. See the inter-operation cases in §7.5. Any S1-only migration whose Slot enum does not yet expose Withdrawn must record the planned forward-compatible gate, not claim S2 lifecycle was already tested.

4. Time model and business transitions

- **Business timezone:** IANA Asia/Bangkok, UTC+07:00 for all stakeholder-facing date/time calculations and display (BR-029/Q29).
- **Storage/transport:** TIMESTAMPTZ instants in PostgreSQL and RFC 3339 ISO-8601 timestamps with an explicit offset on wire, normalized to UTC Z in JSON responses. Reject timezone-free date-time strings in mutation input. Render with an explicit Asia/Bangkok formatter; browser timezone or clock cannot affect eligibility.
- **Intervals:** half-open [startsAt, endsAt); overlap exactly when a.start < b.end AND b.start < a.end. Exact adjacency is permitted for one Tutor's offers and a Customer's active Bookings (Q06/Q28).
- **Tutor creation:** future 60-minute Slot; do **not** silently apply the **new Booking** 60-minute cutoff to Slot creation; proposed hours, horizon and minute grid stay OD-001..003.
- **New Booking (Q05):** S - T_elig >= **60 min**, equality passes. T_elig is a fresh DB-authoritative instant obtained **inside the transaction after relevant lock/contention waits and immediately before the final guarded write**. It is **not** HTTP arrival time, transaction start time, client time or commit time.
- **Customer cancellation (Q07, S2):** start minus authoritative decision >= **30 min**, equality passes; reopen original Slot but Q05 still applies to later claims.
- **Reschedule (Q10/Q11, S2):** both original appointment and destination start are >= **60 min** away at authoritative decision, equality passes; at most one successful reschedule (Q16).
- **Tutor cancellation (Q18, S2):** start minus authoritative decision >= **60 min**, equality passes; requires meaningful nonblank reason and withdraws Slot.
- **Q30:** status Booked/Cancelled only; Past after interval end is derived, never Completed. How *in-progress* lessons fit the S3 summary is still OD-006.

Q05 contention example: 15:00 Slot, request arrives 13:59:59, but waits for a row lock until the authoritative guarded instant is 14:00:01. The request **fails** (59m59s remaining). If the guarded instant is exactly 14:00:00 and all other checks pass, a commit *after* that instant does not itself invalidate eligibility. Always revalidate upon retry of an aborted transaction; do not reuse an earlier T_elig.

Operational precision (OD-005, F-S1-04): The authoritative Q05 guard is **inside the PostgreSQL Booking INSERT trigger**, after the relevant User/Customer and Slot locking protocol in §7.1 and before the Booking row can be accepted; a prior HTTP/controller check is only advisory. Every valid mutation path must acquire the same conflict-serialization rows so competing Booking, cancellation and withdrawal operations cannot produce a *new* conflict-capable wait after this guard. PostgreSQL unique/exclusion checks can still wait implicitly if some writer bypasses the expected lock protocol. An implementation must not

claim success from a T_elig sampled before such an unaccounted wait: its Part E transaction adapter must demonstrate a **no-post-guard-contention invariant** with real DB tests, or explicitly abort and retry the whole transaction with a fresh DB clock on detecting an unaccounted wait. If detection cannot be proven reliably, the design is **blocked for code acceptance** until an alternative DB guard/serialization strategy is verified. Do not replace this with a blanket commit-time cutoff: SRS Q05 permits a genuinely valid guarded check whose commit happens later. See §7.3 and required barrier case VC/TEST-S1-39 in §7.5.

5. Authorization, authentication and data minimization

5.1 Server-enforced role and record matrix

Legend: **own** means tied to the authenticated principal/actual entity ownership, **no** means forbidden, **public policy TBD** must not be interpreted as permitted anonymous access. The stage column is the proposed delivery stage, not a claim that code exists.

Operation	Anonymous	Customer	Tutor/Staff	Stage
Submit credentials on login screen	login only	login	login	S1
Get current identity and logout	no	own	own	S1
Publish/read Tutor's restricted Slots/schedule	no	no	own Tutor schedule only	S1
Discover narrow availability	**no unless approved**	authenticated, privacy-filtered	no Customer privilege by Tutor role	S1
Book an eligible offered Slot	no	as authenticated Customer	no	S1
Read private Customer Booking list/detail	no	owned Bookings only	no global Customer lookup	S1
Read Tutor's attached Booking schedule	no	no	own authorized Tutor schedule only	S1
Cancel or reschedule Customer Booking	no	owned eligible Booking only	no via Customer route	S2
Tutor-cancel assigned Booking with reason	no	no	currently assigned Tutor only	S2
Withdraw unoccupied future Offered Slot	no	no	owner Tutor, **no** if active Booking	S2
Read private cancellation notice/reason	no	addressed Customer / owned cancelled Booking only	no unrelated record or prior-Tutor privilege	S2
Tutor Summary counts and lists	no	no	own Tutor scope only	S3
Any foreign Customer/foreign Tutor record	no	no	no	all

Every record query is scoped on the server by session principal + actual customerId/Slot.tutorId, including nested detail IDs, list filters, pagination, exports (if ever approved), and error branches. For Tutor reads join the Booking's current Slot to its actual Tutor; do not grant global Customer visibility. For historical cross-Tutor movement, keep OD-015 closed-by-explicit-decision **before** revealing anything to a former Tutor. Frontend hiding of controls cannot substitute for API checks.

5.2 Credential and session contract proposal (OD-005 gate)

1. **Accounts:** pre-provisioned Customer/Tutor accounts for Sprint 1. Validate a canonicalized, uniquely indexed email; use Argon2id with unique salt and configurable reviewed work factors. Never store plaintext passwords, seed shared known production passwords, expose hash fields in JSON or log credentials.
2. **Login:** validate input and account active state, verify Argon2id hash, issue a cryptographically random **at least 256-bit opaque session token** (not a predictable UUID). Persist only a SHA-256 digest with userId, creation, expiration and

revocation state. Send raw token only via HttpOnly cookie.

3. **Cookie:** Secure, HttpOnly, SameSite=Lax, Path=/, no Domain in HTTPS production. A non-Secure local-development exception may be used **only** for localhost HTTP and never in production; document the local cookie name/behavior in F-S1-01.
4. **Expiry: OD-005 candidate policy, NOT approved:** eight-hour absolute expiry from createdAt, with no implicit refresh; a protected request checks token digest, expiresAt > server time, revokedAt IS NULL, User.isActive and effective role against persisted state on each request. An expired cookie never restores access, even if the browser retains it.
5. **Revocation and replacement policy (OD-005 proposal; approval required in F-S1-01):** a normal **login** creates one new Session and leaves other valid Sessions unchanged (no hidden rotation); **logout** transactionally revokes only the Session presented by the cookie and acknowledges completion/clears that cookie **after DB commit**. If optional explicit **session rotation** is added later, revoke the old Session and create its replacement in **one DB transaction**, then issue the new cookie only after successful commit; an old token must never work after rotation. A **password/credential change** (not an S1 feature) is proposed to revoke *all* prior Sessions for that User and create one replacement atomically if the authenticated change succeeds. An authorized **compromise response** (not an S1 feature) would revoke *all* Sessions without implicitly creating a new token. **User deactivation** (future administrative action) must deny every token immediately on the next protected request by User.isActive check and should revoke all User Sessions in the same transaction as deactivation. No password-change, compromise or administrative UI/API is authorized by this SDS.
6. **Atomicity/failure semantics:** any failed logout update or optional rotation/change transaction is a non-success, not a claim of revoked/issued state; do not deliver a new session cookie until its persisted record commits. Logout is idempotently safe for a previously revoked presented token (never grants access). Unknown network outcome is reconciled by checking current session validity rather than assuming rollback. Exact TTL, all-session revocation triggers, CSRF-token format and rate limiting remain **OD-005** engineering decisions subject to peer approval.
7. **Browser safeguards:** same-origin production; CSRF token tied to the session plus server Origin/Referer checks for cookie-authenticated mutation routes. Enforce HTTPS and validate reverse-proxy trust configuration. No auth token in browser localStorage or client-controlled role claims.
8. **Failure handling:** use non-enumerating login errors, input size limits and bounded login-attempt protections; specific rate limits/password policy and expiry/refresh decisions require F-S1-01 review under OD-005. Authenticated foreign-record errors disclose no private payload.

Proposed auth negative-test matrix (F-S1-01 STS, VAL-001/002/034/036):

Precondition / action	Expected persisted state and protected API result
Login twice from different browsers (normal login)	Two independent valid Sessions (unless future approved policy changes); logging out one does not revoke the other.
Logout with cookie A, then reuse A	Session A revoked in DB after successful logout transaction; stale A denied on /current-user and every protected resource; cookie B from independent login remains valid.
Optional rotation or successful credential change (future)	One atomic replacement and old-token revocation for rotation, or all old Sessions revoked for credential change; every old cookie denied and only committed new cookie accepted. Inject failure and assert no partially delivered replacement.
Authorized compromise revoke-all (future)	All existing Sessions invalid, no fresh Session created implicitly; old cookies denied.
Deactivate User (future) while a session exists	A protected API call made after deactivation denies access even if revokedAt was not yet observed; deactivation and bulk revocation tested atomically when implemented.
Now >= expiresAt, manually replay still-present cookie	401/non-success and no protected fields; subsequent login requires fresh valid credentials, not implicit renewal.
Invalid/unknown credentials, wrong role, forged owner ID, invalid CSRF or revoked Session	Sanitized non-success without protected payload, credential enumeration, session token or stack trace.

These are **planned API and persistence tests**, not executed results. Explicit test fixtures and error status mapping belong to F-S1-01 Feature SDS/STS; the eight-hour expiration, bulk invalidation and future rotation behaviors are proposed OD-005 choices, not confirmed stakeholder policy.

5.3 Data exposure contract

- **Customer availability:** only an approved safe Tutor label/ID, Slot ID and UTC interval, plus agreed availability indication; **never** another Customer's Booking, email/phone, cancellation reason or full restricted schedule.
- **Customer own Booking:** Booking ID, relevant Tutor/Slot/time, status, and only that Customer's approved history/reason. Cross-customer list/detail guesses reveal no data.
- **Tutor schedule:** only owned Slots and permitted Booking fields. Customer identity display remains unapproved; use the minimum necessary representation and obtain approval in F-S1-05 API/UI contracts.
- **Notice (S2):** only the recipient Customer can reread; reason additionally appears in that Customer's cancelled Booking details. Reading must not delete or automatically age-expire it (Q24/25). Data deletion/retention and prior-Tutor history are open OD-008/015.
- **Logs/errors:** redact cookie, token, password/hash, personal Booking and private reason. Emit non-sensitive correlation IDs and structured failure codes only. No raw SQL errors, internal stack traces or secrets in responses.

6. Shared API and UI contracts

6.1 Shared API conventions (not feature-specific endpoint signatures)

- Same-origin HTTPS JSON; path prefix proposed /api/v1. Auth operations conceptually cover login, current-user retrieval and logout; Slot/availability/Booking endpoints and exact URL/method combinations are **deferred to the relevant Part E api-spec.md**.
- JSON keys use lowerCamelCase; IDs use opaque UUID strings; instants use RFC 3339 UTC strings with Z. Pagination/ordering defaults, safe Tutor labels and detailed request/response DTOs are **not frozen** here (OD-004/009).
- Validate path, query and JSON bodies with Zod and server checks (types, length/size, ISO offset, 60-minute arithmetic, state and identity). Reject unknown privileged identity/status fields; construct writes from trusted session and locked DB records. Parameterize all SQL.
- On successful *mutations*, only return confirmed committed outcomes; do not imply a Booking exists before commit. Read results may become stale; a displayed Available marker is not a lease or lock.
- Standard candidate response envelope:

```
{
  "data": {
    "id": "server-generated-uuid",
    "startsAt": "2026-11-08T08:00:00.000Z"
  },
  "meta": { "requestId": "opaque-correlation-id" }
}
```

```
{
  "error": {
    "code": "SLOT_UNAVAILABLE",
    "message": "This appointment is no longer available.",
    "requestId": "opaque-correlation-id"
  }
}
```

- **Proposed status vocabulary:** 400 malformed input; 401 no valid session; 403 wrong role without record detail; 404 missing/foreign record when existence would disclose privacy; 409 occupied/stale overlap or rejected conflict; 422 well-formed but invalid field/business condition; 429 rate limit; 500/503 internal or transient persistence failure, sanitized. Exact endpoint-specific mapping and stable error codes belong to Feature SDS/API Spec and tests. Error messages must not reveal whether a foreign Booking exists.
- Use request IDs for tracing. UTC wire timestamps round-trip to Bangkok displays exactly; server determines time comparisons, not the browser. Do not serialize Prisma objects/columns wholesale without response allowlists.

6.2 Screen inventory and navigation

Screen / state	Role and stage	Minimum behavior
Login / signed-out	Anonymous, S1	Validated login, generic auth failure, keyboard access and session continuity.
Customer availability	Customer, S1	Tutor + time discovery, loading/empty/no-match/error states, eligibility/refresh indication; bookable-only vs disabled items remains OD-004.
Customer Booking confirmation + My Bookings/detail	Customer, S1	Display only committed Bookings after successful write, own list/detail and accurate Booked/Past; refresh preserves data.
Tutor Slot publish and own schedule	Tutor, S1	Single start input with derived 60-min end, validation/errors, Offered/occupied distinction, own attached Bookings.
Customer cancel/reschedule and history	Customer, **S2 proposed**	Cutoffs, one-success quota, retained Booking ID, cross-Tutor selection, no partial confirmation.
Tutor cancellation/Slot withdrawal	Tutor, **S2 proposed**	Current assignment/ownership, nonblank reason, distinct cancelled vs withdrawn consequences.
In-app notices	Customer, **S2 proposed**	Private read/reread without automatic expiry; no broadcast.
Tutor Appointment Summary	Tutor, **S3 proposed**	Matching category counts and lists, correct owner scope; in-progress bucket OD-006.

Shared UI conventions: accessible labeled controls, visible focus, keyboard use, field-level validation plus server error banners, pending/disabled submit state (not a concurrency proof), explicit empty vs no-results states, retry controls for safe reads, confirmation only after committed write, and responsive phone/tablet/desktop layouts. App role navigation is based on current-user API, but API still enforces every permission. Use consistent loading skeletons/spinners, success, stale/conflict, offline, forbidden and not-found states. Avoid silently clearing a form after transient or validation errors; revalidate server-side on resubmit. Proposed performance (p95 <=2.0 s) and keyboard acceptance thresholds are OD-010 and **not approved NFR gates**.

7. Transaction protocol and three race proofs

7.1 Shared mutation invariants

1. **I-A (Q06):** at most one currently Offered Slot for a Tutor overlaps a given offered interval; one-hour duration, same-Tutor adjacency allowed.
2. **I-B (Q01):** at most one current Booked Booking uses any Slot; a failed attempt leaves no orphan reservation or false success.
3. **I-C (Q28):** one Customer has no overlapping current Booked intervals across Tutors; adjacency allowed.
4. **I-T (Q05/Q29):** time and validity are checked at authoritative post-contention T_elig; exact 60:00 is eligible, 59:59 is not.
5. **I-P (atomic persistence):** all accepted changes are committed together; a rejected, aborted or failed write leaves the previous committed state and non-success response. Past Booked stays Booked.

Global lock hierarchy for every domain mutation (SD-07 / OD-005): lock **all affected User principals in ascending UUID order** (including Customer and the currently assigned/selected Tutors), then **all affected Slot rows in ascending UUID order**, then **existing Booking rows in ascending UUID order**. Lock the required principal rows even if the operation seems to need only the Customer or Tutor: this provides a single agreed order across creation, booking, cancellation, reschedule and withdrawal. PostgreSQL Serializable isolation and the constraints/triggers in §3.3 are mandatory backstops, not substitutes for that order.

Operation	User locks first (ascending UUID)	Slot locks next (ascending UUID)	Booking locks last and guarded change
Tutor Slot creation (S1)	Authenticated Tutor	None (new Slot does not yet exist)	None; insert Offered Slot after Tutor lock, validate no overlap at DB constraint
Customer new Booking (S1)	Authenticated Customer **and selected Slot's actual Tutor**	Selected Offered Slot	No existing Booking required for new claim; read active conflicts after locks, trigger-guarded insert

Operation	User locks first (ascending UUID)	Slot locks next (ascending UUID)	Booking locks last and guarded change
Customer cancellation (S2)	Booking owner Customer **and current Slot's Tutor**	Current Slot	Current Booking; verify owner/status/cutoff; update to Cancelled, leaving Slot Offered
Customer reschedule (S2)	Booking owner Customer, **original Tutor and destination Tutor** (deduplicate/sort)	Original and destination Slots, sorted	Current Booking; validate current assignment and Q10/Q11, update same Booking and unique history atomically
Tutor cancellation (S2)	Current assigned Tutor **and owning Customer**	Current Slot	Current Booking; validate assignment/reason/Q18, set Booking Cancelled **before** withdrawing Slot and inserting private notice in same transaction
Tutor Slot withdrawal (S2)	Owning Tutor	Target Offered Slot	If active Booking discovered, reject; no direct mutation of an occupied Slot, then withdraw it

Non-authoritative discovery pre-read: Customer cancellation, reschedule and Tutor cancellation need the current Booking's Customer/current Slot/Tutor IDs to know which principal and Slot locks to take; booking and reschedule also need the selected destination's real Tutor. First perform an **unlocked, read-only lookup** of Booking/Slot references for lock-set discovery, *not* as an authorization or business-rule decision. Begin (or restart) a Serializable transaction, lock the complete discovered User set and Slot set in the global order, then lock/re-read the Booking and selected Slot(s). Verify that **all** discovered IDs, current assignments/status, relevant owners and intervals are unchanged. If any membership/assignment changed or a relevant row disappeared, abort and retry from a **fresh discovery** with no locks carried over; never add a missing earlier-order User/Slot lock while holding a later-order Booking lock. Only the post-lock state can authorize the actual write. Locking pre-reads are forbidden as the sole proof of current state. Subsequent triggers lock already-held Slot rows, and nonconforming writers must abort on deadlock/serialization/conflict rather than bypass invariants.

Part E lock-order proof obligation: review actual SQL/Prisma query order for **each row of the table**, instrument concurrent transactions that discover changed assignments between pre-read and locked re-read, and demonstrate retries/rollback without partial rows. A missing lock, unconstrained direct SQL writer or unaccounted post-guard wait is a failed proof gate, not an accepted shortcut.

7.2 Tutor Slot creation (Race A; F-S1-02)

1. Authenticate a live Tutor and validate candidate start/derived end from the request (no client tutorId authority).
2. Begin one database transaction (proposed Serializable); lock that Tutor principal row. Recheck active Tutor identity, future time and any *approved* calendar constraints.
3. Insert Offered Slot. PostgreSQL's conditional GiST exclusion rejects a same-Tutor overlapping Offered window even under simultaneous writers. Adjacent windows or same-time offers by **different** Tutors are permitted when otherwise valid.
4. Commit, then return success; on exclusion/serialization conflict, roll back and return a controlled non-success unless a bounded full retry can legitimately succeed. Never update a historical Withdrawn Slot to Offered.

Proof: Q06, FR-006, BR-006/034, NFR-004, F-S1-02, VAL-005, AC-S1-13, VC/TEST-S1-37. Use a synchronized barrier with overlapping starts; inspect committed Slot rows and verify the loser did not create a partial Slot.

7.3 Exclusive Booking (Races B/C and Q05; F-S1-04)

1. Authenticate current Customer and validate request contains a Slot reference, not a spoofable customerId/role/start timestamp.
2. Discover the selected Slot's real Tutor as a **non-authoritative pre-read**. Begin one database transaction with **Serializable isolation** and lock **both Customer and Tutor User rows in ascending UUID order**, followed by the selected Slot row. Revalidate that Slot.tutorId still matches the discovered Tutor and that the Customer remains active; on mismatch abort and restart discovery without adding an out-of-order lock. This serializes same-Customer and same-Tutor write paths under §7.1, not only the selected Slot.
3. Re-read Slot lifecycle Offered, immutable interval, occupancy and the Customer's **all-Tutor** Booked intervals inside the transaction. Compute availability from committed Booking relations, not a stale browser list.

4. Once all conflict-relevant principal/Slot locks are held, perform server-side preliminary validation. For the final authoritative Q05 decision, the **Booking INSERT row trigger** in §3.3 samples PostgreSQL **clock_timestamp() after its required Slot lock**, then evaluates Slot.start - T_elig >= 60 minutes *before accepting the new row*. It uses current locked Offered Slot state and DB-owned interval, not request time or a client field. A preliminary application time check is not T_elig.
5. The shared User/Slot locks and DB trigger prevent ordinary competing writes from introducing a fresh conflict-capable wait *after* T_elig: Race B shares the Slot lock and Race C shares the Customer lock, including with later S2 mutations. If a DB unique/GiST index check nevertheless waits on an unaccounted competing writer, **do not trust the earlier T_elig**. Part E must either prove no such post-guard wait can occur under the actual DB trigger/lock arrangement, or instrument/detect it and abort/retry the complete operation so the new attempt samples a fresh post-contention T_elig. Any attempt that cannot establish this proof is blocked, not marked as a successful Booking. Do not substitute a blanket commit-time time check that would reject an otherwise valid exact-boundary guarded decision.
6. Unique active-slot index blocks a second Customer on the same Slot (B); Customer range exclusion blocks overlapping Booked windows across Tutors (C), including separate application processes. Catch and classify exclusion/unique/serialization errors without treating them as successful claims.
7. Commit the Booking transaction; **only after confirmed commit** send success/Booking ID. A transaction rollback leaves no Booking and does not change occupancy, which is derived from the Booked row.

Proof B: Q01, FR-012, BR-001/034; F-S1-04, VAL-010, AC-S1-25 and VC/TEST-S1-25. **Proof C:** Q28, FR-013, BR-028/034; F-S1-04, VAL-011, AC-S1-26 and VC/TEST-S1-26. **Q05 proof:** FR-011, BR-005, VAL-008/009, AC-S1-35 and VC/TEST-S1-39. Force a **synchronized row-lock wait crossing** the exact 60-minute cutoff and assert the trigger's T_elig occurs *after* that wait. Include a second fixture for an unexpected **index/constraint wait** after preliminary validation, prove it cannot silently reuse a stale T_elig, and inspect the committed DB state. Include same-Customer adjacent intervals as permitted control cases. These are verification obligations, not claims of observed DB behavior.

7.4 Failure, retry, and later transitions

- **Rollback and retry:** handle known PostgreSQL SQLSTATE 40001 (serialization) and 40P01 (deadlock), including their ORM equivalents (Prisma P2034), by rolling back the **whole** transaction, using bounded jittered retries (**three attempts proposed**), reacquiring locks and **resampling T_elig** every time. A unique violation 23505 or exclusion 23P01 indicating a genuine conflicting offer/Booking is a *business conflict*, not permission to spin until one magically succeeds. Exhausted retries produce sanitized non-success. Do not retry non-idempotent effects outside the transaction.
- **Unknown response after possible commit:** network failure is not evidence of rollback. No blind duplicate success or changed Booking ID. In S1, reconcile through authenticated own-Booking reads and preserve invariants; an optional durable idempotency-key/result registry, payload fingerprint, key lifetime and repeated-response behavior remain **OD-013** to approve in F-S1-04/S2 Feature SDS. Never claim exactly-once delivery without such a contract.
- **Injected DB failure:** fail after validation / between scheduled statements, roll back, verify committed Booking and Slot state unchanged and API does not say success (FR-016/018, BR-034, VAL-014).
- **Stale UI:** an earlier Available card can become occupied, withdrawn or too close to start. Reject gracefully and refresh; frontend reservation indicators do not reserve database rows (FR-009/011).
- **Customer cancellation S2:** discover original Slot/Tutor from Booking only to identify the lock set; lock Customer + Tutor principals (UUID order), original Slot, then Booking. Revalidate ownership/current assignment and Q07 at authoritative transaction decision; set Booking Cancelled atomically. The original Offered Slot becomes unoccupied because occupancy is derived; Q05 still gates any new claim (VAL-015/016).
- **Customer reschedule S2:** discover original and destination Slot/Tutor references as a non-authoritative lock set; lock Customer + both Tutor principals (UUID order), **both** Slots (UUID order), then Booking. Revalidate original assignment, destination ownership, both 60-minute cutoffs and no conflicting other Customer bookings; atomically change currentSlotId + interval and append UNIQUE RescheduleHistory. Keep Booking ID; a failed attempt preserves destination, original and one-success allowance (VAL-017..020). On changed discovered IDs restart the full transaction.
- **Tutor cancellation S2:** discover the current Customer/current Slot only to determine locks; lock both User principals, Slot, then Booking per §7.1 and revalidate currently assigned Tutor, Q18 cutoff and nonblank reason; in one transaction set Booking Cancelled **before** Slot Withdrawn and then persist one private Notification. Failure leaves all three prior states, no notice (VAL-021..024).

- **Tutor withdrawal S2:** lock owning Tutor principal, target Slot, revalidate no active Booking and future Offered status, then withdraw. DB lifecycle trigger independently forbids an occupied withdrawal; no reactivation of historical ID. A new distinct Slot may later overlap that withdrawn historical interval (VAL-025..027).
- **Summary S3:** use a consistent snapshot or equivalent single-query predicate for count and list under the same Tutor/Booking filters; no inferred Completed. In-progress classification is gated (VAL-028/029).

7.5 Minimum real-database verification contract

Case	Real-DB setup / observable committed assertion	Owner
Race A (VAL-005)	Same Tutor submits overlapping Offered Slot inserts concurrently under a barrier; at most one currently Offered overlap persists; a distinct Tutor's same-time Slot and same-Tutor adjacency may succeed.	F-S1-02 SDS/STS
Race B (VAL-010)	Two Customers concurrently book the same valid Slot; exactly one Booked row after both settle; no two success responses or partial state.	F-S1-04 SDS/STS
Race C (VAL-011)	One Customer concurrently claims distinct overlapping Slots from two Tutors; never two overlapping Booked rows; adjacent control case allowed.	F-S1-04 SDS/STS
Cutoff after lock wait (VAL-008/009; VC/TEST-S1-39)	Use a real controlled barrier: start Booking at 60m+ while a competing transaction holds the required Customer/Slot row lock, release after the cutoff, assert the INSERT trigger's T_elig is **after** wait and Booking is rejected; run exact 60:00 control and a forced unexpected index wait case to reject stale-time success or block design acceptance until a proven retry strategy exists. Inspect committed rows and timestamps from the DB, not the request-arrival clock.	F-S1-04 SDS/STS; SYS-01 gate
Persistence fault (VAL-014)	Inject post-validation DB write/commit failure; verify no partial committed Booking, false success or Slot occupancy.	F-S1-04 + SYS-01
Auth/privacy (VAL-002/034/036)	Anonymous, wrong-role, foreign-Booking, foreign-Tutor, client-supplied IDs, revoked session; inspect entire payload, not only UI.	F-S1-01/05 + SYS-01
Durability (VAL-007/031)	Create Slot and Booking through actual UI/API/DB; restart app against same DB; retrieve correct owner-scoped state.	F-S1-02/04/05 + SYS-01
Booking vs Slot withdrawal (S2, Q26/27; VAL-025/026 plus VAL-014)	Start an eligible new Booking and unoccupied-Slot withdrawal concurrently with synchronized barrier; force both winning orders. Final state must be **(Offered + one Booked)** or **(Withdrawn + zero Booked)**, never Withdrawn with a Booked row. Include a direct SQL attempt bypassing the service guard; trigger must reject and rollback without ghost occupancy.	F-S1-04 Booking contract (S1 guard); F-S2-05 withdrawal SDS/STS (S2 execution); SYS-01
Reschedule vs Tutor cancellation (S2, Q12/13/17/19/21; VAL-020/022/023)	One Booking is simultaneously moved to a different Tutor and cancelled by its formerly assigned Tutor. Synchronize both orders. Verify same Booking ID, one-or-zero successful RescheduleHistory rows, correct old/new Slot lifecycle, currently authorized Tutor, and **notice/reason only if Tutor cancellation really committed**. Losing operation must leave no partial history, withdrawal or notice.	F-S2-02 + F-S2-03/04 SDS/STS; SYS-01
Customer cancellation vs new Booking (S2, Q07/08/05; VAL-015/016/010)	Original customer cancels sufficiently far ahead (>60m) while a second customer claims the same original Slot. Force cancellation-first and contender-first interleavings; at most one active Booked occupant, existing Booking becomes Cancelled only when its transaction commits, Slot remains Offered, and new Booking succeeds **only after** released occupancy and an independent Q05 check. No false double-success or partial status.	F-S2-01 + F-S1-04 Booking regression SDS/STS; SYS-01
Later regression (VAL-015..029)	S2 cancellation/reschedule/notification and S3 count/list rules; verify required constraints and no identity/time/privacy regression.	Corresponding S2/S3 SDS/STS

Tests must deliberately synchronize competing transactions, inspect **final committed rows**, include a losing contender and fault rollback, and record database version, setup/seed, command, Git SHA, pass/fail/skip counts and evidence. Pure mocks, Promise.all without synchronized interleaving, or HTTP-only checks are **insufficient** for the race obligations. These are **planned tests, NOT executed results**.

8. Reproducible local development and future delivery

8.1 Proposed scripts and configuration (not yet present)

Use one npm-workspaces root and two application packages. Proposed scripts to add in Part E, not commands claimed to work today:

```
npm ci                      # install exact locked project dependency graph
npm run db:up               # start local PostgreSQL (Docker Compose or existing service)
npm run db:migrate          # local: Prisma migrate dev incl. reviewed SQL constraints
npm run db:seed             # idempotent LOCAL/TEST-only Customer/Tutor fixtures
npm run dev                 # web + API; Vite same-origin /api proxy
npm run typecheck
npm run lint
npm run test:unit
npm run test:api
npm run test:db             # real PG + barriers + committed-row assertions
npm run test:e2e            # real UI/API/DB flow
npm run build
npm run db:migrate:deploy   # later release: reviewed Prisma migrate deploy
```

Document and implement each script before presenting it as runnable. Provide README setup prerequisites (Node 24, npm 11, PostgreSQL 18 or agreed compatible version, Docker optional), exact .env.example placeholders, startup/teardown instructions and fresh DB reset procedure **only for disposable development/test DBs**. Never reset a real or shared database. Use separate TEST_DATABASE_URL, migrations and unique test fixtures to prevent contamination. Seed is **idempotent**, uses only fictional records and does not expose known shared/production account credentials. Tests should use isolated DBs and rollback/cleanup of test fixtures.

8.2 Proposed environment variable names (examples, not secrets)

```
NODE_ENV=development
PORT=3000
DATABASE_URL=postgresql://APP_USER:CHANGE_ME@localhost:5432/nuttiew_dev
TEST_DATABASE_URL=postgresql://TEST_USER:CHANGE_ME@localhost:5432/nuttiew_test
SESSION_COOKIE_NAME=nuttiew_session_dev
SESSION_SECRET=GENERATE_A_STRONG_LOCAL_ONLY_SECRET
TRUST_PROXY=false
WEB_ORIGIN=http://localhost:5173
```

All values above are **placeholders**, never deployable credentials. In the proposed opaque-token/digest scheme, a SESSION_SECRET is only required if a separately signed CSRF/session adjunct is approved; do not treat it as a replacement for true random tokens or a reason to store raw sessions. Commit only .env.example and ignore real .env, test output, DB dumps and secrets. Do not place passwords/API keys in sample responses, tests or logs. For deployment use secret management; fail fast when required config is missing. Production cookies require HTTPS, Secure, HttpOnly and a reviewed proxy/host setup.

8.3 Database migration and operational readiness

- Create Prisma 7 schema and adapter configuration, include versioned custom SQL for CHECK, partial unique, GiST exclusion and composite FK. Check extension permissions (btree_gist), upgrade compatibility and migration dry-run on a new DB. Use a single migration authority, not Prisma db push on production.
- Index foreign keys/read paths, including Tutor Slots by owner+startsAt, Booking by customer and current slot, and future Notice recipient/time; verify query plans when data size makes it useful. Avoid storing computed Booked/Past or redundant occupancy counters.
- Preserve existing rows as S2/S3 migrations add lifecycle, history and notice features; use additive/default/backfill strategy, then constraints, never destructive drop-and-recreate. Test up/down or safe forward recovery and backup first; rollback a release by correcting forward schema where unsafe to reverse.
- Separate development/test/prod databases and least-privilege accounts. Production migrations run as privileged release step, API uses narrower runtime credentials. Define backup/restore and secret rotation in Lab 5 operations; no retention or uptime SLA invented at D1.

8.4 Docker/GCP and CI/CD future design only



- **Lab 5 target:** build a production image with Node 24, compile React assets, serve the same-origin site and Express API in a Cloud Run service, connect to a supported Cloud SQL PostgreSQL major over an authenticated/private connector, use Secret Manager and service identity. Confirm Cloud SQL major/extension support, SQL connection pooling and migration permissions before implementation.
- **CI plan:** on each PR run lockfile install, typecheck, lint, build, unit/API tests, and real-DB integration/concurrency suites with an ephemeral PostgreSQL service and migrations; record explicit status. Deploy only reviewed/accepted main releases; enforce branch protection and no secrets in PR logs.
- **Scale:** Cloud Run instances are stateless and may overlap; PostgreSQL unique/exclusion and serializable transaction invariants must hold **across instances**, not only one Node process. Session state remains in DB, never a single-process memory map. Cap DB pool size for serverless instance multiplication.
- **Current evidence:** no app image, deployed Cloud Run, Cloud SQL, GitHub Actions quality gate or successful execution was created by Issue #11. The course's supplied Lecture 11 material frames Docker/CI/CD/GCP as later Lab 5 work, not an assertion of delivery.

9. Requirements-to-design-to-verification traceability

Design responsibility	Requirement / feature IDs (authoritative in SRS and inventory)	Planned checks and downstream contract
Auth, session, role/owner boundaries	FR-001..004; BR-002/033; NFR-001/002/015; F-S1-01	VAL-001/002/034/036; F-S1-01 Feature SDS/STS, session expiry, CSRF, bad roles/ownership
Offered 60-minute Slot / Tutor ownership	FR-005..007; BR-003/004/006/029/035; F-S1-02	VAL-003/004/005; AC-S1-13; VC/TEST-S1-37; migration constraint and Race A
Availability and safe Customer projections	FR-008/009; NFR-007; F-S1-03	VAL-006/009/036; stale/empty/privacy/refresh UI and API
Exclusive Booking and time/concurrency	FR-010..013/018; BR-001/005/028/034; NFR-004/005/006/008; F-S1-04	VAL-007..011/014/030; AC-S1-25/26/35; VC/TEST-S1-25/26/39; Races B/C, T_elig, retry
Persisted private Booking and correct status	FR-014..017/019; BR-030; NFR-003/007; F-S1-05	VAL-012/013/031/036; restart, owner scope, Past not Completed
Cross-cutting system ownership	FR-018/019; BR-029/033/034; NFR-003..008/011/012/013/015; SYS-01	VAL-014/031/034/035/036; migrations, safe error, CI/GCP future plan
Customer cancellation	FR-020/021; BR-007/008; F-S2-01 **proposed**	VAL-015/016; Q05 remains independent after reopen
Cross-Tutor one-success reschedule	FR-022..027; BR-009..016; F-S2-02 **proposed**	VAL-017..020; same Booking ID, at most one history, atomic cross-Slot movement
Tutor cancellation and withdrawal	FR-028..030/033/034; BR-017..020/026/027/032; F-S2-03/05 **proposed**	VAL-021/022/025..027; no occupied withdrawal, reason and distinct ID
Private notice and cancellation reason	FR-031/032; BR-021..025; NFR-014; F-S2-04 **proposed**	VAL-023/024/036; read/reread, no age expiry, data minimization
Tutor-only summary	FR-035/036; BR-030/031; NFR-016; F-S3-01 **proposed**	VAL-028/029/036; consistent counts/lists, in-progress decision gate

Coverage boundary: All stakeholder decisions Q01–Q32 are represented through BR-001..032 with extra BR-033..035 for instructor/security/integrity, and FR-001..036 / NFR-001..016 remain controlled by SRS. The detailed ID-to-feature/VAL crosswalk is in traceability. This SDS adds design mappings and **does not** redefine or renumber any source requirement. VAL, AC and VC/TEST references above are **planned evidence**, not passed tests. Feature-specific exact test lists and acceptance results must be established in each Part E STS.

10. Outstanding gates, review checklist and next authorized work

Gate	Required disposition before the indicated step
OD-001/002/003	Decide Tutor operating hours/days, maximum horizon and start-minute grid **before F-S1-02 final contract**; do not fabricate restrictions.
OD-004	Approve availability display, safe Tutor labels, field allowlists, sorting/filtering and list behavior **before F-S1-03/05 API/UI contracts**.
OD-005	Peer-approve stack integration, proposed eight-hour Session TTL/CSRF policy, specific **current-only logout vs all-session credential/compromise revocation**, atomic rotation, precise API errors, DB lifecycle triggers, lock order, post-guard wait treatment and failure handling **before S1 code**.
OD-006	Classify in-progress appointments for Upcoming/Past summary without creating Completed state **before S3 summary contract**.
OD-007	Q32 already **prohibits** reactivation of withdrawn historical Slot; any exception needs *new* stakeholder authority, not a default.
OD-008/009	Privacy retention/deletion and summary count/list snapshot/pagination **before corresponding S2/S3 contracts**.
OD-010	Proposed keyboard/p95 thresholds and DB race stress sample sizes need approved methods/fixtures; not mandatory until approved.
OD-011/012	Final instructor Master document and formal Theme 1 eligibility **still unverified**, blocking official sign-off; this design may only be reviewed as a draft.
OD-013	Retry idempotency-key shape/window and unknown-commit reconciliation **before Booking/Reschedule Feature SDS/STS**.
OD-014/015	Meaningful Tutor reason validation and previous-Tutor history visibility **before corresponding S2 design contracts**.
Issue #12	Sprint 2/3 assignment and feasible capacity remain proposed pending peer-reviewed three-sprint plan.

Issue #11 review checklist (independent @Chxtamos review required):

1. Confirm the real repo has no implementation and that this PR changes **only** docs/system-sds.md.
2. Challenge proposed Node/React/Vite/Express/Prisma/Postgres versions and Prisma 7 adapter/migration details; verify reproducible future commands are clearly **not yet executable**.
3. Validate ERD cardinalities, no redundant writable Tutor schedule or occupied flag, immutable 60-minute Slot, FKs, conditional indexes/exclusions and S2 history/notice compatibility.
4. Confirm all three race invariants with DB-executed planned test proof, post-lock Q05 clock semantics, equality/adjoining windows, atomic rollback and bounded retry contract.
5. Confirm anonymous/Customer/Tutor role and **record** enforcement, cookie/token hygiene, CSRF protection, privacy-safe response allowlists and error discipline.
6. Confirm required Q07–Q32 later behavior is represented *without* prematurely exposing Sprint 2/3 endpoints, and OD-001..015 / Master/Theme blockers remain visible.
7. Check source links, consistent IDs, no fabricated test/deployment/approval claims and a substantive GitHub review. An AI self-check is not a teammate approval.

Exit / handoff: After a documentation-only diff check and this Issue's scoped review, open a PR from the Issue branch to sprint1, request @Chxtamos and **leave it open**. Only a real teammate's review and an explicitly authorized later merge can satisfy the full Issue DoD. Issue #12 can use this draft for feasibility but cannot claim it is already signed off. Part E should author **versioned feature contracts first**, then implement/test the S1 slice on their own Issue branches.

11. References and provenance

- Product authority: Whole-product SRS, Sprint 1 baseline, Feature inventory, Traceability, AGENTS.md, Issue #11.
- Instructor materials cited by the SRS: Group Work Specification v1, **marked draft**, issued 6 October 2026; Deliverable 1 v1 for 8 November 2026. **Authoritative final Master and Theme allocation not verified** (SRS OD-011/012).
- Version support: Node.js releases, Vite supported releases; npm registry version metadata examined 3 November 2026. These justify *proposed* targets, not installed project libraries.
- PostgreSQL 18 CREATE TABLE / exclusion constraints and range types.
- Prisma 7 transaction isolation and retry; implementation must validate compatibility with the **chosen** Prisma 7 adapter and API.
- No test execution, generated migration, package installation or cloud deployment was performed **as part of authoring this specification**.

Part D: Feasible Three-Sprint Delivery Plan

> **Status: PROPOSED — Issue #12 / Deliverable 1 Part D.** This plan is not a record of feature completion, executed tests, instructor authorization, or accepted release. The team must peer-review dates, capacity and Sprint 2/3 allocations. Product rules Q01–Q32 are controlled by SRS; Sprint 1 identities by requirements; assignments by Feature Inventory, traceability, and architecture by System SDS.

1. Document control, authority, scope

Item	Plan
Issue	#12 — D1-04 Part D
Owner / peer reviewer	@Chxtamos / @Tanaboonnnnn (independent human review)
Product / people	NutTiew one-to-one tutoring appointments; two students
Integration / release	Issue `docs/*` or `feature/*` branch → reviewed PR into `sprint1` → separate reviewed release PR into `main`
Mandatory Sprint 1 checkpoint	**D1: 8 November 2026, 23:59 Asia/Bangkok**
Later checkpoints stated by Issue #12	**D2: 29 November 2026** (Sprint 2 and evidence of Sprint 3 progress); **completion: 13 December 2026** (Sprint 3)
Source limitation	Reviewer-cited Group Work Specification v1, Deliverable 2 v1 and Final Delivery v1 inform these Lab, D2 and final checkpoints. The authoritative final Master and official Theme allocation remain unverified (OD-011/012).
Scope	Schedule, responsibilities, interdependencies, test/review outcomes, risks and fallback; **no app implementation, no simulated test pass, no direct edits to `main` / `sprint1`**

Baseline: At planning time `sprint1` contains `AGENTS.md`, `README.md`, `docs/requirements.md`, `docs/srs.md`, `docs/features.md`, `docs/traceability.md`, and `docs/system-sds.md`; no application scaffold is established. A documentation merge is not a working app or a passing test. The team must build a genuine UI → API → durable DB journey in Sprint 1. The concrete stack in the System SDS is **proposed**, not installed/approved: React/Vite, Express/TypeScript/Node, PostgreSQL/Prisma. The Part E feature contracts must validate integration feasibility before writing code.

2. Sprint goals and non-negotiable scope

Sprint	Product scope (stable IDs)	Objective accepted outcome	Planned verification / release
S1 — D1	**F-S1-01** Authentication; **F-S1-02** Tutor Slots; **F-S1-03** Availability; **F-S1-04** exclusive Booking; **F-S1-05** protected Booking/Tutor reads; **SYS-01** integrity, privacy, persistence and evidence	Real Customer/Tutor login, logout and current identity; backend role/record checks; future single 60-minute Tutor offers; customer discovery, eligible booking, durable own records; Q28 cross-Tutor overlap prevented; all three DB races and Bangkok clock boundary handled; UI/API/DB function end-to-end	VAL-001..014 (especially **Race A VAL-005, Race B VAL-010, Race C VAL-011**), VAL-030/031/034/036 as applicable; AC-S1-01..35 and VC/TEST-S1-01..39 (real PASS/FAIL/SKIP recorded only upon execution); review/merge PR `sprint1` → `main`; D1 evidence and submission
S2 - proposed D2 with Lab 5-7	**F-S2-01** cancellation/reopen, **F-S2-02** single reschedule/history, **F-S2-03** Tutor cancellation, **F-S2-04** private notice, **F-S2-05** Slot withdrawal, and SYS-01 regression	Q07-Q27/Q32, ownership and atomic persistence must work in tested candidate	S2 VAL-015..027 and cross-operation race evidence, peer-reviewed SDS/STS before code; Lab 5-7 evidence; D2 integrated reviewed main/GCP release also includes one implemented/tested S3 workflow

Sprint	Product scope (stable IDs)	Objective accepted outcome	Planned verification / release
S3 - D2 tested increment and mandatory Final completion	**F-S3-01** Tutor-authorized summary: D2 minimum Past Booked count/list, then **Upcoming Booked and Cancelled counts + matching lists** mandatory for Final	By 25 Nov test Past Booked in UI/API/DB; by **30 Nov** complete all three owner-filtered categories with consistent count/list, approved OD-006 in-progress rule, independent review and deployed regression; Final 13 Dec acceptance	Real DB **VAL-028**, **VAL-029**, **VAL-036** fixtures, unauthorized-role/foreign-Tutor negatives, versioned PR/SHA and CI/GCP deployment; no category considered optional polish

Mandatory S1 boundary cannot be traded away: real credentials/session; own roles and record-level checks; exactly 60-minute Slot publishing; Customer availability; Booking with 60-minute cutoff and exclusive real-DB guarantees; Customer/Tutor private reads; persistence across refresh/restart; denied invalid/past/occupied/overlap requests; authenticated end-to-end UI/API/DB; negative and synchronized real-DB tests for Race A/B/C; accepted release on `main`. Demo-only mocks, memory-only records, UI-only blocking, a PR without a release, or tests described but not run do **not** satisfy this scope.

3. Feature dependencies, owners and verification handoffs

Ownership plan: Owner is accountable for getting a change reviewed and verifiable, not sole author of all code. @Chxtamos leads Part D and existing Issues #13/#15/#17; @Tanaboonnnnn leads existing Issues #14/#16/#18. The team coordinates shared architecture and migrations; no Issue assignment changes are made by this plan. These are **planning assignments requiring team agreement**, not a reassignment of existing GitHub Issues. The author of each PR must have the **other human teammate** review. Any actual feature-Issue owner/reviewer already assigned by GitHub prevails until explicitly updated.

Work package	Depends on	Proposed accountable driver / independent reviewer	Exit evidence (not yet executed)
A / B / C baseline	Issue #9 SRS, #10 Feature Inventory, #11 System SDS	Issue-specific existing assignees; both check consistency	Document references and reviewed PRs integrated in `sprint1`; technical choices and OD gates still identified
Shared project scaffold + database (SYS-01)	reviewed SD-01..12 / OD-005; environment and schema/migration decisions	@Tanaboonnnnn / @Chxtamos	Runnable reproducible local install, migrations/seed and data persistence; secrets excluded
F-S1-01 (#13)	System SDS; own **Feature SDS + UI/API/data contracts + STS reviewed before code**	@Chxtamos / @Tanaboonnnnn (existing Issue owner/reviewer)	VAL-001/002/034/036; actual login/logout/role/session tests
F-S1-02 (#14)	F-S1-01; same Part E contract gate; DB exclusion constraint strategy	@Tanaboonnnnn / @Chxtamos (existing Issue owner/reviewer)	VAL-003..005; real-DB overlapping offer Race A and negative cases
F-S1-03 (#15)	F-S1-02, authenticated API; OD-004 safe availability fields	@Chxtamos / @Tanaboonnnnn (existing Issue owner/reviewer)	VAL-006/009/036; stale/too-soon/occupied privacy-safe responses
F-S1-04 (#16)	F-S1-01/02/03; DB uniqueness/exclusion/trigger and OD-005/013 guards	@Tanaboonnnnn / @Chxtamos (existing Issue owner/reviewer)	VAL-007..011/014/030; Race B/C, Q05 wait crossing and rollback/duplicate safety
F-S1-05 (#17)	F-S1-01/04; approved response allowlist	@Chxtamos / @Tanaboonnnnn (existing Issue owner/reviewer)	VAL-012/013/031/036; own Customer/Tutor reads and persistence
F-S2-01/02	F-S1-04/05; S2 Part E contracts and Q07–Q16; OD-013	Team to assign/peer review before S2	VAL-015..020, 030; correct cancelled/reopened or one-history reschedule state
F-S2-03/04/05	F-S1-02/04; cancellation/notice/withdrawal integrity; OD-008/014/015	Team to assign/peer review before S2	VAL-021..027, 036 and Booking-vs-withdraw / reschedule-vs-Tutor-cancel race

Work package	Depends on	Proposed accountable driver / independent reviewer	Exit evidence (not yet executed)
F-S3-01 - full Final scope	F-S1-05, Booked/Cancelled persisted records and S2 lifecycle; OD-006 in-progress classification and OD-009 query snapshot gate	@Chxtamos implementation/test lead; @Tanaboonnnnn independent reviewer (planning assignment)	Past Booked implemented/tested by 25 Nov for D2; prepare reviewed SDS/STS and fixtures for **Upcoming Booked and Cancelled** by 25 Nov, implement/test them on **30 Nov** (counts + matching lists), VAL-028/029/036, regression, PR integration and GCP redeploy before exams; full scope mandatory for Final

Contract-before-code sequence for each Feature: source SRS FR/BR/NFR + accepted AC/Q → versioned Feature SDS (`specification.md` , applicable `ui-spec.md` / `api-spec.md` , data/transaction contract) → STS (`tests.md` , named failing/negative cases, evidence criteria) → *independent PR review* → feature code + tests → actual build/unit/API/DB/E2E results → reviewed feature PR to `sprint1` . A stub file or unreviewed spec does not open the implementation gate. SYS-01's migration/locking/Q05 design requires independent real PostgreSQL proof before Booking claims completion.

4. Sequence, checkpoints and integration strategy

Dates below are **planning allocations**, not claims that tasks were performed or promises of completion. Work is highly compressed: use checkpoints as stop/go decisions and escalate capacity risk, not as permission to skip mandatory acceptance.

Date/window (ICT)	Planned step	Exit / escalation
3 Nov	Review merged Parts A/B/C, approve feasibility/pending OD-005, author/review Part D; list per-feature Issues/contracts; test local Node/Postgres compatibility	Traceability/stack gaps owned; if local DB unavailable, escalate immediately rather than marking setup done
4 Nov	Review System SDS security/schema details and Part E specs/STS for F-S1-01 and F-S1-02; prepare scaffold/migration/seed, auth/slot test fixtures	Contracts reviewed **before code**, migration applies on clean local test DB, no secrets
5 Nov	Complete auth and Tutor Slot vertical slice; real DB and Race A; prepare F-S1-03/04 contract reviews	Negative role/overlap tests with actual committed state; ownership and Q05 blocking issues escalated
6 Nov	Availability + Booking implementation, Q05 guard, Race B/C, fail/duplicate handling, protected reads contract/code	Real DB barriers and persisted rows, no second success; any unproven constraint is a release blocker
7 Nov	End-to-end UI/API/DB journeys, security/private read checks, restart durability, real test evidence; review feature PRs then integrate `sprint1`	All S1 AC/VC mapped to observed pass/fail/skip; blockers fixed, never silently waived
8 Nov before 23:59	Issue #18 (@Tanaboonnnnn owner / @Chxtamos reviewer) independent integration review, separately reviewed `sprint1 → main` release, versioned evidence/submission package: whole-product Parts A–D, per-feature Part E SDS/STS, Part F source/test logs and release SHA, and required D1 PDF/report per authoritative course instructions	Only claim D1 acceptance if main contains the working slice and actual tests/review/release evidence; instructor/Theme gates remain explicit
9-11 Nov - S2 and Lab 5 setup	Both review S2 feature SDS/STS before code. @Tanaboonnnnn leads Docker/GCP/Cloud SQL/secrets; @Chxtamos leads CI and S1 regression planning	Reviewed contracts, migration/seed setup and CI plan; reserve **at least 50% of combined planned capacity** for Lab 5 to 15 Nov; available hours remain unknown
12-15 Nov - Lab 5 due 15 Nov	Both: Docker, CI/CD, GCP Cloud Run with durable Cloud SQL; @Tanaboonnnnn deployment driver and @Chxtamos test/peer-review driver	Version/commit/image SHA, CI run URL/status, GCP URL/version, secret checks, post-deploy auth/Booking and DB restart persistence evidence. Lab 5 due 15 Nov, after D1 but mandatory for D2

Date/window (ICT)	Planned step	Exit / escalation
16-19 Nov - S2 and Lab 6 setup	Both implement reviewed F-S2-01..05 in dependency order; @Chxtamos prepares cross-team UAT scenarios/defect log and @Tanaboonnnnn deployed candidate/fixes	S2 VAL-015..027/DB race and rollback proof as ready; reserve **at least one-third of combined planned capacity** for Lab 6/UAT/retests
20-22 Nov - Lab 6 due 22 Nov	Both run cross-team UAT; @Chxtamos coordinates scenario/evidence/defect log, @Tanaboonnnnn triages repairs and re-tests	Dated UAT outcomes, defect IDs/severity/owners, linked repair PRs and re-test results; feed residual defects into D2 regression
23-25 Nov - D2 S3 working increment plus full-category preparation	@Chxtamos obtains independent peer review of the **full F-S3-01 SDS/STS before code**, covering all three category predicates, same-time snapshot, OD-006 and fixtures. Implement real Tutor-only DB-backed Past Booked count/list via UI/API and prepare shared count/list query/UI and tests for Upcoming and Cancelled. @Tanaboonnnnn reviews	By 25 Nov Past Booked **implemented and tested** for D2, with real DB VAL-028/029/036 and source SHA; complete category test specification and prerequisites for mandatory Upcoming+Cancelled by 30 Nov. Prototype-only does not satisfy D2
26-28 Nov - Lab 7 and D2 candidate	@Tanaboonnnnn leads Lab 7 TA-approved change request, impact and updated reviewed contract, red-green regression, CI/GCP redeploy, five date-stamped sourced metrics and short retrospective; @Chxtamos leads S1/S2/S3 regression and D2 PDF; both review integration/release PR and UAT fixes	By 28 Nov tagged main/GCP candidate with CI run/status, SHA, URL/version, DB durability and S1/S2 + tested S3 Past Booked, plus Lab 7 evidence: TA approval reference, contract impact, red/green test logs, redeploy proof, five dated sourced measurements with units, short retro. Missing approvals/evidence = BLOCKED
29 Nov - D2 and Lab 7 due	Both verify main/GCP candidate and submit D2 self-contained PDF via LEB2; @Tanaboonnnnn checks release and Lab 7 change/metrics/retro, @Chxtamos PDF/evidence/receipt	Integrated main/GCP candidate with tested S1/S2 and **at least one tested S3 Past Booked workflow**, CI/post-deploy persistence and regression, UAT retests and Lab 5-7 evidence. Missing gate = NOT MET; Upcoming+Cancelled remain **mandatory Final**, scheduled 30 Nov
30 Nov - mandatory F-S3-01 completion and release freeze	@Chxtamos implements **Upcoming Booked and Cancelled counts + matching lists** in authorized UI/API/DB based on already-reviewed full F-S3-01 SDS/STS and reusable fixtures; @Tanaboonnnnn independently reviews actual tests/PR. Both integrate into sprint1, independently review sprint1-to-main release, run regression and CI/GCP redeploy; draft final PDF/video script	**By 30 Nov** all **three mandatory FR-035 categories** implemented/tested and deployed: Upcoming/Past/Cancelled fixtures, count=list matching owner/time snapshot, wrong-role/foreign-Tutor denials, approved OD-006 classification/no Completed state, VAL-028/029/036 real DB results, peer review, SHA/CI/GCP and post-deploy persistence/regression. **Critical one-day capacity risk**: reusable query/UI and all-category contract/tests ready by 25 Nov; escalate if infeasible rather than silently defer
1-11 Dec - final exams and release monitoring only	Protect exam time: no new core S3 implementation. @Chxtamos final PDF/video assembly, @Tanaboonnnnn security/release checks and low-risk regression	Full three-category Summary already implemented, tested, reviewed and deployed by 30 Nov; remaining mandatory gaps or unresolved OD-006 explicitly block Final rather than become optional refinements
12-13 Dec - final due 13 Dec	Both verify deployed accepted release with **Upcoming Booked, Past Booked, Cancelled counts and matching lists**, replay real-DB VAL-028/029/036, authorization and time classifications, submit final PDF plus demo video with both members explaining	Full S3 scope, security/regression, release SHA, CI/GCP redeploy proof and submission receipt. A missing mandatory category means Final NOT MET, not last-day implementation

Integration cadence: no feature PR before its reviewed contracts; PRs are independently reviewed (no self-approval), and only accepted feature branches land on `sprint1`. After final S1 verification, open a separate `sprint1 → main` release PR and review it. Avoid simultaneous merge conflicts by naming file/feature ownership and sequencing schema migrations before dependent services. GitHub Project #6 advances `Backlog → Specified → Started → PR Review → Fixing → Done` based on verified work, not calendar dates.

5. Definition of Done and objective evidence

Gate	S1 acceptance and measured evidence	Outcome if absent
Source/contract	Part A/B/C references and applicable **peer-reviewed Part E SDS/STS before implementation**, including stable IDs	Contract/implementation gate **BLOCKED**
Working path	Customer/Tutor UI → authenticated API → durable relational DB, real login/logout/current identity and persisted own records after restart	**NOT DONE** if mock/in-memory/UI-only
Domain	Exactly 60-minute Slots, no same-Tutor active overlap; valid/invalid/occupied/too-soon cutoffs; one Active Booking per Slot; no cross-Tutor Customer overlap	**NOT DONE** if invariant unproven
Security	Backend role + record authorization, sanitized error/data allowlists, credentials hashed, no secrets committed	**NOT DONE** on unauthorized read/write
Race / rollback	Synchronized *real DB* Races A/B/C, Q05 post-wait boundary, rejected writes and injected rollback, final committed rows inspected	**NOT DONE** if HTTP-only/mock-only
Tests	VAL-001..014 and S1-applicable VAL-030/031/034/036; AC-S1-01..35 / VC/TEST-S1-01..39 have actual command, setup, SHA and PASS/FAIL/SKIP records	No manufactured PASS; unresolved required FAIL/NOT RUN blocks release
Integration / delivery	Peer-reviewed feature PRs into `sprint1`; independently reviewed `sprint1 → main` PR; release SHA and D1 artifact/evidence by deadline	Staging/PR alone **does not count as accepted release**

D2 acceptance gate (after D1): Peer-reviewed Part E SDS/STS must precede all S2/S3 code. By 29 Nov produce tested S1+S2 AND at least one **implemented/tested S3** rule in a reviewed integrated **main** release candidate deployed on **GCP via Lab 5 CI/CD**. Record version/tag, SHA, CI run/status, GCP URL/version, post-deploy DB persistence, S1/S2/S3 regression and authorization/negative tests. Lab 6 UAT defects/re-tests and Lab 7 maintenance/metrics feed this evidence. Submit a self-contained **D2 PDF via LEB2**. By 13 Dec provide final PDF and demo video with BOTH students explaining; reserve exams 1-11 Dec. Do not claim these plans are executed tests or deployments; not required at D1. **Separate Final F-S3-01 gate:** D2 Past Booked is only a minimum slice. Complete Upcoming Booked and Cancelled plus Past Booked counts/lists, approved OD-006, real-DB VAL-028/029/036, review, integration, CI/GCP redeploy by 30 Nov; Final 13 Dec requires full summary, not optional enhancements.

6. Capacity, risks, gates and controlled fallback

Two-person feasibility: D1 leaves roughly five calendar days from 3 Nov; after D1, Lab 5/6/7 overlap with mandatory S2 and at least one tested S3 rule before D2. Allocate approximately half of combined planned capacity to Lab 5 through 15 Nov, one-third to Lab 6 through 22 Nov, prioritize Lab 7/main/GCP/PDF on 26-29 Nov and prepare final artifacts before 1-11 Dec exams. Actual student hours are UNKNOWN; these fractions are planning safeguards, not capacity proof. Both students peer-review opposite authored PRs. If core correctness/review/release gates fail, explicitly report NOT MET instead of silently reducing mandatory scope. The 30 Nov full-S3 completion/review/deploy is a one-day critical-path risk; prepare approved contracts, reusable UI/API/DB and fixtures by 25 Nov, record remaining work and escalate lack of capacity instead of claiming acceptance.

Risk / open gate	Owner / checkpoint	Containment and defensible fallback
OD-011/012: final instructor Master and formal Theme eligibility unverified	@Tanaboonnnnn obtains instructor/TA evidence; @Chxtamos compares before formal D1 sign-off	Present drafts as provisional, record source/version/delta; **do not** assert instructor sign-off
OD-005: proposed stack, session, DB triggers/locking and Q05 post-contention proof not yet accepted/executed	@Tanaboonnnnn design lead + @Chxtamos review, before Part E implementation	Probe real DB early; choose reviewed simpler implementation only if it still proves Race A/B/C and Q05; **never** remove DB integrity checks
OD-001..004: Tutor hours/horizon/grid, availability safe fields unresolved	Product decision owner/team before F-S1-02/03 final contracts	Implement only confirmed constraints; defer optional filtering and non-approved input restrictions, not mandatory Slot/browse outcomes

Risk / open gate	Owner / checkpoint	Containment and defensible fallback
OD-013: duplicate/unknown-response Booking retry contract	F-S1-04 owner with peer review before Booking SDS/STS sign-off	Preserve at-most-one and reconcile with authorized own reads while token/replay protocol remains explicitly proposed; don't claim exactly-once response
OD-006/008/009/014/015: in-progress summary, retention, reporting snapshot, meaningful reason, prior-Tutor privacy	Relevant S2/S3 feature owner and stakeholder/team before affected contract	Mark blocking decisions and do not invent product rules; no unauthorized historical visibility
Missing local DB/toolchain or incompatible Postgres extension/ORM migration	Shared SYS-01 owner, setup checkpoint	Prioritize reproducible dev DB and constraints, temporarily replace an unworkable ORM detail **via reviewed design delta**, not with memory-only or unguarded writes
Late failed Race A/B/C, Q05, auth, persistence, or release checks	Both people, S1 test/release gates	Stop false acceptance, record FAIL/NOT RUN and remediate smallest cause; escalate deadline impact. **Do not quietly cut mandatory S1 feature**
Reviewer unavailable/merge queue and limited two-person capacity	Each PR author requests other teammate promptly; daily check-in	Keep PRs small, parallelize independent reviewable contracts, capture unresolved blockers; no self-approve or skip release PR
Missing Lab 5/6/7 evidence, UAT re-tests, or D2 main/GCP candidate	@Tanaboonnnnn deployment and Lab 5/7, @Chxtamos Lab 6/UAT/regression	Reserve Lab capacity and capture exact failed gates; no false D2 deployment or passed-test claims
Missing full FR-035 category set or exam capacity	@Chxtamos implementation/tests, @Tanaboonnnnn independent reviewer/redeploy; all categories due 30 Nov	Deliver Past Booked for D2 25 Nov, prepare full SDS/STS/tests beforehand; Upcoming+Cancelled implementation, regression, review and GCP redeploy on 30 Nov. Never call mandatory categories optional polish
Optional UI polish, benchmarks/accessibility thresholds not approved	Owner TBD / OD-010	Defer nonessential polish and unapproved thresholds after mandatory S1 correctness and tested end-to-end flow; document what remains

Fallback hierarchy: (1) resolve source/data/security blockers; (2) protect mandatory S1 real UI/API/durable DB and Race A/B/C; (3) protect D2 tested S2 features, one implemented/tested S3 rule, Lab 5-7 and deployed main/GCP candidate; (4) simplify ONLY optional UI polish/refinements with peer/team approval; (5) if a required checkpoint fails, state NOT MET and preserve failing evidence rather than asserting acceptance. **FR-035 Upcoming/Past/Cancelled are three mandatory Final categories and cannot be deferred as optional UI polish.**

7. Coverage and approval checklist

- **S1 features:** F-S1-01..05 + SYS-01, all mandatory Sprint 1 acceptance/test IDs `AC-S1-01..35`, `VC/TEST-S1-01..39`, Q01/Q02/Q03/Q04/Q05/Q06/Q28/Q29/Q30 as applicable; planned VAL-001..014, VAL-030/031/034/036.
- **S2 proposed:** F-S2-01..05 and VAL-015..027, VAL-030/036; S1 authorization/time/atomicity regression; Q07..Q27/Q32 and relevant privacy rules.
- **S3 proposed: F-S3-01 Past Booked** implemented/tested for D2 minimum by 25 Nov, then **Upcoming Booked and Cancelled counts + matching lists** implemented/tested, peer-reviewed, regression checked and deployed by **30 Nov**, before exam period. Real DB **VAL-028**, **VAL-029**, **VAL-036**, Tutor ownership, consistent state/time and OD-006 approval gate; all three mandatory for Final.
- **System NFR:** NFR-001..008/011/012/014/015/016 mapped to sprint features; NFR-009/010 proposed OD-010 thresholds, not approved. NFR-013 requires **Lab 5 Docker/CI/GCP by 15 Nov** and a **main/GCP deployed candidate by D2 29 Nov**, NOT by D1; Lab 6 UAT/retests due 22 Nov and Lab 7 maintenance/metrics due 29 Nov. D2 PDF via LEB2, final PDF and both-student demo video due 13 Dec according to reviewer-cited v1 handouts.
- **Explicit remaining gates:** OD-001..015 tracked in SRS; Issue #12 planning review does not close them automatically, does not approve new product rules and does not certify instructor/Theme eligibility.
- **Approval:** The two students must review and accept this allocation as a plan. @Tanaboonnnnn supplies substantive peer review on the Issue #12 PR into `sprint1`; @Chxtamos cannot self-approve or self-merge. The actual accepted D1 code/release and test evidence belong to later feature/integration Issues, not this Part D document.

8. References

- Issue #12 — Part D plan
- Whole-product SRS, especially §§3, 10, 13, 15
- Sprint 1 requirements and AC/VC IDs
- Feature inventory and cross-sprint traceability
- System SDS, especially §§1, 3, 5, 7–10
- AGENTS.md: local issue branch, review, validation and acceptance gates

PR #24 source provenance: This revision follows reviewer citations of group_work_specs_v1.pdf section 4, project_deliverable_2_v1.pdf sections 2/3.3/3.5, and project_final_delivery_v1.pdf. The complete handouts were not independently opened for this edit; final instructor Master and Theme sign-off remain open (OD-011/012).

Final F-S3-01 acceptance boundary: The D2 Past Booked count/list is only a partial milestone. Final demands DB-derived owner-filtered counts **and corresponding lists** for Upcoming Booked, Past Booked and Cancelled. Complete the remaining two on 30 Nov, test all three with VAL-028/029/036 against real DB fixtures and wrong-role/foreign-record negatives, uphold same time/visibility snapshot and explicit OD-006 policy; peer-reviewed feature PR to sprint1, reviewed release to main, CI/GCP post-deploy regression. If missing, Final is NOT MET.

Lab 7 v1 proof obligation (per second review): TA-approved change request with source/approval reference -> impact analysis and reviewed updated contract -> failing-then-passing regression evidence -> CI/CD redeploy with runtime SHA/URL -> **five dated metrics** with units and sources -> brief team retrospective. No fabricated TA approval or metrics.

Part E: Sprint 1 Feature Specifications & Software Test Specifications

> **Completeness Notice:** Every reviewed and accepted Sprint 1 Part E contract (`specification.md` , `ui-spec.md` , `ui-decision-2026-11-05.md` , `api-spec.md` , `data-spec.md` , `transaction-spec.md` , and `tests.md`) is embedded **verbatim in full** below so this PDF stands alone without external repository links.

E.1 F-S1-01: Authentication, Sessions & Rate Limiting

E.1.1 F-S1-01 Authentication Feature SDS (specification.md)

> **Part E contract; not implemented or runtime-tested.** Issue #13, owner @Chxtamos, reviewer @Tanaboonnnnn. Decision approval from PR #25 Conversation #6624817784: **OD-005-A/B/C/E and Auth UI direction APPROVED; OD-005-D APPROVED (PR #25 reviewer comment #6625521193)** reviewed and accepted in PR #25 comment #6625521193. The reviewed and approved Part E PR #25 was merged into `sprint1` before Part F implementation. Instructor-final Master/Theme gates remain separate.

Authority and scope

Follow Sprint 1 requirements FR-S1-01..04, AC-S1-01..07, BR-S1-04/16/17/20, NFR-S1-01/02/07/08 and VC-01..07/31/35; whole-product SRS FR-001..004, BR-002/033, VAL-001/002/034/036; System SDS SD-09 and section 5; Feature Inventory; three-sprint plan. Tutor and Staff are one business role. This feature owns **login, current trusted identity, logout, sessions, and reusable server access guards**; Slot/Booking persistence and actual row-level proofs depend on Issues #14–17.

Proposed architecture and decisions (OD-005)

Single-origin React/Vite client and Express/TypeScript backend with durable PostgreSQL User and Session records (Prisma adapter/SQL migration details to verify before code). Credentials are Argon2id salted hashes. Valid login obtains 32 random CSPRNG bytes; the opaque cookie contains the raw token while the database stores **only** a unique SHA-256 token digest. Set cookie **after** Session insert commit. Client role, claimed owner IDs, headers or query parameters never grant authority.

Topic	Proposed Part F behavior	Review decision
Cookie	`nuttiew_session` ; `HttpOnly; Secure; SameSite=Lax; Path=/` ; no Domain; localhost HTTP only uses `nuttiew_session_dev` with Secure=false	Production HTTPS, proxy setup, allowed origins
Lifetime	**Eight-hour absolute** lifetime from DB creation; no rolling renewal; check `expiresAt > serverNow` , `revokedAt IS NULL` , active User and role on every request	**OD-005-A APPROVED as decision; implementation/tests NOT RUN** (PR #25 comment #6624817784)
Login	Existing Customer/Tutor accounts only, normalized unique email, generic error for unknown/wrong/inactive, no new cookie on failure; separate logins create independent sessions	Hash cost, rate limiting, input bounds, seed policy
Logout	CSRF-protected POST; atomically revoke **presented session only**; send 204 and expire cookie **after DB commit**; replay old cookie denied; independent other session stays valid	**OD-005-B APPROVED as decision; implementation/tests NOT RUN**
CSRF	Approved Session-bound HMAC token with same-origin Origin/Referer rules and read-only token bootstrap; no cross-origin credentialed CORS; matched on unsafe requests	**OD-005-C APPROVED as decision; implementation/tests NOT RUN**
Error/privacy	Uniform invalid-login result; no email enumeration, raw secrets, stack/SQL error, foreign resource details; no session token in JSON/localStorage/logs	Exact status/code response schema in API contract

Topic	Proposed Part F behavior	Review decision
Future	Password reset, role switching, rotation/revoke-all administration and S2/S3 operations not implemented	Separate approved feature contract

If Session insert or logout revocation fails, **do not claim success**. An expired/revoked token is never implicitly refreshed. Each protected request loads a live Session and active User from persistent DB; after restart the same valid cookie continues until actual expiry. Session identity must derive from the database, not the client.

Server-side permission boundary

Action	Anonymous	Customer	Tutor	Record-level predicate
Login	Credential exchange	Allowed	Allowed	Valid existing active account
Own identity/logout	401	Own session	Own session	Valid unexpired/unrevoked Session joined to active User
Tutor Slot create/read	401	403	Own schedule only	Create `tutorId` from principal; read SQL where `Slot.tutorId = principal.id`
Customer Booking create/read	401	Own only	403/general reads denied	`customerId = principal.id` from session, not request
Tutor Booking/schedule read	401	403	Own only	Derive Tutor from persisted Slot and filter by principal
Foreign record	401	No protected data	No protected data	Privacy-safe 404 for foreign/missing ID

`requireSession`, `requireRole`, `requireOwnedResource` and owner-filtered DB queries are interface intentions; their actual domain integrations require Issues #14–17. Hidden buttons, forged `role`, `customerId` or `tutorId` cannot replace backend checks. Avoid existence leaks via foreign-record responses.

Part E → Part F gate

OD-005 decision sheet — A/B/C/D/E approved decisions; Part F tests tracked separately

The explicit human decision is recorded in PR #25 Conversation comment **#6624817784**: **OD-005-A/B/C/E and Auth UI direction APPROVED**, **OD-005-D APPROVED (PR #25 reviewer comment #6625521193)**. The updated in-flight reservation tradeoff was expressly approved by @Tanaboonnnnn in PR #25 comment #6625521193. **Approval of an engineering decision is not proof of a passing implementation**; Part E was independently reviewed and merged into `sprint1`; Part F acceptance and tests remain separately reviewable. Instructor-final Master/Theme gates remain separate. Detailed protocols: API, Data, STS.

Decision	Concrete proposed value/behavior	Approver / gate
OD-005-A Session TTL	Absolute **28,800 seconds (8 hours)**, measured from persisted `createdAt`, no sliding refresh; strict `expiresAt > DB/server now`, `revokedAt IS NULL` and `User.isActive` for every protected call; cookie Max-Age **<= 28800**	**APPROVED decision** by @Tanaboonnnnn, comment #6624817784; Part F tests NOT RUN
OD-005-B Logout	**Current presented Session only**, DB revocation commit **before** 204/clear-cookie, captured-cookie replay 401; another Session B remains usable; DB write fault must not be reported as success	**APPROVED decision** by @Tanaboonnnnn, comment #6624817784; Part F tests NOT RUN
OD-005-C CSRF and origin	**Session-bound HMAC CSRF (approved in PR Conversation comment 6624817784):** create and persist a nonsecret 32-byte `csrfNonce` once at login; derive a stable MAC from managed server HMAC key + Session ID + nonce using the canonical expression **outside this GFM table** in API/Data. Repeated read-only `GET /auth/csrf` returns the same MAC without DB mutations; multiple tabs work. Unsafe methods constant-time verify against the live Session, and A's token fails for B; logout/expiry revokes it. No per-GET rotation. Same-origin Origin/Referer fallback and cross-site safeguards remain as documented; managed versioned keys require secure overlap and retirement	**APPROVED by @Tanaboonnnnn in PR Conversation comment 6624817784**; implement/test in Part F, AUTH-13/17/18/21

Decision	Concrete proposed value/behavior	Approver / gate
OD-005-D Rate limit	**Reviewer-requested revision, APPROVED design in PR #25 reviewer comment #6625521193:** 15-minute PostgreSQL-backed HMAC-keyed account and trusted-IP buckets. Per account allow **5 failed credential attempts/15min** (successful logins do not increase failure count); per trusted IP allow **20 total login attempts/15min** (success and failure both counted). Before Argon2, reserve one IP attempt and one **account in-flight permit** atomically under row locks, but do **not** increment account failure count until the credential result is known. On failed credential validation commit one account failure and release its permit; on successful validation release the permit **without incrementing failures**. Enforce the account failure quota against **failed + in-flight reservations** so simultaneous attempts across instances cannot bypass five. Do not hold DB row locks during Argon2. A rejected attempt returns generic 429 and never creates a Session. Fixed UTC 15-minute windows, managed HMAC key, trusted proxy allowlist, fail-closed DB errors and nonenumerating responses remain required. See Data contract for transaction boundaries, window rollover and crash-recovery policy	@Tanaboonnnnn requested **5 failed-only/account + 20 total/IP** in PR #25 Conversation comment 6624817784; **OD-005-D APPROVED (PR #25 reviewer comment #6625521193)**, revised atomic permit protocol and all-valid saturation tradeoff expressly approved in PR #25 comment #6625521193; implementation verification tracked in AUTH-20/22
OD-005-E Input bounds	JSON request body **<= 4 KiB**; email **3–254 Unicode code points** after trimming, normalized lowercase and valid email syntax; password **1–128 UTF-8 bytes**, no trimming or Unicode normalization of secret; strict only `email` and `password` keys; malformed/oversized → 400/413 and no DB Session insert. Avoid timing-dependent account enumeration (dummy Argon2 verification for unknown account)	**APPROVED decision** by @Tanaboonnnnn, comment #6624817784; Part F tests NOT RUN

Pre-code acceptance record (PR #25 Conversation comment 6624817784, 4 Nov 2026): @Chxtamos = proposal owner; @Tanaboonnnnn confirmed **OD-005-A (8h absolute TTL), B (current-session logout), C (HMAC CSRF/Origin), E (input bounds) and Auth UI direction APPROVED** in that comment, and **OD-005-D APPROVED (PR #25 reviewer comment #6625521193)** (failed-only account threshold, total trusted-IP threshold). This document reflects those stated decisions, not an implementation/test pass. **The revised OD-005-D permit protocol was expressly approved by @Tanaboonnnnn in PR #25 comment #6625521193 and Part E was merged into `sprint1`.** This approval is a design decision, not Part F verification or a finding that AUTH-01..23 all pass. Source baseline remains SD-09, FR-S1-01..04 and OD-005. If browser/CI clients lack acceptable Origin/Referer for unsafe requests, reject rather than bypass security.

Contract links: API, UI, persistent data, STS/tests. Human teammate must approve session TTL, CSRF, logout scope, rate limits, response schemas and the documents before coding. Later Part F work starts on a fresh branch from reviewed `sprint1`, first records a failing test where feasible, implements genuine UI→API→PostgreSQL, and publishes real test logs and a separately reviewed code PR. This document has **NOT RUN** status for every app/DB test and does **not** close Issue #13.

> **Approval-state update (2026-11-04):** @Tanaboonnnnn expressly approved OD-005-D, including the temporary sixth-correct-login 429 tradeoff, in PR #25 comment #6625521193, with follow-up review confirmation in PR #25 review #5765005104. The approved Part E contract was merged into sprint1. This corrects historical pending-status text only; implementation and the AUTH-01..23 tests are independently tracked in Part F verification ledger.

E.1.2 F-S1-01 Authentication UI Specification (ui-spec.md)

> Part E UX **contract**, not implemented UI or test evidence. **Auth UI design direction APPROVED** by @Tanaboonnnnn in PR #25 Conversation comment #6624817784; implementation, browser review and AUTH-15/23 remain NOT RUN. OD-005-D approved in PR #25 reviewer comment #6625521193; Part E merged into sprint1; Part F verification remains separate. See SDS, API, STS. Role navigation does not authorize an API request.

UI state	Action	Observable expected behavior	Trusted server interaction
Signed out	Open app	Labeled email/password form, no protected data or role selector; login is permitted	`GET /api/v1/auth/me` missing cookie → 401; show login
Pending	Submit credentials	Disable duplicate submit while pending, visible status, do not print password	`POST /auth/login` (same origin)

UI state	Action	Observable expected behavior	Trusted server interaction
Customer success	Valid Customer login	Show server-issued **own account reference plus effective Customer role**, not just generic "Customer". Proposal: "บัญชี: ...a1b2c3d4 · Customer", where suffix is last 8 hex characters of authenticated UUID (short label, not unique security identifier)	`GET /auth/me` derives `id` and `role` from DB Session/User, never client input or stale state
Tutor success	Valid Tutor login	Show **own account reference plus Tutor role** and own schedule navigation. After switching account, no previous C1/T1 identity persists	Same trusted `/auth/me` response, domain authorization still enforced by API
Bad credentials	Unknown/wrong/inactive account	Generic identical error, no account enumeration, no protected data	401 `LOGIN_FAILED`
Bad form	Blank/invalid inputs	Keyboard-accessible field errors; server revalidates	400 `INVALID_INPUT`
Reload	Active/expired Session	Loading until current-user response; active survives refresh; revoked/expired loses identity	`GET /auth/me` then authorized or signed out
Logout	Authenticated principal clicks logout	Obtain in-memory CSRF token, post logout; clear UI identity after **confirmed 204 only**. On network/DB error show uncertainty and retry rather than falsely claim revocation	`GET /auth/csrf` → `POST /auth/logout`
Forged navigation	Customer attempts Tutor URL or Tutor A accesses Tutor B ID	UI access-denied state with no private data; backend still applies 403/404	Role and DB owner checks in later Issues #14–17

Interaction and security requirements

Use native `<label>` elements for email/password, `autocomplete=email` and `current-password`, visible focus and keyboard Tab/Enter flow, live status/error announcement, no user-selectable role, no credential or raw session storage in URLs, localStorage, analytics or logs. Because the cookie is HttpOnly, JavaScript never reads bearer secret; CSRF value is memory-only and cleared after sign-out. Avoid showing cached protected identity after 401 or after a stale request finishes late. If logout commit fails, do not present successful sign-out based solely on clearing frontend state.

Future UI evidence (Part F only)

Record real browser/API/DB verification of Customer and Tutor login, wrong password, trusted role on refresh, logout and replay denial, keyboard navigation and forged access attempts. Screenshots of a hidden Tutor button are **not** a backend permission test. Actual screenshots, Playwright results and integration evidence are **NOT RUN** at this Part E phase.

Auth-only visual direction — APPROVED design decision, implementation/tests pending

The approved visual direction is a **calm tutoring/scheduling utility**, not a landing or marketing page, scoped to **F-S1-01 only**. @Tanaboonnnnn approved the direction and tokens in PR #25 Conversation comment #6624817784; this is a **decision approval**, not evidence that a browser-rendered UI exists or passes accessibility checks. Part F must implement and verify it with AUTH-15/23 after the now-completed OD-005-D approval and Part E merge gates. These visual decisions are team technical/UX choices, not new instructor business rules.

Element	Desktop	Mobile and acceptance
Anatomy	Small text-only NutTiew wordmark, concise sign-in heading, subdued helper, left-aligned visible email/password labels and inputs, one prominent primary action, quiet real help text only	Keep same order; no role picker, imaginary schedules, testimonials, hero or extra CTAs
Layout	Warm neutral page, readable **400–440px maximum** form column with content left-aligned, centered within available canvas; **no nested cards**	At 320/375px use 16px horizontal gutters and full remaining width; at 768/1280px preserve readable column, no horizontal overflow
Spacing and corners	4/8/12/16/24/32px spacing scale; 8px label gap, 16px field gap, 24–32px section separation, 8px radius	Input/button internal padding 12–16px; target size **at least 44 by 44 CSS px**; avoid clipping or overlapping text

Element	Desktop	Mobile and acceptance
Typography	Thai/English legible stack: Noto Sans Thai, Sarabun, system-ui, sans-serif; heading 24px/1.3 weight 600; body 16px/1.5; labels 14–16px/1.5	Input font >=16px; keep bilingual text readable without truncation and clear focus/navigation order
Semantic colors	**--auth-bg #F6F4EF** warm neutral, **--auth-surface #FFFFFF**, **--auth-text #262B28** graphite, **--auth-muted #4F5853**, **--auth-accent #285C4B** single muted pine green, **--auth-border #77877B** visible input boundary, **--auth-focus #285C4B**, **--auth-error #9C2B2B** (error status only)	Verify actual text/background pairs **>=4.5:1 normal text**, functional input border against white AND warm canvas **>=3:1 non-text contrast** (WCAG 2.2 SC 1.4.11); >=2px focus indicator; white-on-accent primary action; never convey errors by color alone

Anti-template gate: do not use generic purple/blue gradients, glassmorphism, glow/blur floating blobs, gratuitous 3D stock mascots, decorative emoji, card-in-card layouts, fictional schedules/testimonials, or generic landing hero/CTA sections. Ordinary accessible icons and focus states are allowed when functional. Do not redesign Booking/Tutor pages in Issue #13.

Auth form accessibility and failure recovery — concrete acceptance rules

- Inputs must be **type=email and type=password**, with visible associated labels and **autocomplete=email/current-password**; permit paste and password-manager autofill. A show/hide-password control is a type=button control with accessible name reflecting current action, aria-pressed state, retained input value and focus; it must never accidentally submit.
- Field-specific errors must appear adjacent to fields, linked with **aria-describedby** and **aria-invalid**. For failed client validation, move focus to the first invalid input. For failed server submit, focus the short role=alert summary and announce it to a screen reader. Generic login copy must **not** distinguish unknown vs inactive account vs wrong password (e.g. "อีเมลหรือรหัสผ่านไม่ถูกต้อง").
- All visible interactive controls, including submit, show/hide and logout, must have **>=44 x 44px** usable targets. Confirm visible **focus-visible**, keyboard Tab/Enter navigation, touch usability and no role picker. UI navigation never substitutes backend ownership checks.
- Loading and duplicate submit: show a visible pending state and disable repeat submit only during active request; after failure/timeout, return to an actionable retry state. Do not leave an indefinite spinner. Do not show an authenticated screen from a client-selected role.
- Reload, expiry and stale responses: on refresh hide protected content until a fresh /auth/me 200; on 401, clear in-memory identity and CSRF and render signed-out. Use request generation IDs or abort/ignore-stale logic so a late successful login/me response cannot restore identity after logout, 401 or a newer auth transition.
- Uncertain logout: when network/503 fails, do not claim revocation; stop protected interactions and display a short retry/check-session message. Only confirmed logout 204 or fresh /auth/me 401 can finalize the signed-out state. Discard late old-identity responses. No password/session token in console, URL or browser storage.
- **AC-S1-01/02 visible own identity:** after trusted 200 `/auth/me`, render a **server-derived short UUID account reference (last 8 hex digits, explicitly labeled "บัญชี") plus effective role**, not simply a Customer/Tutor role tag shared by all accounts. A shortened UUID is only a privacy-preserving visual hint and may collide; never use it for authorization or claim global uniqueness. The backend continues to use the full ID. Part F must check that seeded C1/C2 and T1/T2 test accounts yield distinguishable visible references; if not, submit any proposed longer UUID display for peer review, not add email/displayName without privacy approval. Clear both reference and role during reload, 401, logout and session switch, then render only from the latest `/auth/me` response. **Auth UI direction was approved** in comment #6624817784; the exact rendered identity label still requires AUTH-23 visual/functional verification in Part F.

> **Approval-state update (2026-11-04):** @Tanaboonnnnn expressly approved OD-005-D, including the temporary sixth-correct-login 429 tradeoff, in PR #25 comment #6625521193, with follow-up review confirmation in PR #25 review #5765005104. The approved Part E contract was merged into sprint1. This corrects historical pending-status text only; implementation and the AUTH-01..23 tests are independently tracked in Part F verification ledger.

E.1.3 F-S1-01 Authentication API Contract (api-spec.md)

> **Part E contract; not implemented or runtime-tested.** See Feature SDS, UI, data and STS. PR #25 Conversation #6624817784 **approved OD-005-A/B/C/E and Auth UI decisions**; revised **OD-005-D APPROVED (PR #25 reviewer comment #6625521193)** review accepted in PR #25 comment #6625521193. `/api/v1` follows System SDS §6. Part E was reviewed and merged into `sprint1` before Part F.

Shared transport and payload

- JSON UTF-8, strict allowlisted inputs, bounded body/email/password sizes; response `Cache-Control: no-store`; browser credentials sent via same-origin cookie automatically. No role or owner parameters accepted as authority.
- Success identity allowlist: `{ "user": { "id": "<UUID>", "role": "Customer" | "Tutor" } }`; never expose password hash, session ID, digest, token or other Customer/Tutor data. Email is excluded pending explicit approved UI need.
- Error envelope: `{ "error": { "code": "CODE", "message": "Generic safe description" }, "requestId": "<opaque id>" }`. No SQL errors, stack traces, raw credentials, account enumeration or private IDs.

Method / path	Request / session check	Success	Proposed errors / privacy
`POST /api/v1/auth/login`	`{ "email": string, "password": string }` only; check same-origin `Origin`, active User and Argon2id hash	200 identity DTO; `Set-Cookie` only after PostgreSQL Session insert commits	400 `INVALID_INPUT`; uniform 401 `LOGIN_FAILED` for wrong, unknown and inactive; 403 `ORIGIN_DENIED`; approved 429 throttling; safe 503 on DB failure; no auth cookie on failure
`GET /api/v1/auth/me`	Live session cookie; read-only	200 identity DTO derived from persisted User/Session	401 `UNAUTHENTICATED` on missing, expired, revoked or inactive; no private payload
`GET /api/v1/auth/csrf`	Live cookie plus same-origin request; token never stored in browser localStorage	200 `{ "csrfToken": "<opaque per-session value>" }` and `no-store`	401 if no session; 403 for invalid cross-origin; lifetime/representation **OD-005**
`POST /api/v1/auth/logout`	Live cookie, allowed Origin and valid `X-CSRF-Token`; no other request fields	204 empty body after **transactional current-session revocation commit**, expire presented cookie	401 if session invalid; 403 `CSRF_DENIED` if invalid/missing Origin/CSRF; 503 DB failure **without falsely clearing live session or reporting success**

Cookie, authorization and replay contract

Production cookie is `nuttiew_session` with `HttpOnly; Secure; SameSite=Lax; Path=/`, no Domain; max age cannot exceed Session expiry. **OD-005-A 8-hour absolute TTL and OD-005-B current-session-only logout are APPROVED decisions (PR #25 comment #6624817784); implementation and tests are NOT RUN.** Only localhost HTTP may set `Secure=false` with distinct `nuttiew_session_dev`; never do so for remote HTTP. The server generates 32 bytes with a CSPRNG and stores only its SHA-256 digest (not the raw cookie) in DB. Every protected request checks active User plus unexpired, nonrevoked Session against server time. Logout must be **replay resistant**: manually restoring old cookie still returns 401; other independent sessions stay live.

OD-005-C Origin/CSRF decision APPROVED by @Tanaboonnnnn in PR #25 comment #6624817784; implementation/tests NOT RUN: Do **not** require an `Origin` header on read-only `GET /api/v1/auth/csrf`, because same-origin browser GET may omit it. This GET requires a valid live cookie and same-origin/no-credentialed-CORS deployment; when `Sec-Fetch-Site` exists and equals `cross-site`, reject 403. **Stable derived CSRF lifecycle:** on login store a **32-byte random** `Session.csrfNonce` **(nonsecret)**, not a one-way hash of an unavailable raw CSRF token. A separately managed **256-bit-minimum** `AUTH_CSRF_HMAC_KEY` derives the response `base64url(HMAC-SHA256(key, 'NutTiew-CSRF-v1' || 0x00 || Session UUID bytes || csrfNonce))`. Repeated GET requests for a live Session derive the **same token without changing DB state**; both tabs and in-flight requests keep working. Server reconstructs the expected MAC from presented Session, rejects malformed header, and **constant-time compares** decoded `X-CSRF-Token` to the expected MAC on unsafe operations. A's token cannot authorize B even for the same User; expired/revoked Sessions fail first. CSRF token valid until session expiry/revocation or an explicitly reviewed key rotation. No raw token in DB/logs/browser persistent storage, and `Cache-Control: no-store` for GET; session-

issued nonce and production key management/rotation are specified in Data and SDS. **Part E OD-005-D was approved by @Tanaboonnnnn (PR #25 comment #6625521193) and the approved Part E was merged into `sprint1`; Part F implementation and test acceptance are reviewed separately.**

For `POST /auth/login`, `POST /auth/logout` and other unsafe cookie-mutating methods: compare `Origin` (scheme, host, port) to a **configured canonical web origin**, never to untrusted `Host` or forwarded headers. If `Origin` is present but foreign/malformed/`null`, **403** even if `Referer` appears local. If `Origin` is absent, permit **only** a syntactically valid `Referer` with matching canonical origin and `Sec-Fetch-Site` not `cross-site`; with neither header or forged/cross-site metadata, **403**. Any explicit `Sec-Fetch-Site: cross-site` also denies. Logout additionally requires `X-CSRF-Token` bound to the current Session before revocation. No credentialed cross-origin CORS. **OD-005-C decision approved**, browser/security execution remains pending Part F; see SDS decision sheet and AUTH-13/17.

Login input contract (OD-005-E APPROVED in PR #25 Conversation comment 6624817784): body **<=4 KiB**, only `email` and `password`; trimmed lowercased email **3–254 code points** with email syntax validation; password **1–128 UTF-8 bytes**, unchanged and never trimmed. Malformed shape 400, oversized body 413; identical 401 `LOGIN_FAILED` for unknown, wrong and inactive credentials with dummy Argon2 verification for unknown identity to reduce enumeration.

Reviewer-requested OD-005-D rate-limit policy (APPROVED design, PR #25 comment #6625521193; Part F implementation evidence separate): PostgreSQL 15-minute fixed UTC windows, two HMAC-keyed buckets: **5 FAILED credential verifications/account/15min**, with successful login **not counted as a failure**, and **20 TOTAL login attempts/trusted IP/15min**, including successful and failed requests. To avoid a race, a short admission transaction locks both rows in a stable order; rejects on `account.failed + account.inFlight >= 5` or `ip.attempts >= 20` with indistinguishable **429** `RATE_LIMITED`; otherwise increments **IP total attempts** and reserves **account in-flight permit** and commits before Argon2. After credential verification, a separate short row-locked transaction releases the same permit and increments account **failed** only for wrong/unknown/inactive credentials. The Session and success 200/Set-Cookie must not be issued until that final transaction successfully commits and permits were settled; DB failure returns sanitized 503, no auth cookie. Untrusted proxy headers never choose the IP. A successful result spends one IP request but zero account failure quota. Reservation lease expiry and fixed-window rollover are defined in Data atomic protocol; AUTH-20/22 prove failed-only counters and concurrent upper bounds. **OD-005-D design approval was explicitly granted in PR #25 comment #6625521193; Part F evidence, including AUTH-20/22, still requires independent review.**

Future dependent domain endpoint rules (not implemented here)

Issue #14/17 Tutor routes require `requireSession` → `requireRole('Tutor')` → owner-filtered query (`Slot.tutorId`/associated Booking's Slot Tutor ID equals principal). Issue #16/17 Customer routes require `Customer` and `Booking.customerId === principal.id`, with foreign and missing resource IDs both 404. On wrong role return 403; missing session return 401. Reject forged `role`, `tutorId`, `customerId`, `userId` or arbitrary principal headers at every path. This contract does **not** create those Slot/Booking endpoints.

Verification status: endpoint implementations, persistence and API tests **NOT RUN**. Corresponding scenarios are specified in STS.

> **Approval-state update (2026-11-04):** @Tanaboonnnnn expressly approved OD-005-D, including the temporary sixth-correct-login 429 tradeoff, in PR #25 comment #6625521193, with follow-up review confirmation in PR #25 review #5765005104. The approved Part E contract was merged into sprint1. This corrects historical pending-status text only; implementation and the AUTH-01..23 tests are independently tracked in Part F verification ledger.

E.1.4 F-S1-01 Identity & Session Data Contract (data-spec.md)

> **Part E data contract — no migration created or executed.** Based on System SDS §3/5 and Feature SDS. PR #25 Conversation comment #6624817784 **APPROVED OD-005-A/B/C/E and Auth UI decisions**; **OD-005-D APPROVED (PR #25 reviewer comment #6625521193)** approved in PR #25 comment #6625521193. Part E contains design contracts only; its approved changes were merged into `sprint1` before Part F. Part F migration and test evidence is tracked separately in `verification-ledger.md`.

Field	Data / constraints	Security and semantics
`User.id`	UUID PK	Trust server-generated principal identity
`User.email`	normalized unique text, canonicalization reviewed before implementation	Private credential identifier; not returned by minimal current-user DTO
`User.passwordHash`	nonnull Argon2id encoded hash with distinct random salt	Never plaintext, reversible, in API JSON or logs
`User.role`	enum `Customer` / `Tutor` (Tutor = Staff)	Authoritative DB role, not client form/header/body
`User.isActive`	boolean, active for auth	Any inactive user denied on next protected request
`User.createdAt/updatedAt`	persisted timestamptz	Not part of minimal identity response
`Session.id`	UUID PK	Internal, not exposed as bearer cookie
`Session.userId`	indexed FK → User.id	Authorization joins actual principal
`Session.tokenDigest`	unique 32-byte SHA-256 digest from 256-bit random token	Raw token lives only in HttpOnly cookie, never persisted
`Session.createdAt/expiresAt`	server timestamptz, expiresAt > createdAt; **8h absolute TTL approved**	Server clock, no sliding renewal; **OD-005-A decision APPROVED**, implementation/tests NOT RUN
`Session.revokedAt`	nullable timestamptz	Logout transaction revokes presented Session only; no reuse after commit
`Session.csrfNonce`	32-byte random BYTEA, generated once at login, not exposed; **not a CSRF token or digest**	Nonsecret per-session HMAC input; repeated GET derives stable token without DB write
`Session.csrfKeyVersion`	short key-identifier of managed HMAC key version	At login snapshot current key version; old version can be verified until all dependent Sessions expire, then retire; no secret stored in DB

Atomic lifecycle: create Session row and commit **before** sending authenticated cookie; on logout mark current row revoked and commit **before** responding 204 or expiring cookie. On failed DB write report sanitized failure and preserve actual session state. Expired/revoked/inactive rows fail identity checks after app restart; cleaning expired rows does not restore access.

OD-005-C decision APPROVED (PR #25 Conversation #6624817784); stable, reproducible, read-only GET; implementation/tests NOT RUN: At successful login generate a **32-byte CSPRNG nonce** and persist it as `Session.csrfNonce` (32 bytes, **nonsecret**, never included in API responses), plus `Session.csrfKeyVersion` for the current managed key. Maintain separate **256-bit minimum** `AUTH_CSRF_HMAC_KEY` in deployment secret manager, not source/DB/client. For live Session S derive `T = base64url(HMAC-SHA256(key[S.csrfKeyVersion], UTF8("NutTiew-CSRF-v1") || 0x00 || UUIDbytes(S.id) || S.csrfNonce))`. The server **does not store** `T` **or its hash**. Each authorized `GET /auth/csrf` recomputes and returns exactly the same T while Session/key version remain unchanged; zero database mutation and no per-GET rotation, so two tabs and requests already in flight retain valid tokens. Unsafe cookie-authenticated POST recomputes T from the currently presented live Session and constant-time verifies the decoded 32-byte MAC against the supplied `X-CSRF-Token` (strict base64url syntax/length); tokens from Session A are invalid for B. Expiry, logout revocation and inactive User invalidate T because Session authorization fails first. `GET /auth/csrf` may omit Origin and rejects explicit cross-site Fetch Metadata; browser must enforce no-store, no cross-origin credentialed CORS, no token in logs/storage. Unsafe requests require exact canonical Origin or same-origin Referer fallback only when Origin absent. **Key rotation:** new Sessions use the new key ID; keep old key securely available to validators for at most one 8h Session TTL after the last Session with old ID was minted, then retire. Unknown/missing key version fails closed; key exposure requires incident revocation/rotation plan, not a silent deploy.

PostgreSQL limiter schema, OD-005-D (APPROVED design, PR #25 comment #6625521193): use a scalar PostgreSQL `BYTEA` with an explicit **32-octet CHECK**, **not** `BYTEA[32]` (which denotes an array and does not enforce digest length). Proposed DDL for `AuthAttemptBucket`:

```
CREATE TABLE "AuthAttemptBucket" (
  "kind" TEXT NOT NULL CHECK ("kind" IN ('account', 'ip')),
  "keyDigest" BYTEA NOT NULL CHECK (octet_length("keyDigest") = 32),
  "windowStart" TIMESTAMPTZ NOT NULL,
  "attempts" INTEGER NOT NULL DEFAULT 0 CHECK ("attempts" >= 0),
  "inFlight" INTEGER NOT NULL DEFAULT 0 CHECK ("inFlight" >= 0),
  "updatedAt" TIMESTAMPTZ NOT NULL,
  PRIMARY KEY ("kind", "keyDigest", "windowStart")
);
CREATE INDEX "AuthAttemptBucket_windowStart_idx"
  ON "AuthAttemptBucket" ("windowStart");
```

The `kind` column is **TEXT with an explicit CHECK** (not pseudo-SQL `ENUM account/ip`); any equivalent Prisma migration must retain the PostgreSQL CHECK constraints and exact composite PK. This table definition is **a reviewable proposal, not a migration executed in Part E**. Use HMAC-SHA256 with a **separate secret** `AUTH_RATE_KEY` (32+ random bytes managed in Secret Manager) over domain-separated UTF-8 `account:<canonicalEmail>` or `ip:<trustedCanonicalIP>`; do not persist enumerable raw/hash-only email or client IP. The server derives trusted client IP from directly observed socket / **explicitly allowlisted ingress proxy**, never arbitrary client-provided `X-Forwarded-For`; deployment review must approve the proxy chain.

Counter interpretation (OD-005-D, revised per Conversation comment 6624817784, APPROVED design in PR #25 reviewer comment #6625521193): `kind='account'` stores `attempts` **= committed failed credentials only**, never successful logins; `kind='ip'` stores `attempts` **= every admitted login**, successful or failed. Account `inFlight` tracks admitted but unsettled credential checks, not failures, and prevents a burst from bypassing five failures. The account window is the UTC fixed quarter-hour chosen **at admission**; subsequent settle uses that same `windowStart`, even after wall-clock rollover. Only authorized PostgreSQL writes change buckets. Retain composite PK, digest-length CHECK, and consistent row locks.

Reservation persistence for crash/retry safety: an `AuthAttemptReservation(id UUID PK, accountKeyDigest BYTEA NOT NULL CHECK(octet_length(accountKeyDigest)=32), ipKeyDigest BYTEA NOT NULL CHECK(octet_length(ipKeyDigest)=32), windowStart TIMESTAMPTZ NOT NULL, admittedAt TIMESTAMPTZ NOT NULL, leaseUntil TIMESTAMPTZ NOT NULL, status TEXT NOT NULL CHECK(status IN ('pending','failed','success','abandoned')))` records each admitted verification. Account/IP bucket rows refer to the same fixed window and key digests; a reservation ID is internal, not accepted from the browser. An application instance holds an in-memory reference to its UUID for the one login operation, not an authorization token. Database cleanup is forbidden while pending reservations exist for its window. **Lease is 60 seconds after admission**, pending expiry is not permission to assume a password was correct: expired pending reservations are settled as `abandoned` **without increasing account failed count**, releasing account `inFlight`; IP total remains spent. Requests still executing after a lease expired must **fail without session issuance**, even if password verifies; they must not settle twice. Sweep stale reservations under row locks at admission/recovery, never with non-atomic read→write, and verify worker timeouts prevent long-running stale success.

Exact atomic phases (no PostgreSQL row lock across Argon2):

1. **Admit, short SERIALIZABLE or row-locked transaction:** resolve canonical email/verified trusted client IP and UTC 15-min window. Insert account and IP buckets using `INSERT ... ON CONFLICT DO NOTHING`; lock bucket rows in deterministic `(kind,keyDigest,windowStart)` order. Reconcile *expired pending* reservations for the account bucket under the same lock (decrement `inFlight` and mark `abandoned`, never lower failure count). Reject with generic 429 when **account.attempts + account.inFlight >= 5** or **ip.attempts >= 20**. Otherwise increment **account.inFlight by one** and **IP.attempts by one**, insert `pending` reservation, COMMIT. Even if IP is full, failed-account count does not change; a denied request cannot create Session. If a transaction retries due to deadlock/serialization conflict, retry the **whole transaction** with bounded backoff and unique reservation ID; never count the same reservation twice.
2. **Verify outside transaction:** run Argon2id (including dummy hash for unknown accounts), using bounded worker timeout. Unknown, wrong and inactive User all count as credential **failure**. A correct active User counts as **success**. Nothing may send a success cookie yet.
3. **Settle, short row-locked transaction:** lock the same admission-window account bucket, plus reservation row in a consistent order; require `pending`, unexpired `leaseUntil`, and the matching UUID/window. **Failure:** decrement `inFlight`, increment account `attempts` exactly once, mark `failed`, commit; return uniform 401. **Success:** decrement `inFlight`, do **not** increment account attempts, mark `success`, and insert durable Session **in that same transaction**; commit before 200/Set-Cookie. If reservation has already expired/abandoned, reject safe 503 without issuing a session even if credentials

were valid. If a DB write fails, no success response/cookie; the remaining pending lease is recovered later. No duplicate settle or Session from replayed reservation.

4. **Recovery and windows:** after restart the next admission reconciles overdue `pending` records **under locks**. A reservation always settles in its original bucket even across window rollover; new requests use the new aligned bucket. Reconciliation never converts abandoned requests into a credential failure. Store recent expired-window records long enough to reconcile (e.g. >30 minutes) and keep old keys/rows until no pending records. A fixed window can admit up to five failures near the end and five in the next window: **not sliding-window protection**. This account lockout and temporary all-valid saturation tradeoff was approved under OD-005-D in PR #25 comment #6625521193; Part F verification remains separate.

Privacy and failure behavior: A rejected login returns the same 429 envelope for unknown, wrong, inactive and existing accounts. The service must perform the same keyed bucket lookup/admission logic for any syntactically valid email, whether a User exists or not; do not expose account existence through error body/headers or obviously different code paths. Use HMAC-SHA256 with a **separate secret** `AUTH_RATE_KEY` (32+ random bytes managed in Secret Manager) over domain-separated `account:<canonicalEmail>` and `ip:<trustedCanonicalIP>`; raw email/IP must not be written to the limiter. `X-Forwarded-For` is only trusted from explicitly allowlisted ingress proxies. Database outages fail closed with sanitized 503. **OD-005-D is an approved Part E design** (PR #25 reviewer comment #6625521193). Part F implements the protocol, with selected PostgreSQL integration and stress tests verified; the full AUTH-01..23 acceptance suite remains PARTIAL, and this text does not assert deployment approval.

Bounds (OD-005-E): JSON body <=4 KiB, email 3–254 Unicode code points after trim/lowercase and syntax validation, password 1–128 UTF-8 bytes without normalization, no unknown keys. Reject malformed/oversized input before authentication and never create a Session. Dummy Argon2 verification for unknown account avoids obvious quick-return enumeration; transport normalization and limits must be human-approved.

Demo seed plan: generate two independent Customer and two Tutor accounts, each password supplied from a local ignored test-only secret or generated at bootstrap then hashed; record only safe account identifiers and role. Seed inactive/expired/revoked Session cases for STS. Never commit password examples, seed known production passwords, .env, token, private key or database dump. No unreviewed administrator, registration, password-change or bulk-revocation features. Slot/Booking data and migrations belong to Issues #14–17 and are only cross-feature authorization dependencies.

> **Approval-state update (2026-11-04):** @Tanaboonnnnn expressly approved OD-005-D, including the temporary sixth-correct-login 429 tradeoff, in PR #25 comment #6625521193, with follow-up review confirmation in PR #25 review #5765005104. The approved Part E contract was merged into sprint1. This corrects historical pending-status text only; implementation and the AUTH-01..23 tests are independently tracked in Part F verification ledger.

E.1.5 F-S1-01 Authentication Software Test Specification (tests.md)

> **Proposed E-0.1 / all cases NOT RUN.** Review alongside SDS, API, UI and Data before implementing Feature code. Issue #13 requires separately reviewed Part E and Part F PRs; planning tests cannot satisfy the implementation DoD.

Proposed harness and recorded evidence

Part F uses real PostgreSQL with applied migrations, generated non-committed test-only secrets, two independent Customers C1/C2 and two Tutors T1/T2, inactive User, expired/revoked Sessions and sessions A/B. API tests with Supertest/Vitest, selected real-browser Playwright/UI, DB row inspections and selected service/unit checks. Tool and package versions must be confirmed during implementation; the repo currently has **no app, installed test suite, migration, or test DB proven running**.

For each real run preserve exact command, source SHA, environment/Node/PostgreSQL/migration identity, sanitized request/result logs, PASS/FAIL/SKIP counts, expected vs observed DB Session rows and response fields. A mock-only authorization unit test **does not prove** UI→API→persistent DB. Domain rows and endpoints in Issues #14–17 are a **later shared integration gate**, never silently counted PASS at Part E stage.

Planned tests (no results claimed)

ID	Trace identifiers	Setup and action	Expected API, database and UI result	Execution stage
AUTH-01	FR-S1-01, FR-001, AC-S1-01/02, VC-01, VAL-001	Login C1 and T1 using correct pre-provisioned credentials	200 own trusted role; distinct HttpOnly cookies; two committed Session digest rows and active User join	Part F API/DB
AUTH-02	FR-S1-01, AC-S1-03, VC-02, VAL-001	Unknown email, wrong password, inactive user	Same generic 401 response; no authenticated cookie or new Session; no user enumeration	Part F API/DB
AUTH-03	FR-S1-02, FR-002, VC-03, VAL-001/002	Valid C1/T1 cookie, then forged role/userId headers/query/body	GET /auth/me returns only DB-derived own id/role; no hash, token, foreign identity	Part F API/DB
AUTH-04	FR-S1-03, AC-S1-04, VC-04, VAL-001/002	Save cookie A, get valid CSRF and Origin, logout, replay saved A	DB revokedAt committed; 204 only after commit; replay A denied 401 and no protected fields	Part F API/DB
AUTH-05	FR-S1-03, VC-04, VAL-002	Login twice A/B, logout A, request with B	A denied, B still valid, ordinary logout did not revoke B	Part F API/DB; OD-005
AUTH-06	FR-S1-02/03, VC-04/05, VAL-002	Expired/revoked session or inactive User; restart app with persistent DB	Every stale credential denied; live B continues until expiry across restart	Part F API/DB
AUTH-07	FR-S1-04, AC-S1-05, VC-05, VAL-002/036	Anonymous calls current-user and protected Slot/Booking routes	401 with no private payload or mutation; domain route checks run only after Issues #14–17	Auth now / domain later
AUTH-08	FR-S1-04, BR-S1-17/20, AC-S1-06, VC-06, VAL-002/036	C1 forges Tutor Slot create or role=Tutor	Backend 403, no Slot row committed; UI hiding Tutor control insufficient	Cross-feature #14
AUTH-09	FR-S1-04, BR-S1-16/20, AC-S1-07, VC-07/31, VAL-002/036	Tutor T1 guesses Tutor T2 schedule/Booking IDs	Privacy-safe denial/404, no T2 payload or mutation	Cross-feature #14/#17
AUTH-10	FR-S1-04, NFR-S1-01, VC-31/35, VAL-002/036	Customer C1 requests C2 Booking ID/list	404 foreign/missing indistinguishable, no private Booking fields	Cross-feature #16/#17
AUTH-11	NFR-S1-02/08, VC-35, VAL-034	Inspect real User, Session and logs/tracked config	Argon2id salted hash, SHA-256 digest only, no plaintext passwords/bearer tokens or tracked secrets	Part F API/DB + Git
AUTH-12	OD-005, FR-S1-03, VAL-002	Inject Session INSERT fault at login or revocation UPDATE fault at logout	No success/session cookie before insert commit; no false 204/cookie clear if revoke fails; DB remains source of truth	Part F DB failure
AUTH-13	OD-005, VAL-002/036	Cross-origin login, missing/wrong CSRF/Origin on logout; later unsafe domain routes	403 before mutation, no foreign fields, no new Session	Part F auth / domain later
AUTH-14	OD-005, VC-04, VAL-002	At expiresAt boundary, repeat logout, concurrent replay of old A	Exactly-at-expiry denied; revoked A never regains access	Part F DB/clock

ID	Trace identifiers	Setup and action	Expected API, database and UI result	Execution stage
AUTH-15	AC-S1-01/02/03/04, NFR-S1-09 proposed; auth visual direction and WCAG 2.2 SC 1.4.11	At **320/375/768/1280px** verify login, error, pending, Customer/Tutor success, expiry/reload/logout-uncertain, show/hide password, keyboard Tab/Enter, password-manager autofill/paste, delayed/duplicate submit and stale responses	No horizontal overflow; >=44x44px targets, normal-text contrast >=4.5:1; **visible form-input boundaries and functional non-text focus cues >=3:1 against adjacent white/warm surfaces** (review token `#77877B`), visible focus and screen-reader error announcement. Capture screenshots or human visual sign-off with reviewer/date/SHA; inspect anti-template criteria. **NOT RUN**	Part F UI / human visual review
AUTH-16	BR-S1-20, FR-S1-04, VAL-036	Test auth guard and owner predicates with client-supplied impersonation fields	Ignore client claims; service checks authenticated principal and owner-filtered DB predicates	Part F auth + domain later
AUTH-17	OD-005-C, VAL-002/036	Create Session A with `csrfNonce` and key version, same-origin `GET /auth/csrf` **without Origin**; repeat GET 10x with two tabs, including concurrent GET requests and a restarted API server; cross-site, anonymous and foreign credentials cases	**Identical decoded 32-byte MAC on every live A GET** with **zero Session writes** (compare before/after DB nonce/version/updatedAt), no `Set-Cookie` rotation, no-store; explicit `Sec-Fetch-Site: cross-site` 403; anonymous 401; no credentialed cross-origin CORS	Part F API/CSRF + real DB
AUTH-18	OD-005-C, VAL-002/036	Sessions A/B of same User; two tabs fetch A token, race two unsafe A POST requests against logout A while B stays live. Cross-check exact Origin, missing Origin + local Referer, forged/null Origin, missing both, cross-site Fetch Metadata, invalid/other-session CSRF	A token works for repeated eligible A POST until logout transaction commits; all requests using A **after revocation** 401 (even if valid MAC); A MAC fails on B; B's own token works. Explicit hostile combinations 403 before mutation; DB end state has only intended committed effects and revoked A; no unauthorized success	Part F API/CSRF + concurrency
AUTH-19	OD-005-A/B, FR-S1-03, AC-S1-04, VAL-001/002	Advance controlled authoritative time to 28,799s, 28,800s, 28,801s after creation; logout A while independent B exists; inject logout DB failure	Absolute 8h expiry boundary, no rolling refresh, A replay 401 after committed logout, B still live if unexpired, failure never falsely reports 204	Part F clock/API/DB, requires OD-005 approval
AUTH-20	OD-005-D/E, AC-S1-03, VAL-001/034	Test 4KiB boundary/oversize JSON, unknown keys, email 2/3/254/255 code points and password 0/1/128/129 UTF-8 bytes. On separate fresh buckets run **three successful active-User logins followed by five wrong-password attempts** for the same account; then sixth invalid attempt. Run unknown/inactive account identities and 20 valid-shaped attempts from one trusted IP to distinct accounts	400/413 does not create Session. **Three successes: account failed count stays 0, IP total rises 3**; five invalid attempts: account failed count rises to 5, IP total rises 5 more; next account attempt 429 (even with correct password), independent of prior successes. IP quota counts **all** 20 admitted requests and denies 21st with 429. Confirm generic failure/429 envelope, no existence leak, HMAC bucket keys, trusted proxy restrictions. OD-005-D and OD-005-E are **APPROVED design decisions** (PR #25 comment #6625521193), while Part F AUTH-20 test evidence still requires independent review	Part F API/DB, planned
AUTH-21	OD-005-C, FR-S1-03, VAL-002/036	Concurrent A GET/unsafe POST from two tabs, restart server, create separate Session B, advance to expiry, logout A; simulate key-version transition with approved old-key overlap	A token remains reproducible under same nonce/version across tabs/restarts; no GET writes or unexpected token invalidation. B token differs and A's token rejected on B; A rejected after expiry/logout. Old session version works until expiry under retained key; missing key version fails closed. Log Session nonce/key version/digest **metadata only**, never raw token/key	Part F API/DB/CSRF, planned

ID	Trace identifiers	Setup and action	Expected API, database and UI result	Execution stage
AUTH-22	OD-005-D, VAL-001/034/036	Synchronized **30–50 login requests on >=2 API instances** targeting same account/IP: all wrong, all correct and mixed passwords. **Use an Argon2 test barrier to hold exactly five all-correct requests in-flight** (`account.failed=0` , `account.inFlight=5` , IP bucket 5), then submit a sixth **correct-password** request from a trusted IP with remaining IP quota. Release barrier, finish five successful verifications, and retry the sixth after settlement **within the same 15-min account window**. Also test multiple accounts on one IP, one account across IPs, forged X-Forwarded-For, crash/lease expiry and UTC rollover	**Temporary concurrency cap acceptance:** while `failed=0` and `inFlight=5` , the sixth correct login MUST return generic **429** `RATE_LIMITED` with **no Session/cookie and no extra account failure or IP reservation** (because admission rejects atomically). This is temporary *capacity saturation*, **not** five failed logins. Once the first five settle successfully, `inFlight=0` , account `failed` remains **0**; retrying the sixth **must be admitted** if IP total is below 20, credentials are correct and the lease/window is valid, and creates its Session after commit. UI may invite manual retry once outstanding work finishes; **no automatic background retry** and no promise of an exact wait interval. Record observed 429/retry sequence, before/after counters, responses, and IP/account bucket snapshots; retry during ongoing saturation may also receive 429. With 5 committed failures, retry is still 429 for that window even after inFlight=0. Across 30–50 requests, assert committed failed <=5 and `failed + inFlight <=5` at all times; success never increments failed; IP admits <=20 total (success + failure). Persisted reservations settle once, crash/60s leases reconcile without negative inFlight or stale Session; DB errors fail closed with sanitized 503. **The all-valid saturation availability tradeoff was expressly approved as a Part E design in PR #25 reviewer comment #6625521193.** Part F has run two-process all-correct/all-wrong HTTP stress and a permit saturation/retry DB case; deterministic Argon2 barrier, mixed-credential stress and forced-crash cases remain **NOT RUN**, so AUTH-22 remains **PARTIAL**.	Part F multi-instance/PostgreSQL stress, planned
AUTH-23	AC-S1-01/02, FR-S1-02, VAL-001/002	Seed C1/C2 and T1/T2 with distinct UUIDs. Sign in sequentially, reload, switch sessions, delayed old `/auth/me` response and logout, verify visible account reference	UI shows **current server-derived short UUID account reference + effective role** and not generic role alone; C1 != C2, T1 != T2 for selected fixtures, no old ID/role after reload, 401 or switch. No added email/displayName/PII. Correlate visible label with returned id/role and backend DB; if 8-hex suffix collides, require approved longer UUID display	Part F browser/UI, planned

Auth visual review checklist (Part F evidence, not current approval)

At **320, 375, 768, 1280 CSS px**, inspect typography Thai/English, form width/padding/button, labels and errors without overflow, responsive keyboard/touch affordances, screenshots for signed out/loading/invalid/Customer/Tutor/expired/logout-uncertain. Assert computed text contrast >=4.5:1, **non-text input border and focus cues >=3:1 against each adjacent background** (white and warm neutral), focus-visible and all action targets >=44px; screen-reader test must announce field and generic auth errors. Test password-manager autofill, paste, show/hide accessible name/pressed state and duplicate submission under slow network. Test current account reference+role for C1/C2/T1/T2 and clear on session switch/401/logout. Simulate stale `/auth/me` success arriving after a newer 401/logout, and network outage during logout; no old identity or false signed-out success. Require recorded **human visual review/sign-off** (name, date, SHA) or actual screenshot audit when UI exists; a documented style proposal is not evidence that rendered UI passes. Explicitly reject unapproved generic purple gradients/glass cards/glow blobs/3D decorations/fake schedules.

Negative-response/privacy assertions

For each 401/403/404/429/503 failure inspect status and sanitized error envelope; assert no `passwordHash` , session or CSRF token, SQL/stack details, foreign identity or Booking data. A Set-Cookie **clearing** an invalid token is not a new authenticated

cookie. Verify failed writes leave no new Session, Slot or Booking row. 403 wrong-role and privacy-safe 404 missing/foreign-row are distinct.

Independent review / go-no-go checklist

- [x] Per **@Tanaboonnnnn's PR #25 Conversation decision 6624817784**, **OD-005-A, B, C, E and Auth UI Direction are APPROVED as documented**. **OD-005-D APPROVED (PR #25 reviewer comment #6625521193)**, with reviewer-mandated **5 FAILED logins/account/15min (do not count successful logins)** plus **20 TOTAL attempts/trusted-IP/15min (count both success and failure)**. The human reviewer **expressly accepted the revised reservation/atomic settlement and recovery protocol** in PR #25 comment #6625521193; do not mistake design approval for a passed Part F implementation or all AUTH tests. Retain cross-origin/CSRF security, strict 4KiB/email/password bounds, API-safe errors and human review/merge gate. Author may not self-approve.
- [ ] Trace AC-S1-01..07, FR-S1-01..04, BR-S1-04/16/17/20, NFR-S1-01/02/07/08 and VAL-001/002/034/036 to named cases above, noting cross-feature tests are NOT RUN until domain routes exist.
- [x] Reviewed and approved Part E PR #25 was merged into `sprint1` before F-S1-01 Part F implementation. Reference PR #25 and approval comment #6625521193; does not indicate Part F tests have passed.
- [ ] On separate Part F branch from latest `sprint1`, write failing tests first, implement real UI/API/PostgreSQL, run actual tests, publish source SHA/results and open independent feature-code PR for human review. **Issue #13 remains open until implementation and review/merge evidence exist.**

> **Approval-state update (2026-11-04):** @Tanaboonnnnn expressly approved OD-005-D, including the temporary sixth-correct-login 429 tradeoff, in PR #25 comment #6625521193, with follow-up review confirmation in PR #25 review #5765005104. The approved Part E contract was merged into sprint1. This corrects historical pending-status text only; implementation and the AUTH-01..23 tests are independently tracked in Part F verification ledger.

E.2 F-S1-02: Tutor Slot Availability & Scheduling

E.2.1 F-S1-02 Tutor Slot Feature SDS (specification.md)

> **UX-only amendment proposed 2026-11-05 (owner-directed, peer acceptance pending):** Replace only the Tutor screen's former narrow form/list presentation with a NutTiew navy/orange 24-hour client-side calendar plus complete private chronological agenda. See dated UX decision, new UI contract, and UX-R1…UX-R6 STS.
> **Unchanged:** Auth/Customer visual design, Tutor role/owner/session/CSRF, exactly 60-minute future Offered Slot semantics, PostgreSQL race protection, strict POST + full-list GET contract, DTO fields, Asia/Bangkok timestamps, Option A no hour/day/horizon/grid constraints, safe uncertain-write reconciliation, and denial of cross-Tutor private data. Calendar day navigation is a pure read-only **client-side projection**, never a business rule or a new GET filter. This amendment has **not** yet received independent peer approval; the baseline Part E history below is preserved.

> **Mandatory sequential amendment gate (new UI direction):** this specification/UX decision/STS update is **Part E only**. Obtain @Chxtamos peer acceptance and merge this docs-only amendment into `sprint1` before opening a separate Part F Dashboard code PR from the updated base. This preserves the original Issue #14 two-PR rule.
> **Additional UX-R5 obligations:** initial/retry GET 503 must not assert an empty schedule; unknown count remains unknown until 200. On Logout click, hide account identity and all private Slots immediately and suspend protected interactions, while retaining the previous Tutor ID **only internally** for uncertain Session recheck. A confirmed Logout 204 or `/auth/me` 401 finalizes signed-out. Only a fresh same-Tutor `/auth/me` 200 **followed by fresh own-list GET 200** may restore the Dashboard and actions; otherwise remain hidden/blocked. Owner-selected 768px two-day / 1280px three-day / mobile agenda design, full-own-list summary and WCAG 2.2 AA target are detailed in dated UX decision.
> See UI contract and UX-R5-GET / UX-R5-LOGOUT STS. No backend/API/DB changes.

> **DOCS-ONLY NEW DASHBOARD CONTRACT / AWAITING PEER ACCEPTANCE.** Issue #14, owner @Tanaboonnnnn, reviewer @Chxtamos. **Historical initial Part E baseline:** `sprint1` `6fd3931` (PR #25/#26), from before Slot implementation. **Current PR #30 base:** `sprint1` `ed451be2ca3e366bdfc1900f4c4697aa7e50b6a0`, with the previously reviewed and merged Slot feature from PR #27/#28 already implemented and tested. This amended UI contract itself neither runs nor approves new Dashboard implementation; its review/merge precedes the *new* Part F UI code-only PR.

Sources and traceability

Authority: Sprint 1 requirements Q02/Q03/Q04/Q06/Q29, FR-S1-04..07/18/19, BR-S1-02..06/11/12/17/18/20/21, AC-S1-08..14, VC/TEST-S1-08..14/37; SRS FR-004..007, BR-002/003/004/006/029/033/034/035 and OD-001..005/011/012; System SDS §§3.3/4/7.2; Feature Inventory; Traceability; Auth Part E and merged PR #26; AGENTS.

The Tutor (=Staff) owns their schedule. A Slot is one explicitly dated 60-minute `Offered` interval; Tutor Schedule is an owner-filtered read projection, **not** another mutable table. Publication state does not imply unoccupied after Booking exists.

Scope

In Part F, an authenticated active Tutor creates exactly one valid **future** 60-minute Slot for **their own** identity and reads persisted own Slots on refresh/restart. The Sprint 1 API surface is **POST /api/v1/tutor/slots** and **GET /api/v1/tutor/slots** only: no individual-detail GET, pagination or filters. Any query parameter, including `?tutorId=other`, is rejected, not used to choose an owner. Reads use authenticated `principal.userId`, ordered by `startsAt,id` ascending, with the fixed safe DTO in HTTP contract. Reject malformed/past requests, wrong role/owner, conflicting same-Tutor offers and simultaneous overlapping creates. Adjacent same-Tutor Slots and identical hours from distinct Tutors are permitted. Use Asia/Bangkok business time.

Out of scope: Customer availability #15, Booking and Q05 60-minute new-Booking cutoff #16, protected Booking views #17, release #18, bulk/recurring slots, edit/delete/withdraw, cancellation, notifications, summary and payments. Future `Withdrawn` compatibility is not permission to implement Sprint 2.

Invariants and user journey

1. Tutor logs in using existing Auth, opens My Availability and sees only own committed Slots.
2. Tutor chooses Bangkok calendar date and **any minute** (`HH:mm`, e.g. 13:17); UI derives end = start + 60m. API accepts strict RFC3339 instants with explicit offset/Z and **zero seconds + zero fractional seconds only** (optional 1–3 all-zero fractional digits). Reject nonzero seconds/fractions without rounding; this transport precision does **not** add an hourly/quarter-hour start grid. Server ignores any client role or owner.

3. API checks DB-backed Session/Tutor role, Origin and HMAC CSRF. Transaction locks User principal and rechecks active Tutor and future time **after contention** using DB clock; inserts `Offered` Slot under `principal.userId`.
4. PostgreSQL CHECK enforces exactly 60m; conditional GiST EXCLUDE prevents two currently `Offered` Slots of the same Tutor overlapping. A versioned **database trigger** rejects normal SQL UPDATE/DELETE/TRUNCATE in Sprint 1 and direct INSERT into non-`Offered` lifecycle. COMMIT before 201.
5. Rejected mutation rolls back and does not change existing Slot rows. GET returns owner-filtered durable data.

Use half-open intervals `[start,end)`: overlap iff `a.start < b.end AND b.start < a.end`. 13:00–14:00 + 13:30–14:30 conflict; 13:00–14:00 + 14:00–15:00 allowed. Two Tutors can each offer 13:00–14:00.

SLOT-I1: exactly 60m and immutable identity/owner/interval, enforced against ordinary direct SQL by the Sprint 1 trigger (all UPDATE/DELETE/TRUNCATE rejected). **I2:** no overlapping currently Offered Slots per Tutor, even under Race A. **I3:** active Tutor-only identity from Session. **I4:** strictly future by authoritative post-lock DB time; no Q05 Booking cutoff on Slot creation. **I5:** no partial writes/false success; restart durability. **I6:** UTC storage and Bangkok display. **I7:** no Withdrawn insertion/transition in Sprint 1; later approved Sprint 2 transition is one-way `Offered -> Withdrawn`, never `Withdrawn -> Offered` on an old ID, with separately verified migration and tests. Privileged DDL/superuser tampering is outside the ordinary DML invariant and must be controlled by DB privileges.

Original Slot AC mapping — historical Part E verification plan

The AC-to-VC mapping below was written as the **original Slot Part E test plan**, **before the Slot feature was implemented.** It is not a statement that these tests remain unexecuted.** The merged Slot Part F work (PR #28) already has recorded real API/PostgreSQL/browser executions, including failures, reruns, exact commands and remaining limitations in the Slot Part F verification ledger. Consult that ledger and the relevant GitHub CI run for actual outcomes at their respective commit SHAs; this historical table is a traceability map, not a live PASS/FAIL dashboard.

The **new Tutor Calendar Dashboard amendment** and its `UX-R1…UX-R6`, `UX-R5-GET-01/02` and `UX-R5-LOGOUT-01/02` acceptance checks are separate **planned Part E requirements** in the amended STS. This docs-only PR neither implements nor executes those new Dashboard/Logout checks; subsequent Part F code and independent peer review remain necessary. Human WCAG 2.2 AA signoff is likewise **NOT RUN**.

AC	Expected	Test
AC-S1-08	One future owned 60m Slot persisted/read on refresh	VC-S1-08/14
AC-S1-09	Same-Tutor positive overlap rejected	VC-S1-09
AC-S1-10	Same-Tutor adjacency accepted	VC-S1-10
AC-S1-11	Different Tutors same hour accepted	VC-S1-11
AC-S1-12	Invalid/past input rejected without mutation	VC-S1-12
AC-S1-13	Race A: never two overlapping committed Offered rows	VC-S1-37

AC	Expected	Test
AC-S1-14	Browser timezone cannot change Bangkok appointment	VC-S1-13

Related product validations: VAL-002/003/004/005/012/013/031/036. Cross-feature Auth cases AUTH-07/08/09/16 become testable via real Slot routes in Part F.

Decision record — scope owner direction and outstanding approvals

Recorded 2026-11-04 (Asia/Bangkok). The Issue #14 requester/owner explicitly selected **Option A** for the **Sprint 1 implementation scope** in the project conversation. This is **owner direction**, not independent stakeholder/instructor approval and not proof of 24/7 Tutor availability. Peer reviewer @Chxtamos must explicitly accept/reject this scoped no-extra-restriction decision in GitHub; stakeholder policy remains unresolved until a separate approval or requirements revision.

Decision	Sprint 1 include/exclude decision from owner	Approval status / future change gate
OD-001 / TBD-S1-01 working days/hours/holidays	**EXCLUDE** any extra hours, weekday or holiday restriction; include strictly future, valid 60m Slot rule only.	Owner chose A 2026-11-04; **peer review pending**; no 24/7 stakeholder promise. New restriction requires reviewed Q/FR/BR/AC/VC update.
OD-002 / TBD-S1-02 maximum advance horizon	**EXCLUDE** a new 7/14/30-day or other business maximum; include only confirmed future-time check.	Owner chose A 2026-11-04; **peer review pending**; stakeholder maximum policy still unresolved.
OD-003 / TBD-S1-03 start-minute grid	**EXCLUDE** :00-only, quarter-hour and any other start grid; **include** every `HH:mm` minute as valid if the Slot otherwise meets rules.	Owner chose A 2026-11-04; **peer review pending**; minute-level **transport precision** (zero seconds/fractions) is a separate, proposed engineering validation, not a business grid.
OD-004 / TBD-S1-04 Tutor read interface	**PROPOSE** POST+own-list GET only, no detail GET or list filtering/pagination in Sprint 1; reject all query params, order `startsAt ASC,id ASC`, allowlist `id,startsAt,endsAt,lifecycle,occupancy`, empty array allowed.	Proposed engineering contract **pending @Chxtamos review**; Customer discovery #15 has separate interface.
OD-005 / TBD-S1-05 transaction/error/security	**PROPOSE** `SERIALIZABLE` + Tutor User lock, DB CHECK/GiST + Sprint 1 immutability/lifecycle trigger, bounded transient retries, shared Auth CSRF/Origin and error envelope.	Proposed engineering contract **pending @Chxtamos review**; actual migrations and race tests are Part F only.
OD-011/012 final Master/Theme 1	No additional claim.	Final instructor/TA specification and formal theme assignment **unverified**; must not be treated as sign-off.

Changes after this decision: additions to working hours, horizon or minute grid need a dated stakeholder/team rule, updated requirement IDs and fresh peer review; neither a form default nor an API validator may silently introduce them.

Definition of Done / two-PR gate

Part E: five cross-linked versioned docs (UI, HTTP, data, STS, this SDS) with decisions and Race A proof plan. Human @Chxtamos review + approved documentation PR merged into `sprint1` before feature code.

Part F: separate feature branch from latest `sprint1`; TDD RED→GREEN, real UI→Express→PostgreSQL, native constraint/transaction, negative auth/privacy, synchronized Race A, restart and fault tests, actual command/result/SHA evidence and peer-reviewed code PR into `sprint1`. No fabricated PASS and no direct main/sprint1 pushes. `sprint1→main` release belongs to #18.

E.2.2 F-S1-02 Tutor Availability UI Specification (ui-spec.md)

> **2026-11-05 proposed visual-contract amendment (pending peer review):** The feature owner selected the @Chxtamos Tutorix-inspired **NutTiew Tutor Calendar Dashboard** as the new Tutor presentation. See dated UX decision. The older approved form+list content below remains a *historical Part E baseline*; its Auth palette and prohibition on calendar presentation are **proposed to be superseded for the Tutor view only**. The HTTP/DB/security/UTC/ ownership/business-rule contract is **not** being superseded. Do not label this amendment peer-approved until review actually occurs.

> **Review sequencing:** this is a new **docs-only Part E proposal**; it must receive peer approval and merge into `sprint1` before the separate Tutor Dashboard Part F implementation PR. Original Part E PR #27 / Part F PR #28 remain historical, previously merged baselines. A combined docs-and-code PR is not a substitute for this gate.

Amendment: Tutor Calendar Dashboard candidate

- Desktop uses a navy `#202B44` sidebar, a bright canvas `#F7F8FB`,

white surfaces, orange `#F47736` call-to-action with the **explicit

contrast-safe foreground/border/focus pairing below**, and

deliberate dark text/focus. These tokens are **scoped to the Tutor

dashboard**; existing Auth Login/Customer palette remains unchanged.

- **Required CTA pairing (WCAG 2.2 AA design target, not human signoff):**

button fill `#F47736`, **dark-ink label/icon** `#172137` (measured

5.76:1 against the orange fill), and a **1 CSS px solid border

`#D96123`** to distinguish the component from adjacent surfaces

(**3.48:1** against canvas `#F7F8FB`; **3.69:1** against white).

The keyboard focus indicator is **3 CSS px solid `#AA4519` with

`outline-offset: 3px`** outside the button/border, contrasting

5.53:1 with the canvas or **5.87:1** with white. These values

are calculated from the specified sRGB tokens; Part F must verify

computed browser colors, focus visibility, hover/disabled/error

states and adjacent backgrounds. **Do not use white text on

`#F47736` (only 2.79:1), or the unbordered orange fill as the

only visible control boundary** (2.62:1 against canvas, 2.79:1

against white). This pairing does not by itself certify WCAG AA.

- Sidebar contains only functional **ตารางเวลาของฉัน**, **บัญชีของฉัน**

and **ออกจากระบบ**. Account data and role come from trusted `/auth/me`.

Do **not** construct fake Students, Revenue, Messages, Bookings,

notifications, learner names, courses or rescheduling.

- **บัญชีของฉัน** links to the in-page `#account-details` section,

not a new route or editable profile. That section displays exactly

Tutor role and a shortened account ID derived from trusted

`GET /api/v1/auth/me` `user.id` and `user.role`; it must not surface

unverified login-form email or invented profile fields. Hide the

section while Logout is pending or uncertain, along with all other

identifying UI.

- Header: **ตารางเวลาติวของฉัน**, role/account reference, `Asia/Bangkok`

and `+ เพิ่มช่วงเวลาติว` that takes the user to the inline form.

- Calendar: accessible **24-hour** local timeline, **3 days at

widths >1060px **(including desktop 1280px),** 2 days at 621–1060px**

(including tablet 768px). At **320/375px and all widths <=620px**,

the complete Agenda replaces the timeline. Responsive breakpoint

checks must include 620/621 and 1060/1061px. Slot cards use genuine

Offered records and computed minute-exact offsets (no snapping), including 13:17, 13:59,

and a 23:30→00:30 date crossing. Hour marks are presentation ticks.

Previous/Next/Today/day picker only change local view state and never

append filters to `GET /api/v1/tutor/slots`.

- Mobile 320/375: no horizontal overflow; use a complete **agenda/list

fallback **instead of a squeezed timeline. The** รายการ Slot ทั้งหมด**

agenda is **available after successful authenticated GET 200** and

preserves `startsAt ASC, id ASC` for all returned private Slots,

including dates outside the current calendar. Hide the Agenda on

initial load/GET error, Logout pending/uncertain or account switch;

do not silently paginate, crop or hide other times when authorized.

- **Counts must not be conflated:** the summary **ช่วงเวลาที่เปิดไว้ทั้งหมด**

is the total number of Slots from the **full unfiltered own-list GET

across all dates**, unaffected by Prev/Next/Today/day selection.

ช่วงที่แสดง counts distinct committed own Slots that intersect

the displayed Calendar viewport only; a Slot spanning midnight is

counted once per viewport even if rendered in two day columns.

Both metrics are **unknown** `–` **until a successful GET 200**, and

must never falsely show zero during GET/Retry failures.

- The create form retains native labeled Bangkok date and any-minute

time, derived read-only +60-minute end, POST security, real loading,

conflict (409), invalid (400/422), and uncertain 503/network states. After success or ambiguous write, reconcile with owner-only GET. The old T1 authenticated 200 must never reappear after T2 login.

- **Read-side state machine:** until the authenticated own-list GET returns 200, the schedule is **unknown**, not empty. Initial load shows progress, initial GET 503/network failure or retry 503 shows an accessible error with a working **โหลดอีกครั้ง** control, **hides** the empty/full Agenda and Calendar, and shows unknown `–` instead of an unverified zero summary. Only a confirmed 200 + `data: []` may show **ยังไม่มีช่วงเวลาติวที่เปิดไว้** and total zero; a confirmed 200 with rows restores all committed own Slots. These rules apply to both initial load and later refresh failures and must prevent stale old-Tutor data from reappearing.
- **Auth/logout guard inherited from F-S1-01 and owner decision 2026-11-06: **starting** synchronously on Logout click**, unmount or hide the Tutor account reference, signed-in status, private Slot cards, Calendar and Agenda. Immediately block protected actions (including the create form, header CTA and any pending CSRF-to-POST flow), invalidate stale UI request generations and prevent any **new** Slot POST after Logout begins. Any POST sent *before* Logout may already have committed; do not claim a server rollback from UI alone. Retain the prior Tutor ID **only in internal non-rendered state**.

UI hidden does not equal Session revoked: only confirmed Logout HTTP 204 or a fresh `/api/v1/auth/me` 401 finalizes signed-out status. If Logout returns network/503/timeout uncertainty, private data stays hidden and protected actions stay blocked; show an accessible **ลองออกจากระบบอีกครั้ง / ตรวจสอบเซสชัน** recovery path. A fresh `/api/v1/auth/me` HTTP 200 with **exactly the same Tutor ID and Tutor role** permits a *new* authenticated own-list GET; restore the Dashboard and protected interactions **only after that GET succeeds**. A 401 clears internal identity, a different identity cannot unlock prior Tutor data, and recheck 503/network keeps the Dashboard hidden/blocked. Old T1 authorized HTTP 200 responses must never repaint private content after Logout or account switch.

- **For state consistency:** `suspended/uncertain` is a third presentation state distinct from both `authenticated/visible` and

`confirmed signed-out`. Do not display authenticated identity while suspended; do not falsely claim Logout succeeded after an unknown response. The earlier Part E state table below is historical and the amended logout rows directly below supersede its old generic "Logout or account switch" wording for the new Tutor Dashboard.

- Required new test IDs: **UX-R1…UX-R6** from the dated decision. Add screenshots from *real DB-backed browser sessions* at 320, 375, 768 and 1280, and keyboard/contrast evidence. The new design targets **WCAG 2.2 Level AA **(all applicable A/AA criteria), measured at** 4.5:1 normal text / 3:1 large text**,** 3:1 non-text components and required visual states**, visible/unobscured keyboard focus, AA **24x24 CSS px minimum target or normative exceptions** with a stricter project **44x44px primary-control** target, and usable 320 CSS px reflow without unexpected two-axis scrolling. Record automated samples separately from complete manual assessment. For read-side failure and logout guard tests, use **UX-R5-GET-01/02** and **UX-R5-LOGOUT-01/02** in STS. **Human WCAG 2.2 AA signoff remains NOT RUN until a person performs it**.

> **PROPOSED DASHBOARD UX AMENDMENT / NO NEW IMPLEMENTATION IN THIS DOCS-ONLY PR.** The existing Slot API, DB and Auth implementation from merged PR #28 already exists. The new dashboard/logout acceptance cases in this amendment are *planned*, not executed or accepted by this documentation alone. Issue #14, @Tanaboonnnnn owner, @Chxtamos reviewer. See SDS and owner decision, HTTP, Data, STS, Auth UI direction.

Screen and navigation

"My Availability / ตารางเวลาติวของฉัน" is an authenticated Tutor-only view. Use the current `GET /api/v1/auth/me` trusted DB-derived role and session identity; do not invent a role selector. Reuse approved NutTiew look from Auth Part E: warm neutral background `#F6F4EF`, graphite text `#262B28`, restrained pine accent `#285C4B`, Thai/English readable typography, visible focus and minimal functional layout. Keep the Tutor screen separate from existing Auth-only `apps/web/src/main.tsx`; avoid unrelated Auth redesign.

Minimum surface: a create form, a derived 60-minute end preview, and a chronological **full own-list** of current Tutor's persisted Slots ordered `startsAt ASC,id ASC`. The Sprint 1 UI calls **only POST /api/v1/tutor/slots and GET /api/v1/tutor/slots**. No detail view/`:id` route, filters, pagination, date-range selector, drag-and-drop week calendar, bulk/recurring generation, delete/edit/withdraw or Customer Booking UI in #14. A list is sufficient if it meets accepted flows; this does not establish a future unlimited-scale guarantee.

Form controls

Control	Visible behavior	Validation / authority
Role/account display	Latest server-issued Tutor identity; authorized route	Display is not backend authorization

Control	Visible behavior	Validation / authority
Bangkok calendar date	Labeled `YYYY-MM-DD` selection	**Owner chose Sprint 1 Option A**: no extra workdays, holidays or max advance horizon (OD-001/002); still strictly future
Start time	Labeled `HH:mm`, Bangkok civil time, **any minute** such as 13:17	No top-of-hour/quarter-hour grid (OD-003). UI has minute precision only; API seconds/fraction must be **zero**, not silently rounded
End time	Read-only start + exactly 60m, including cross-midnight day rollover	Client cannot submit arbitrary duration/end
Create Slot	Visible pending state; prevent accidental double-click	Database remains authoritative for concurrent writes
Schedule rows	Bangkok date/time, lifecycle `Offered`, safe derived `occupancy`	No other Customer names/emails; exactly the HTTP DTO allowlist. Offered does not imply Customer-bookable under Q05
Error feedback	Keep user input and show actionable safe error	Never show false success after 409/422/503

Timezone and precision contract: combine Bangkok civil date and `HH:mm` explicitly with `Asia/Bangkok` or a verified `+07:00` offset; construct a valid RFC3339 timestamp with seconds `:00`, optional fractional `.000`, and explicit offset; convert to UTC `.000Z` before POST. **Do not** rely on `new Date('YYYY-MM-DDTHH:mm')` device-local parsing. Format returned UTC with `Intl.DateTimeFormat` and `timeZone:'Asia/Bangkok'` or a tested equivalent. 13:00:00 and 13:00:00.000 are accepted, **13:00:30 and 13:00:00.500 rejected without rounding** by API even when sent outside this form. Any HH:mm minute (13:00, 13:17, 13:59) is allowed if otherwise future/valid. Test UTC/Tokyo and cross-midnight 23:30→00:30 Bangkok.

Observable UI states

State	Behavior
Auth pending	Do not show private schedule until `/auth/me` confirms Tutor
Loading	Display status, not fake appointments
Empty (authenticated own-list GET 200 with `data: []` only)	Show "ยังไม่มีช่วงเวลาติวที่เปิดไว้" and usable creation form; never infer empty from a failed/unknown GET
Editing	Show selected Bangkok start and derived end
Creating	Clear pending indicator, one request in flight; no phantom success
201 Created	Show committed confirmation and refresh own-list from server
409 Overlap	"ช่วงเวลานี้ทับกับเวลาที่เปิดไว้แล้ว"; retain fields, focus error
400/422 Invalid	Field-linked invalid/past message, no Slot added
401/403	Clear private list, signed-out or access denied; no unauthorized data
403 CSRF/Origin	No success; recover session/CSRF where allowed
POST 503/network timeout	Indeterminate write outcome: show uncertainty, refresh own-list before a duplicate submit; GET 503 is a separate read-side unknown state, not empty
Logout starts (new Tutor dashboard amendment)	Immediately hide old account ID, signed-in status, Calendar and Agenda; block Create and any new protected POST synchronously, invalidate old requests; retain prior Tutor ID internally only for recheck, do **not** assert signed-out yet
Logout response 503/network/timeout	Keep all private surfaces hidden and protected actions blocked; announce outcome unknown and expose retry Logout / check Session, not fake Logout success
Fresh same-Tutor `/auth/me` 200 after uncertain Logout	Re-read complete authenticated own-list GET; restore Tutor identity, Calendar, Agenda and actions **only after successful owner GET**. Failed own-list GET retains privacy/blocked state
Fresh `/auth/me` 401 or confirmed Logout HTTP 204	Finalize signed-out, clear internal identity and private data, prevent stale T1 responses

State	Behavior
Different account or failed `/auth/me` recheck	Never unlock the previous Tutor; clear or retain only hidden internal comparison state as appropriate, all private surfaces hidden and protected writes blocked
Account switch without Logout	Invalidate previous owner generation, immediately hide old identity and Slots and use only the newly verified account's authenticated own-list

The current Auth UI has request-generation guards for stale `/me` responses. The new Tutor view must use equivalent lifecycle cancellation/generation checks. **Required privacy scenario:** T1 GET starts but its response is held; the user logs out or switches to T2; then release the old T1 success response. The UI must **never re-display T1 Slots** in T2's/current signed-out view; authenticated owner-list GET for T2 must include only T2 rows. This is a planned deterministic E2E negative, **not** a passed test.

Accessibility / responsive acceptance

- 320/375px: usable single-column form/list; 768/1280px: readable restrained page width; no horizontal overflow.
- Labeled date/time inputs, `aria-invalid` and associated `aria-describedby` messages, accessible alert/status updates and keyboard order.
- At least 44x44px interactive targets, visible `:focus-visible` and adequate contrast checked rather than assumed. No color-only conflict message.
- Avoid faux calendars/testimonials, generic hero pages, unnecessary visual ornament or decorative random UI.
- Part F evidence: real screenshots at 320/375/768/1280, manual keyboard/contrast check with reviewer/date/SHA, UTC vs Bangkok browser test, create/error/empty/refresh/restart flows.

Review decisions

Decision state: owner @Tanaboonnnnn selected Option A on 2026-11-04 for **Sprint 1 scope only**: no additional workday/hour, advance horizon or start-minute grid constraint; not a 24/7 stakeholder claim. The minute-level input/transport precision is an independent engineering validation proposal. Peer reviewer @Chxtamos must accept/reject that policy scope and the **no-detail, no-filter, no-pagination own-list**/DTO/ordering contract, session failure recovery and shared Auth visual design. No UI or automated test result is claimed by this document.

E.2.3 F-S1-02 UX Decision Amendment — Tutor Calendar Dashboard (ui-decision-2026-11-05.md)

Decision owner: @Tanaboonnnnn (requested in conversation on 2026-11-05).

Visual direction source: @Chxtamos's detailed *Tutorix-inspired Calendar Dashboard*

change-request text, reproduced in the owner's accepted task instructions;

the screenshot itself is **not available as a verifiable GitHub artifact**.

Review status: owner-selected design direction; **this is a docs-only

Part E amendment proposed for independent peer acceptance and merge before

any new Part F dashboard implementation PR may be accepted**. It does not

retroactively validate work previously started in a mixed docs/code PR.

Baseline: original Slot Part E PR #27 and Part F PR #28 (merged into `sprint1`

at `ed451be2ca3e366bdfc1900f4c4697aa7e50b6a0`).

Two-PR gate for this new visual change: review and merge the *new*

docs-only UX amendment into `sprint1` first. Only then branch the

Tutor Dashboard implementation from that updated `sprint1` and open a

separate code PR. The earlier combined PR #29 did not satisfy this

sequencing requirement and cannot count as the accepted Part E gate.

Superseded visual layout, retained domain contract

The previous *Tutor screen* used the approved Auth warm-neutral/pine palette and a narrow create-form-first full-list layout. Replace **only the Tutor presentation** with a navy/orange scheduling dashboard: a genuine sidebar, account details, a 24-hour date timeline with **three day columns at 1280px and wider**, two day columns at 621–1060px** (including 768px), read-only arbitrary-minute 60-minute Slot blocks, an inline create panel, and a complete all-slots chronological agenda. **At 320/375px and up to 620px**, replace the timeline with the full accessible agenda.

All navigation is real. No fake Students, Revenue, Messages, Bookings, notification counts, subjects, learners, reschedule or demo appointments.

Not superseded: the F-S1-01 Login/Auth appearance, trusted identity, session handling, the F-S1-02 API and DB contract, immutable Slot semantics, the Option A absence of working hours/advance-horizon/grid limitations, UTC transport/Asia-Bangkok display, cross-midnight precision, full private own-list, network-uncertainty reconciliation and stale-success protection.

Owner-selected CTA contrast pairing (Tutor view only): use orange `#F47736` with dark text/icons `#172137` (**5.76:1** on orange), `#D96123` border (**3.48:1** against `#F7F8FB` canvas; **3.69:1** against white), and a `3px` `#AA4519` keyboard focus outline offset outward by `3px` (**5.53:1** against canvas, **5.87:1** against white). Do **not** use white text on orange (**2.79:1**) or rely on the orange fill alone for component visibility (**2.62:1** against canvas). Ratios are token calculations for the design contract, **not evidence of a complete WCAG audit**; verify rendered styles and states in Part F. This does not recolor the previously approved Auth/Customer UI.

Ownership and display boundaries

- Only `GET /api/v1/tutor/slots` (full owner DTO list, no query) and `POST /api/v1/tutor/slots`; calendar navigation is in memory and never changes server-side selection. The agenda shows **every** returned Slot.
- **Functional account link:** `บัญชีของฉัน` navigates to the in-page `#account-details` section, which displays **only the Tutor role and a shortened account ID supplied by trusted `/api/v1/auth/me`**. No separate profile route, profile-edit capability, email pulled from a

login form, or additional personal fields are authorized by this scope.

- Calendar day positions use **real minute offsets**, not 15/30/60-minute snapping. The hour grid marks time visually but imposes no creation rule. Days cover 00:00 through 24:00, with cross-midnight slices appearing on their respective dates.
- The **overview total** counts **every persisted Slot in the complete authenticated Tutor owner-only GET response across all dates**, even outside the current Calendar viewport. It never changes when the user navigates days. The Calendar-only **"ช่วงที่แสดง"** metric is a separate number: count of distinct own Slots intersecting the currently displayed day columns (a cross-midnight Slot counts once per viewport, not twice per rendered day fragment).
  Both numbers derive from real returned Slots only **after successful GET**. On initial GET failure/retry failure show a retryable error, suppress Calendar and Agenda (including false empty claims), and display an unknown count (`–`). Empty means confirmed **200 with** `data: []` only. The UI must label
  `Offered` as an opened time, **not** as a confirmed Customer Booking. Until Issue #16, occupancy is the existing safely derived `Unoccupied` .
- **Owner-selected logout privacy (2026-11-06):** from the instant Logout begins, **hide the account identity/status and all private Slot cards, Agenda and Calendar** and block protected Slot mutation controls. Retain the previous Tutor ID **only in internal non-rendered state** for comparison against a later `/api/v1/auth/me` response; this hidden state is **not** a claim that the server Session was revoked. If Logout is uncertain (503/network/timeout), show an accessible retry-logout/check-session path with private content still hidden. Restore access **only** after a fresh authenticated `/auth/me` 200 confirms the **same Tutor ID** and a fresh owner-list GET succeeds; 204 Logout or `/auth/me` 401 finalizes signed-out state. If a recheck fails or confirms another identity, continue to block the old Tutor.
- No new API parameters, endpoints, schema, booking, withdrawal, editing, deletion, bulk operation, recurrence, working-hours rule, or role selector.

Accessibility and acceptance

The redesign targets **WCAG 2.2 Level AA** across all applicable Level A and AA success criteria, including:

normal text contrast **>=4.5:1**, large text **>=3:1** (SC 1.4.3),
non-text UI controls/necessary state indicators **>=3:1** against
adjacent colors (SC 1.4.11), complete keyboard operation (2.1.1),
visible keyboard focus (2.4.7) not entirely obscured (2.4.11),
target sizes **>=24x24 CSS px or the defined AA exceptions**
(2.5.8), and functional **320 CSS px reflow without two-dimensional
scrolling** except for intrinsic two-dimensional content (1.4.10).
Project rule: actionable primary controls meet the stricter
44x44 CSS px target. Verify labels, status text,
errors and color-independent meaning; reduced motion must not hide
focus or essential information. This is an acceptance **target, not a
claim of conformance**. Automated samples do not substitute for a
complete human keyboard/contrast/reflow accessibility signoff.
Color is never the sole Slot status indicator. The 24-hour timeline is a *visual
projection*: every Slot remains readable in its accessible agenda row.
UX-R1: desktop navy sidebar, white canvas, orange CTA with the
specified dark-ink/border/focus contrast pairing,
real Tutor role/ID and no fictitious navigation or figures.
UX-R2: real Slot clock placement at arbitrary minutes (13:17, 13:59);
cross-midnight and all-day (24-hour) access without work-hour restrictions.
UX-R3: date navigation locally changes only the calendar; full own-list
agenda remains complete, ordered, and owner-private.
UX-R4: at 320/375px render full mobile agenda, at 768px
render a **two-day timeline**, at 1280px render **three days**;
test 620/621px and 1060/1061px breakpoints; verify keyboard labels,
WCAG 2.2 AA text/non-text contrast, visible unobscured focus,
targets and 320px reflow without horizontal overflow.
UX-R5: unchanged secure create/CSRF/GET refresh, 409/422/503 recovery
and old-Tutor delayed-200 safety. Specifically, GET 503 then failed retry
must not display empty state, and authorized GET 200 restores genuine
nonempty/empty own-list. **Logout pending/uncertain** immediately hides
private identity and Slots and suspends protected mutations, retaining
the old Tutor ID internally only for safe comparison. It must provide
an actionable retry/check-session path and only re-enable on a fresh
`/api/v1/auth/me` 200 confirming the same Tutor plus a successful fresh
owner-list GET; 204/401 finalizes signed-out without falsely claiming

logout success after 503.

UX-R6: evidence must be from actual app, real PostgreSQL and Browser, not a mock screenshot; include automatic checks and explicitly distinguish human accessibility verification not performed.

Approval gate: the owner has selected this direction. @Chxtamos must accept and merge **this docs-only Part E amendment first**. No new implementation PR can claim the pre-code gate is met before that merge. Human WCAG 2.2 AA/keyboard/contrast/reflow signoff is a separate **NOT RUN** manual gate, not implied by passing Playwright.

E.2.4 F-S1-02 Slot HTTP Contract (api-spec.md)

> **REVISED PROPOSED CONTRACT / NOT IMPLEMENTED / independent peer review required.** Issue #14. Cross-references: Feature SDS, UI, Data, STS, Auth Contract. Engineering choices below require @Chxtamos review; they are **not** instructor-approved endpoint signatures.

Transport, authorization and ownership

Use the existing same-origin Express `/api/v1` JSON stack, DB-backed `requireSession`, `requireRole('Tutor')`, `getPrincipal(req)`, safe error envelope, per-request requestId and Cache-Control:no-store from PR #26. All Slot writes derive `tutorId` from trusted `principal.userId`; never accept client owner/role/lifecycle/end/duration. Restrict own-list SQL by `tutorId=principal.userId`.

Unsafe POST requires **canonical Origin/Referer and Sec-Fetch-Site checks plus current Session-bound X-CSRF-Token** obtained from `GET /api/v1/auth/csrf`. Do not duplicate an inconsistent CSRF implementation; small reusable guard may be extracted from Auth while retaining old Auth behavior/tests. Browser cookies remain HttpOnly.

Request timestamps require RFC3339 explicit offset or Z; do **not** interpret timezone-free date strings using browser device zone. **Minute-level protocol:** seconds **must be 00**, fractional seconds may be absent or **1–3 zero digits** (`.0` , `.00` , `.000`); reject any nonzero seconds/fractions without rounding, and reject malformed calendar dates, offsets and missing seconds. This is a **technical precision boundary**, not a business minute grid: 13:00, 13:17 and 13:59 are all valid proposed starts if future. DB stores UTC instants and API returns ISO UTC `Z` with `.000` ; UI displays Asia/Bangkok.

Positive/negative wire examples (proposed):

Submitted timestamp (illustrative future date)	Decision / normalized instant
`2026-11-06T13:00:00+07:00` or `2026-11-06T13:00:00.000+07:00`	Accept; `2026-11-06T06:00:00.000Z`
`2026-11-06T13:17:00+07:00`	Accept; `2026-11-06T06:17:00.000Z` (no top-of-hour grid)
`2026-11-06T13:00:30+07:00`	Reject 400 INVALID_INPUT; no silent floor/round
`2026-11-06T13:00:00.500+07:00`	Reject 400 INVALID_INPUT; no silent floor/round
`2026-11-06T13:00+07:00` , `2026-02-30T13:00:00+07:00` , timezone-free string	Reject 400 INVALID_INPUT

The examples describe **precision**, not approval for those dates to be within Tutor working hours. All accepted timestamps must additionally pass the **strictly-future DB-time test**.

Proposed endpoints

Method/path	Request	Success	Enforcement
`POST /api/v1/tutor/slots`	Exactly `{"startsAt":"2026-11-06T06:00:00.000Z"}`; no query parameters	201 committed Slot DTO	Session, Tutor, Origin, CSRF, future at DB decision, exactly 60m, native exclusion
`GET /api/v1/tutor/slots`	**No** query parameters, including `tutorId`	200 own-only array (empty `[]` valid), ordered `startsAt ASC,id ASC`	Session, Tutor, SQL `WHERE "tutorId"=principal.userId`

OD-004 design decision proposed for approval: No `GET /api/v1/tutor/slots/:id` in Sprint 1; no list filter, date range, pagination, cursor or query-based owner selector. An unknown query parameter on POST or GET (notably `?tutorId=T2`) returns **400 INVALID_INPUT** with no data. Unregistered detail paths do not retrieve a foreign/missing Slot and return 404 (no detail endpoint exists); their response body is not a defined Slot-detail DTO. The full own-list is a **Sprint 1 minimum**, not an unlimited scalability/retention guarantee; introduction of pagination or detail access requires a separately reviewed interface revision. Customer browse/read functionality is owned by #15, and protected Booking reads by #17.

Example request

```
POST /api/v1/tutor/slots
Origin: http://localhost:5173
Content-Type: application/json
Cookie: nuttiew_session_dev=<opaque>
X-CSRF-Token: <session-bound>

{"startsAt":"2026-11-06T06:00:00.000Z"}
```

This is 13:00 Bangkok, ending 14:00 Bangkok, for illustration only.

Example response after confirmed COMMIT

```
{
  "data": {
    "id": "c62bfb8b-2530-4a7c-979c-a9a6d16be0df",
    "startsAt": "2026-11-06T06:00:00.000Z",
    "endsAt": "2026-11-06T07:00:00.000Z",
    "lifecycle": "Offered",
    "occupancy": "Unoccupied"
  },
  "meta": {"requestId":"opaque-request-id"}
}
```

The **entire allowed Slot DTO** is exactly `id` (UUID), `startsAt` and `endsAt` (UTC ISO `.000Z`), `lifecycle` (currently `Offered`), and `occupancy` (`Unoccupied` or `Occupied`). No `tutorId`, User email, Customer ID, Booking payload or extra database columns are serialized. The Tutor identity is already trusted from `/api/v1/auth/me`. Own-list returns `{"data":[<same-safe-slot-DTO>],"meta":{"requestId":"..."}}`, or `{"data":[],"meta":{"requestId":"..."}}` when empty. Sort ascending `startsAt` then UUID `id`, no pagination/filter. `occupancy` is **derived**, not a mutable `Slot.isBooked` column: until #16 Booking exists each new offer is `Unoccupied`; after #16 it must be recomputed from committed active `Booked` rows and can be `Occupied` without changing `lifecycle`. Offered publication is distinct from Customer-bookable eligibility (#15). This field set/order is frozen **as a review proposal**, not peer-approved yet.

Proposed safe errors

Condition	Status / code	Required DB outcome
No valid Session	401 UNAUTHENTICATED	No mutation/private data
Customer tries Tutor routes	403 FORBIDDEN	No mutation
Client tries owner/role/endsAt/unknown extra POST keys	400 INVALID_INPUT	No mutation

Condition	Status / code	Required DB outcome
Any GET/POST query parameter, including forged `?tutorId=T2`	400 INVALID_INPUT	No foreign response or mutation
Malformed/offsetless/missing start, nonzero seconds/fraction, invalid calendar/offset	400 INVALID_INPUT	No mutation and no rounding
Valid but nonfuture start at post-lock DB clock	422 SLOT_INVALID_TIME	No mutation
Overlap of currently Offered same-Tutor Slots	409 SLOT_OVERLAP	Prior Slot unchanged
`GET /api/v1/tutor/slots/:id` (not a Sprint 1 route)	404 (unregistered route)	No Slot-detail lookup or foreign ownership side channel
Missing/wrong CSRF or Origin	403 CSRF_DENIED / ORIGIN_DENIED	No mutation
DB retry exhausted / unavailable	503 SERVICE_UNAVAILABLE	No false 201; uncertain responses reconciled via own GET
Unexpected internal failure	500 INTERNAL_ERROR	No raw SQL/stack/secrets

Errors use the existing envelope `{"error":{"code":"...","message":"..."},"requestId":"..."}`; proposed SLOT-specific codes require peer sign-off. A genuine exclusion SQLSTATE 23P01 is conflict, not infinite retry; 40001/40P01 may trigger bounded **whole-transaction** retry. Never claim exactly-once HTTP delivery without approved idempotency policy.

Review gate

Peer-review the **two-route-only** choice, minute-precision/no-rounding examples, exact allowlist and ascending order, explicit **no pagination/filter** policy, strict query rejection, trusted Tutor ownership, CSRF/Origin reuse and status mapping. Test anonymous, wrong-role, forged GET owner query, late own-list response after logout/account switch, stale cookies, overlap/race and no unsafe successful response. All tests planned, **not executed** in Part E.

E.2.5 F-S1-02 Slot Data & Transaction Contract (data-spec.md)

> **PROPOSED SQL, NOT APPLIED OR TESTED.** Issue #14. See SDS, HTTP, UI, STS and whole-product System SDS §§3.3, 7.1–7.2.

Existing DB baseline

Merged PR #26 includes Prisma `User`, `Session`, `AuthAttemptBucket`, `AuthAttemptReservation` and auth migration `20261104000000_auth`; it does **not** include Slot or Booking. Preserve merged auth migration unchanged. Add a **new forward-only Slot migration** and Prisma Slot model/User relation. Auth transactions use `pg` Pool, Prisma for schema/migration; one Slot mutation must use **one pg PoolClient**, not independent Prisma/pg transactions falsely described as atomic.

Proposed Slot model

Field	Type / policy
`id`	UUID PK, immutable
`tutorId`	UUID NOT NULL FK to `User.id`, owner from Session
`startsAt`, `endsAt`	TIMESTAMPTZ NOT NULL, exact one hour apart
`lifecycle`	Offered now, Withdrawn reserved for Sprint 2
`createdAt`	DB-generated timestamp
`withdrawnAt`	NULL for Offered; later Sprint 2 history

Tutor Schedule is `WHERE tutorId=principal.userId`, not a separate writable Schedule table. Once #16 exists, occupancy derives from active Booked rows, not a stored `isBooked` flag. **Sprint 1 read contract:** only the authenticated Tutor's full owner-list GET (no `:id` route, query filters or pagination); strict-query rejection prevents client-supplied `tutorId` from changing owner scope. Sort by `"startsAt" ASC,"id" ASC` and project **only** the approved-for-review DTO fields. `occupancy` is computed as Unoccupied at #14 because no Booking relation exists yet, and must become derived from active Booked rows in #16.

Proposed migration fragment (requires peer review and disposable DB proof)

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE "Slot" (
  "id" UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  "tutorId" UUID NOT NULL REFERENCES "User"("id") ON DELETE RESTRICT,
  "startsAt" TIMESTAMPTZ NOT NULL,
  "endsAt" TIMESTAMPTZ NOT NULL,
  "lifecycle" TEXT NOT NULL DEFAULT 'Offered'
    CHECK ("lifecycle" IN ('Offered','Withdrawn')),
  "createdAt" TIMESTAMPTZ NOT NULL DEFAULT now(),
  "withdrawnAt" TIMESTAMPTZ,
  CONSTRAINT slot_exactly_one_hour
    CHECK ("endsAt" = "startsAt" + INTERVAL '60 minutes'),
  CONSTRAINT slot_lifecycle_consistent
    CHECK (
      ("lifecycle"='Offered' AND "withdrawnAt" IS NULL)
      OR ("lifecycle"='Withdrawn' AND "withdrawnAt" IS NOT NULL)
    )
);
CREATE INDEX "Slot_tutorId_startsAt_idx"
  ON "Slot" ("tutorId","startsAt","id");
ALTER TABLE "Slot" ADD CONSTRAINT offered_slot_no_overlap_per_tutor
  EXCLUDE USING gist (
    "tutorId" WITH =,
    tstzrange("startsAt","endsAt",'[)') WITH &&
  )
  WHERE ("lifecycle"='Offered');

-- Sprint 1 ordinary-DML invariant: only new Offered Slots may be inserted.
-- Every UPDATE/DELETE/TRUNCATE is denied at the database, not just in Express.
CREATE FUNCTION slot_guard_s1() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'INSERT' THEN
    IF NEW."lifecycle" <> 'Offered' OR NEW."withdrawnAt" IS NOT NULL THEN
      RAISE EXCEPTION 'Sprint 1 only permits Offered Slot inserts'
        USING ERRCODE = '23514', CONSTRAINT = 'slot_s1_insert_lifecycle';
    END IF;
    RETURN NEW;
  END IF;
  RAISE EXCEPTION 'Slot mutation is forbidden in Sprint 1'
    USING ERRCODE = '23514', CONSTRAINT = 'slot_s1_immutable';
  RETURN NULL; -- Unreachable: every non-INSERT raises an exception.
END;
$$;

CREATE TRIGGER slot_s1_insert_guard BEFORE INSERT ON "Slot"
  FOR EACH ROW EXECUTE FUNCTION slot_guard_s1();
CREATE TRIGGER slot_s1_update_delete_guard BEFORE UPDATE OR DELETE ON "Slot"
  FOR EACH ROW EXECUTE FUNCTION slot_guard_s1();
CREATE TRIGGER slot_s1_truncate_guard BEFORE TRUNCATE ON "Slot"
  FOR EACH STATEMENT EXECUTE FUNCTION slot_guard_s1();
```

Notes: Auth migration already creates `pgcrypto`/UUID generation, but `btree_gist` installation rights must be verified on actual Postgres 17/18 and future deployment. Prisma cannot fully express conditional exclusion or PL/pgSQL guards: preserve native SQL in later migrations. Do **not** add a time-relative `CHECK startsAt>now()`, as a historical valid Slot eventually becomes past while remaining persisted. A FK alone does not enforce authenticated Tutor ownership: verify/lock User in the write transaction. The application exposes **no** update/delete/withdraw route in Sprint 1.

Database-level operation / transition matrix (P1 review decision)

Operation attempted by ordinary SQL/API	Sprint 1 result	Expected DB proof
`INSERT` as `Offered` with NULL `withdrawnAt`, future 60-minute interval, valid owner	**Allowed** only after all server authorization/time and native constraints; direct SQL has **no Session ownership proof**	New row committed if checks pass
`INSERT` with `Withdrawn` and a timestamp, or a non-NULL `withdrawnAt`	**Denied** by insert trigger	SQLSTATE **23514**, constraint `slot_s1_insert_lifecycle`; zero rows
`UPDATE` any existing row (including `id`, `tutorId`, `startsAt`, `endsAt`, `lifecycle`, `withdrawnAt` or no-op update)	**Denied** unconditionally by row-level trigger	SQLSTATE **23514**, constraint `slot_s1_immutable`; row unchanged
`DELETE` any Slot	**Denied** by row-level trigger	SQLSTATE **23514**, constraint `slot_s1_immutable`; row still present
`TRUNCATE` Slot	**Denied** by statement trigger; may additionally lack privilege for runtime DB role	SQLSTATE **23514** (owner in test) or **42501** (insufficient privilege), table unchanged
`Offered -> Withdrawn` or `Withdrawn -> Offered`	**Neither permitted in Sprint 1**; no `Withdrawn` row can be written normally	Attempted UPDATE rejected as above; direct reactivation fixture deferred until S2
Sprint 2 reviewed `Offered -> Withdrawn`	**NOT IMPLEMENTED**; requires a separate migration replacing the restrictive trigger, one-way transition with non-NULL `withdrawnAt` and an atomic no-active-Booking check	New S2 tests/review, not #14 PASS
Sprint 2 attempted `Withdrawn -> Offered` (same ID), change ID/owner/start/end, or DELETE	**Must remain denied** even after S2 one-way migration	Future S2 direct-SQL negative tests; **NOT RUN** in #14

The explicit SQLSTATE values above come from **proposed** `RAISE EXCEPTION` clauses, not a claim that this migration has been applied. PostgreSQL privileged DDL can disable/drop triggers; runtime role must not have schema/table ownership, superuser, or trigger-disabling privileges in a production-equivalent environment. Part F must verify the actual role/permission setup; a deployment that uses a schema-owner runtime connection cannot claim resistance to malicious DDL. Privileged test teardown must use an **isolated disposable DB/schema**, not `DELETE FROM "Slot"` that the invariant correctly rejects.

Transaction and Race A

1. Verify `requireSession`, Tutor role, Origin and Session CSRF at HTTP edge. Strictly parse **valid** RFC3339 start with explicit timezone: seconds exactly `:00`, optional fractional part 1–3 **zero digits only**; e.g. `13:17:00Z` or `13:17:00.000+07:00` permitted, `13:00:30` / `13:00:00.500` rejected **without rounding**, as are invalid civil dates/offsets. Derive +60m end on server.
2. Begin **SERIALIZABLE** transaction on one `pg.PoolClient`; don't reuse generic Auth `transaction()` helper that begins with default isolation.
3. Lock `User` row `SELECT id,role,isActive FROM "User" WHERE id=$1 FOR UPDATE` with `principal.userId`. Recheck active and Tutor role **after** lock. Current transaction's locking hierarchy matches System SDS User-before-Slot.
4. Sample fresh DB `clock_timestamp()` after lock. Require start strictly future; **do not impose Q05 Booking lead time** or unapproved working hours/horizon/grid.
5. Insert Offered row using trusted tutorId. Duration CHECK and same-Tutor Offered GiST EXCLUDE are last-resort DB safeguards, including two concurrent writers/instances. Distinct Tutors and adjacent half-open intervals remain allowed.
6. COMMIT **before** 201. On any failure, ROLLBACK and release. Map exclusion `23P01` to safe 409 Conflict; `40001` serialization / `40P01` deadlock eligible for **full fresh transaction** bounded retry (3 attempts proposed), not infinite retry of an actual overlap. DB unavailable/exhaustion -> safe non-success; uncertain client-side response can require owner GET reconciliation.

Verification obligations

- Fresh disposable DB: apply Auth then Slot migration, inspect native constraints and indexes, regenerate Prisma client; record DB version, actual SQLSTATEs and application behavior.
- Real Race A with two independent connections, explicit lock/barrier ensuring contention: capture A/B `pg_backend_pid()`, observe `pg_blocking_pids(B_pid)` **containing A_pid** from an independent observer while A holds the User lock; then release A. Require both winner orders, observed HTTP statuses and SQLSTATE (if thrown), committed post-race rows and invariant check. Also test direct SQL bypass of HTTP precheck; random `Promise.all()` alone is insufficient. Avoid a timeout-only or sleep-only "race" proof.
- Prove 59m duration direct SQL rejected, same-Tutor positive overlap rejected, adjacency and different-Tutor same hour accepted. Inject transactional fault, confirm zero partial commits and no 201.
- Check **own-list** SQL owner filter, strict-query denial of `GET /api/v1/tutor/slots?tutorId=T2`, response allowlist, no foreign Customer/Booking information, and no `GET :id` route. Test stale owner-list response arriving after logout/account switch (old T1 payload must never render under T2), role mutation, revoked session and CSRF failures.
- Retain current `apps/api/package.json` behavior but **wire new Slot DB tests** into `npm run test:db` discovery; current command enumerates Auth tests, so a new file could otherwise be omitted silently.
- Record exact `npm ci`, `db:generate`, `db:migrate`, `typecheck`, `build`, `test`, `test:db`, `test:e2e` and Race A results by SHA in Part F; **none are run by Part E**.

Review-required engineering decisions

Peer review the proposed `slot_guard_s1` trigger and operation matrix, SQLSTATE 23514 assertions, future one-way migration gate, actual runtime DB privilege separation, lock ordering and Serializable behavior; accept/reject no-detail/no-pagination/no-filter own-list and minute-precision transport. No blanket architectural refactoring of #13.

E.2.6 F-S1-02 Tutor Slot Software Test Specification (tests.md)

> **UX delta proposed 2026-11-05:** preserve SLOT-01…SLOT-20 below;
> add these *new, not-yet-executed-at-design-time* dashboard checks from
> the dated UX decision. The older Part E
> run ledger is historical; actual new executions belong in the Part F
> verification ledger, not this planned list.

ID	New dashboard evidence to collect
UX-R1	Logged-in Tutor sees original NutTiew identity, navy/orange shell, no fake data or actions; `บัญชีของฉัน` navigates to in-page `#account-details` showing **only role and shortened ID from trusted /auth/me**; Customer sees original authorized Auth page
UX-R2	Real PG Slot at Bangkok 13:17 and 13:59 positions at exact minute-offset, 23:30→00:30 splits over two dates; all 24 hours accessible
UX-R3	Prev/Next/Today and day picker change local view without changing GET URL; all returned Slot IDs remain in chronological agenda. **Overview count** equals full own-list Slot count across **all dates** and is constant across viewport navigation; **ช่วงที่แสดง** separately counts distinct Slots intersecting the current window (including cross-midnight only once per window)
UX-R4	Real app **320/375px = full mobile Agenda** (`<=620px`), **768px = two-day timeline** (`621–1060px`), **1280px = three-day timeline** (`>1060px`); check 620/621 and 1060/1061 boundaries; screenshots, 320 CSS px reflow/no unexpected horizontal scroll, keyboard-only nav and visible unobscured focus, valid labels/status, primary controls **>=44x44 CSS px** and applicable WCAG 2.2 AA target size **>=24x24 CSS px or normative exception**. Record separate **>=4.5:1 normal text**, **>=3:1 large text**, **>=3:1 UI/non-text** computed contrast for all critical states. Automated subset is not conformance; manual human WCAG 2.2 AA signoff **NOT RUN** until actually assessed
UX-R5	Re-run existing SLOT-09/12/14/15/18/20, including real 200-after-switch privacy and uncertain-POST reconciliation, with new shell
UX-R5-GET-01	Authenticated Tutor with 4 committed own Slots: initial GET 503 → retry GET 503 show loading/error and retry only, **no empty Agenda/Calendar/zero**; next genuine GET 200 restores all 4 Slot IDs, sorted and private
UX-R5-GET-02	Authenticated Tutor with **no** committed Slot: initial GET 503 → retry GET 503 must not claim an empty schedule; only authorized GET 200 with `data: []` may show true empty state and 0 total

ID	New dashboard evidence to collect
UX-R5-LOGOUT-01	T1 starts Logout: hold CSRF and Logout requests, then inject HTTP 503, network loss or timeout. **Immediately hide T1 account ID, signed-in status, Calendar, Agenda and private cards; block Create/header CTA and any new protected POST before and after uncertainty**. Attempt creation and verify **zero new POST /api/v1/tutor/slots after Logout begins**, no newly committed Slot row. A pre-Logout in-flight POST may have committed and is not asserted rolled back. A failed `/auth/me` recheck keeps content hidden/actions blocked; do not falsely claim Logout succeeded
UX-R5-LOGOUT-02	After T1 uncertain Logout, retry Logout or `/auth/me` session check. A **fresh /auth/me 200 matching exact internal T1 ID and Tutor role** must be followed by fresh owner-list GET 200 **before** restoring T1 display or Create; failed owner GET stays hidden/blocked. Fresh Logout 204 or `/auth/me` 401 finalizes signed-out; different account or recheck 503/network stays blocked; stale T1 authorized 200 never repopulates hidden content
UX-R6	Real rendered screenshots from DB-backed Tutor account; no invented data or unimplemented domain action

Provenance / test gate: the above are **Part E planned acceptance checks**, not outcomes of this docs-only PR. The implementation PR must link these stable IDs to real Playwright/DB evidence with RED→GREEN for unsafe Logout and GET states, rerun the original SLOT-01…20 and Auth regressions, and record exact commit SHA and hosted CI. Do not mark this design accepted before peer review and merge.

WCAG evidence gate: the Part F verification report must distinguish (a) automated measured contrast/keyboard/viewport samples, (b) manual inspection of all applicable WCAG 2.2 A/AA criteria including SC 1.4.3/1.4.10/1.4.11, 2.1.1, 2.4.7/2.4.11 and 2.5.8, and (c) explicit NOT RUN items with reviewer, date and exact implementation SHA. The old handful of 4.5:1 samples does **not** establish WCAG 2.2 AA.

For UX-R4, validate the CTA's full rendered pairing (not just its label): `#172137` text/icons on `#F47736` fill (target calculated **5.76:1**), `#D96123` visible border versus both `#F7F8FB` canvas and white (calculated **3.48:1 / 3.69:1**), and `#AA4519` `3px` offset keyboard focus outline against both backgrounds (calculated **5.53:1 / 5.87:1**). Verify hover, disabled, keyboard, zoom and any additional component states using **actual computed browser colors**. White text on orange and borderless orange on canvas/white are **not accepted**. This is a future Part F test obligation, not a test PASS recorded by this Part E PR.

> **NEW DASHBOARD UX TESTS PLANNED / NOT RUN IN THIS DOCS-ONLY PR.** The original SLOT-01…SLOT-20 design and earlier implementation/tests from merged PR #28 are historical, **not missing** in today's `sprint1`. Issue #14, owner @Tanaboonnnnn, reviewer @Chxtamos. Source contracts: SDS / owner policy decision, HTTP, UI, Data / DB trigger matrix.
>
> **Historical baseline only:** `sprint1` `6fd3931` was the **original Part E snapshot before Slot implementation** and had Auth tests but no Slot code/migration/test. **Current PR #30 base:** `sprint1` `ed451be2ca3e366bdfc1900f4c4697aa7e50b6a0` **already contains** the original Slot API, DB migration and tests from merged PR #27/#28. The new dashboard/Logout cases above are not claimed executed by this documentation PR.

Test harness, data and TDD evidence

Use **clean disposable** real PostgreSQL with Auth migration plus **new** Slot migration, test-only accounts T1/T2 (Tutor) and C1 (Customer), authenticated session cookies and session-bound CSRF, Supertest/Vitest, at least two independent `pg` connections plus an observer connection, Playwright for real Tutor flow. Test dates are future relative to **database authoritative time**, not hardcoded as perpetually valid. Never commit passwords/keys. Test names must match stable AC/VC and execute through actual npm scripts. Because Slot direct DELETE/TRUNCATE is intentionally blocked, isolate and roll back fixtures or drop an **entire test-owned DB/schema as privileged teardown**; do not depend on deleting persisted production-like Slot rows for test cleanup.

Record **RED** first (missing route/schema/constraint causes expected failure), **GREEN** after implementation, command, exact SHA, PG/Node version, migration revision, PASS/FAIL/SKIP and final DB rows. UI-only warnings and `Promise.all()` without verified contention do not prove Race A.

Original Slot STS cases — historical Part E test plan

The `SLOT-01` through `SLOT-20` cases below were **planned before the

original Slot implementation**, and many were subsequently exercised

by the reviewed Part F work (PR #28). This section records the original

expected observations, not the current execution state.

For actual command output, PASS/FAIL/rerun counts and limitations at

their recorded commit SHAs, see the separate

original Slot Part F verification ledger.

The **new Dashboard UX and Logout tests above are separately PLANNED

and have not been executed by this docs-only PR**.

STS	Trace IDs	Setup/action	Required observable result
SLOT-01	AC-S1-08, VC/TEST-S1-08, VAL-003	T1 authenticated, POST future Bangkok 13:00	201, one DB Offered Slot owned T1, end exactly 14:00
SLOT-02	AC-S1-08, VC/TEST-S1-14, VAL-031	T1 GET own list (no filters), refresh/restart API with same DB	Same committed Slot ID and UTC times, own account from `/auth/me`; exact safe DTO, `startsAt,id` ascending, no in-memory substitute
SLOT-03	AC-S1-09, VC/TEST-S1-09, VAL-004	T1 already offers 13:00–14:00, proposes 13:30–14:30	409; old row unchanged, no second row
SLOT-04	AC-S1-10, VC/TEST-S1-10	T1 creates 13:00–14:00 then 14:00–15:00	Both rows committed; adjacency permitted

STS	Trace IDs	Setup/action	Required observable result
SLOT-05	AC-S1-11, VC/TEST-S1-11	T1/T2 both create same 13:00–14:00 future window	Both succeed, distinct owner rows
SLOT-06	AC-S1-12, VC/TEST-S1-12	Missing/invalid/offsetless timestamps, invalid civil date/offset, past/equal-now, extra client owner/duration	Defined 400/422; no new/modified row; no silent timestamp normalization
SLOT-07	AC-S1-05/06, VC/TEST-S1-05/06, AUTH-07/08	Anonymous/Customer tries Tutor POST and protected GET	401/403, no private response or write
SLOT-08	AC-S1-07, VC/TEST-S1-07/31, AUTH-09/16	T1 GET `/tutor/slots?tutorId=T2`, T1 GET unregistered `/:id` for T2/missing UUID, T1 POST forged `tutorId` / `role`	GET forged query **400**, no T2 rows; no detail route, both detail paths 404/no Slot payload; strict POST 400, T2 unchanged
SLOT-09	AC-S1-14, VC/TEST-S1-13, VAL-013	Open same Slot in Bangkok, UTC and Tokyo browser zones; cross midnight	Same intended Bangkok date/hour, exact 60m, same UTC instant
SLOT-10	AC-S1-13, **VC/TEST-S1-37**, VAL-005	Two same-Tutor overlapping creates with deterministic barriers; capture A/B `pg_backend_pid()` and verify observer `pg_blocking_pids(B_pid)` includes A	At most one conflicting Offered row committed; loser non-success; log true lock wait, HTTP outcomes, SQLSTATE and no ghost row
SLOT-11	AC-S1-13, VC/TEST-S1-37	Reverse A/B winner order using two API instances/independent DB connections; direct SQL bypass of HTTP guard	Native exclusion prevents overlap; record observed blocker/wait, SQLSTATE (often 23P01/40001 depending order), committed rows and exact winner
SLOT-12	NFR-S1-11, BR-S1-18/034	Inject SQL failure before commit, simulate DB unavailable	Rollback, zero partial data, safe 503/non-success, no false 201
SLOT-13	NFR-S1-01/08, AUTH-13/16	Invalid/missing CSRF, wrong Origin, revoked cookie	Denied, DB unchanged; sanitized error without secrets
SLOT-14	AC-S1-08/14, FR-S1-19	Real Tutor UI login→create→refresh→full own-list (no detail/filter/page); process restart	Database-backed end-to-end path persists; safe DTO/occupancy (Unoccupied at #14), deterministic order
SLOT-15	Proposed NFR-S1-09/VAL-033	Keyboard, contrast/targets, viewports 320/375/768/1280; 409/422/503	Visible focus/alerts, no overflow, error recovery and no false success
SLOT-16	SLOT-I1/I2, VAL-004/005	Direct SQL 59-minute insert, overlap insert; inspect pg constraint/index	CHECK and GiST exclusion deny invalid committed rows
SLOT-17	SLOT-I1/I3/I4/I7	Lock Tutor, deactivate/role change; direct `INSERT Withdrawn`, `UPDATE` each of `id/tutorId/startsAt/endsAt/lifecycle/withdrawnAt` and a no-op update, `DELETE`, `TRUNCATE`	API rejects invalid owner; S1 DB trigger rejects INSERT Withdrawn/UPDATE/DELETE with **23514** and identified constraint; TRUNCATE **23514/42501**, unchanged committed row counts; no S2 reactivation PASS claim
SLOT-18	SLOT-I5, retry safety	Duplicate POST/unknown network outcome	At most one overlapping offer, GET used for reconciliation, no false exactly-once promise

STS	Trace IDs	Setup/action	Required observable result
SLOT-19	AC-S1-12/14, VC/TEST-S1-12/13, OD-003	Send RFC3339 future `13:00:00` / `13:00:00.000` / `13:17:00` with offsets, and `13:00:30` / `13:00:00.500`, leap-day/invalid-day, missing offset; run in UTC/Tokyo browser zone	Accept second/fraction **zero** and **any minute**; 400 for nonzero seconds/fraction/invalid civil time; no rounding/DB row; Bangkok hour unchanged
SLOT-20	AC-S1-07, AUTH-09/16, privacy	Hold a successful T1 owner-list GET; logout or switch to T2 while pending; release old T1 response after T2/current signed-out state	Old T1 Slots **never reappear**; T2 own-list contains only T2; stale GET cannot update protected view after session change

Deterministic Race A procedure

1. Open A/B on **independent** PostgreSQL sessions. Both target overlapping Offered intervals of T1 on an approved future Bangkok date. Prepare T1 identity/auth fixtures.
2. A begins SERIALIZABLE, captures `SELECT pg_backend_pid() AS a_pid` and locks T1 `User FOR UPDATE`; hold A at an **explicit test-only barrier** before COMMIT. B begins with an independently checked-out `pg.PoolClient` / second API instance; a test-only instrumentation seam records B's `pg_backend_pid()` **before** it attempts the same User lock. This seam must not become a production HTTP test hook.
3. Using an independent observer connection, repeatedly query `SELECT pg_blocking_pids($1::int) AS blockers` with B PID under a **bounded test timeout**, until it includes A PID (optionally corroborate `pg_stat_activity.wait_event_type='Lock'`). **Fail the race test** if no actual blocking pair is observed; do not proceed on guessed timing/sleep alone.
4. Release A, let it INSERT+COMMIT, then allow B to resume/full-transaction retry. B either returns a safe conflict or aborts with an attributable SQLSTATE, never commits overlapping Offered state. Record both request statuses (201 vs 409/transaction error as contracted), observed SQLSTATE(s), A/B backend PIDs, blocker observation and retry count.
5. From a **fresh post-commit observer connection**, query `SELECT id,"tutorId","startsAt","endsAt","lifecycle" FROM "Slot" WHERE "tutorId"=$1 ORDER BY "startsAt",id` and independently assert no pair of Offered half-open ranges intersects; record row IDs/counts. Reverse winner order; repeat direct SQL inserts on separate connections that bypass Express to prove GiST backstop. Controls: same-Tutor adjacency and different-Tutor same-hour must both pass.
6. Inject a DB failure before COMMIT and verify rollback plus safe response; restart API and re-read committed rows.

Important: Test with real PostgreSQL, not SQLite, only ORM mock, or HTTP status without DB assertions. A `23P01` conflict is not a successful retry; bounded `40001` / `40P01` retries start new transactions and must still fail an actual overlap. Tests **must** prove B waited on A, not only that two requests were launched.

Original privacy and DB immutability procedures (historical Part E plan)

The procedures below were originally written **before the Slot feature

was implemented**, not as a claim that the original privacy or

immutability tests are still unexecuted. Actual Slot Part F evidence,

including executed SLOT-17/19/20 cases and residual manual/production

verification gaps, is recorded in the

verification ledger. Do not infer new

Dashboard UX acceptance from these historical checks.

- **SLOT-08:** authenticate as T1. Request `GET /api/v1/tutor/slots?tutorId=T2`, `?page=1` or another unknown query. All return 400 INVALID_INPUT and no foreign rows; own unfiltered GET returns only T1 rows. `GET /api/v1/tutor/slots/{T2SlotId}` and an unknown UUID both return 404 because **there is no detail route**, not because a foreign record was looked up. A forged `tutorId` in POST is a 400. No query can change `WHERE tutorId=principal.userId`.
- **SLOT-20:** with T1 signed in, intercept/hold a genuine successful T1 GET while the component is mounted. Trigger Logout (or switch auth identity to T2), clear protected state and issue T2 own-list if applicable. Release the **old 200** response **after**

logout/new T2 success. Assert no T1 row IDs/time appear; verify T2 can see only T2 data. Repeat with a 401 or network failure from the stale request.

- **SLOT-17:** on an Offered fixture row use real SQL to attempt UPDATE ID, owner, start, end, lifecycle, withdrawnAt, and no-op UPDATE; ordinary UPDATE/DELETE must throw **SQLSTATE 23514 /** `slot_s1_immutable`; INSERT Withdrawn must throw **23514 /** `slot_s1_insert_lifecycle`; TRUNCATE must throw **23514** (or runtime **42501**). Compare pre/post committed row images/count. **Withdrawn→Offered cannot be executed in Sprint 1** because Withdrawn creation and updates are forbidden; mark it **DEFERRED TO SPRINT 2**, not PASS. Sprint 2 migration must explicitly permit one-way Offered→Withdrawn and separately prove no reactivation. Check production-equivalent DB runtime credential cannot disable the protective trigger via DDL; if it can, report the privilege gap rather than claiming immutable DB protection.
- **SLOT-19:** check that the UI produces `HH:mm -> HH:mm:00.000+07:00`, API validates full real civil date+offset, accepts every minute (including 13:17), rejects 13:00:30/13:00:00.500 without rounding and that cross-midnight +60m retains date meaning under UTC/Tokyo browser emulation.

Feature integration limits

AUTH-07/08/09 and domain part of AUTH-16 are exercised with actual Slot routes in #14. Customer availability and stale selection belong to #15; Booking cutoff Q05 and races B/C belong to #16; Customer-private Booking reads belong to #17; final integrated `sprint1→main` release to #18. Do not report those as #14 PASS.

Original pre-Part-F run checklist — historical planning snapshot

**The status cells below describe the original pre-implementation

checklist, not the present Slot feature or its current PASS/FAIL state.**

The original test commands and Race A/immutability/precision/privacy

cases were subsequently executed in Part F; the authoritative

observed outcomes, any failures, reruns and limitations are in the

original Slot Part F verification ledger.

The new Dashboard UX-R1…R6, GET/Logout acceptance tests, and any

future regression reruns must be documented **separately** in their

own Part F evidence. Human WCAG signoff remains **NOT RUN** until

performed, and any stakeholder policy approval remains a separate gate.

Original planned command/check	Historical Part E state — consult linked Part F ledger for actual result
`npm ci` / `npm run db:generate` / `npm run db:migrate` (fresh DB)	Planned in original Part E; check actual Part F ledger
`npm run typecheck` / `npm run build`	Planned in original Part E; check actual Part F ledger
`npm test`, `npm run test:db` with Slot test discovered	Planned in original Part E; check actual Part F ledger
`npm run test:e2e` including Tutor UI	Planned in original Part E; check actual Part F ledger
Real DB Race A observed `pg_blocking_pids()` / direct SQL / rollback / restart	Planned in original Part E; check actual Part F ledger
SLOT-17 immutability trigger and Sprint 1 mutation matrix	Planned in original Part E; check actual Part F ledger
SLOT-19/20 precision and stale protected GET tests	Planned in original Part E; check actual Part F ledger
Human WCAG/keyboard/contrast signoff (separate manual gate)	**NOT RUN** for the new Dashboard; no human conformance claim
Stakeholder/instructor OD policy approvals	Separately gated; no approval implied by test execution

Original pre-code review gate (historical): @Chxtamos reviewed

the original Slot Part E contract through PR #27 before the Slot

implementation in PR #28. For the **new** Tutor Calendar Dashboard,

the amended Part E in PR #30 still requires independent approval and

merge **before** a new code-only Part F PR; this historical checklist

does not satisfy that new gate.

E.3 Shared Booking Data Foundation (D1-08A)

E.3.1 D1-08A Shared Booking Data Foundation SDS (specification.md)

> **PROPOSED CONTRACT — awaiting human peer review.** Issue #33. Contract owner: @Chxtamos. Required peer reviewer / F-S1-04 owner: @Tanaboonnnnn. This Part E change is documentation only. It does not add a Prisma model, migration, endpoint, UI, fixture, or test implementation; it does not claim that any database test has run.

1. Purpose and authority

Define one shared, durable PostgreSQL/Prisma Booking representation for:

- #15 — F-S1-03 Customer Availability, which reads persisted Bookings to exclude occupied Slots and the authenticated Customer's overlapping appointments.
- #16 — F-S1-04 Exclusive Booking, which owns Booking creation and the application transaction/lock/retry protocol. The shared database guard trigger is owned by #33 Part F, not duplicated in #16.

The approved Part E contract for #15 selected Route B: this shared foundation must be reviewed and merged separately before either feature implements Booking-dependent behavior. This document is the proposed contract for that review; it is not itself approval.

Source hierarchy:

1. Sprint 1 requirements: Q01, Q05, Q28, Q30; FR-S1-10..13; BR-S1-08..10, BR-S1-13, BR-S1-19; AC-S1-19, AC-S1-25, AC-S1-26, AC-S1-35; NFR-S1-03..05 and NFR-S1-12.
2. System SDS: §§3.2–3.3 (entities and native PostgreSQL constraints), §4 (time/state semantics), §§7.1 and 7.3 (lock/transaction responsibilities), §7.5 (real-database verification).
3. #15 accepted Feature SDS and data contract and data spec, merged by PR #32.
4. #16 issue scope. Its own Part E contract must still define the Booking API/transaction and OD-013 retry behavior; this foundation does not pre-empt those choices.

2. Verified baseline at the Part E branch point

This contract branch is based on `sprint1` commit `98d984169fff6b8ec0ff467c15abb441ca5cadd7` (PR #32 merge).

- `apps/api/prisma/schema.prisma` contains `User`, `Session`, `AuthAttemptBucket`, `AuthAttemptReservation`, and `Slot`; there is no `Booking` model.
- Existing migrations are `20261104000000_auth` and `20261105000000_tutor_slots`; neither creates a Booking table.
- The Auth migration installs `pgcrypto`. The Tutor Slots migration installs `btree_gist`, creates the immutable Sprint 1 Slot table, and enforces same-Tutor Offered-Slot non-overlap.
- `Slot` currently has UUID `id` and `tutorId`, `startsAt` / `endsAt` as `TIMESTAMPTZ(6)`, `lifecycle` defaulting to `Offered`, `createdAt`, and nullable `withdrawnAt`. Sprint 1's Slot triggers reject ordinary updates/deletes and `TRUNCATE`; the current migration does not enable a Withdrawn lifecycle transition.
- `User.role` is persisted as text with the existing Customer/Tutor constraint. A foreign key to `User` alone does not prove that the referenced account has the Customer role or is active; the application must check both using the authenticated principal.

- The System SDS targets PostgreSQL 18.x. No database migration or PostgreSQL compatibility test is executed by this documentation-only PR; the actual database version and `btree_gist` privileges must be recorded during Part F.

Do not edit, squash, replace, or regenerate the existing Auth/Slot migrations as part of this foundation.

3. Proposed shared data contract

The canonical field list, relation semantics, SQL constraints, and Prisma mapping are in data-spec.md. The core decisions proposed for peer acceptance are:

ID	Proposed contract	Rationale / trace
BDF-D01	`Booking.id` is a server-generated UUID primary key and an **application-level immutable identity**. PostgreSQL enforces primary-key uniqueness and non-nullability but the primary key alone does not reject an ID-changing `UPDATE`. Sprint 1's append-only guard separately rejects Booking `UPDATE` / `DELETE` / `TRUNCATE` as lifecycle boundaries; this is not a dedicated ID-immutability guarantee or ID-specific trigger.	Stable identity for persistence and later authorized retrieval; FR-S1-10, FR-S1-14.
BDF-D02	`customerId` is a required FK to `User.id`; delete/update is restricted. The service additionally verifies `role = Customer` and `isActive = true` from the authenticated Session.	Preserve ownership and do not trust client identity; FR-S1-04/10, BR-S1-13.
BDF-D03	`currentSlotId` is a required FK to the selected `Slot`. The current Tutor is derived from `Slot.tutorId`; Booking does not duplicate `tutorId`.	Avoid two mutable sources of Tutor ownership; System SDS §3.2.
BDF-D04	Persist `startsAt` and `endsAt` as `TIMESTAMPTZ(6)` snapshots copied from the selected Slot, never accepted from the client. A composite FK `(currentSlotId, startsAt, endsAt) → Slot(id, startsAt, endsAt)` enforces that the snapshot matches the immutable Slot.	Required by System SDS §3.3 and #15 data-spec; prevents a Booking interval drifting away from its Slot.
BDF-D05	`status` is text constrained to exactly `Booked` or `Cancelled`; new Sprint 1 Booking writes begin as `Booked`. Only `Booked` is active/occupying. `Cancelled` is retained for the shared state model and isolated DB tests; no cancellation workflow is added in Sprint 1.	Q30, BR-S1-08/09/10. `Past` is derived display classification, not a status.
BDF-D06	Intervals are valid only when `startsAt < endsAt` and use half-open `[start,end)` semantics.	BR-S1-09/10; adjacent appointments are allowed.
BDF-D07	Parent deletion and primary-key mutation are restricted for both Customer and Slot references.	Preserve Booking history and stable references; no deletion/retention policy is invented.
BDF-D08	PostgreSQL partial unique and exclusion constraints are the final integrity boundary for active Slot exclusivity and Customer overlap.	FR-S1-12/13, BR-S1-08/09/19, NFR-S1-04.
BDF-D09	Add a forward-only migration; current baseline has no Booking rows, so no Booking backfill is required. Existing Auth/Slot data and constraints must remain unchanged.	Safe migration from the verified `sprint1` baseline.
BDF-D10	Store `createdAt` and `updatedAt` as `TIMESTAMPTZ(6)`. `createdAt` is DB-defaulted; Prisma `@updatedAt` maintains `updatedAt` for application updates. These are metadata, not booking eligibility clocks.	Consistent with the System SDS entity model; Q05 continues to use its separate authoritative `T_elig`.
BDF-D11	**#33 Part F owns one versioned `booking_guard_s1()` function and its two PostgreSQL trigger registrations** in the shared Booking migration (remediated in Issue #39): a row-level `BEFORE INSERT OR UPDATE OR DELETE` trigger and a statement-level `BEFORE TRUNCATE` trigger. INSERT enforces the System SDS User → Slot lock hierarchy at the database level by acquiring `User(customerId) FOR UPDATE` before locking `Slot` and before index insertion in `BEFORE ROW` trigger, eliminating direct-SQL GiST exclusion index contention, reverse wait, and `40P01` deadlock for concurrent application transactions; it then applies the Offered-Slot/Q05 guard after the Slot lock; every UPDATE/DELETE/TRUNCATE is rejected with SQLSTATE `0A000`. **#16 owns the application transaction adapter**, including the System SDS User → Slot → Booking lock order, Serializable transaction and no-post-guard-contention proof or whole-transaction retry. #16 must not create a duplicate Booking guard.	System SDS §§3.3, 7.1 and 7.3; BR-S1-07/13/18/19; AC-S1-25/26/35; VC/TEST-S1-25/26/39; Issue #39.

ID	Proposed contract	Rationale / trace
BDF-D12	**Sprint 1 Booking is append-only after INSERT:** no row `UPDATE`, `DELETE`, or table `TRUNCATE`. #33 Part F enforces this with the row-level and statement-level guards in BDF-D11, following the existing Slot migration pattern. Deployment must separate the application runtime role from the migration/table-owner role; runtime is non-owner/non-superuser, has no `UPDATE` / `DELETE` / `TRUNCATE` or DDL/trigger-management privileges, and cannot bypass triggers through replication-role settings. No routine application/admin deletion exception is part of Sprint 1. BDF-24..26 verify the runtime role, both destructive operations, and final committed occupancy.	Preserve committed Booking history and derived Slot occupancy; prevent direct-SQL deletion from releasing a Slot without an approved lifecycle action.

All BDF-D01..D12 entries are **proposals awaiting human review**, not stakeholder decisions or accepted implementation facts. BDF-D11 resolves trigger ownership; BDF-D12 makes the Sprint 1 append-only/role boundary explicit. Both must be accepted by the consuming feature owners before any migration is authored.

4. Integrity and responsibility boundaries

4.1 Database-owned invariants

The database must reject:

- missing or invalid Customer/Slot references;
- a non-positive Booking interval or a snapshot that differs from its referenced Slot;
- an unsupported Booking status;
- a second `Booked` row for the same `currentSlotId`;
- any positive-duration overlap between `Booked` intervals belonging to the same `customerId`, including across different Tutors;
- every Booking `UPDATE` or `DELETE` in Sprint 1, and every Booking-table `TRUNCATE`, including direct SQL.

`Cancelled` rows do not occupy a Slot and do not participate in the Customer-overlap exclusion. A `Booked` row remains Active after its end time; elapsed time does not cancel or release it.

4.2 Application and transaction responsibilities

- #16 derives `customerId` from the live authenticated Session, reads the selected Slot and its actual Tutor from PostgreSQL, copies the persisted Slot interval, and sets the initial status to `Booked`. Client-supplied Customer/Tutor IDs, status, interval, or cutoff timestamps are not authoritative.
- The database FK does not enforce `User.role` or `isActive`; #16 must validate those within its agreed transaction/locking protocol.
- The unique and exclusion constraints protect committed state even when concurrent requests arrive through separate API processes. Application error mapping must treat constraint/serialization conflicts as non-success and must never report a Booking before commit.
- **#33 Part F owns the database guard artifact.** Its forward-only migration installs one `booking_guard_s1()` function with a row-level `BEFORE INSERT OR UPDATE OR DELETE` trigger and a statement-level `BEFORE TRUNCATE` trigger, following the existing Slot migration's row/statement guard pattern. The function rejects every `UPDATE` and `DELETE` at entry with SQLSTATE `0A000`, including `Cancelled → Booked`, `Booked → Cancelled`, moves and metadata updates; the statement trigger rejects every `TRUNCATE` with the same SQLSTATE. For a `Booked` INSERT, the function first acquires `User(customerId) FOR UPDATE` in `BEFORE ROW` trigger (enforcing the System SDS User → Slot lock hierarchy for direct-SQL writers before tuple insertion into unique or GiST exclusion indexes, preventing reverse wait and `40P01` deadlocks), then locks and re-reads the referenced Slot and rejects a missing or non-`Offered` Slot; it samples PostgreSQL `clock_timestamp()` only after acquiring both locks and enforces Q05 (`startsAt - T_elig >= 60 minutes`). The composite FK remains the final interval-snapshot equality constraint. The app runtime role must be separate from the migration/table owner, non-owner/non-superuser, lack UPDATE/DELETE/TRUNCATE and DDL/trigger-management privileges, and have no trigger-bypass capability. No Booking API or application service is added to #33.
- **#16 owns application transaction coordination, not a second copy of the trigger.** Its Part E contract must specify the System SDS global lock order: all affected User rows (Customer and selected Slot's actual Tutor) in ascending UUID order, then all affected Slot rows in ascending UUID order, then existing Booking rows in ascending UUID order; use Serializable

isolation as specified. The service must acquire the Slot lock before the guarded INSERT so the trigger re-locks an already-held row. #16 must prove no conflict-capable unique/exclusion wait occurs after `T_elig`; if it cannot prove that invariant, it must abort/retry the entire transaction and sample a fresh DB clock. The trigger and indexes are database backstops, not substitutes for the application lock protocol.

- `Booking.id` immutability remains an application-level identity invariant: #16 generates it server-side and any future supported update path must not accept a replacement ID. The PostgreSQL primary key alone is not an immutability guard. Sprint 1's blanket Booking mutation prohibition (UPDATE/DELETE/TRUNCATE) is a separate lifecycle rule, not a dedicated ID-immutability trigger or ID-specific DB test.
- **Sprint 1 Booking rows are append-only:** the database rejects UPDATE/DELETE/TRUNCATE, while the application runtime role is not permitted to UPDATE/DELETE/TRUNCATE or alter/disable the guards. The migration/table-owner role is separate and is not used by API runtime processes.
- #15 is read-only: it consumes `status = Booked`, `currentSlotId`, and the persisted interval for availability predicates. It must not create/update Booking records or add `Slot.isBooked`.

4.3 Explicit exclusions

No Booking HTTP route/controller/service, UI, private Booking retrieval, cancellation/reschedule operation, Booking deletion/truncation workflow, `RescheduleHistory`, `Notification`, cancellation actor/reason, idempotency registry, payment, attendance, `Completed` state, or Slot withdrawal workflow is included. OD-013 retry-response semantics remain owned by #16. Issue #15's OD-002 60-day availability horizon is not a Booking column or a change to Q05.

5. Migration and compatibility plan

Part F of #33 may begin only after the Part E contract is accepted by both consuming feature owners **and** the #16 Part E transaction contract has been reviewed so the trigger and application use one agreed lock protocol.

1. Accept this Part E contract with both #15 and #16 feature owners.
2. Complete and peer-review #16 Part E's application transaction contract, including the §7.1 User → Slot → Booking lock order, Serializable behavior, retry semantics and the real-DB no-post-guard-contention proof strategy. This is a design dependency, not permission to implement the Booking API early.
3. Start from the then-current remote `sprint1`; re-check the schema/migration baseline. Add a new timestamped Prisma migration and matching model/relations; do not rewrite or regenerate existing Auth/Slot migrations.
4. In that **#33 Part F migration**, create the shared Booking table/constraints and the `booking_guard_s1()` function with row-level INSERT/UPDATE/DELETE and statement-level TRUNCATE trigger registrations described in BDF-D11/D12 and data-spec.md §7. Provision a separate non-owner application runtime role with no UPDATE/DELETE/TRUNCATE or DDL/trigger-management privileges; do not create a duplicate Booking guard in #16.
5. Confirm `btree_gist` is installed and usable by the migration role on the actual target PostgreSQL version. The current Slot migration already installs it; any repeated extension statement must be idempotent and must not drop the extension.
6. Run **BDF-01..BDF-22 and BDF-24..BDF-26 as the #33 pre-merge foundation proof gate** on real PostgreSQL, including fresh/upgrade migration tests, direct-SQL Offered/UPDATE/DELETE/TRUNCATE-guard tests, Q05 boundary and post-lock wait tests, synchronized DB-level race tests, runtime-role privilege checks, and inspection of final committed rows. Record PostgreSQL version, extension version/availability, exact commands, source SHA, and PASS/FAIL/SKIP/BLOCKED results. **BDF-23 is not part of this pre-merge gate** because it requires the complete #16 application transaction adapter.
7. Review and merge the #33 Part F migration into `sprint1` only after BDF-01..BDF-22 and BDF-24..BDF-26 pass; record its exact merged schema SHA in Issues #15 and #16. Only then may #15/#16 implement Booking-dependent application behavior against that accepted SHA.
8. After the merged schema SHA exists and the #16 Part F transaction adapter is implemented, #16 runs **BDF-23 as the joint post-merge integration gate** using the actual application adapter and real PostgreSQL barriers. Record the evidence in #16 and cross-link it from #33; do not claim BDF-23 PASS before this point. BDF-23 does not block the #33 migration PR from merging.
9. The complementary Slot lifecycle trigger for `Offered → Withdrawn` / no reactivation is a separate Sprint 2 migration and **must exist before any Sprint 2 withdrawal workflow is enabled**; it is not part of this Sprint 1 Booking foundation.

6. Peer-review checklist and open integration gate

The required reviewer @Tanaboonnnnn is also the provisional owner of #16, so this review must explicitly confirm that the shared fields/constraints satisfy both #15 and #16. Please review:

- [ ] BDF-D01..D12: field list, status semantics, interval snapshot, timestamps, ID-immutability boundary, trigger ownership, append-only DELETE/TRUNCATE guard and no duplicated Tutor ownership.
- [ ] Both FK delete/update policies and the composite Slot-window FK.
- [ ] Partial unique index and GiST exclusion semantics, including concurrent writes and adjacent intervals.
- [ ] `btree_gist` availability/permissions on the actual target PostgreSQL deployment.
- [ ] BDF-D11/D12: #33 Part F owns the shared guard function, row/statement triggers and append-only policy; #16 owns the application lock/transaction/retry adapter. Confirm the exact merge sequence and no duplicate guard is planned.
- [ ] #16 Part E records the System SDS User → Slot → Booking lock order and no-post-guard-contention / whole-transaction retry strategy before #33 Part F migration implementation.
- [ ] #33 Part F passes BDF-01..BDF-22 and BDF-24..BDF-26 before migration merge, including Offered/Q05, UPDATE/DELETE/TRUNCATE guards, `Cancelled → Booked`, Q05 60:00/59:59, post-lock cutoff crossing and runtime-role privilege boundary.
- [ ] #16 Part F owns BDF-23 after the #33 schema SHA is merged; this joint adapter proof is cross-linked from #33 and is not a pre-merge blocker for the #33 migration PR.
- [ ] No accidental Sprint 2 workflow, API/UI, `Slot.isBooked`, or OD-013 retry decision.

P1 resolution proposed: #33 Part F owns the one Booking guard function plus row-level INSERT/UPDATE/DELETE and statement-level TRUNCATE trigger registrations; it must pass BDF-01..BDF-22 and BDF-24..BDF-26 before its migration PR merges. The row/statement guards reject UPDATE/DELETE/TRUNCATE, while the separate non-owner app runtime role has no destructive DML or trigger/DDL bypass privileges. BDF-24/25 prove that committed Booking rows and derived occupancy survive attempted DELETE/TRUNCATE; BDF-26 proves the runtime-role boundary. #16 Part E/F owns the application lock order, transaction adapter and no-post-guard-contention / full-retry proof; #16 runs BDF-23 only after the #33 schema SHA is merged. The order is #34 Part E acceptance → #16 Part E acceptance → #33 Part F migration + BDF-01..BDF-22 and BDF-24..BDF-26 proof/merge → #15/#16 application implementation → #16 BDF-23 joint integration proof. BDF-23 is not a prerequisite for the #33 migration PR merge. This remains a proposed contract pending human acceptance; no implementation or test result is claimed here.

7. Traceability

Foundation invariant	Requirements / tests
One active Booking per Slot	FR-S1-12, BR-S1-08/19, AC-S1-25, BDF-10/11
One Customer cannot overlap active intervals across Tutors	FR-S1-13, BR-S1-09/10/19, AC-S1-26, BDF-12/13/14
Slot interval snapshot and valid references	FR-S1-10/16, System SDS §3.3, BDF-05..09
Cancelled is non-occupying; Past is not a state or completion	Q30, BR-S1-14, BR-S1-22, BDF-04/15
#33 pre-merge DB durability, concurrency and append-only evidence	NFR-S1-03/04/05/12, BDF-01..22 and BDF-24..26
#16 post-merge application/DB contention integration	System SDS §§7.1/7.3, BDF-23 (after #33 schema SHA)
Availability read predicates	#15 AVA-02, AVA-15, AVA-16, AVA-17 after the merged schema SHA is recorded

8. Part E Definition of Done

- [ ] The required human peer reviewer has reviewed and accepted the common data shape and invariants for both #15 and #16.
- [ ] BDF-D11 trigger ownership, BDF-D12 append-only/role boundary, merge order, lock-order dependency and real-PostgreSQL proof gates are explicitly accepted by both feature owners.

- [ ] Any review changes are incorporated into the contract documents and the final accepted decisions are recorded in the PR.
- [ ] Part F migration work remains a separate reviewed change; no migration or implementation result is claimed by this Part E PR.

Overall Issue #33 remains open until its Part F migration and real-PostgreSQL verification acceptance criteria are completed, reviewed, merged, and the exact schema SHA is published to #15 and #16.

E.3.2 D1-08A Shared Booking Data Specification (data-spec.md)

> **Proposed mapping and SQL intent only. NOT an applied migration, Prisma schema change, or PostgreSQL test result.** See foundation SDS, test plan, and whole-product System SDS §§3.2–3.3.

1. Canonical Booking fields

Field	PostgreSQL / Prisma type	Nullability / default	Contract
`id`	`UUID` / `String @db.Uuid`	NOT NULL; server-generated UUID	Stable identity and **application-level immutable invariant**. PostgreSQL's primary key enforces uniqueness and NOT NULL but does not itself prevent an ID-changing `UPDATE`. Sprint 1's append-only Booking guard separately prohibits UPDATE, DELETE and TRUNCATE; this is a lifecycle boundary, not a dedicated ID-immutability guarantee.
`customerId`	`UUID` / `String @db.Uuid`	NOT NULL	FK to `User.id`; derived from authenticated Customer Session by #16, never accepted as an owner claim from the client.
`currentSlotId`	`UUID` / `String @db.Uuid`	NOT NULL	FK to the selected `Slot`; identifies the currently assigned Slot.
`startsAt`	`TIMESTAMPTZ(6)` / `DateTime @db.Timestamptz(6)`	NOT NULL	Persisted copy of the selected Slot's `startsAt`; not client input.
`endsAt`	`TIMESTAMPTZ(6)` / `DateTime @db.Timestamptz(6)`	NOT NULL	Persisted copy of the selected Slot's `endsAt`; not client input; must be greater than `startsAt`.
`status`	`TEXT` / `String`	NOT NULL; default `Booked`	CHECK permits exactly `Booked` or `Cancelled`. Only `Booked` occupies/conflicts.
`createdAt`	`TIMESTAMPTZ(6)` / `DateTime @db.Timestamptz(6)`	NOT NULL; `now()`	Database creation timestamp.
`updatedAt`	`TIMESTAMPTZ(6)` / `DateTime @db.Timestamptz(6)`	NOT NULL; initially `now()`	Prisma `@updatedAt` for application updates; metadata only, not a business clock or audit log.

`tutorId` is intentionally **not** stored on Booking. The current Tutor is `Slot.tutorId` through `currentSlotId`. This prevents a duplicated Tutor reference from diverging from the selected Slot. The authenticated principal's role/active status remains an application authorization check; a normal FK cannot enforce a cross-table `User.role = 'Customer'` condition.

`Booking.id` is generated by the server/database on creation and is never accepted as mutable input. The #16 application contract must not expose an operation that replaces it; any future supported update path must preserve the original ID. This is an **application-level identity guarantee**, not a dedicated PostgreSQL ID-immutability guarantee. The separate Sprint 1 append-only prohibition below is a lifecycle boundary, not an ID-specific guard; no ID-specific trigger or SQL test is included in this foundation.

2. Relationship and delete/update policy

Relationship	Cardinality	Proposed FK	Delete / update
`Booking.customerId → User.id`	One User to many Bookings	`booking_customer_fk`	`ON DELETE RESTRICT ON UPDATE RESTRICT`
`Booking.(currentSlotId, startsAt, endsAt) → Slot.(id, startsAt, endsAt)`	One Slot to many historical Bookings; zero or one active `Booked` Booking	`booking_current_slot_window_fk`	`ON DELETE RESTRICT ON UPDATE RESTRICT`

The composite Slot FK requires a matching unique key on `Slot(id, startsAt, endsAt)`. `Slot.id` is already the primary key; the composite unique key exists specifically so PostgreSQL can enforce that the persisted Booking interval always matches its referenced Slot. It also prevents Slot time changes while a Booking references that exact interval. Existing Sprint 1 Slot immutability remains unchanged.

Restricting parent deletion preserves referenced Booking history and avoids cascading away a Customer's or Tutor's appointment record. Separately, Sprint 1 Booking rows themselves are append-only: UPDATE/DELETE are rejected by a row-level guard, TRUNCATE is rejected by a statement-level guard, and the application runtime role has no UPDATE/DELETE/TRUNCATE or DDL/trigger-management privileges. No routine application/admin exception may delete or truncate a Booking in Sprint 1. This is an engineering persistence boundary, **not** a general product data-retention policy. Any future anonymization, deletion, or archival workflow requires a separate decision and migration. Parent IDs are stable; they are not edited in normal operation.

3. Status and interval semantics

- `Booked` is the only Active/occupying state. A `Booked` row remains Active even after `endsAt`; elapsed time may be displayed as `Past`, but does not update status, imply attendance, create `Completed`, or release the Slot.
- `Cancelled` is a valid shared state and is excluded from both active constraints. No Customer/Tutor cancellation action is introduced in Sprint 1 by this foundation. Cancelled rows in foundation tests are isolated test fixtures; cancellation workflows and metadata remain later-scope work.
- The database must enforce `startsAt < endsAt`.
- Intervals use half-open `[startsAt, endsAt)` semantics. Positive-duration overlap is exactly `existing.startsAt < candidate.endsAt AND candidate.startsAt < existing.endsAt`. Adjacent intervals are allowed.
- The interval is copied from the actual selected Slot after it is read/locked by #16. A composite FK, not application convention alone, guarantees that `Booking.startsAt/endsAt` equal the referenced Slot's values.

4. Proposed Prisma mapping

The following is a **review target**, not code to paste without running Prisma validation against the exact project version. SQL-only partial/exclusion constraints are specified in §5 and must not be removed by a later generated migration.

```
model User {
  // Existing fields/relations remain unchanged.
  customerBookings Booking[] @relation("BookingCustomer")
}

model Slot {
  // Existing fields/relations remain unchanged.
  bookings Booking[] @relation("BookingCurrentSlot")

  // Required target key for the composite Booking FK.
  @@unique([id, startsAt, endsAt], map: "slot_id_window_unique")
}

model Booking {
  // Stable identity; BDF-D01 is application-level. Sprint 1 separately rejects Booking UPDATE/DELETE/TRUNCATE.
  id            String   @id @default(uuid()) @db.Uuid
  customerId    String   @db.Uuid
  currentSlotId String   @db.Uuid
  startsAt      DateTime @db.Timestamptz(6)
  endsAt        DateTime @db.Timestamptz(6)
  status        String   @default("Booked")
  createdAt     DateTime @default(now()) @db.Timestamptz(6)
  updatedAt     DateTime @default(now()) @updatedAt @db.Timestamptz(6)

  customer User @relation(
    "BookingCustomer",
    fields: [customerId],
    references: [id],
    onDelete: Restrict,
    onUpdate: Restrict,
    map: "booking_customer_fk"
  )
  currentSlot Slot @relation(
    "BookingCurrentSlot",
    fields: [currentSlotId, startsAt, endsAt],
    references: [id, startsAt, endsAt],
    onDelete: Restrict,
    onUpdate: Restrict,
    map: "booking_current_slot_window_fk"
  )

  @@index([customerId], map: "Booking_customerId_idx")
  @@index([currentSlotId], map: "Booking_currentSlotId_idx")
}
```

Do not add a Prisma `@@unique([currentSlotId])`: that would incorrectly prevent a Cancelled historical Booking and a later Booking from referring to the same Slot. The active-only uniqueness rule is a SQL partial unique index.

5. Proposed PostgreSQL migration intent

This fragment records the intended table/index constraints for peer review. It is **not** a complete timestamped migration and has not been run. The #33 Part F migration must also install the `booking_guard_s1()` function and row-level/statement-level trigger registrations specified in §7; they are deliberately not represented as completed SQL implementation here. Reconcile exact constraint names, Prisma migration output, extension/role privileges, and trigger behavior before acceptance.

```
-- Already installed by the merged Tutor Slots migration on the normal chain.
-- Keep this idempotent for a fresh/verified baseline; never DROP the extension.
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- PostgreSQL requires a unique referenced key for the composite FK.
ALTER TABLE "Slot"
  ADD CONSTRAINT slot_id_window_unique
  UNIQUE ("id", "startsAt", "endsAt");

CREATE TABLE "Booking" (
  "id" UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  "customerId" UUID NOT NULL,
  "currentSlotId" UUID NOT NULL,
  "startsAt" TIMESTAMPTZ(6) NOT NULL,
  "endsAt" TIMESTAMPTZ(6) NOT NULL,
  "status" TEXT NOT NULL DEFAULT 'Booked',
  "createdAt" TIMESTAMPTZ(6) NOT NULL DEFAULT now(),
  "updatedAt" TIMESTAMPTZ(6) NOT NULL DEFAULT now(),
  CONSTRAINT booking_status_check
    CHECK ("status" IN ('Booked', 'Cancelled')),
  CONSTRAINT booking_positive_interval
    CHECK ("startsAt" < "endsAt"),
  CONSTRAINT booking_customer_fk
    FOREIGN KEY ("customerId")
    REFERENCES "User" ("id")
    ON DELETE RESTRICT ON UPDATE RESTRICT,
  CONSTRAINT booking_current_slot_window_fk
    FOREIGN KEY ("currentSlotId", "startsAt", "endsAt")
    REFERENCES "Slot" ("id", "startsAt", "endsAt")
    ON DELETE RESTRICT ON UPDATE RESTRICT
);

-- Full indexes also support parent-FK checks for historical Cancelled rows.
CREATE INDEX "Booking_customerId_idx"
  ON "Booking" ("customerId");
CREATE INDEX "Booking_currentSlotId_idx"
  ON "Booking" ("currentSlotId");

-- Race B: one active Booking at most for one exclusive Slot.
CREATE UNIQUE INDEX booking_one_active_per_slot
  ON "Booking" ("currentSlotId")
  WHERE "status" = 'Booked';

-- Race C: one Customer cannot have positive-duration overlapping active windows.
-- [start,end) allows exact adjacency. btree_gist supplies UUID equality in GiST.
ALTER TABLE "Booking"
  ADD CONSTRAINT booked_customer_no_overlap
  EXCLUDE USING gist (
    "customerId" WITH =,
    tstzrange("startsAt", "endsAt", '[)') WITH &&
  )
  WHERE ("status" = 'Booked');
```

Constraint behavior matrix

Attempt	Expected database result
Insert valid `Booked` row with real Customer, Slot and matching interval	Accept one committed row.
Insert valid `Cancelled` fixture for a real Customer/Slot	Accept; it does not occupy or conflict.
Insert `startsAt >= endsAt`	Reject by `booking_positive_interval`.
Insert unsupported status	Reject by `booking_status_check`.
Insert unknown Customer or Slot	Reject by the corresponding FK.
Insert Booking interval that differs from the referenced Slot	Reject by `booking_current_slot_window_fk`.

Attempt	Expected database result
Insert second `Booked` row for same Slot	Reject by `booking_one_active_per_slot`, including concurrent attempts.
Insert overlapping `Booked` interval for same Customer on another Tutor's Slot	Reject by `booked_customer_no_overlap`, including concurrent attempts.
Insert adjacent `Booked` interval for same Customer	Accept if all other constraints pass.
Insert overlapping row after prior row is `Cancelled`	Accept if all other constraints pass.
Delete referenced User or Slot / update referenced primary key or Slot window	Reject by restrictive FK.
`DELETE FROM Booking` in Sprint 1	Reject by the row-level `BEFORE DELETE` guard (`0A000`); runtime role also lacks DELETE privilege (`42501`).
`TRUNCATE Booking` in Sprint 1	Reject by the statement-level `BEFORE TRUNCATE` guard (`0A000`); runtime role also lacks TRUNCATE privilege (`42501`).

6. Index and extension notes

- `Booking_customerId_idx` and `Booking_currentSlotId_idx` are full B-tree indexes. They also cover FK checks for Cancelled historical rows; the active-only partial index does not.
- `booking_one_active_per_slot` is partial on `status = 'Booked'`; cancelled history may share a Slot reference.
- `booked_customer_no_overlap` is a GiST exclusion constraint over Customer UUID equality and the half-open timestamp range. `btree_gist` is required for the UUID equality operator class. Verify the extension and its version/permissions on the actual PostgreSQL target; do not assume a local extension install proves managed-database availability.
- The exclusion constraint's GiST index also supports the overlap predicate; any extra query index must be justified by real `EXPLAIN (ANALYZE, BUFFERS)` evidence, not added speculatively.
- `Slot(id, startsAt, endsAt)` composite unique key is required for the composite FK even though `id` is already unique.

7. Booking trigger ownership and Sprint 1 guard

7.1 Explicit owner and merge order (BDF-D11)

Decision proposed for peer acceptance: one `booking_guard_s1()` function and its two trigger registrations are database artifacts owned by **Issue #33 Part F** and installed by the shared Booking migration: (1) a row-level `BEFORE INSERT OR UPDATE OR DELETE` trigger and (2) a statement-level `BEFORE TRUNCATE` trigger. Both use the same guard function; #16 does not create a second guard migration. **Issue #16 owns the application transaction adapter** that acquires the System SDS locks before the INSERT guard runs and proves the post-guard contention behavior. This split follows the System SDS §§3.3, 7.1 and 7.3 and Issue #33's shared database-enforced foundation scope.

The required sequence is:

1. Both #15 and #16 feature owners accept this #33 Part E contract.
2. #16 Part E is reviewed and records the exact application lock order, Serializable transaction, retry behavior and no-post-guard-contention proof strategy. The #33 migration must implement against that agreed protocol.
3. #33 Part F adds the Prisma Booking model, forward-only SQL migration, integrity constraints, the shared Booking guard function/trigger registrations, and the application-role privilege boundary; it then runs the **#33 pre-merge foundation proof set BDF-01..BDF-22 plus BDF-24..BDF-26** in tests.md. BDF-23 is explicitly excluded from this pre-merge gate.
4. The reviewed #33 Part F PR may merge into `sprint1` only after BDF-01..BDF-22 and BDF-24..BDF-26 pass; record its exact merged schema SHA in Issues #15 and #16.
5. Only after that schema SHA exists may #15/#16 begin Booking-dependent application implementation. #16 Part F then owns and runs **BDF-23 as a joint integration gate** against its complete application transaction adapter. #16 must not duplicate the trigger in its own migration.

7.2 Required Booking trigger behavior in #33 Part F

The migration must install one shared PostgreSQL `booking_guard_s1()` trigger function with two registrations on `Booking`: (1) a row-level `BEFORE INSERT OR UPDATE OR DELETE` trigger (not a column-limited UPDATE trigger), and (2) a statement-level `BEFORE TRUNCATE` trigger. The row-level trigger's first operation for **every** `UPDATE` **or** `DELETE` must be to reject the operation with stable SQLSTATE `0A000` (`feature_not_supported`) before inspecting mutable row values, acquiring a Slot lock, or evaluating Q05. This blocks `Cancelled → Booked`, `Booked → Cancelled`, Slot/window moves, Customer reassignment, ID replacement, metadata-only updates and direct row deletion. The statement-level trigger rejects every `TRUNCATE` statement with the same SQLSTATE; a row-level DELETE trigger alone is insufficient because TRUNCATE does not fire row triggers. No Sprint 1 application path or direct-SQL path may update/delete/truncate a Booking row.

For an `INSERT` whose `NEW.status = 'Booked'`, the trigger must (remediated in Issue #39):

1. Lock the Customer User row with `PERFORM 1 FROM public."User" AS u WHERE u."id" = NEW."customerId" FOR UPDATE;` in `BEFORE ROW` trigger. If `customerId` does not exist, the query returns zero rows without failing, allowing the composite FK (`booking_customer_fk`) to raise standard foreign key violation `23503`. Acquiring the Customer User lock inside `BEFORE ROW` trigger guarantees that direct-SQL writers acquire `User(customerId)` before inserting uncommitted tuples into table heap, unique index, or `booked_customer_no_overlap` GiST exclusion index, preventing post-`T_elig` reverse wait and `40P01` deadlock on concurrent application transactions.

1. Lock the referenced Slot row with `SELECT ... FOR UPDATE` and re-read its persisted lifecycle and interval in the writer's transaction.
2. Reject if the Slot does not exist, is not currently `Offered`, or its interval does not match the Booking snapshot. The composite FK independently enforces interval equality and remains mandatory.
3. Only after both User and Slot locks have been acquired, sample PostgreSQL `clock_timestamp()` as `T_elig` and reject unless `Slot.startsAt - T_elig >= interval '60 minutes'`. Equality passes; 59:59 fails. Do not accept a client-supplied or application-precomputed timestamp as authoritative.
4. Raise a non-success database error on rejection so the entire transaction rolls back. The trigger is a final DB guard for direct SQL/application bypass; it does not replace service authorization, the application lock order or the unique/exclusion constraints.

An `INSERT` of a valid `Cancelled` test fixture remains possible and does not occupy a Slot; the blanket UPDATE prohibition prevents that fixture (or any other row) from being transitioned to `Booked`. The DELETE and TRUNCATE guards preserve committed Booking history and prevent occupancy from disappearing without a reviewed lifecycle action. This closes the direct-SQL `Cancelled → Booked` Q05 bypass and deletion/truncation bypass without adding a Sprint 2 cancellation/reschedule workflow. These table-wide mutation prohibitions are Sprint 1 lifecycle boundaries, not a dedicated `Booking.id` immutability policy. Any future update/cancellation/reschedule/deletion behavior requires its own reviewed contract and migration before enablement.

7.3 #16 application transaction responsibility

Before issuing the guarded INSERT, #16 must use the System SDS global hierarchy: lock all affected User principals (authenticated Customer and selected Slot's actual Tutor) in ascending UUID order, then all affected Slot rows in ascending UUID order, then existing Booking rows in ascending UUID order. Use Serializable isolation as specified. The selected Slot must already be locked by the application transaction when the trigger executes, so the trigger's `SELECT ... FOR UPDATE` does not introduce a new lock-order inversion or an unaccounted post-`T_elig` wait.

The #16 Part E/Part F proof must show that no conflict-capable unique/exclusion wait can occur after the trigger samples `T_elig`. If an unaccounted wait is possible, the complete transaction must abort and retry with all locks reacquired and a fresh DB clock; never reuse the earlier `T_elig` or substitute a blanket commit-time cutoff. The actual no-post-guard-contention or full-transaction retry behavior is a real-PostgreSQL acceptance gate, not a documentation-only claim.

7.4 Application runtime database role boundary

The deployed application runtime role must be distinct from the migration/table-owner role. It must not own `Booking`, be a superuser, inherit membership in the migration/owner role, or have DDL/trigger-management privileges. It must not have `UPDATE`, `DELETE`, or `TRUNCATE` privileges on `Booking`, and must not be able to set `session_replication_role` to bypass

triggers. It retains only the read/insert privileges needed by the application. Migration credentials are used only by the migration/deployment path and are never used by API runtime processes. No application, operations, or administrator role is authorized to delete/truncate Sprint 1 Booking rows. The migration/table-owner role is for schema changes only and is not used for runtime or data-purge operations. Any future deletion, retention, or archival workflow requires a separate reviewed decision and migration; no emergency trigger-bypass exception is assumed.

BDF-24 and BDF-25 test the real application runtime role's privilege denial and independently exercise the row/statement guards with a non-owner, non-bypass DML probe role. BDF-26 verifies role ownership, membership, DDL/trigger and replication-bypass boundaries. If the deployed environment cannot provide this separation, the foundation gate is BLOCKED and must not be reported PASS.

7.5 Later Slot lifecycle trigger boundary

The complementary Slot `BEFORE UPDATE` lifecycle guard for `Offered → Withdrawn`, rejection of occupied withdrawal and permanent rejection of `Withdrawn → Offered` is **not part of this Sprint 1 foundation migration**, because the Sprint 1 Slot trigger does not permit a Withdrawn transition and #33 excludes the Sprint 2 withdrawal workflow. A separate reviewed Sprint 2 migration must install and real-DB-test that guard **before** any withdrawal feature is enabled. Do not defer or omit the Sprint 1 Booking row/statement guards or runtime-role boundary described above.

8. Migration/backfill constraints

- Use a new forward-only migration after the existing Auth and Tutor Slots migrations.
- Include the `booking_guard_s1()` function and its row-level/statement-level trigger registrations from §7 in the #33 Part F migration; provision the runtime role separately from the migration/table owner; do not create a duplicate Booking guard in #16.
- The verified baseline contains no Booking table/rows; no backfill or synthetic production data is needed.
- Provision separate migration/table-owner and application runtime roles. The runtime role must not own Booking, be superuser, inherit the owner role, hold UPDATE/DELETE/TRUNCATE or DDL/trigger-management privileges, or bypass triggers.
- Preserve all existing User, Session, AuthAttempt, Slot rows, indexes, triggers and constraints. Do not edit old migration files.
- Do not add `Slot.isBooked` or another denormalized occupancy flag. Occupancy is derived from `Booking.status = 'Booked'` and `currentSlotId`.
- No Booking API, application service, UI, cancellation/reschedule workflow, history, notification, or OD-013 idempotency mechanism belongs in this migration.
- Run both fresh-database and exact-baseline upgrade tests on disposable PostgreSQL databases. The database version and extension availability are evidence fields in tests.md.

E.3.3 D1-08A Shared Booking Data Test Specification (tests.md)

> **PLANNED TESTS ONLY — NOT IMPLEMENTED OR RUN.** Passing these cases requires real PostgreSQL and inspection of committed rows/constraint outcomes; Prisma mocks or HTTP-only assertions are insufficient. **Gate split:** BDF-01..BDF-22 and BDF-24..BDF-26 are the #33 Part F pre-merge foundation gate. BDF-23 is a separate #16 Part F joint integration gate, run only after the #33 migration is merged and its exact schema SHA is recorded; BDF-23 must not block the #33 migration PR merge. See foundation SDS and data spec.

1. Test environment and evidence

- Use a disposable PostgreSQL database, not a shared or production database.
- Record `SELECT version()`, server major/minor, installed `btree_gist` extension version, migration-owner and application-runtime role names/privileges, Node/Prisma versions, migration SHA, source SHA, fixture IDs, exact command, and PASS/FAIL/SKIP/BLOCKED result. Never expose credentials.
- Run one migration test from an empty database and another from the exact `sprint1` migration chain. Do not use a manually approximated schema.
- Use synthetic Customers and Tutors only. Create actual committed Slot and Booking rows satisfying all FKs/constraints.

- For concurrency tests, use separate PostgreSQL connections and explicit synchronization barriers. `Promise.all` without a controlled interleaving is not sufficient. Inspect the final committed rows after both transactions settle.
- Tests that use a Cancelled Booking must create an isolated `Cancelled` fixture with a direct INSERT inside the disposable database. Do not use UPDATE or DELETE to create or mutate a fixture, and never TRUNCATE to reset it: the Sprint 1 Booking guards reject these operations. UPDATE/DELETE/TRUNCATE appear only as explicit negative-test actions in BDF-20 and BDF-24/25.
- Verify that the application runtime role is distinct from the migration/table-owner role and is not owner/superuser; it has no UPDATE/DELETE/TRUNCATE or DDL/trigger-management privilege and cannot set `session_replication_role` to bypass triggers.
- No test case below is a PASS in this Part E PR.

2. Planned database test cases

ID	Trace	Setup / action	Required observable result
BDF-01	NFR-S1-12	Apply the agreed migration chain to an empty disposable PostgreSQL database.	Migration completes; Prisma schema and SQL constraints agree; Booking table, FKs, checks and indexes exist.
BDF-02	NFR-S1-03/12	Build a disposable database by applying every current `sprint1` migration, then apply the Booking foundation migration.	Upgrade succeeds without manual schema repair.
BDF-03	NFR-S1-03/05	Compare Auth/Slot rows and schema objects before/after the foundation migration.	Existing User, Session, AuthAttempt and Slot data, triggers, indexes and constraints are preserved; no destructive old-migration edit.
BDF-04	Q30, BR-S1-14, BR-S1-22	Insert a valid `Booked` row and a valid `Cancelled` test row with matching real Customer/Slot references; after the Booked interval ends, read the stored state and derived display classification.	Both statuses persist exactly; `Booked` remains the business status after elapsed time, `Past` is display-only, and `Cancelled` is historical/non-occupying.
BDF-05	Data contract	Insert `startsAt = endsAt` and `startsAt > endsAt`.	Both reject at PostgreSQL `booking_positive_interval`; no row commits.
BDF-06	Data contract	Insert a status outside `Booked` / `Cancelled`.	Reject at `booking_status_check`; no row commits.
BDF-07	FR-S1-10	Insert with a non-existent `customerId`.	Reject by `booking_customer_fk`; no row commits.
BDF-08	FR-S1-10	Insert with a non-existent `currentSlotId`/Slot-window reference.	Reject by `booking_current_slot_window_fk`; no row commits.
BDF-09	System SDS §3.3	Insert a Booking whose `currentSlotId` is real but whose `startsAt` or `endsAt` differs from that Slot.	Reject by the composite Slot-window FK; persisted Slot/Booking data remain unchanged.
BDF-10	FR-S1-12, BR-S1-08	Sequentially insert two `Booked` rows for the same Slot and different Customers.	First commits; second is rejected by `booking_one_active_per_slot`; exactly one active row remains.
BDF-11	AC-S1-25, NFR-S1-04	Two Customers concurrently insert a `Booked` row for the same Slot under a barrier.	At most one transaction commits; exactly one active row remains; no two success outcomes or partial claims.
BDF-12	FR-S1-13, BR-S1-09	One Customer has a `Booked` interval on Tutor A; insert a positive-overlap Booking for a distinct Tutor B Slot.	Second row is rejected by `booked_customer_no_overlap`; first row remains unchanged.
BDF-13	AC-S1-26, NFR-S1-04	One Customer concurrently inserts Bookings for distinct overlapping Slots owned by two Tutors under a barrier.	At most one overlapping active row commits; final committed state contains no same-Customer overlap.

ID	Trace	Setup / action	Required observable result
BDF-14	BR-S1-10	One Customer has `[13:00,14:00)` and inserts `[14:00,15:00)`.	Both commit if all other constraints pass; exact adjacency is not an overlap.
BDF-15	Q30, #15 AVA-17	In separate controls: (a) Customer A has a Cancelled row on Slot S and Customer B books Slot S; (b) Customer A has a Cancelled interval on Tutor A and books an overlapping Slot on Tutor B.	Both new Booked rows commit; Cancelled history neither occupies the Slot nor participates in Customer overlap.
BDF-16	Data contract	Attempt to delete a referenced Customer/Slot or mutate a referenced parent primary key/Slot window.	Operations reject and parent/Booking remain intact. Record the actual guard/constraint: the existing Slot immutability trigger may reject Slot mutations before the composite FK is evaluated; verify the FK definition independently. `Booking.id` mutation is intentionally not a DB test because BDF-D01 defines its immutability at the application boundary only.
BDF-17	#15 AVA-02/15/16/17	Query persisted rows using `status = 'Booked'`, `currentSlotId`, and half-open interval predicates.	Occupied Slot and same-Customer overlap predicates match actual committed data; adjacent and Cancelled controls are handled correctly.
BDF-18	System SDS §3.3	Verify `btree_gist` exists and supports the UUID equality/range exclusion migration under the actual migration role.	Extension creation/use succeeds on the recorded target version without dropping an existing extension; missing privilege/version is BLOCKED, not silently skipped.
BDF-19	NFR-S1-12	Inspect Prisma model, migration SQL, and database catalog after generation/deploy.	Relations/types/indexes match; Prisma schema drift or a generated migration dropping SQL-only partial/exclusion constraints fails the test.
BDF-20	System SDS §3.3; BDF-D11	In a disposable DB, create a controlled non-`Offered` Slot fixture using privileged setup only, restore normal Slot guards, then use a non-owner DML probe role with INSERT/UPDATE privileges but no DDL/trigger bypass to attempt a direct-SQL `Booked` INSERT against it. Separately, INSERT a valid `Cancelled` fixture for an `Offered` Slot whose start is less than 60 minutes away, then use the same probe role to attempt direct SQL `UPDATE status = 'Booked'` (the `Cancelled → Booked` Q05-bypass case). Also attempt a `Booked → Cancelled` or Slot/window UPDATE as a control.	The trigger rejects the non-`Offered` `Booked` INSERT and rejects **every UPDATE** with SQLSTATE `0A000`, including `Cancelled → Booked` even though Q05 would fail; no row changes state, no moved/active Booking commits, and the original fixture rows remain intact. This proves the guard independently of the #16 API.
BDF-21	Q05; System SDS §§3.3/4; BDF-D11	In the disposable DB, exercise the Q05 comparison with fixed DB-side `startsAt` / `T_elig` instants at exactly 60:00 and 59:59; separately execute the actual trigger path with the non-owner DML probe role and real PostgreSQL clock; verify it reads `clock_timestamp()` only after the Slot lock, never a caller-supplied time.	The exact comparison accepts 60:00 and rejects 59:59; the trigger integration proves the authoritative time source/order. If the exact-boundary subcase cannot be exercised without bypassing the production decision path, mark that subcase BLOCKED rather than PASS.
BDF-22	System SDS §§3.3/7.1/7.3; BDF-D11	On independent PostgreSQL sessions, hold the selected Slot row lock in transaction A. Start a direct Booking INSERT using the non-owner DML probe role in transaction B while the Slot has more than 60 minutes remaining, then release A only after the cutoff has passed. Capture both backend PIDs, `pg_blocking_pids`, trigger-observed `T_elig`, SQLSTATE and final rows.	B waits on the Slot lock; after the lock is acquired the trigger samples a fresh `T_elig` and rejects the now-too-late Booking. No row commits. This proves the trigger does not sample Q05 before its own Slot-lock wait.
BDF-23	System SDS §§7.1/7.3; #16 Part E/F; BDF-D11	**Run only after #33 Part F is merged and its exact schema SHA is recorded.** Run the complete #16 transaction adapter against real PostgreSQL with independent sessions and synchronized same-Slot and same-Customer/cross-Tutor attempts. Instrument lock waits and trigger `T_elig`; include a control that forces an unexpected conflict-capable index/constraint wait.	Prove all conflict-capable waits occur before `T_elig`; otherwise the complete transaction aborts/retries with all locks reacquired and a fresh DB clock. Inspect final committed rows and prove no stale-eligibility success, duplicate claim or partial write. This is a **post-merge joint integration gate**, owned by #16; it is not part of the #33 pre-merge gate (BDF-01..22 and BDF-24..26).

ID	Trace	Setup / action	Required observable result
BDF-24	BDF-D12; Sprint 1 append-only contract	Commit a valid `Booked` fixture and confirm its Slot is occupied. Using the actual application runtime role, attempt direct `DELETE FROM "Booking" WHERE id = ...`; then, in a disposable test database, use a non-owner DML probe role with DELETE privilege but no trigger/DDL bypass to exercise the row-level guard.	The runtime role is denied by privilege (`42501`); the probe role is rejected by the `BEFORE DELETE` guard (`0A000`). After rollback and a fresh connection, the committed Booking row still exists and the Slot remains occupied. No cancellation/deletion endpoint is involved.
BDF-25	BDF-D12; Sprint 1 append-only contract	Commit a valid `Booked` fixture and confirm its Slot is occupied. Using the actual application runtime role, attempt `TRUNCATE TABLE "Booking"`; then, in a disposable test database, use a non-owner DML probe role with TRUNCATE privilege but no trigger/DDL bypass to exercise the statement-level guard.	The runtime role is denied by privilege (`42501`); the probe role is rejected by the `BEFORE TRUNCATE` statement guard (`0A000`). After rollback and a fresh connection, the committed Booking row still exists and the Slot remains occupied. This proves the statement guard is not replaced by a row-level DELETE trigger.
BDF-26	BDF-D12; least-privilege deployment boundary	Connect as the configured application runtime role and inspect `current_user`, role attributes/membership, table ownership and effective privileges. Attempt `ALTER TABLE "Booking" DISABLE TRIGGER ALL` and `SET session_replication_role = replica`; inspect whether the role owns the table, is superuser, inherits the migration/owner role, or has DDL/trigger-management privileges.	Runtime role is distinct from the migration/table owner, is not owner/superuser, has no UPDATE/DELETE/TRUNCATE or DDL/trigger-management privileges, cannot disable the guards or set replication-role bypass, and retains only required application SELECT/INSERT access. Any missing separation is BLOCKED, not PASS.
BDF-28	Issue #39 remediation; BDF-D11	Targeted red→green 2-writer proof (Adapter protocol + direct-SQL Rogue): Part 1 reproduces the legacy canonical #33 baseline in a disposable schema showing Adapter blocks on Rogue in GiST (reverse wait); Part 2 proves that under migration `20261109000000_booking_guard_user_lock`, Rogue is blocked inside `booking_guard_s1()` on `User(customerId) FOR UPDATE` before index entry, Adapter INSERT completes with zero reverse wait (`adapterReverseBlockers = []`), delays across 60m cutoff and commits successfully ("valid 60:00 guard may still commit later"), Rogue unblocks and fails with `23P01` (never `40P01` deadlock), and exactly 1 Booked row commits.	**PASS**

3. Constraint outcome interpretation

Expected PostgreSQL classes for isolated negative cases:

- CHECK violations: SQLSTATE `23514`.
- Foreign-key violations: SQLSTATE `23503`.
- Unique partial-index conflicts: SQLSTATE `23505`.
- Exclusion conflicts: SQLSTATE `23P01`.
- The Sprint 1 Booking row guard raises SQLSTATE `0A000` (`feature_not_supported`) for UPDATE/DELETE and the statement-level TRUNCATE guard raises the same code; BDF-20, BDF-24 and BDF-25 must verify the guards. The application runtime role's denied UPDATE/DELETE/TRUNCATE privileges raise `42501`.
- Serializable/deadlock outcomes may surface as `40001` / `40P01` if the accepted transaction strategy uses them; #16 must classify and handle them according to its reviewed contract.

For concurrent cases, do not require one exact error code when PostgreSQL may report a serialization failure under the selected isolation/locking design. The acceptance oracle is the final committed state plus correctly classified non-success for the losing operation.

4. Tests owned outside this foundation

Behavior	Owner / evidence
Customer role, active account and Session-derived `customerId`	#16 API/transaction tests; the FK alone does not enforce role.

Behavior	Owner / evidence
Booking Offered-state, Q05 database guard and Sprint 1 row-level UPDATE/DELETE prohibition (`BEFORE INSERT OR UPDATE OR DELETE` , 60:00/59:59, Slot-lock wait, `Cancelled → Booked` rejection)	#33 Part F migration and BDF-20..22; all are pre-merge foundation gates and direct-SQL tests must prove the trigger independently of the API.
Booking DELETE/TRUNCATE guard and application runtime role boundary	#33 Part F migration/deployment contract and BDF-24..26; pre-merge gates prove actual runtime privilege denial, row/statement trigger rejection and committed occupancy preservation.
Application lock order and no-post-guard-contention / full-transaction retry proof	#16 Part E/F and BDF-23; run only after #33 Part F is merged and the schema SHA is recorded. BDF-23 is not a prerequisite for #33 migration merge.
`Booking.id` immutability	Application-level invariant only (BDF-D01): #16 generates the ID server-side and accepts no ID replacement. Any future update/reschedule path must prove it preserves the original ID; no DB immutability guard or SQL negative test is claimed by #33.
Duplicate/uncertain HTTP request response semantics (OD-013)	#16 Part E decision and #16 API tests; not decided here.
Customer availability 60-day horizon and safe response allowlist	#15 AVA tests; OD-002 is confirmed as 60 days in Issue #15.
Browser/UI, Bangkok rendering, session switch and accessibility	#15 UI/API/E2E tests; not a database foundation test.
Customer/Tutor Booking retrieval and record-level privacy	F-S1-05 owner contract/tests; no retrieval endpoint is added here.
Sprint 2 cancellation/reschedule/withdrawal/notification and Slot lifecycle transition triggers	Separate Sprint 2 contracts and migrations; this Part E does not implement them.

5. Part F evidence ledger template

The implementation PR must replace this planned status with actual results; do not copy `PASS` without command output and final database inspection.

Evidence field	Required value
PostgreSQL server	Exact `SELECT version()` output
Extensions	`btree_gist` version and migration-role privilege evidence
Database roles	Application runtime role, migration/table-owner role, role memberships, ownership and effective Booking privileges
Destructive-operation proof	DELETE/TRUNCATE runtime denial, guard-probe SQLSTATEs, committed Booking row and occupied Slot after rollback
Base schema	Exact `sprint1` migration head SHA
Migration	New migration path and SHA
Test source	Commit SHA and test file paths
Commands	Exact fresh DB, baseline upgrade, and test commands
Results	Count of PASS / FAIL / SKIP / BLOCKED; link logs/evidence
Concurrency	Barrier setup, connections, final committed rows, losing error classification
Review	Human reviewer and accepted schema SHA

Part E status: BDF-01..BDF-26 are planned, with the execution split above. #33 Part F must pass BDF-01..BDF-22 and BDF-24..BDF-26 before its migration PR merges; #16 Part F runs BDF-23 after the merged schema SHA exists. No migration, database test, API, UI, or concurrency result is claimed by this documentation-only PR.

E.4 F-S1-03: Customer Availability Browse & Filter

E.4.1 F-S1-03 Customer Availability Feature SDS (specification.md)

> **Part E documentation contract only.** Issue #15; provisional owner @Chxtamos; independent peer reviewer @Tanaboonnnnn. Decision states below reflect the peer-review decision record in PR #32 comment #6683929084. These are engineering/design decisions, not new stakeholder business rules. This change does not implement UI/API code, add a migration, execute tests or close the issue.

1. Authority and scope

Sources: Sprint 1 requirements (FR-S1-08/09/13/18, AC-S1-15..18, BR-S1-06/07/09/10/11/12/15/17/20/21, NFR-S1-01/07/08/09/10/12, Q28), SRS, System SDS, Feature Inventory, Traceability, AGENTS.md, accepted Auth contract and Tutor Slot contract.

Outcome: an authenticated Customer discovers a real eligible Tutor Slot, identifies the Tutor using an approved public display name when its data gate is cleared (otherwise an opaque Tutor reference), sees Bangkok start/end time, and selects that offer without seeing another Customer's private Booking data.

In scope: Customer-only read access to current Offered Slots; Customer-specific bookability projection; safe Tutor/Slot references; a conditional Tutor public-display-name data gate; UTC API and Asia/Bangkok UI time; privacy allowlist; loading/empty/error/retry/session-change states; planned API, DB, browser, timezone and privacy tests.

Out of scope: Booking create/confirm/history/detail (#16/#17); Tutor Slot management/dashboard (#14); public/anonymous discovery; any unapproved Tutor profile/name/email/phone or Customer identity; subject, rating, price, location, messaging, payments, notifications, cancellation/reschedule, recurring Slots; new Tutor hours, weekdays, holidays, maximum horizon or start-minute grid; any mutable Slot.isBooked flag or Booking schema owned by #15.

Part E is documentation only. Part F must be a separate code-only branch/PR after this contract is peer-accepted.

2. Dependencies and verified repository facts

Issue #13 Auth and Issue #14 Tutor Slots/Dashboard are closed. PR #31 is merged into sprint1. This branch is based on sprint1 SHA 551a9469e3d3a7e8e3ae004a382862c4f3e87e0e.

Selected sequencing route — B: a separately scoped shared data foundation. At this base, `apps/api/prisma/schema.prisma` contains User, Session, AuthAttemptBucket, AuthAttemptReservation and Slot, but no Booking model/table. `apps/api/src/slots.ts` currently projects Tutor own-list occupancy as Unoccupied because #16 has not introduced Booking; that projection is not a Customer-availability occupancy source.

The team selected a separate Issue/PR for the shared Booking data foundation, with schema, migration, ownership and sequencing reviewed before either feature relies on it. The minimum persisted fields needed by this contract are `id`, `customerId`, `currentSlotId`, `startsAt`, `endsAt` and `status` (`Booked` / `Cancelled`); `currentSlotId` is the occupancy relation and the persisted Booking interval is the Customer-overlap source. Booking mutation remains owned by #16 and must not move into #15.

Foundation tracking (not yet created): proposed implementation owner @Chxtamos; peer reviewer @Tanaboonnnnn; dependency links #15 and #16; exact foundation Issue/PR URL and reviewed migration/schema SHA are **TBD** until that separate work is created and accepted. This PR does not create the Issue/PR or approve a migration. Until the shared foundation is reviewed, merged and its exact SHA recorded against the Part F branch, occupancy-dependent Part F work is a hard **STOP GATE**. Do not invent `Slot.isBooked`, assume all Slots are unoccupied, or create a partial Booking schema under #15.

3. Actors and journey

Actor	Result
Anonymous	401; no offer data
Authenticated Customer	200 safe availability projection
Authenticated Tutor	403; no Customer availability payload
Server	Derives Customer identity from live Session; derives eligibility from persisted Slot/Booking and database time

1. Customer opens the availability screen.
2. UI verifies GET /api/v1/auth/me and requires trusted role Customer.
3. UI calls the Customer availability GET with the same-origin HttpOnly Session cookie.
4. Server enforces Session and Customer role, reads eligible offers and serializes only the approved allowlist.
5. UI displays an approved Tutor public display name only after the separate data-approval gate; otherwise it displays the opaque Tutor reference. It also displays Bangkok start/end. Selecting a Slot changes transient UI state only; it is not a Booking or reservation.
6. Later #16 receives only the selected Slot ID and independently rechecks offer state, occupancy, Q05 and all Booking rules in its authoritative transaction. A prior availability response is never a lock, lease or guarantee.

4. Eligibility and time

For one read, `T_view` is one database-authoritative instant sampled for the availability query. Under the confirmed Customer-specific bookable-only policy, include a Slot only when:

1. lifecycle is Offered;
2. no active Booking with status Booked references that Slot;
3. startsAt - T_view is at least 60 minutes (60:00 included; 59:59 excluded);
4. no Active Booking owned by the authenticated Customer overlaps this Slot, even when the existing Booking belongs to a different Tutor; and
5. the persisted Slot interval and Tutor ownership remain valid under the accepted Slot contract.

Customer-specific overlap predicate (BR-S1-09/10, Q28): derive the owner only from `principal.userId` supplied by the live Session. Exclude a candidate Slot when an existing Booking has `customerId = principal.userId`, `status = Booked`, and its persisted interval overlaps the candidate using half-open intervals: `booking.startsAt < slot.endsAt AND slot.startsAt < booking.endsAt`. A Cancelled Booking does not conflict; exact adjacency is allowed. This is an eligibility predicate only: the query may test existence but must not return or disclose any matching Booking or Customer data. The same Customer's overlapping Booking on another Tutor must make the candidate unavailable to that Customer; another Customer's Booking must not affect this Customer's overlap predicate.

T_view is an indicative read timestamp, **not** #16's transactional T_elig. Booking T_elig must be sampled by the authoritative Booking transaction after relevant contention waits, immediately before the guarded write. The availability UI must never imply a displayed Slot is reserved or guaranteed to remain bookable.

PostgreSQL stores TIMESTAMPTZ; API returns RFC 3339 UTC instants ending Z; UI formats explicitly with Asia/Bangkok regardless of device timezone. Never use browser time or a client timestamp for eligibility. OD-002 (maximum advance horizon) remains OPEN: this contract does not choose an upper bound and does not claim an approved unlimited horizon. Resolve OD-002 before Part F; do not silently add a horizon, hours, weekday, holiday or minute-grid policy.

A Slot can become occupied or cross Q05 after T_view. If #16 later rejects a stale selection, UI shows non-success and refreshes; #15 does not create a fake Booking endpoint.

5. Peer-review decision record

The following states record the decisions in PR #32 comment #6683929084. “Confirmed” means the team engineering/design choice recorded by peer review; it does not claim a new stakeholder business rule or implementation/test completion.

ID	Decision / contract	Status
OD-AV-01 Display	Customer-specific bookable-only: return/display only Slots that pass current Slot offer, occupancy, Q05 and the authenticated Customer's BR-S1-09/10 overlap rules at `T_view`.	**CONFIRMED design choice**; not a Booking guarantee.
OD-AV-02 Tutor identity	Desired UI identity is an approved Tutor public display name. Do not read/add/publish a name until its source, owner/consent, field, migration and allowlist are approved. Until then use opaque `tutorRef`.	**CONDITIONAL DATA-APPROVAL GATE**.

ID	Decision / contract	Status
OD-AV-03 List	No filters or pagination in Sprint 1; stable order `startsAt ASC → tutorRef ASC → slotId ASC`.	**CONFIRMED technical constraint**; not an unlimited-horizon decision.
OD-AV-04 Clock	One PostgreSQL `T_view` per read; `meta.evaluatedAt` is present for both populated and empty success responses.	**CONFIRMED**; not a substitute for #16 `T_elig`.
OD-AV-05 Occupancy	Exclude a Slot through active `Booking.status = Booked` and `Booking.currentSlotId = Slot.id`; never store duplicate `Slot.isBooked`. Customer overlap is a separate owner-derived interval predicate.	**CONFIRMED invariant**; schema-dependent Part F STOP GATE.
OD-AV-06 Error/privacy	Reuse existing API/session security, Customer role, session binding, request ID/error envelope, `Cache-Control: no-store`, and distinguish error from empty.	**CONFIRMED**.
OD-AV-07 Performance	Record p95 under a reproducible Part F workload; ≤2.0s remains a proposed reference only.	**NON-BLOCKING MEASUREMENT**; no pass/fail threshold approved.
OD-AV-08 Visual	Customer UI remains light/readable but shares NutTiew brand, typography, related colors/accents, spacing, components, focus and navigation patterns with Tutor Dashboard. Role-specific layouts are allowed within one design system.	**CONFIRMED design direction**.
OD-002 Horizon	No maximum advance horizon, working hours, weekday, holiday or minute-grid rule is selected.	**OPEN**; resolve before Part F.
#15 ↔ #16 data foundation	Use a separately scoped shared Booking data-foundation Issue/PR (route B); record owner, reviewer, dependency links and exact accepted schema SHA.	**SELECTED; NOT CREATED**. Part F STOP GATE remains.
NFR-S1-09 accessibility	Plan keyboard, focus, announcements and responsive checks; collect actual evidence in Part F.	**TEST PLAN ACCEPTED**; no WCAG 2.2 AA or 44px conformance claim.

Conditional Tutor public-display-name gate

- **Proposed source:** an explicit public display name supplied/maintained by the Tutor; never derive it from `User.email`, login credentials or a guessed profile field.
- **Proposed field/type:** nullable `User.tutorDisplayName` (`String?` / PostgreSQL `TEXT`), used only for Tutor accounts. This is a candidate contract, not an approved schema change.
- **Data owner and permission:** the Tutor is the source/data owner and must explicitly opt in to customer-facing display. The product/stakeholder owner must record approval for publication; @Chxtamos owns this feature contract and @Tanaboonnnnn is the peer reviewer. Record the actual approving stakeholder and consent rule before migration.
- **Migration/seed:** only a separate reviewed forward-only migration after the data gate is accepted; no default, no backfill from email, and existing Tutors remain `NULL`. Seed only clearly test-only synthetic values in isolated test fixtures; never present fixture names as real Tutor identity.
- **API/UI allowlist:** until the gate is cleared, omit `tutorDisplayName` and show `tutorRef`. After approval, add only the approved nullable field to the response allowlist and render it only when present; never add email, phone or other profile fields.
- **Privacy evidence:** planned AVA-18 verifies no name is inferred from email, no unapproved field is returned, and—only after approval—the approved display field is the sole additional Tutor identity field. AVA-05 continues to inspect the full response allowlist.

6. Acceptance and traceability

Source	F-S1-03 outcome	Planned STS
FR-S1-08 / AC-S1-15	Customer identifies a specific Tutor using approved display name after its gate, otherwise opaque reference, plus Slot start/end	AVA-01, AVA-05, AVA-18
FR-S1-09 / AC-S1-16	Anything shown as bookable is current, unoccupied, outside Q05 cutoff and non-overlapping for this Customer	AVA-02, AVA-03, AVA-15, AVA-16, AVA-17

Source	F-S1-03 outcome	Planned STS
AC-S1-17	Stale offer is revalidated by #16; no false Booking success	AVA-11
AC-S1-18 / BR-S1-12	Bangkok time meaning is independent of device locale	AVA-04
BR-S1-06/07/08/09/10/11/15/17/20/21	Distinct Tutor offers, Slot occupancy, Q05/future-time eligibility, Customer-specific non-overlap and adjacency, role and privacy	AVA-02/03/05/06/07/15/16/17
FR-S1-18	Failed/unauthorized reads communicate non-success without private disclosure or false availability	AVA-05/06/10
NFR-S1-01/08	Deny anonymous/wrong role; no secret or private Booking leakage	AVA-05/06/12
NFR-S1-07/12	Reproducible evidence, stable IDs and reviewed PR	Part F ledger/PR
NFR-S1-09	Keyboard, focus, announcements and responsive behavior are planned for manual/browser verification	AVA-13
NFR-S1-10	Record p95 and workload evidence; ≤2.0s is non-blocking and not an approved threshold	AVA-14
OD-AV-08	Customer and Tutor screens share NutTiew brand, typography, spacing, colors/accents, components, focus and navigation patterns with role-specific layouts	AVA-19

Whole-product trace anchors: FR-008/009, NFR-007/009/010, VAL-006/009/033/036, VC-S1-13/TEST-S1-13, VC-S1-15/TEST-S1-15, VC-S1-22/TEST-S1-22 and VC-S1-36/TEST-S1-36. These are planned links, not test results.

7. Definition of Done

Part E

- [ ] Five cross-linked documents exist: Feature SDS, UI spec, API spec, data spec and STS.
- [ ] FR-S1-08/09 and AC-S1-15..18 map to explicit rules, response fields, UI states and tests.
- [ ] PR #32 peer decisions are recorded with explicit CONFIRMED / CONDITIONAL / OPEN / STOP GATE states; no engineering choice is mislabeled as stakeholder business approval.
- [ ] Customer-specific bookable-only predicate includes same-Customer cross-Tutor overlap and half-open adjacency; owner comes only from the live Session.
- [ ] Empty success response always contains DB `meta.evaluatedAt` and `Cache-Control: no-store`; query returns one metadata row even when there are no offers.
- [ ] Tutor public display name remains behind its source/consent/schema/migration/allowlist gate; no email-derived or invented production name is exposed.
- [ ] Response allowlist excludes Customer identity/Booking data, credentials, Session fields and unrestricted Tutor fields.
- [ ] Shared data-foundation route B is recorded with proposed owner/reviewer/dependencies; exact Issue/PR and schema SHA remain pending, so Part F is blocked until accepted. No fake occupancy flag is added.
- [ ] Markdown links/structure and git diff --check pass.
- [ ] PR contains only the five F-S1-03 Markdown files, targets sprint1, links #15 and requests @Tanaboonnnnn.
- [ ] PR stays open for independent human review; no self-approval or self-merge.

Part F — separate implementation PR

- [ ] Start from updated sprint1 after Part E acceptance and the separately reviewed/merged shared Booking data foundation; record its exact schema SHA.
- [ ] Implement Customer-only UI → Express API → PostgreSQL read projection from real Slot and Booking rows.
- [ ] Verify occupied exclusion, Customer-specific cross-Tutor overlap and adjacency, Q05 boundary, approved Tutor-name allowlist gate, Bangkok rendering, auth/role denial, empty/error/retry and session-change privacy.
- [ ] Run keyboard/focus/announcement/responsive checks and record evidence; do not claim WCAG 2.2 AA or a 44px target pass without the corresponding assessment.

- [ ] Record real commands/results, DB/migration, source SHA and hosted CI.
- [ ] Open separate code-only PR into sprint1 and obtain peer review.

Verification boundary: this Part E change is documentation only. No build, API test, migration, Booking query, browser E2E, performance benchmark or accessibility audit is claimed as run or passed.

E.4.2 F-S1-03 Customer Availability UI Specification (ui-spec.md)

> **Part E UI contract only; no UI implementation/browser test.** Customer-specific bookable-only and the shared NutTiew design direction are recorded as peer decisions in Feature SDS. Tutor display name remains conditional on its separate data-approval gate. See API, Data, STS.

1. Screen and visual direction

Screen: **ช่วงเวลาติวที่พร้อมให้เลือก** — authenticated Customer availability discovery.

This is a focused scheduling utility, not a marketing page or a visually separate application. Customer Availability and the Tutor Calendar Dashboard share one NutTiew design system while allowing role-specific layouts.

Shared product token	Customer Availability application
Brand / navigation	Text-only NutTiew wordmark; shared navy `#202B44` for brand/navigation and the same active-navigation, account and logout patterns as Tutor Dashboard
Canvas / surface	Light canvas `#F7F8FB`, white surface `#FFFFFF`; no nested cards
Text	Dark ink `#172137`; muted text must remain contrast-safe on the shared light surfaces
Primary action	Orange `#F47736` with dark-ink label/icon `#172137` and border `#D96123`; do not use white text on orange
Focus	Shared visible focus ring `3px solid #AA4519` with `outline-offset: 3px`; verify it remains unobscured in every state
Typography	Noto Sans Thai, Sarabun, system-ui, sans-serif; use the same type hierarchy as Tutor Dashboard
Spacing / corners	Shared 4/8/12/16/24/32px spacing scale and 8px radius; align component padding and row rhythm with Tutor Dashboard

Use the same reusable button, status, focus and navigation patterns as Tutor Dashboard. Desktop navigation follows the same NutTiew shell; only Customer-authorized destinations are shown (availability, account and logout), never Tutor-only or unimplemented menus. At mobile widths use the same accessible collapsed-navigation behavior. Keep the Customer content light and readable; do not create a second brand palette with an unrelated accent.

No gradients, glassmorphism, hero banners, fake Tutor profiles, testimonials, ratings, prices, message panels, decorative emoji or card-in-card layouts. The orange CTA pairing and focus values are design tokens from the Tutor UI contract, not proof of WCAG conformance; Part F must verify actual rendered states and contrast.

2. Screen anatomy

1. Small text-only NutTiew wordmark.
2. Heading: เลือกช่วงเวลาติว.
3. Helper: appointment times are displayed in Asia/Bangkok.
4. Optional ตรวจสอบล่าสุด timestamp from response meta.evaluatedAt.
5. Chronological offers grouped by Bangkok calendar date.
6. Each offer shows an approved Tutor public display name when the OD-AV-02 data gate is cleared; otherwise it shows the stable opaque Tutor reference. Also show Bangkok date, 24-hour start/end and a เลือกช่วงเวลานี้ action.
7. Selected-offer summary uses the same approved name/fallback reference and time, and must state that selection **is not a Booking**.

Selecting an offer stores slotId in transient UI state only. It does not POST, reserve, occupy or confirm a Booking. The later #16 flow owns confirmation and must revalidate the selected Slot on the server.

Display policy OD-AV-01

Confirmed: Customer-specific bookable-only. Show only offers the API evaluated as bookable for the authenticated Customer at `meta.evaluatedAt`: current Offered Slot, not occupied by an Active `Booked` row, at least 60 minutes before start, and not overlapping that Customer's own Active Booking even if the existing Booking belongs to another Tutor. Exact adjacency is allowed. Do not render unavailable offers or disclose why another Slot was excluded.

The maximum advance horizon remains open under SRS OD-002. The UI must not invent a date cap or imply that an unlimited horizon has been approved; update the visible range only after the team records that decision.

3. UI states

State	Trigger	Required behavior
Session loading	/auth/me pending	Hide offers; show progress
Signed out	/auth/me returns 401	Clear offers/selection; show Auth
Wrong role	Principal is not Customer	Access denied; do not call/render Customer availability
Initial loading	Customer verified; availability GET pending	Show progress, not empty/zero
Populated	200 with offers	Render authorized offers in server order
Confirmed empty	200 with data: []	Show ยังไม่มีช่วงเวลาติวที่พร้อมให้เลือก
Read error	Network/timeout/500/503	Show error and โหลดอีกครั้ง; never show empty/zero
Selection	Customer selects an offer	Mark selected; state no Booking was created
Stale selection	Later #16 rejects it	Show non-success, clear/mark stale selection, refresh; never show Booked
Session changed	Logout/account switch/binding mismatch	Immediately clear offers/selection and ignore late old-session responses

Only a successful authorized 200 with data: [] may render confirmed-empty. A failed request or malformed payload is not an empty result.

4. Interaction and accessibility

- Semantic headings, native buttons, visible labels and logical Tab/Enter order.
- Every offer is keyboard selectable; selected state is communicated with text/semantics, not color alone.
- Visible :focus-visible; focus must not be obscured. Move focus to an error summary for blocking failures where appropriate.
- Announce loading, result count, empty and error transitions without repeatedly announcing the entire list.
- Primary controls target at least 44×44 CSS px as a design target only; this is not a verified pass. Part F records actual target-size, contrast, focus, zoom and 320 CSS px reflow evidence.
- NFR-S1-09 test planning is accepted, but this specification does not claim WCAG 2.2 AA conformance. Do not report WCAG AA or 44px as passed without the corresponding manual/browser evidence.

5. Responsive behavior

Viewport	Proposed layout
320/375 CSS px	Single-column list, 16px side gutters, no horizontal overflow
768 CSS px	Readable constrained list; no unnecessary calendar grid
1280 CSS px	Centered readable column; rows may place metadata/action in two columns

Preserve the same chronological offer set and field visibility at every viewport. Do not crop offers or hide Tutor/time on mobile.

6. Privacy and content rules

Show only the approved Tutor public display name when the OD-AV-02 gate is cleared; otherwise show the opaque Tutor reference, plus Slot date/time. The server-side overlap check may use the authenticated Customer ID but must not reveal whether a matching Active Booking exists, which Tutor owns it, or any Booking/Customer details. Never derive a name from email or invent a name. Do not show/fetch Customer identity, emails/phones, Booking IDs/reasons, Tutor email, credentials/session fields or full private Tutor schedule. No unapproved profile fields, subjects, ratings, prices, locations, images or unrelated availability. No Booking confirmed/Booked/Reserved state in #15.

7. Acceptance mapping

Criterion	UI requirement
AC-S1-15	Selectable approved Tutor display name after its data gate, otherwise opaque Tutor reference, and Bangkok start/end
AC-S1-16	Occupied/inside-cutoff offers and offers overlapping this Customer's Active Booking are excluded from the bookable-only list
AC-S1-17	Selection is not reservation; #16 revalidates and UI shows non-success
AC-S1-18	Asia/Bangkok rendering independent of device timezone

Part F must capture real DB-backed browser evidence at 320/375/768/1280 CSS px and test keyboard selection, loading/empty/error/retry, 401/403, session switch, timezone and stale #16 response. None has been run in Part E.

E.4.3 F-S1-03 Customer Availability HTTP/API Contract (api-spec.md)

> **Part E docs-only contract; not implemented/runtime-tested.** Peer-review decisions are recorded in Feature SDS §5. See UI, Data, STS, Auth API, Tutor Slot API.

1. Transport and authorization

- Same-origin /api/v1 JSON stack. This GET has no request body.
- Use the existing HttpOnly Session cookie, requireSession and requireRole(Customer).
- Principal comes only from getPrincipal(req) and persisted live Session/User. Never accept customerId, tutorId, role, owner, status, time or eligibility from query/header/body as authority.
- Anonymous: 401 UNAUTHENTICATED. Authenticated Tutor: 403 FORBIDDEN.
- Read-only GET: no CSRF token or unsafe-method Origin check. Preserve same-origin cookie behavior and no credentialed cross-origin CORS.
- Set Cache-Control: no-store. Return the same server-derived X-NutTiew-Session-Binding header as /api/v1/auth/me and Tutor own-Slot GET so UI rejects late data from a different Session, including another Session for the same User.
- Reuse existing request ID and error envelope; do not fork Auth/session logic.

2. Endpoint

Method/path	Query	Success	Enforcement
GET /api/v1/customer/availability	No query parameters (confirmed OD-AV-03)	200 safe projection; `data: []` is a valid confirmed-empty result and still includes `meta.evaluatedAt`	Live Session, Customer role, Customer-specific read-time eligibility, allowlist, no-store

Unknown query parameters (customerId, tutorId, from, to, page, cursor, limit or anything else) return 400 INVALID_INPUT under confirmed OD-AV-03. Any future filter/pagination requires a reviewed contract amendment; do not infer a maximum horizon/date window.

Success response — Customer-specific bookable-only

```
{
  "data": [
    {
      "slotId": "c62bfb8b-2530-4a7c-979c-a9a6d16be0df",
      "tutorRef": "4a6f3e1b-58f0-4d6e-9e4c-1c2a5b6d7e8f",
      "startsAt": "2026-11-08T06:00:00.000Z",
      "endsAt": "2026-11-08T07:00:00.000Z"
    }
  ],
  "meta": {
    "requestId": "opaque-request-id",
    "evaluatedAt": "2026-11-08T04:30:00.000Z"
  }
}
```

Values are illustrative. Before the response is written, `data[].startsAt`, `data[].endsAt` and `meta.evaluatedAt` must all be normalized to UTC with exactly three fractional-second digits (`YYYY-MM-DDTHH:mm:ss.sssZ`). The data projection uses explicit UTC conversion/formatting (`AT TIME ZONE 'UTC'` followed by `to_char`) before constructing JSON; raw PostgreSQL `timestamptz` values must never be serialized into JSON using the connection's session `TimeZone`. The API sends those normalized strings unchanged. `evaluatedAt` is database `T_view`. Every row is bookable for the authenticated Customer at that evaluation instant only; it is not a reservation or guarantee that a later Booking succeeds. `tutorRef` is the current identity fallback because no Tutor display-name field is approved in the base schema.

An empty successful response has the same envelope and metadata, with `data: []` and a non-null RFC 3339 UTC `meta.evaluatedAt` normalized by the same explicit UTC formatting rule. The endpoint sets `Cache-Control: no-store` for populated and empty 200 responses (and for errors). The timestamp comes from the same database statement/read evaluation as the offer list; it is not omitted or generated from the application/browser clock when there are no offers. Populated and empty responses must be byte-for-byte stable in timestamp representation when the PostgreSQL connection `TimeZone` changes between `UTC` and `Asia/Bangkok` for the same persisted instants.

```
{
  "data": [],
  "meta": {
    "requestId": "opaque-request-id",
    "evaluatedAt": "2026-11-08T04:30:00.000Z"
  }
}
```

Exact response allowlist

Field	Type	Meaning
data	array	Eligible offers under the confirmed unfiltered S1 list
data[].slotId	UUID string	Persisted Slot ID; identifier for later Booking selection
data[].tutorRef	UUID string	Opaque Tutor reference derived from Slot.tutorId
data[].tutorDisplayName	Not currently returned; conditional optional string only after OD-AV-02 data approval	Explicit Tutor-provided public display name; never derived from email. Until approval, this property is absent and the UI uses `tutorRef`.
data[].startsAt	RFC 3339 UTC string ending Z	Persisted Slot start
data[].endsAt	RFC 3339 UTC string ending Z	Persisted Slot end
meta.requestId	opaque string	Existing request correlation ID
meta.evaluatedAt	RFC 3339 UTC string ending Z	Database read-evaluation instant T_view

Never serialize Prisma/SQL rows wholesale. Exclude customerId, Customer name/email/phone, Booking ID/status/reason, Tutor email/phone, unapproved Tutor profile/name fields, password hash, Session ID/token/CSRF, other DB columns, SQL errors and

stack traces. Bookable-only response needs no bookable boolean. The only possible later addition is the specifically approved `tutorDisplayName` field after its data gate and migration are accepted.

OD-AV-01 is confirmed as Customer-specific bookable-only. Any later change to show unavailable offers requires a new reviewed contract amendment; it must never disclose whether unavailability is due to another Customer's Booking or the Q05 cutoff.

3. Ordering and read semantics

Confirmed Sprint 1 order: `startsAt ASC`, `tutorRef ASC`, `slotId ASC`. No client-selected order, filters or pagination. OD-AV-03 is a technical list constraint only; OD-002 (maximum advance horizon) remains OPEN. The absence of filters/pagination is not an approved unlimited-horizon policy or a guarantee for unlimited data volume.

Confirmed OD-AV-01/04: sample one PostgreSQL `T_view` for the read. Under Customer-specific bookable-only, a row must be lifecycle Offered, have no active Booked row for its Slot ID, satisfy startsAt - T_view >= 60 minutes, and have no overlapping Active Booking owned by `principal.userId`. The owner is derived from the live Session; it is never accepted from a query parameter, header or body. For the Customer-overlap check, only `status = Booked` rows with `customerId = principal.userId` participate, using persisted Booking interval fields and the half-open predicate `booking.startsAt < slot.endsAt AND slot.startsAt < booking.endsAt`. Cancelled rows do not conflict and adjacent intervals are allowed. This predicate checks existence only and never returns Booking details. Exactly 60:00 is included; 59:59 is excluded. Never use browser time or HTTP arrival time. T_view is not #16 T_elig; #16 repeats the authoritative checks after contention waits.

4. Safe errors

Condition	Status/code	Behavior
Missing/expired/revoked Session	401 UNAUTHENTICATED	No offer data
Authenticated wrong role	403 FORBIDDEN	No offer data
Unknown/malformed query	400 INVALID_INPUT	No query-dependent data
PostgreSQL unavailable/transient read failure	503 SERVICE_UNAVAILABLE	Sanitized error; no empty-list fallback
Unexpected internal failure	500 INTERNAL_ERROR	Sanitized; no SQL/stack details

Error envelope follows the existing API: `{ error: { code, message }, requestId }`. Keep `Cache-Control: no-store` on every success and error response, including 401/403/400/500/503. A failed request is not equivalent to 200 with `data: []`.

5. Endpoint boundary

No POST, Booking mutation, reservation hold, Booking confirmation or idempotency key in #15. Do not reuse Tutor own-schedule route. No owner selector, filter, detail route or private Booking read. Selection carries only slotId to #16; #16 owns transactional recheck and Booking outcomes.

6. Implementation dependency and verification status

The current sprint1 schema has no Booking model. The Slot-occupancy and Customer-overlap `NOT EXISTS` predicates against active Booked rows are **logical contracts**, not executable against the current schema. Do not ship as "all Slots are unoccupied" or add `Slot.isBooked`. The selected route is a separately scoped shared Booking data-foundation Issue/PR (route B); its Issue/PR and accepted schema SHA are not yet created/recorded. Keep Part F blocked until that foundation is reviewed, merged and its exact SHA is recorded as described in Feature SDS §2.

Conditional Tutor display-name data gate (OD-AV-02)

- Candidate source: a public display name explicitly supplied and maintained by the Tutor; never infer it from `User.email` or login identity.
- Candidate field: nullable `User.tutorDisplayName` (`String?` / PostgreSQL `TEXT`), used only for Tutor accounts. The current schema has no such field.
- Data owner: the Tutor; customer-facing publication requires explicit Tutor opt-in and recorded product/stakeholder approval. @Chxtamos owns this contract; @Tanaboonnnnn peer-reviews it. The actual approving stakeholder and consent rule must

be recorded before migration.

- Migration: a separate reviewed forward-only migration only after approval; no default and no email backfill. Existing rows remain `NULL`; seed only clearly test-only synthetic values in isolated fixtures.
- Current response: omit `tutorDisplayName`; return `tutorRef`. Only after approval may the exact allowlist add the nullable display-name field. Never add email, phone or unrestricted profile data.
- Planned privacy test: AVA-18 verifies no name is inferred from email, no unapproved field appears, and—after approval only—the approved field is the sole additional Tutor identity property.

This document defines the Part E endpoint contract only. No route, query, API test, browser test or performance result is claimed to exist or pass.

E.4.4 F-S1-03 Customer Availability Data Contract (data-spec.md)

> **Logical read contract only.** No Prisma model, SQL migration, Booking table, index, Tutor-name column or runtime query is created here. Peer-review decision states and the selected shared-data route are recorded in Feature SDS. See API, UI and STS.

1. Existing records at the Part E base

Verified sprint1 base: 551a9469e3d3a7e8e3ae004a382862c4f3e87e0e.

Record	Relevant fields	F-S1-03 use
User	id, role, isActive; no display-name field exists	Source of opaque Tutor reference; `tutorDisplayName` is only a conditional future candidate, not present in the base schema
Slot	id, tutorId, startsAt, endsAt, lifecycle	Candidate offer, owner and UTC interval
Session	persisted user/session/expiry/revocation data	Auth middleware derives Customer principal; Session fields never enter response
Booking	**Not present in current Prisma schema/migrations**	Required future occupancy source; see §2

The current Slot table has a Tutor owner index (tutorId, startsAt, id), but no isBooked column. The Tutor own-list DTO temporarily returns occupancy Unoccupied because #16 has not introduced Booking; that is not a valid Customer-availability occupancy source after Booking exists.

2. Booking relationship and selected sequencing gate

The System SDS describes Booking fields including customerId, currentSlotId, startsAt, endsAt and business status (Booked or later Cancelled). A current S1 Booked row is an Active Booking and occupies its referenced Slot. The separately reviewed shared data foundation must expose these persisted fields so #15 can check both Slot occupancy and Customer-specific overlap; #15 still does not create or mutate Booking records.

Required logical relation: Booking.currentSlotId = Slot.id AND Booking.status = Booked.

Required Customer-specific relation: `Booking.customerId = principal.userId AND Booking.status = Booked AND Booking.startsAt < Slot.endsAt AND Slot.startsAt < Booking.endsAt`. `principal.userId` is obtained from the authenticated Session, not request input. Booking interval values are the persisted snapshot of its current Slot interval. This is the half-open `[start,end)` overlap rule from requirements.md §3: a Customer's Active Booking with Tutor A excludes an overlapping offer from Tutor B; exact adjacency does not conflict. Cancelled rows are excluded. The query uses `NOT EXISTS` only and never projects the conflicting Booking, Customer ID or reason.

Neither relation can be queried in the current schema because the shared Booking data foundation has not yet been created. The team selected route B to break the #15/#16 dependency without moving Booking mutation into #15. Part E records the correct data source but does not invent Slot.isBooked, assume all Slots remain unoccupied, or create a partial Booking schema under #15.

Selected route B — separate shared data foundation: create a dedicated Issue/PR for the shared Booking schema/migration, with #15 and #16 as dependency links. Proposed implementation owner is @Chxtamos and peer reviewer is @Tanaboonnnnn. The exact foundation Issue/PR, accepted migration and schema SHA are not yet created/recorded; this Part E PR does not create or approve them. Part F remains a hard **STOP GATE** until the shared foundation is reviewed, merged and the exact schema SHA is recorded. Booking mutation remains in #16; #15 only reads the persisted rows.

3. Proposed logical read projection

The following is **pseudocode**, not executable SQL against the current base. `$1` is a bound parameter populated only from `principal.userId` after `requireSession` / `requireRole(Customer)`; it is never copied from client input. The statement returns exactly one aggregate row even when `eligible_slots` is empty, so the API can always build `meta.evaluatedAt` from the same T_view.

```
WITH view_clock AS MATERIALIZED (
  SELECT clock_timestamp() AS evaluated_at
),
eligible_slots AS MATERIALIZED (
  SELECT
    s."id" AS "slotId",
    s."tutorId" AS "tutorRef",
    s."startsAt" AS "startsAt",
    s."endsAt" AS "endsAt"
  FROM "Slot" AS s
  CROSS JOIN view_clock
  WHERE s."lifecycle" = 'Offered'
    AND s."startsAt" >= view_clock.evaluated_at + INTERVAL '60 minutes'
    AND NOT EXISTS (
      SELECT 1
      FROM "Booking" AS occupied
      WHERE occupied."currentSlotId" = s."id"
        AND occupied."status" = 'Booked'
    )
    AND NOT EXISTS (
      SELECT 1
      FROM "Booking" AS customer_conflict
      WHERE customer_conflict."customerId" = $1
        AND customer_conflict."status" = 'Booked'
        AND customer_conflict."startsAt" < s."endsAt"
        AND s."startsAt" < customer_conflict."endsAt"
    )
)
SELECT
  COALESCE(
    jsonb_agg(
      jsonb_build_object(
        'slotId', eligible_slots."slotId",
        'tutorRef', eligible_slots."tutorRef",
        'startsAt', to_char(
          eligible_slots."startsAt" AT TIME ZONE 'UTC',
          'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"'
        ),
        'endsAt', to_char(
          eligible_slots."endsAt" AT TIME ZONE 'UTC',
          'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"'
        )
      )
      ORDER BY eligible_slots."startsAt" ASC,
               eligible_slots."tutorRef" ASC,
               eligible_slots."slotId" ASC
    ) FILTER (WHERE eligible_slots."slotId" IS NOT NULL),
    '[]'::jsonb
  ) AS "data",
  to_char(
    view_clock.evaluated_at AT TIME ZONE 'UTC',
    'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"'
  ) AS "evaluatedAt"
FROM view_clock
LEFT JOIN eligible_slots ON TRUE
GROUP BY view_clock.evaluated_at;
```

Intent: sample DB clock once as T_view; use persisted Slot lifecycle/interval; exclude active Booked rows on the candidate Slot; exclude any overlapping Active Booking belonging to the authenticated Customer (including another Tutor); return only the safe projection; order deterministically. The outer query starts from the one-row `view_clock` CTE and LEFT JOINs the candidate set, then aggregates with a filtered `jsonb_agg` and `COALESCE`. Therefore it returns one row with `data = []` and the same non-null `evaluatedAt` when no candidate exists. This is one SQL statement, not a second clock read or an application-clock fallback. **Before values enter JSON, every timestamp is explicitly normalized to UTC with `AT TIME ZONE 'UTC'` and formatted as `YYYY-MM-DDTHH:mm:ss.sssZ` by `to_char`; no raw `timestamptz` is passed to `jsonb_build_object`.** This makes the wire representation independent of the PostgreSQL connection `TimeZone`. Exact SQL/Prisma query and read-isolation require peer review and a real PostgreSQL test after Booking schema exists. This pseudocode has not been run.

4. Time and timezone

- PostgreSQL stores startsAt/endsAt as TIMESTAMPTZ.
- The SQL response projection converts `startsAt`, `endsAt` and `T_view` to UTC before JSON construction: `to_char(value AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"')`. The wire values have exactly three fractional-second digits and end in `Z` (for example `2026-11-08T06:00:00.000Z`). Never put a raw `timestamptz` directly into `jsonb_build_object` or rely on the connection `TimeZone` to serialize it.
- API returns these already-normalized UTC RFC 3339 strings unchanged; it does not reinterpret a timezone-dependent database timestamp string.
- UI always formats Asia/Bangkok (UTC+07:00), independent of browser timezone.
- T_view is the single PostgreSQL time for this read; startsAt - T_view >= 60 minutes is the confirmed Customer-availability read rule and equality passes.
- A Customer's own `Booked` rows are checked by persisted `[startsAt, endsAt)` intervals; only strict overlap is excluded, so adjacency passes. The principal comes from the Session.
- #16 Booking write uses its own authoritative T_elig after contention waits; it must not trust T_view, evaluatedAt, browser time or prior availability.
- OD-002 (maximum advance horizon) remains OPEN. This contract does not choose an upper bound and does not imply an approved unlimited horizon. Resolve OD-002 before Part F; do not invent work-hour/day, holiday or start-minute rules either.

5. Response projection and privacy

Property	Source	Type	Privacy
slotId	Slot.id	UUID string	Safe selection ID, not Booking ID
tutorRef	Slot.tutorId	UUID string	Current opaque Tutor reference; no email/name/phone
tutorDisplayName	Not present in the current schema; conditional future `User.tutorDisplayName`	Nullable string only after separate data approval and migration	Not returned before approval; never derived from email; Tutor opt-in and publication approval required
startsAt	Slot.startsAt	UTC ISO string	Required time
endsAt	Slot.endsAt	UTC ISO string	Required time
meta.evaluatedAt	T_view	UTC ISO string	Read evaluation time, not Booking guarantee
meta.requestId	Existing middleware	opaque string	Correlation only

Never select/serialize Customer ID/name/email/phone, Booking ID/status/reason, Session ID/token/CSRF, password hash or arbitrary Prisma columns. Request fields cannot choose another Customer/Tutor owner. Tutor UUID is not a credential and grants no authorization; UI compaction is presentation only.

The API must set `Cache-Control: no-store` for populated and empty success responses and every error response. `meta.evaluatedAt` is sourced from the one-row DB clock aggregate in §3 even when `data = []`; do not fall back to an application/browser clock.

Conditional Tutor public-display-name source and migration gate

- **Current source fact:** `User` has no display-name/profile field in the reviewed base schema. `email` is an authentication identifier and is never a display-name source.
- **Candidate source/field:** an explicit public display name supplied and maintained by the Tutor, stored as nullable `User.tutorDisplayName` (`String?` in Prisma / PostgreSQL `TEXT`), used only for Tutor accounts. This field is a proposal only; it is not added by this PR.
- **Data owner/permission:** the Tutor owns and supplies the value and must explicitly opt in to customer-facing display. The product/stakeholder owner must record publication approval and consent wording. @Chxtamos is the feature-contract owner; @Tanaboonnnnn is the peer reviewer. The actual approving stakeholder and consent rule must be recorded before any migration.
- **Migration/seed:** add the nullable field only through a separate reviewed forward-only migration after approval; no default, no backfill from email, and existing Tutor rows remain `NULL`. Use synthetic names only in isolated test fixtures clearly marked test-only; never make fixture data appear as a real Tutor identity.
- **DTO rule:** until the gate is cleared, omit `tutorDisplayName` entirely and return `tutorRef`. After approval, add only this explicitly approved field to the response allowlist; keep email, phone and other profile fields excluded.
- **Privacy test:** AVA-18 checks absence before approval and checks the exact approved field/allowlist after approval; it also verifies that an email value is never used as a fallback name.

6. Storage and mutation boundary

F-S1-03 is read-only: no record writes, tables, columns, migrations, indexes, triggers or occupancy denormalization. It does not alter F-S1-02 Slot owner, immutability, duration or overlap constraints. Booking persistence and transaction integrity belong to #16.

7. Part F data verification obligations

1. Seed at least two Tutors and two Customers with test-only opaque IDs.
2. Create real future Offered Slots for multiple Tutors, including same-time offers from different Tutors.
3. With no active Booking, verify expected offers.
4. With a committed Booked row for one Slot, verify it is excluded while other offers remain; inspect full JSON for no Customer/Booking leakage.
5. Verify T_view equality (60:00 included) and below-boundary (59:59 excluded) with controlled DB time.
6. Verify deterministic order, UTC serialization and identical Bangkok display from UTC/other-timezone browsers.
7. Inject read failure; verify no false empty list; retry successfully.
8. Verify role/session isolation, session binding and no stale data after logout/account switch.
9. Verify a Customer's overlapping Active Booking on Tutor A hides an overlapping free Slot on Tutor B for that Customer (AVA-15).
10. Verify exact adjacency is allowed under `[start,end)` (AVA-16); Cancelled rows and another Customer's Active Booking do not create a conflict for the current Customer (AVA-17).
11. Verify Tutor display-name field is absent until the data gate is accepted; after approval, only the approved `tutorDisplayName` field is projected and no email/phone or unapproved profile data leaks (AVA-18).

All are PLANNED / NOT RUN in Part E. Occupancy- and Customer-overlap-dependent checks remain BLOCKED until the separate shared Booking data foundation is reviewed, merged and its exact schema SHA recorded. The positive Tutor-name projection check remains BLOCKED until its separate data-approval and migration gate is cleared.

E.4.5 F-S1-03 Customer Availability Software Test Specification (tests.md)

> **Planned STS only.** All cases are NOT RUN by this documentation PR. Booking-query cases require the separately reviewed shared data foundation selected in Feature SDS §2; only AVA-11 requires the Issue #16 Booking endpoint/transaction. Tutor-name positive-path checks require the separate OD-AV-02 data approval and migration gate.

1. Traceability and planned cases

STS ID	Trace	Scenario	Required result	Dependency
AVA-01	FR-S1-08, AC-S1-15, VC/TEST-S1-15, VAL-006	C1 requests availability with an eligible future Slot	200; approved Tutor display name if gate is cleared, otherwise opaque Tutor reference; Slot ID and Bangkok start/end identifiable; deterministic order	Shared Booking data foundation + #13/#14; name field remains gated
AVA-02	FR-S1-09, AC-S1-16, BR-S1-08/21, VC/TEST-S1-22	One Slot has a committed active Booked row; another is free	Occupied Slot not returned as bookable; other offer remains; no Customer/Booking details	Reviewed shared Booking foundation migration + committed test-only Booking rows in disposable PostgreSQL; no #16 endpoint required
AVA-03	FR-S1-09, AC-S1-16, BR-S1-07/11, VC/TEST-S1-17/18, VAL-008/009	Evaluate a past Slot, exactly 60:00 and 59:59 before start using controlled DB time	Past/59:59 excluded; exactly 60:00 included; browser clock irrelevant	Shared Booking data foundation + DB clock harness
AVA-04	AC-S1-18, BR-S1-12, VC/TEST-S1-13	Request the same populated fixture with one controlled identical DB `T_view` and PostgreSQL connection `TimeZone` set to `UTC` and `Asia/Bangkok`; also render in Bangkok, UTC and another browser timezone	Wire `startsAt`, `endsAt` and `meta.evaluatedAt` are byte-for-byte identical UTC `...sssZ` strings under both DB settings; Bangkok display is the same intended local date/time under every browser timezone	Reviewed shared Booking foundation migration + populated disposable-DB fixture; DB-session and browser timezone matrix
AVA-05	FR-S1-08, NFR-S1-01/08, NFR-007, VAL-036	Inspect full JSON with multiple Customers/Tutors	Exact allowlist only; no Customer PII, Booking ID/status/reason, email, Session/token/hash, SQL or stack; Tutor identity is `tutorRef` until name approval	Shared Booking data foundation; name field remains gated
AVA-06	BR-S1-17, NFR-S1-01, VC/TEST-S1-05/06, VAL-002	Anonymous and Tutor call Customer endpoint	Anonymous 401; Tutor 403; no offer data	#13/#14 merged
AVA-07	BR-S1-20, NFR-S1-01	Customer adds customerId/tutorId/role/page/arbitrary query	400 INVALID_INPUT; identity/owner scope cannot be changed	#13/#14 merged
AVA-08	AC-S1-15, NFR-S1-07	Seed same-time offers from different Tutors	Stable startsAt/tutorRef/slotId order; no duplicates/hidden rows	Shared Booking data foundation + #14 merged
AVA-09	AC-S1-15, NFR-S1-07/08, VAL-036	With a controlled identical DB `T_view` and no eligible offers, repeat the empty query with PostgreSQL connection `TimeZone` set to `UTC` and `Asia/Bangkok`	Both wire responses return `data: []`, the same non-null `meta.evaluatedAt` formatted as UTC `...sssZ`, and `Cache-Control: no-store`; the aggregate still returns one metadata row; errors are never mapped to empty	Reviewed shared Booking foundation migration + empty disposable-DB fixture; DB-session timezone matrix
AVA-10	FR-S1-18, NFR-S1-07/08, VAL-036	GET/retry fails 503/network then later succeeds	Error/retry visible; no empty/zero claim during failure; successful retry restores results	#13/#14 merged

STS ID	Trace	Scenario	Required result	Dependency
AVA-11	AC-S1-17, FR-S1-09/11, BR-S1-13/21, VC/TEST-S1-22	Customer selects offer; it becomes occupied or enters Q05 cutoff before Booking	#16 rejects without Booking/false success; UI shows non-success and refreshes	**#16 Booking transaction/endpoint required**
AVA-12	NFR-S1-01/08, BR-S1-20, VAL-036	Hold old Customer 200; logout/switch Session before releasing it	Old response never repaints; clear offers/selection on Session binding mismatch	#13/#14 merged
AVA-13	NFR-S1-09, VAL-033	Keyboard-only browse/select through loading/populated/empty/error at 320/375/768/1280 CSS px	Logical focus order, visible/unobscured focus, semantic loading/result/empty/error announcements, responsive reflow, actionable retry and measured target dimensions; 44px is a design target, not a pass without evidence	Part F manual/browser check
AVA-14	NFR-S1-10, VC/TEST-S1-36	Benchmark local seeded availability GET after warm-up	Record machine/DB/sample/p95/source SHA and workload; ≤2.0s is reference-only and must not determine pass/fail	Part F measurement; no threshold gate
AVA-15	FR-S1-09/13, AC-S1-16, BR-S1-09, Q28, VAL-011	C1 has Active Booked interval 13:00–14:00 with Tutor A; a free Tutor B Slot is 13:30–14:30	Exclude Tutor B Slot for C1 because the authenticated Customer already has an overlapping Active Booking on another Tutor; do not return the conflicting Booking or Customer details	Reviewed shared Booking foundation migration + committed test-only Booking rows in disposable PostgreSQL; no #16 endpoint required
AVA-16	FR-S1-09/13, AC-S1-16, BR-S1-10, Q28	C1 has Active Booked interval 13:00–14:00; a free Tutor B Slot is exactly 14:00–15:00	Include the candidate because `[start,end)` intervals are adjacent, not overlapping; strict predicate permits equality at the boundary	Reviewed shared Booking foundation migration + committed test-only Booking rows in disposable PostgreSQL; no #16 endpoint required
AVA-17	FR-S1-09/13, BR-S1-09/10, Q28	A free candidate Slot overlaps either a Cancelled C1 Booking or an Active Booking of C2 on a different Slot/Tutor	Include the candidate for C1 in both controls; only `customerId = principal.userId AND status = Booked` participates in the Customer-overlap predicate. A separate Active Booking on the candidate Slot itself must still exclude it for everyone under AVA-02	Reviewed shared Booking foundation migration + committed test-only Booking rows in disposable PostgreSQL; no #16 endpoint required
AVA-18	FR-S1-08, NFR-S1-01/08, VAL-036	Inspect Tutor identity before and after OD-AV-02 data approval	Before approval, no `tutorDisplayName` field and no email-derived name; after approved migration/consent, only the approved nullable display-name field may be added to allowlist; no email/phone/profile leakage	Current schema for negative case; separate data approval + migration for positive case
AVA-19	OD-AV-08, AC-S1-15	Compare Customer Availability with Tutor Dashboard at desktop and mobile breakpoints	Shared NutTiew wordmark, typography, spacing, brand/nav colors, button/focus components and navigation behavior; only role-authorized destinations differ	Part F browser visual review

2. Fixtures and execution method

- Use disposable real PostgreSQL with accepted Auth/Slot migrations and the reviewed Booking migration when available.
- Seed test-only C1/C2 Customers and T1/T2 Tutors; do not commit credentials, tokens, real records or DB dumps.

- Seed future Slots relative to DB time, not fixed dates that expire.
- For AVA-02/15/16/17, apply the reviewed shared Booking foundation migration to a disposable PostgreSQL database, then create actual persisted `Booked` / `Cancelled` rows with test-only seed SQL/fixtures that satisfy the schema's constraints. Commit the rows in the DB transaction and query them directly; these read-predicate cases do not call or wait for the #16 Booking endpoint. Never simulate occupancy with Slot.isBooked, bypass DB constraints, or fabricate response-only Booking objects.
- For AVA-09 use the same reviewed foundation in a disposable DB and a controlled DB clock; test both populated/empty projection semantics under connection `TimeZone = 'UTC'` and `'Asia/Bangkok'`.
- For AVA-11 use the real #16 Booking endpoint and its committed transaction outcome; never simulate a successful or rejected Booking in the UI.
- For AVA-15/16/17 use actual committed `Booked` and `Cancelled` rows for two Customer principals and multiple Tutors; assert owner filtering and half-open boundaries from persisted Booking intervals, never from fake Slot flags.
- For AVA-18 do not derive a display name from email. Run the positive allowlist case only after the Tutor-provided field, opt-in, approving stakeholder and forward-only migration are accepted; test fixtures must be explicitly synthetic/test-only.
- For AVA-19 compare actual Customer/Tutor browser renders and computed shared design tokens; do not infer visual consistency from source-only screenshots or claim accessibility conformance from token comparison.
- API tests inspect full response body/headers; for AVA-04/09 set PostgreSQL connection `TimeZone` to `UTC` and `Asia/Bangkok` while holding persisted instants (and `T_view` for AVA-09) constant, then assert identical wire timestamps for `startsAt`, `endsAt` and `meta.evaluatedAt`, each matching `YYYY-MM-DDTHH:mm:ss.sssZ`. Playwright/browser tests separately verify Bangkok rendering across browser timezones.
- Use controlled DB time or deterministic query seam for 60:00/59:59; no sleeps or browser clock.
- Record exact command, environment, Node/PostgreSQL version, migration, source SHA and PASS/FAIL/SKIP/BLOCKED plus observed rows.

3. Error/UI observations

API outcome	UI behavior
200 with offers	Render approved Tutor display name only after its gate, otherwise `tutorRef`; preserve server order
200 with data: []	Confirmed-empty state; metadata still includes `evaluatedAt` and response is `no-store`
401	Clear protected data and return to signed-out Auth
403	Access denied; no availability data
400	Safe invalid-request state; do not auto-retry unchanged input
500/503/network/timeout	Error + retry; never empty/success
Later #16 stale/occupied/cutoff rejection	Non-success; refresh; do not claim Booking
Session-binding mismatch/account switch	Clear offers/selection and ignore stale responses

4. AC coverage

Acceptance criterion	STS
AC-S1-15	AVA-01, AVA-05, AVA-06, AVA-08, AVA-09, AVA-18
AC-S1-16	AVA-02, AVA-03, AVA-15, AVA-16, AVA-17
AC-S1-17	AVA-11
AC-S1-18	AVA-04

5. Part E boundary and Part F evidence

- AVA-01..19 are planned; none is a test PASS from this Part E PR.

- All DB-backed availability-query cases (AVA-01..05, AVA-08..10, AVA-12 and AVA-15..17) are BLOCKED until the separately reviewed shared Booking data foundation is merged and its exact schema SHA is recorded. AVA-02/15/16/17 can then use committed test-only rows in a disposable DB without the #16 endpoint. AVA-11 alone additionally requires the #16 Booking transaction/endpoint; #16 owns Booking mutation/revalidation.
- AVA-18 positive-path display-name verification is BLOCKED until the Tutor data source, opt-in/publication approval, migration and allowlist amendment are accepted. Its pre-approval absence/email-leakage checks remain planned.
- AVA-13/19 are manual/browser checks; AVA-14 records p95 only. No WCAG 2.2 AA, 44px target or ≤2.0s performance pass is claimed or used as a gate here.
- No test, migration, API route, query, browser run, benchmark or manual accessibility audit is executed by this STS.
- Part F may run Auth/Slot regressions independently, but must not claim the full Customer availability query works until the shared Booking foundation exists. Then run AVA-01..10, AVA-12 and AVA-15/16/17 with real data; run AVA-11 after the #16 Booking endpoint exists; run AVA-18 positive path only after the name-data gate; and perform AVA-19 browser design-system comparison plus timezone/responsive checks at 320/375/768/1280 CSS px and keyboard/focus/announcement review. Record actual commands/results/screenshots and the exact implementation/schema SHAs.
- Do not claim full WCAG 2.2 AA without a complete manual assessment.

E.5 F-S1-04: Customer Exclusive Booking & Concurrency Protection

E.5.1 F-S1-04 Exclusive Booking Feature SDS (specification.md)

> **Documentation contract only; not implemented, executed or peer-approved yet.** Issue #16; owner @Chxtamos; independent reviewer @Tanaboonnnnn. This Part E records the user's Sprint 1 Booking decision for OD-013 and selects the PostgreSQL transaction design for review.

1. Authority and baseline

Sources: Sprint 1 requirements (FR-S1-10..13, 16, 18..19; BR-S1-01, 07..13, 18..21; NFR-S1-01, 03..08, 11..12; AC-S1-19..29 and AC-S1-35; VC/TEST-S1-16..27, 33, 38, 39), SRS (Q01/Q02/Q05/Q28..Q30, J-03, OD-013), System SDS (§§3.2–3.3, 6, 7.1, 7.3–7.5), Feature Inventory, and the merged #33 Foundation SDS, data contract and BDF STS.

Accepted dependencies: Auth #13, Tutor Slots #14, Customer Availability #15.

Verified branch point: `sprint1` at `ea36ef6fd3ea7e2c2582489d1326adbd335452ee`. PR #34 is merged at this commit. It accepts #33 **Part E documentation only**, not a Booking schema/migration SHA. #33 Part F must merge and publish its exact schema SHA before #16 Part F implements Booking-dependent code.

Ownership: #33 owns the canonical Booking model, migration, constraints, one `booking_guard_s1()` trigger and runtime-role boundary. #16 owns the Booking API/application transaction adapter and post-merge BDF-23. #15 remains read-only; #17 owns protected Booking retrieval. #16 must not duplicate #33's model, migration or trigger.

2. Outcome and journey

An authenticated Customer requests one selected Slot. The server derives Customer identity from the live Session, actual Tutor and interval from the persisted Slot, revalidates eligibility under the transaction protocol, and commits one `Booked` row. A new Booking is confirmed only after PostgreSQL acknowledges COMMIT.

1. Customer selects an offer in #15; selection is transient UI state, not a reservation.
2. Customer confirms one selected Slot; the request contains only `slotId`.
3. API authenticates a live Customer Session and derives owner from `principal.userId`.
4. Transaction discovers the actual Tutor, acquires all affected User rows in ascending UUID order and then all affected Slot rows in ascending UUID order, locks and revalidates the authenticated Customer's current Session on the same connection, and only then reads/replays or inserts Booking. After a recognized unique/exclusion failure and ROLLBACK, a fresh READ

COMMITTED reconciliation revalidates the current User's active Customer role, then the exact Session, before any Booking read.

5. New row is inserted through the shared #33 guard; Q05 is decided using DB `T_elig` after the Slot lock.
6. Only after COMMIT does API return 201. A repeated same-Customer/same-Slot request returns 200 and the original Booking ID without another INSERT.
7. Invalid, occupied, overlapping, too-soon, unauthorized or failed attempts never produce false success or partial Booking.

3. Scope

In scope: Customer-only Booking creation; request/response/error contract; trusted owner derivation; canonical #33 data mapping; PostgreSQL SERIALIZABLE and the User → Slot → Booking domain lock order, with the authenticated Customer Session as an auth sub-lock after User/Slot locks and before Booking access; bounded full-transaction retry; Q05 and no-post-guard-contention proof; natural-key replay; confirmation/pending/conflict/uncertain UI states; planned STS mapped to existing AC/VC IDs.

Out of scope: #33 schema/model/migration/trigger implementation; #15 availability reads; #17 protected Booking list/detail; cancellation, reschedule, withdrawal, notices, payment, attendance, `Completed` or S2/S3 behavior; any Idempotency-Key, registry table, payload fingerprint, replay TTL/window column, new Booking column or new natural-key DB constraint.

4. Invariants

Invariant	F-S1-04 contract
Ownership	`customerId = principal.userId` from live active Customer Session; never trust client owner/role.
User authorization	Booking belongs to the current active User only while `User.isActive=true` and `User.role='Customer'`. After post-ROLLBACK, revalidate the latest User row from the original `principal.userId` before Session or Booking access; missing/inactive → 401, active wrong role → 403, no Booking DTO.
Session validity	HTTP `requireSession` is preliminary only. Lock `principal.sessionId` on the SERIALIZABLE mutation connection after User and Slot lock waits; recheck `revokedAt IS NULL` and `expiresAt > clock_timestamp()` before replay/INSERT. After a recognized `23505` / `23P01` ROLLBACK, a fresh READ COMMITTED reconciliation locks/revalidates current User first, then the same Session before any Booking read/200/409; invalid Session returns 401 with no DTO.
Tutor / Slot	`currentSlotId` is selected persisted Slot. Tutor derives through `Slot.tutorId`; Booking does not duplicate `tutorId`.
Interval	Copy persisted Slot `startsAt` / `endsAt`; never accept either timestamp or duration from client.
State	New Sprint 1 Booking status is `Booked`; `Past` is display-only and does not release occupancy.
Exclusivity	At most one Active (`Booked`) row per Slot.
Customer overlap	No positive-duration overlapping Active Bookings for one Customer across Tutors; half-open `[start,end)` means adjacency is allowed.
Q05	At authoritative DB `T_elig`, `Slot.startsAt - T_elig >= 60 minutes`; exact 60:00 passes, 59:59 fails.
Atomicity	No success before COMMIT; rejected/aborted attempt leaves no partial Booking or phantom occupancy.
Lifecycle	Sprint 1 Booking is append-only; this feature performs INSERT only.

5. Decision record — OD-013, Sprint 1 Booking only

USER-APPROVED — 7 November 2026: use **Natural-key idempotency without an Idempotency-Key**. The existing active Booking pair `(authenticated customerId, slotId)` identifies a repeated request:

- First successful creation: **201 Created** with the committed Booking ID.
- Same authenticated Customer submits the same Slot again after a successful commit: **200 OK** with the **original Booking ID**; do not insert a second row.
- Different Customer attempts a Slot already occupied by another Customer: **409 Conflict**, without disclosing the other Customer or Booking.
- Same Customer requests a different Slot that overlaps an existing Active Booking: **409 Conflict**.

- If the first response is lost or COMMIT outcome is unknown, retry the **same POST with the same** `slotId`. If the original row committed, reconcile it and return its original ID.

The pair is an application-level lookup/reconciliation key, **not** a new persisted key or database unique constraint. No key format, registry table, fingerprint or replay TTL is added. The committed #33 Booking row is the durable source of truth. A matching row qualifies for replay only while its status is `Booked`; future cancellation/rebooking behavior is outside this Sprint 1 contract.

This resolves the **Booking-create portion** of SRS OD-013 for F-S1-04. OD-013 remains open for future S2 reschedule/cancel operations until those features have their own reviewed contracts. The decision is recorded in Issue #16.

Replay remains valid after later Tutor account changes: if the same-pair `Booked` row is already committed, an active Customer with a live Session still receives 200 and the original Booking ID even if that Slot's Tutor is subsequently deactivated or changes away from role `Tutor`. Current Tutor `role='Tutor'` / `isActive=true` eligibility is enforced only when attempting a **new INSERT**. An ineligible Tutor makes a new claim return generic 409 `SLOT_UNAVAILABLE`, without disclosing Tutor identity/status; it does not revoke or hide an existing Booking replay.

6. Engineering selections for peer review

- PostgreSQL **SERIALIZABLE** for every Booking mutation attempt.
- Preserve the System SDS **domain** lock order: all affected User rows in ascending UUID order → all affected Slot rows in ascending UUID order → existing Booking rows in ascending UUID order only where needed. After all User and Slot locks, lock only the current Customer `Session` row (`principal.sessionId`) as an authentication sub-lock before any Booking read/lock; this does not reverse the domain order. Hold it through COMMIT. No path may lock Session and then request User, Slot or Booking locks. Existing F-S1-02 takes User → Session but creates a new Slot row without acquiring an existing Slot row lock, so it does not invert this row-lock order. A new Booking INSERT has no pre-existing Booking row to lock.
- Retry the whole transaction for SQLSTATE `40001`, `40P01` or ORM-equivalent `P2034`; maximum **three total mutation attempts**, with fresh discovery/locks and fresh `T_elig` on every new INSERT attempt.
- `23505` / `23P01` are not blind mutation retries. Only structured tuples `23505 / booking_one_active_per_slot` and `23P01 / booked_customer_no_overlap` enter reconciliation; unknown/missing/mismatched tuples fail closed as 500 `INTERNAL_ERROR` without Booking reads. After ROLLBACK, a fresh READ COMMITTED connection locks/revalidates current User (`isActive`, `role='Customer'`) first, then the exact Session (`userId`, `revokedAt`, DB-clock `expiresAt`) before any Booking read. Missing/inactive User → 401; active wrong role → 403; invalid Session → 401; DB validation unavailable → 503. Only a currently active Customer with a live Session may receive 200 original ID for a same-pair committed row, or the cause-specific 409 if no same-pair row exists.
- Revalidate the live Session on the same SERIALIZABLE connection **after all User and Slot lock waits**: lock `principal.sessionId` with `FOR UPDATE`; verify `userId = principal.userId`, `revokedAt IS NULL` and `expiresAt > clock_timestamp()` before replay/INSERT. A revocation committed during either lock wait is observed after the wait or causes `40001` and a fresh transaction; expiry during a wait fails the DB-clock check. Never return 200 replay or 201 from an invalid Session. A missing exact Session row/owner mismatch at the transactional lock returns 401 immediately; only a PostgreSQL `40001` caused by a concurrent row change/stale snapshot is retried. After Customer/Session authorization, same-pair replay is checked before current Tutor eligibility; an inactive/non-Tutor account rejects only a new claim with 409 `SLOT_UNAVAILABLE`.
- Reuse Auth OD-005-C: obtain the stable CSRF token from `GET /api/v1/auth/csrf`, then call `GET /api/v1/auth/me` again and compare the returned `X-NutTiew-Session-Binding` and identity with the initial result; the CSRF response itself contains only `csrfToken` and is not a binding source. If they differ, discard the token and restart bootstrap. Require `X-NutTiew-Expected-Session-Binding` on the Booking POST; a Session switch after the final recheck must fail before mutation as `SESSION_BINDING_MISMATCH` or `CSRF_DENIED`, then reset bootstrap. The UI must discard late responses after its auth/session epoch changes, including a different Session for the same User.
- #33 trigger uses PostgreSQL `clock_timestamp()` after its Slot lock. Application/browser time, HTTP arrival, transaction-start `now()`, #15 `T_view` and commit time are not authoritative. Non-Offered and Q05 trigger rejections have distinct stable SQLSTATE/constraint pairs: `P0B01 / booking_slot_not_offered` and `P0B02 / booking_q05_min_notice` respectively; #33 Part F must implement and publish this dependency.

- BDF-23 must prove no conflict-capable unique/exclusion wait can occur after `T_elig`, or prove the complete transaction aborts/retries with fresh locks and DB time. A stale-time INSERT that commits after an unaccounted post-guard wait fails the design gate.

These are engineering selections pending independent @Tanaboonnnnn review and real-PostgreSQL proof, not executed results or new stakeholder business rules.

7. Acceptance and traceability

Existing criterion	Contract interpretation	Planned STS
AC-S1-19	One eligible request commits one `Booked` row with authenticated Customer, actual Slot/Tutor relationship and copied interval.	BOOK-01
AC-S1-20 and AC-S1-21	Exact 60:00 passes; 59:59 fails at trigger `T_elig`.	BOOK-02/03
AC-S1-22	Another Customer cannot claim an occupied Slot; no private owner data is returned.	BOOK-04
AC-S1-23 and AC-S1-24	Positive same-Customer overlap across Tutors is denied; adjacency is allowed.	BOOK-05/06
AC-S1-25 and AC-S1-26	Race B (two Customers/one Slot) and Race C (one Customer/two overlapping Tutors) have separate barriers and final-row assertions.	BOOK-10/11
AC-S1-27 / VC-S1-33	Stale/invalid selection and persistence failure never produce phantom success or inconsistent occupancy.	BOOK-07/08/12/21/26/27
AC-S1-28	Customer/Tutor identity is derived from Session/Slot, not request fields.	BOOK-09
BR-S1-13 / OD-005-A/B/C	Current active Customer User and Session remain authorized after User/Slot waits and are revalidated again after ROLLBACK before reconciliation; Session-bound CSRF and binding prevent stale-session mutation or late-response disclosure.	BOOK-26..30
AC-S1-29 / VC-S1-38	Same-pair replay returns 200 and same Booking ID only for a currently active Customer with a live Session; post-ROLLBACK reconciliation returns 401/403 for invalid current User authorization and 401 for revoked/expired Session. Unknown constraint tuples never produce replay.	BOOK-13..16, BOOK-30
AC-S1-35 / VC-S1-39	Request arriving before cutoff but waiting until after cutoff is rejected by post-lock DB `T_elig`.	BOOK-17/19

Related requirements: FR-S1-10..13, 16, 18..19; BR-S1-01, 07..13, 18..21; NFR-S1-01, 03..08, 11..12; Q01/Q02/Q05/Q28/Q29/Q30; VAL-007..011/014/030. Tests are planned / NOT RUN in Part E.

8. Definition of Done

Part E — specification-only PR

- [ ] Feature SDS, UI, API, data, transaction and STS documents are cross-linked and consistent.
- [ ] User-approved OD-013 Sprint 1 Booking decision is recorded without closing future S2 retry decisions.
- [ ] PostgreSQL SERIALIZABLE, User → Slot → Session(auth sub-lock) → Booking acquisition, bounded whole-transaction retry and Q05 authority are explicit; after post-ROLLBACK, current User authorization is revalidated before Session, and both are revalidated on a fresh READ COMMITTED connection before Booking reads/200/409.
- [ ] Auth CSRF is acquired through the existing endpoint, tied to the current Session binding, and stale responses are ignored after logout/account/session switch.
- [ ] Non-Offered and Q05 trigger failures have distinct, stable #33 SQLSTATE/constraint mappings and separate STS assertions; unique/exclusion reconciliation allowlists only the exact canonical SQLSTATE/constraint pairs and fails closed for unknown constraints.
- [ ] Same-pair `Booked` replay is checked before occupied/Q05 rejection; returns original ID without INSERT.
- [ ] No-post-guard-contention / BDF-23 is a hard proof gate and is not claimed to have passed.
- [ ] AC-S1-19..29/35 and VC/TEST-S1-16..27/33/38/39 are traceable to planned cases.

- [ ] No Prisma schema, migration, API implementation, UI implementation or executable test is added.
- [ ] Markdown links/structure/IDs and `git diff --check` pass.
- [ ] PR targets `sprint1`, links #16, requests @Tanaboonnnnn and stays open for independent human review.

Part F — separate implementation PR

Only after Part E acceptance and #33 Part F schema merge: consume exact schema SHA; implement Customer POST/transaction adapter against the one canonical #33 model/guard; TDD authorization/replay/conflict/Q05/Race B/C/rollback; run BDF-23 with real PostgreSQL; record commands, environment, schema/source SHA and actual results; open a separate reviewed code PR into `sprint1`.

Verification boundary: Part E is documentation only. No build, migration, application test, API call, browser test or BDF case is claimed as executed.

E.5.2 F-S1-04 Customer Booking Confirmation UI Specification (ui-spec.md)

> **Part E UI contract only; no component, route or browser test is implemented.** This flow completes the selection hand-off from F-S1-03 Availability; it does not turn selection into a reservation until the server commits.

1. Screen and shared design system

Screen: **ยืนยันการจองช่วงเวลาติว** — confirm one selected offer as an authenticated Customer.

Reuse the NutTiew design system shared with Customer Availability and Tutor Dashboard; do not introduce a separate booking palette.

Token	Booking confirmation use
Brand/navigation	Text-only NutTiew wordmark; shared navy `#202B44`
Canvas/surface	Light canvas `#F7F8FB`, white surface `#FFFFFF`; no nested cards
Text	Dark ink `#172137`; muted text must remain contrast-safe
Primary action	Orange `#F47736`, dark-ink label/icon `#172137`, border `#D96123`; never white text on orange
Focus	Visible ring `3px solid #AA4519`, `outline-offset: 3px`; never clipped
Typography	Noto Sans Thai, Sarabun, system-ui, sans-serif
Spacing/corners	Shared 4/8/12/16/24/32px scale and 8px radius

Keep Tutor label/reference and Bangkok appointment date/time clear. No fake profile, rating, price, payment, contact, testimonial, message panel, decorative emoji or unrelated content.

2. Journey and content

1. Call `GET /api/v1/auth/me` with the existing HttpOnly Session cookie. Require role `Customer`; capture the response's `X-NutTiew-Session-Binding` value with the identity.
2. Call `GET /api/v1/auth/csrf` with the same cookie and obtain the stable Session-bound HMAC token. Its current response contains only `csrfToken`; it does **not** return `X-NutTiew-Session-Binding`, so do not claim to compare a binding from this response. Keep the token in memory only. Then call `GET /api/v1/auth/me` again with the same cookie and compare its returned binding and identity with the first `/auth/me` result. If either changed (for example, Session A changed to Session B between the two GETs), discard the token and captured state and restart identity/CSRF bootstrap. The CSRF read-only GET may omit `Origin` under the Auth contract, but explicit `Sec-Fetch-Site: cross-site` is rejected. If the cookie changes after this final `/auth/me` check but before POST, the captured expected binding and Session-bound CSRF token remain the final server-side guard: `SESSION_BINDING_MISMATCH` or `CSRF_DENIED` must reject before mutation; clear identity/CSRF state and restart bootstrap.
3. Customer selects a Slot in #15. Selection is transient state, **not** a Booking.

4. Confirmation summary shows only Tutor display name if #15's data gate permits it (otherwise opaque Tutor reference), Bangkok date and 24-hour start/end.
5. Primary action: **ยืนยันการจอง**. Send only `slotId` in JSON, plus `X-CSRF-Token` and the captured `X-NutTiew-Expected-Session-Binding`; apply the existing canonical Origin/Referer and Fetch Metadata rules. Never send `customerId`, `tutorId`, Booking ID, status, interval or cutoff time.
6. Capture the current in-memory auth/session epoch, binding and `slotId` when POST starts. Apply a response only if all still match the current auth context; otherwise discard it without displaying the old Booking ID or details.
7. Show confirmed state only after valid 201 or natural-key replay 200 for the still-current Session.
8. Do not add a private Booking list/detail route; #17 owns protected retrieval.

The Session binding is a non-credential response/request consistency value, not authorization and not a replacement for the HttpOnly cookie or CSRF token.

3. UI states

State	Trigger	Required behavior
Session loading	`/auth/me` pending	Hide private selection details; show progress
CSRF loading	`/auth/csrf` pending for the captured Session binding	Keep confirmation disabled; retain token only in memory
CSRF/session mismatch	Binding changes between `/auth/me` and `/auth/csrf` or POST returns `SESSION_BINDING_MISMATCH`	Discard token/selection tied to old binding; refresh identity and CSRF before a new confirmation
Signed out	`/auth/me` returns 401	Clear selection/pending state; show Auth
Wrong role	Principal is not Customer	Deny flow; do not submit
Ready	Customer verified and #15 offer selected	Show summary and one enabled confirm action
Submitting	POST in flight	Disable repeated activation; show progress; retain exact `slotId`
New success	201 after COMMIT	Show "จองสำเร็จ"; display returned Booking ID and confirmed time
Replay success	200 same-pair replay	Show same Booking ID and indicate booking already exists; no new local item
Conflict	409 occupied Slot / non-Offered Slot / Customer overlap	No success; explain selected time is unavailable; refresh #15
Not eligible	422 Q05 minimum-notice cutoff (59:59 or less)	No success; explain offer can no longer be booked; refresh
Unknown outcome	Network loss, timeout or `503 BOOKING_OUTCOME_UNKNOWN`	Do not assert success/failure; preserve same `slotId`; offer **ส่งคำขอเดิมอีกครั้ง**
Service unavailable	503 known no-commit result	Safe error; retain selection for explicit retry
Session changed	Logout, login, account switch, or Session A → Session B (including two Sessions for the same User)	Increment auth/session epoch; clear private summary, selected Slot, pending intent, CSRF token and binding; ignore any late response whose captured epoch/binding differs. Never render its Booking ID/details in the new Session.

4. Natural-key retry interaction

- Pending intent is current authenticated Customer plus selected `slotId`; no Idempotency-Key is generated or stored.
- After network failure or ambiguous COMMIT, retry the **same POST body**. Do not silently substitute a newly selected Slot.
- While outcome is unknown, do not offer a different Slot as a replacement for the unresolved intent. First retry/reconcile the original request.
- Same-pair existing `Booked` row returns 200 and original Booking ID; never generate a new ID locally.

- A different Slot is a distinct request and must pass all normal eligibility/overlap rules.
- If page closes and in-memory selection is lost, this contract does not invent a client key or server registry. Once #17's protected own-Booking read is integrated, it provides the normal post-refresh recovery view; #16 alone does not claim cross-reload recovery.
- Clearing UI state on logout/account switch is a privacy boundary, not proof that an already-sent request rolled back. If the old request committed before logout, the commit is not undone; its late response is still discarded after the auth/session epoch changes. If a stale binding from Session A is sent with Session B's cookie, the API rejects it before mutation.

5. Accessibility and interaction

- Semantic heading, visible text labels and native buttons for confirm/retry.
- Keyboard operation, logical Tab order and visible unobscured focus.
- Selected/pending/success/error states are communicated in text/programmatic status, not color alone.
- Announce submitting, confirmed, replayed, conflict and unknown outcome with a polite live region.
- Button disabling is usability feedback, never the concurrency guard.
- NFR-S1-09 remains a proposed usability target. This Part E does **not** claim WCAG 2.2 AA, 44px target-size or rendered contrast conformance; record actual evidence in Part F.

6. Privacy and acceptance mapping

Never render Customer ID/email/phone, Session/CSRF data, credentials, another Customer's Booking or private Tutor fields. Conflict messages must not reveal who occupies a Slot. Do not store credentials, Session tokens or CSRF tokens in browser storage.

Criterion	UI obligation
AC-S1-19/20/21	Confirm only from server's committed/eligibility outcome
AC-S1-22/23/24	Safe conflict/adjacency outcome; no third-party disclosure
AC-S1-25 and AC-S1-26	Truthful under concurrent outcomes; disabled button is not race safety
AC-S1-27 / VC/TEST-S1-33	No phantom success after stale selection or persistence failure
AC-S1-28	No owner/Tutor fields sent from client
AC-S1-29 / VC-S1-38	Same-Slot retry reconciles to original Booking ID
AC-S1-35 / VC-S1-39	Pre-cutoff click cannot override post-wait Q05 rejection

---

E.5.3 F-S1-04 Customer Booking HTTP/API Contract (api-spec.md)

> **Proposed API contract for independent review; not implemented or runtime-tested.** See Feature SDS, UI, data, transaction protocol, STS, and the accepted Auth API contract.

1. Transport and authorization

- Same-origin JSON under existing Express `/api/v1`; reuse live-session middleware, request ID, safe error envelope and `Cache-Control: no-store`.
- Require a live active Session and `requireRole('Customer')`. Derive `customerId` only from `principal.userId`; derive actual Tutor/interval from PostgreSQL.
- The HTTP `requireSession` /role check is preliminary, not sufficient for reconciliation: the SERIALIZABLE mutation locks/revalidates Customer and Tutor Users, then the exact `principal.sessionId` after all User/Slot waits. If a recognized `23505` / `23P01` causes ROLLBACK, fresh READ COMMITTED reconciliation must lock/revalidate the current User (`isActive` and `role='Customer'`) first, then the exact Session (`userId` , `revokedAt` , DB-clock `expiresAt`), before any Booking read/200 or business 409. Missing/inactive User → 401; active non-Customer → 403; invalid Session → 401; see transaction contract.

- Reuse Auth OD-005-C without a new CSRF scheme: obtain the stable token from `GET /api/v1/auth/csrf` using the same HttpOnly cookie; require `X-CSRF-Token` on POST. The CSRF token is valid only for the Session that issued it.
- Require `X-NutTiew-Expected-Session-Binding`, captured from `X-NutTiew-Session-Binding` on `GET /api/v1/auth/me`. It must match the current `sessionBinding(userId, sessionId)`; missing or mismatched binding returns 403 `SESSION_BINDING_MISMATCH` before mutation. This is a consistency guard, not an authentication credential.
- Unsafe POST applies the Auth canonical Origin/Referer rules and rejects explicit `Sec-Fetch-Site: cross-site`; no credentialed cross-origin CORS. The read-only `GET /api/v1/auth/csrf` may omit Origin but rejects explicit cross-site Fetch Metadata, as defined by Auth #13.
- JSON UTF-8; maximum body 4 KiB; strict key allowlist; no query parameters.
- Client must not submit `customerId`, `tutorId`, role, Booking ID, status, `startsAt`, `endsAt`, duration or `T_elig`.
- No `Idempotency-Key` is required or interpreted. Replay uses authenticated `(customerId, slotId)`.
- Browser sends the existing HttpOnly Session cookie automatically; CSRF token is held in memory only, never in JSON or persistent browser storage.

2. Endpoint

Method/path	Request	Success	Enforcement
`POST /api/v1/customer/bookings`	Exactly `{"slotId":"<UUID>"}`; `X-CSRF-Token` and `X-NutTiew-Expected-Session-Binding` headers; no query parameters	201 new committed Booking; 200 same-pair replay	Customer Session/role, Origin/CSRF/binding, User → Slot → Session locks, SERIALIZABLE, shared #33 guard and constraints

No GET/detail/list route is defined here. Protected Booking reads belong to #17; Customer Availability remains #15.

Request

```
POST /api/v1/customer/bookings
Origin: http://localhost:5173
Content-Type: application/json
Cookie: nuttiew_session_dev=<opaque>
X-CSRF-Token: <session-bound>
X-NutTiew-Expected-Session-Binding: <value-from-auth-me>

{"slotId":"c62bfb8b-2530-4a7c-979c-a9a6d16be0df"}
```

`slotId` must be a valid UUID. Unknown JSON keys, missing/null `slotId`, malformed JSON, oversized body or query parameters return 400 `INVALID_INPUT` (or existing shared 413 body-limit response). No owner or appointment interval is accepted from caller.

3. Success response

New Booking — 201 Created, only after COMMIT acknowledgement

```
{
  "data": {
    "id": "9b4d0c3f-0d4f-4c3a-a18d-8b4c9a6d52e1",
    "slotId": "c62bfb8b-2530-4a7c-979c-a9a6d16be0df",
    "startsAt": "2026-11-08T06:00:00.000Z",
    "endsAt": "2026-11-08T07:00:00.000Z",
    "status": "Booked"
  },
  "meta": {"requestId":"opaque-request-id"}
}
```

Natural-key replay — 200 OK, original Booking ID

Same DTO and same `id` as previously committed Booking; no new INSERT or Booking ID.

```
{
  "data": {
    "id": "9b4d0c3f-0d4f-4c3a-a18d-8b4c9a6d52e1",
    "slotId": "c62bfb8b-2530-4a7c-979c-a9a6d16be0df",
    "startsAt": "2026-11-08T06:00:00.000Z",
    "endsAt": "2026-11-08T07:00:00.000Z",
    "status": "Booked"
  },
  "meta": {"requestId":"new-request-id"}
}
```

Exact DTO allowlist: Booking `id`, `slotId`, UTC `startsAt`, UTC `endsAt`, business `status` and opaque request ID. Exclude `customerId`, `tutorId`, profiles, Session/CSRF fields, SQL metadata and arbitrary ORM columns. Tutor derives through Slot; #17 owns richer protected reads.

All success/error responses set `Cache-Control: no-store`. Timestamps are UTC RFC 3339 with exactly three fractional digits and `Z`; UI formats them as Asia/Bangkok. The authenticated response must not include Session/CSRF values or owner IDs. The client applies a result only if its captured auth/session epoch and binding still match; a late response is ignored after logout or Session/account switch.

4. Safe error contract

Condition	HTTP / code	Required state
Missing, expired, revoked Session, exact transactional Session row absent/owner-mismatched after the HTTP guard, or missing/inactive User (including during post-ROLLBACK revalidation)	401 `UNAUTHENTICATED`	No mutation/private data/Booking DTO; do not read/replay Booking data
Authenticated active User whose current role is not Customer (including during post-ROLLBACK revalidation)	403 `FORBIDDEN`	No mutation/private data/Booking DTO
Invalid/missing/extra request field, malformed UUID/JSON, oversized body, query parameter	400 `INVALID_INPUT` (or shared 413)	No mutation
Missing Slot	404 `SLOT_NOT_FOUND`	No Booking/private information
Slot occupied by another Customer, Slot lifecycle is not `Offered`, or the current Tutor is inactive/no longer role `Tutor` for a **new claim**	409 `SLOT_UNAVAILABLE`	Existing row preserved; no new Booking; no owner/Tutor-status disclosure
Customer has a different overlapping Active Booking	409 `CUSTOMER_OVERLAP`	No conflicting row
Q05 fails at DB `T_elig` (59:59 or less)	422 `BOOKING_NOT_ELIGIBLE`	No Booking/occupancy
Invalid/foreign Origin, explicit `Sec-Fetch-Site: cross-site`, missing/wrong/old-Session CSRF	403 `ORIGIN_DENIED` / `CSRF_DENIED`	No mutation
Missing or mismatched `X-NutTiew-Expected-Session-Binding`	403 `SESSION_BINDING_MISMATCH`	No mutation; refresh identity and CSRF bootstrap
Serialization/deadlock retry budget exhausted	503 `TRANSACTION_RETRY_EXHAUSTED`	All attempts rolled back
DB unavailable before mutation, during known rollback handling or during fresh User/Session reconciliation	503 `SERVICE_UNAVAILABLE`	No false success; never skip live-Session validation
COMMIT acknowledgement lost / outcome unknown	503 `BOOKING_OUTCOME_UNKNOWN` if possible; otherwise network failure	Do not claim rollback/success; retry same POST/ `slotId`
Unexpected internal failure	500 `INTERNAL_ERROR`	No raw SQL/stack/secrets

Tutor eligibility is checked only for a new INSERT, after active Customer/live-Session authorization and the exact same-pair replay lookup. A previously committed same-pair `Booked` row still returns 200 with its original ID if the Tutor was later deactivated or changed role; current Tutor ineligibility must not invalidate that replay. For a new claim, an inactive or non-`Tutor` account maps to the generic 409 `SLOT_UNAVAILABLE` without disclosing Tutor identity or account state.

#33 Part F must emit these stable trigger diagnostics so #16 can map by cause without guessing or exposing raw DB errors:

#33 Part F INSERT guard cause	Required PostgreSQL diagnostic	#16 HTTP mapping
Slot lifecycle is not `Offered`	Custom SQLSTATE `P0B01`; constraint diagnostic `booking_slot_not_offered`	409 `SLOT_UNAVAILABLE`
Q05 minimum-notice check fails at `T_elig`	Custom SQLSTATE `P0B02`; constraint diagnostic `booking_q05_min_notice`	422 `BOOKING_NOT_ELIGIBLE`

The SQLSTATEs are stable application-facing trigger contracts (valid five-character custom codes); the diagnostic constraint names are stable identifiers, not raw exception text. #33 owns their implementation. Unknown/unmapped trigger errors fail closed as 500 `INTERNAL_ERROR` and are never guessed to be Q05. BDF-D11's `0A000` remains reserved for forbidden UPDATE/DELETE/TRUNCATE, not INSERT eligibility.

Only these structured SQLSTATE/constraint pairs may enter the unique/exclusion reconciliation path; inspect driver metadata (`code` and `constraint` / `constraint_name`), never parse error text:

Exact PostgreSQL tuple	Meaning / mapping when no same-pair replay exists
`23505 / booking_one_active_per_slot`	409 `SLOT_UNAVAILABLE`
`23P01 / booked_customer_no_overlap`	409 `CUSTOMER_OVERLAP`

Any missing, unknown or mismatched tuple (including another `23505`, such as a Booking primary-key violation) rolls back and fails closed as 500 `INTERNAL_ERROR`; it must not enter reconciliation, return 200 or be guessed to be a business conflict. These exact identifiers are owned by #33 Part F and must match its canonical schema.

For a recognized tuple only, ROLLBACK and discard the failed SERIALIZABLE snapshot. On a fresh connection, begin a separate `READ COMMITTED` reconciliation transaction. First lock the current `User` row for `principal.userId` with `FOR UPDATE` and revalidate its latest committed state: missing/inactive → 401 `UNAUTHENTICATED`; active but current role is not Customer → 403 `FORBIDDEN`. Do not read Session or Booking data on either failure. Then lock the exact `Session` row for `principal.sessionId` and `principal.userId` with `FOR UPDATE`, and check `revokedAt IS NULL AND expiresAt > clock_timestamp()`; missing, mismatched, revoked or expired Session → 401 `UNAUTHENTICATED`. Only an active Customer with a live Session may perform one exact-pair `Booked` read: a committed same-pair row with matching interval returns 200 and its original ID; otherwise return the safe 409 mapped from the allowlisted original tuple above. The reconciliation transaction takes no Slot or Booking row locks and performs no mutation. If User/Session DB validation is unavailable, return 503 and fail closed. Do not spin on a genuine conflict.

5. Boundary and review gate

Only this POST is introduced. No Booking GET/list/detail, cancellation, reschedule, deletion or Slot mutation route is added. Peer review endpoint path, request allowlist, 201/200 replay semantics, cause-specific 409/422 mapping, allowlisted SQLSTATE/constraint reconciliation, in-transaction and post-ROLLBACK User/Session revalidation, Auth CSRF/session-binding reuse, DTO privacy, timestamp format and exact #33 trigger diagnostic dependency. All API cases remain PLANNED / NOT RUN in Part E.

E.5.4 F-S1-04 Booking Data Contract (data-spec.md)

> **Consumes #33's shared contract; does not define or implement a second schema.** No Prisma model, SQL migration, trigger, index or database test is added by Issue #16 Part E.

1. Canonical source and ownership

Canonical Booking fields, relations and integrity decisions are owned by #33 Foundation SDS, data specification and BDF STS, merged by PR #34 at `ea36ef6fd3ea7e2c2582489d1326adbd335452ee`.

That is a documentation merge, **not** a future schema/migration SHA. #16 Part F waits for #33 Part F to merge and records its exact schema SHA before implementing Booking-dependent code/tests.

2. Canonical Booking mapping

Field	Canonical type/source	#16 write rule
`id`	Server/database-generated UUID PK	Never accept from request; return only after COMMIT or same-pair replay
`customerId`	UUID FK to `User.id`	Live authenticated Customer `principal.userId`
`currentSlotId`	UUID FK to `Slot.id`	Validated request `slotId`
`startsAt` / `endsAt`	`TIMESTAMPTZ(6)`	Copy persisted Slot interval; never client input
`status`	Text: `Booked` / `Cancelled`	New Sprint 1 row always `Booked`
`createdAt` / `updatedAt`	`TIMESTAMPTZ(6)` metadata	Not an eligibility clock

There is **no Booking `tutorId` field**. Current Tutor derives from `Slot.tutorId` through `currentSlotId`. Composite Slot-window FK enforces that copied Booking interval matches the immutable Slot.

3. Database invariants consumed

#33 Part F owns and must implement BDF-D01..D12:

- Required Customer/Slot references, restrictive parent delete/update behavior and positive interval.
- Supported `Booked` / `Cancelled` status; only `Booked` occupies/conflicts.
- Composite Slot-window FK matching `currentSlotId`, `startsAt`, `endsAt`.
- Partial unique `booking_one_active_per_slot` for one Active Booking per Slot.
- GiST exclusion `booked_customer_no_overlap` for positive-duration same-Customer Active interval overlap across Tutors; half-open adjacency is allowed.
- One shared `booking_guard_s1()` row-level `BEFORE INSERT OR UPDATE OR DELETE` trigger and statement-level `BEFORE TRUNCATE` trigger, plus runtime-role boundary.
- Sprint 1 append-only lifecycle. #16 performs INSERT only; no UPDATE/DELETE/TRUNCATE operation exists in this feature.

#16 must not duplicate model, trigger, migration, occupancy field or Tutor reference.

4. Natural-key replay mapping — no schema addition

Application lookup pair:

```
(authenticated principal.userId, requested slotId)
```

Exact replay predicate:

```
customerId = principal.userId
AND currentSlotId = slotId
AND status = 'Booked'
```

Rules:

1. Query this pair only after the transaction has acquired ordered User and Slot locks, locked/revalidated the current Customer Session, and rechecked Session validity after the final lock wait.
2. Check it **before** occupied-Slot rejection and before a new-booking Q05 check. A matching committed `Booked` row is already successful: return 200 and original ID without INSERT or new `T_elig`.
3. The pair is not a new composite UNIQUE constraint, column, Idempotency-Key, registry, fingerprint or TTL.
4. Only `Booked` qualifies. A future `Cancelled` row is not a replay; S2 rebooking semantics need their own contract.
5. If no matching row exists, apply Slot occupancy, Customer overlap and guarded INSERT checks. Another Customer's occupied Slot or a different overlapping Slot for this Customer is a conflict.
6. Only the exact structured tuples `23505 / booking_one_active_per_slot` and `23P01 / booked_customer_no_overlap` are recognized as Booking business conflicts. Capture SQLSTATE and constraint metadata; never parse error text. Any

unknown/missing/mismatched tuple (including a different `23505`) rolls back and returns safe 500 `INTERNAL_ERROR` without entering replay reconciliation.

7. For a recognized tuple, ROLLBACK and discard the failed snapshot; on a fresh connection begin a READ COMMITTED reconciliation transaction. Lock the current User row for the original `principal.userId` first and revalidate latest `isActive`/role: missing or inactive → 401; active but no longer Customer → 403. Only then lock the exact authenticated Session (`principal.sessionId` + `principal.userId`) with `FOR UPDATE` and check `revokedAt IS NULL` and `expiresAt > clock_timestamp()`; missing/revoked/expired → 401. Do not read Session/Booking after failed User authorization, or Booking after failed Session authorization.
8. Only an active Customer with a live Session may read the exact pair: return 200 for a committed same-pair `Booked` row with matching interval; otherwise return safe 409 mapped only from the recognized original tuple (`SLOT_UNAVAILABLE` for `23505` / `booking_one_active_per_slot`; `CUSTOMER_OVERLAP` for `23P01` / `booked_customer_no_overlap`). This fresh transaction takes no Slot/Booking row locks and performs no mutation. If User/Session validation is unavailable, return 503 and fail closed.

The existing Booking row is the only durable replay record; retrying same request creates no duplicate active row/history.

5. Read/write and privacy boundary

- F-S1-04 creates a Booking only. #15 availability remains read-only; #17 owns protected Customer list/detail and Tutor schedule Booking reads.
- Do not add `Slot.isBooked`, duplicate Tutor identity, Booking registry or second occupancy source.
- Create DTO contains only Booking ID, Slot ID, copied UTC interval, `Booked` status and request ID. Exclude Customer/Tutor IDs, profiles, contacts, credentials, Session/CSRF values and arbitrary ORM fields.
- Reuse only existing Auth fields for post-rollback revalidation: `User.id`, `User.role`, `User.isActive`, then `Session.id`, `Session.userId`, `Session.revokedAt` and `Session.expiresAt`. Read/lock the User before Session on the fresh READ COMMITTED connection. No User/Session column, Booking field, migration or second authentication schema is added by #16 Part E.
- Conflict responses must not reveal identity or Booking ID of Customer occupying a Slot.
- No retention/deletion behavior is added; BDF-D12 append-only/runtime-role rules apply.

6. Verification boundary

BDF-01..26 remain planned in merged #33 Part E. #33 Part F must run pre-merge BDF-01..22 and BDF-24..26. #16 Part F runs BDF-23 only after exact #33 schema SHA is merged. The exact unique/exclusion constraint identifiers above must be verified against #33 Part F before implementation; a mismatch blocks this adapter rather than widening the allowlist. No Booking schema, query, migration or database result is claimed here.

E.5.5 F-S1-04 PostgreSQL Booking Transaction, Lock & Retry Contract (transaction-spec.md)

> **Selected engineering design for peer review.** Application transaction contract only, not a migration or SQL implementation. #16 owns this adapter; #33 owns the canonical Booking model and single DB guard. Align with System SDS §§7.1/7.3–7.5, BDF-D11/D12 and BDF-23.

1. Selected transaction mechanism

- PostgreSQL **SERIALIZABLE** for each Booking mutation attempt.
- One attempt uses one checked-out PostgreSQL connection (F-S1-02 uses one `pg.PoolClient` for atomic mutation). Do not mix a Prisma transaction and independent `pg` connection and call them one transaction.
- Begin each attempt with `BEGIN ISOLATION LEVEL SERIALIZABLE`; all locks, reads and INSERT use that same connection.
- Explicit row locks follow System SDS global order. Unique/exclusion constraints and #33 trigger are mandatory backstops, not replacements for application locks.
- Commit before returning 201. Rollback, conflict, retry exhaustion or ambiguous COMMIT must never be represented as confirmed new Booking.

2. Lock-set discovery and global order

The selected Slot's actual Tutor ID determines the full User lock set. First perform an **unlocked, read-only discovery query** for selected Slot `id` and `tutorId`. This is not authorization, a booking check or source of interval values. Missing Slot → safe 404.

For every attempt, acquire locks in this exact order:

Order	Rows	New Booking operation
1	Affected `User` rows, ascending UUID	Authenticated Customer and selected Slot's actual Tutor; deduplicate IDs
2	Affected `Slot` rows, ascending UUID	Selected Slot
3	Current Customer `Session` row	`principal.sessionId`, matched to `principal.userId`; lock only this authenticated Session after **all** User and Slot waits and before any Booking read/lock
4	Existing `Booking` rows, ascending UUID	No existing Booking is mutated by this INSERT-only operation. Read conflict predicates after User/Slot/Session checks; no nonexistent Booking row can be locked.

The System SDS **domain** lock hierarchy remains User → Slot → Booking. The current Customer Session row is a subordinate authentication lock after the User and Slot phases but before any Booking access; it does not reverse the domain order. No transaction may lock Session and then request a User, Slot or Booking lock. The existing logout path updates only its Session row and does not acquire User/Slot locks: if logout commits while Booking is waiting for either User or Slot, the later Session lock/revalidation observes revocation or the stale SERIALIZABLE attempt aborts and retries with a fresh snapshot. If Booking obtains the Session lock first, logout waits and is serialized after the Booking transaction. Hold the Session lock through COMMIT/ROLLBACK.

Post-rollback reconciliation is a separate non-mutating authorization/read transaction: it locks the current User row first, then the exact current Session row, and then performs a plain MVCC Booking read (no Booking row lock). It acquires no Slot or Booking row locks. The User → Session → Booking-read sequence preserves the User-before-Session order and cannot invert the User → Slot → Booking domain lock hierarchy. It must never initiate another Booking mutation.

Use deterministic ordered locking or explicit per-ID locks whose acquisition order is verified; do not rely on incidental query-plan order. After User locks, immediately revalidate Customer `role='Customer'` / `isActive=true`. For the selected Tutor, retain the current locked User state but defer the `role='Tutor'` / `isActive=true` eligibility decision until after Session revalidation and the exact-pair replay lookup. A committed same-pair replay belongs to an existing Booking and must remain available to an active Customer with a live Session even if the Tutor was subsequently deactivated or changed role. Only when no qualifying replay exists may an ineligible current Tutor reject a new claim with safe 409 `SLOT_UNAVAILABLE`, without disclosing Tutor/User status. If the lock set/assignment differs, abort and restart from fresh discovery; never add an earlier-order User or Slot lock while holding a later-order Session/Booking lock.

The Session is an existing Auth record, not a Booking data addition. The principal and `sessionId` come from the HTTP guard; never accept either from the request body. The locked Slot row, not discovery pre-read or client data, supplies interval and Tutor association.

3. Ordered transaction algorithm

1. **HTTP boundary:** require live Session, active Customer role, allowed Origin, Session-bound CSRF and a matching `X-NutTiew-Expected-Session-Binding`. This is preliminary authentication; it does not replace transactional revalidation. Strictly parse one UUID `slotId`; reject extra body keys/query parameters.
2. **Discovery:** read selected Slot ID/Tutor ID without locks only to discover the User lock set. Missing Slot → 404.
3. **Begin:** check out one DB connection and start a fresh SERIALIZABLE transaction.
4. **Lock Users:** lock Customer and actual Tutor User rows in ascending UUID order. Immediately revalidate Customer `role='Customer'` / `isActive=true`. Retain the selected Tutor's locked current `role` / `isActive` state, but do not reject on Tutor eligibility yet; defer that decision until after Session validation and the same-pair replay lookup in steps 6–8.
5. **Lock Slot:** lock the selected Slot row; re-read/compare Tutor ID, lifecycle and interval. Persisted locked state is authoritative.

6. **Lock current Session after the relevant waits:** on the same connection, lock `principal.sessionId` with `SELECT ... FROM "Session" WHERE id = principal.sessionId AND userId = principal.userId FOR UPDATE`. Do this only after all affected User and Slot locks have been acquired. If this exact lookup returns no row (including an owner mismatch), ROLLBACK and return 401 `UNAUTHENTICATED` immediately; do not read Booking data and do not classify absence as a retryable abort. If PostgreSQL instead raises SQLSTATE `40001` because a concurrent Session update/delete or stale SERIALIZABLE snapshot invalidated the row operation, ROLLBACK and retry the whole transaction under §6.1; the fresh attempt must re-read the Session and return 401 if it is now absent. Never continue using the HTTP middleware's earlier Session snapshot.
7. **Revalidate Session with DB time:** after the Session row lock is acquired, execute a fresh same-connection check requiring `revokedAt IS NULL AND expiresAt > clock_timestamp()`. If no live matching row exists, ROLLBACK and return 401 `UNAUTHENTICATED`; do not read/replay Booking data. This catches revocation during User/Slot waits and expiry during any preceding wait. Keep the Session lock through transaction end so a concurrent logout is serialized before or after the Booking, never halfway through its authorization decision.
8. **Natural-key replay first:** query `Booking` for exact `(customerId=principal.userId, currentSlotId=slotId, status='Booked')`. If found and interval matches locked Slot, COMMIT the read-only transaction and return 200 with that Booking ID, even if the locked Tutor is now inactive or no longer has role `Tutor`. Do this **before** new-claim Tutor eligibility rejection, occupied-Slot rejection and new-booking Q05. This exception applies only to the already committed same-pair Booking; it does not authorize a new INSERT for an ineligible Tutor.
9. **New-claim Tutor eligibility:** only if step 8 found no same-pair replay, require the selected Tutor's current locked User row to have `role='Tutor'` and `isActive=true`. If the Tutor is inactive or has another role, ROLLBACK and return generic 409 `SLOT_UNAVAILABLE`; do not disclose Tutor identity/status and do not create a Booking. This check never invalidates an existing same-pair replay.
10. **New-claim conflicts:** if any other `Booked` row occupies Slot, return 409 `SLOT_UNAVAILABLE`. If Customer has a different `Booked` interval with positive overlap (`existing.startsAt < slot.endsAt AND slot.startsAt < existing.endsAt`), return 409 `CUSTOMER_OVERLAP`. Exact adjacency is allowed.
11. **Guarded INSERT:** insert one row with server/DB-generated ID, authenticated Customer ID, locked Slot ID, persisted Slot interval and status `Booked`. #33 `booking_guard_s1()` locks/re-reads Slot (already held), then samples DB `clock_timestamp()` as authoritative `T_elig`; enforce `startsAt - T_elig >= interval '60 minutes'`. Exact 60:00 passes; 59:59 fails. Do not substitute application time, PostgreSQL transaction-start `now()`, HTTP arrival, #15 `T_view` or commit time.
12. **Commit:** COMMIT on the same connection. Return 201 with persisted Booking ID only after COMMIT acknowledgement.
13. **Release:** on error, ROLLBACK when connection is usable, release it and classify per §6. No external side effect runs inside transaction.

Same-pair replay is checked only after Customer authorization, live Session revalidation and all relevant User/Slot lock waits. It returns the original committed record before current Tutor eligibility is applied to a new claim, before occupied-Slot rejection and before new-booking Q05; it does not INSERT and does not need new `T_elig`. Every new INSERT must pass current Tutor eligibility and the DB trigger.

4. Natural-key replay and uncertain COMMIT

Only replay identity:

```
(authenticated customerId, slotId)
```

Situation	Result
Valid new request commits	201 with new Booking ID
Same Customer repeats same Slot after prior commit	200 with exact original Booking ID; no INSERT
Different Customer submits occupied Slot	409 `SLOT_UNAVAILABLE`; do not reveal owner
Same Customer submits a different overlapping Slot	409 `CUSTOMER_OVERLAP`
Same Customer submits a different non-overlapping Slot	Distinct request; evaluate normal eligibility

Situation	Result
Existing same-pair row is `Cancelled`	Not a Sprint 1 replay; future rebooking behavior is outside this contract

A broken connection/timeout around COMMIT does not prove rollback. Server must not claim 201 or silently start a second mutation in the same request. If possible, return 503 `BOOKING_OUTCOME_UNKNOWN`; otherwise client sees network failure. Client retries exact same POST body. A fresh transaction returns 200 if original row committed, or creates one row if it did not. No key registry or TTL is required.

SERIALIZABLE snapshot caveat: a concurrent request may take its snapshot before waiting on Customer/Slot lock; lock release does not refresh that snapshot. It may not see the first transaction's newly committed row and PostgreSQL may instead raise 40001 or a unique/exclusion conflict. Do not assume post-lock SELECT sees a newer snapshot. Full mutation retry and post-rollback exact-pair reconciliation in §6 must return 200 for the same pair only after fresh DB active-Customer User and live-Session checks. If that Session was revoked or expired before reconciliation, return 401 with no Booking DTO; never create another row or report a permanent conflict for a valid same-pair commit.

5. Race and Q05 safety / no-post-guard-contention proof

Race B — two Customers, one Slot

Both attempts lock their own Customer and the same Slot before conflict checks/INSERT. Slot lock serializes compliant API writers. After winner commits, loser sees occupied state on a fresh attempt or loses to unique constraint; at most one active row can commit.

Race C — one Customer, two Tutors

Both attempts include same Customer User row in affected User lock set, even though selected Tutors/Slots differ. UUID-sorted User locking serializes Customer's Booking writes before guarded INSERT. GiST exclusion remains final DB backstop; no positive-duration overlapping active rows may commit. Adjacency remains allowed.

Q05 — post-contention eligibility

Application acquires known conflict-serialization User/Slot locks before guarded INSERT. #33 trigger's Slot `FOR UPDATE` must re-lock a row already held, so it creates no new Slot wait after `T_elig`. Only trigger-side DB `clock_timestamp()` after that lock is authoritative. A request received at 13:59:59 that waits until 14:00:01 for a 15:00 Slot must fail; a valid 60:00 guard may still commit later.

Mandatory no-post-guard-contention proof

Unique active-Slot index and same-Customer GiST exclusion may wait on another transaction after the BEFORE INSERT trigger has sampled `T_elig`. Normal API lock protocol is intended to make every compliant conflicting writer acquire same Customer and/or Slot lock before guarded INSERT: Race B shares Slot; Race C shares Customer; all affected User rows precede Slot rows.

This intent is **not proof** against direct SQL or a writer bypassing the application protocol. BDF-23 must use independent PostgreSQL sessions, backend PIDs, `pg_blocking_pids`, trigger-observed `T_elig`, constraint outcomes and final committed rows. Force an unexpected unique/exclusion index wait after preliminary validation and establish one of:

1. Prove no conflict-capable wait can occur after `T_elig` under actual #33 trigger/role/writer arrangement; or
2. Demonstrate complete transaction aborts before it can commit a stale-time INSERT, then restarts with all locks reacquired and fresh DB `T_elig`.

If a forced post-guard wait can resolve and allow an INSERT to commit using stale `T_elig`, BDF-23 fails. Do not treat SERIALIZABLE alone, application pre-check, random `Promise.all()`, timeout-only test or blanket commit-time cutoff as proof. If neither outcome is demonstrated, #16 Part F is blocked and #33/#16 lock/guard contract must be revised through review.

6. Retry, conflict and failure classification

6.1 Full transaction retry

Retry known aborts only: SQLSTATE `40001` (serialization), `40P01` (deadlock), ORM equivalent `P2034` if surfaced, or changed/missing **User/Slot lock-set member** between discovery and locked re-read after abort. An exact transactional Session

lookup that returns no row or an owner mismatch is a terminal 401, not a retry condition. A concurrent Session change that PostgreSQL reports as `40001` remains retryable; the fresh attempt must re-read and return 401 if the Session is then absent.

Policy: maximum **three total mutation attempts**, including first. Each retry starts a new transaction/snapshot, rediscovers Slot's actual Tutor, reacquires all locks in order and reruns checks. Every retry reaching a new INSERT invokes trigger and samples fresh `T_elig`. Use short capped exponential jitter; never retry indefinitely. Exhaustion → 503 `TRANSACTION_RETRY_EXHAUSTED`; every failed attempt rolls back.

6.2 Unique/exclusion conflict and reconciliation

Only these structured PostgreSQL error tuples are eligible for the unique/exclusion reconciliation path; capture the driver's SQLSTATE and constraint metadata (for example, `code` plus `constraint` / `constraint_name`), never parse human-readable error text:

SQLSTATE	Exact constraint identifier owned by #33	Original conflict cause
`23505`	`booking_one_active_per_slot`	Slot already has an active Booking → `SLOT_UNAVAILABLE`
`23P01`	`booked_customer_no_overlap`	Customer has a different overlapping Active Booking → `CUSTOMER_OVERLAP`

Any missing, unknown or mismatched SQLSTATE/constraint pair (including a different `23505` such as a primary-key violation) is **not** a recognized business conflict: ROLLBACK and return safe 500 `INTERNAL_ERROR`; do not reconcile, read Booking data, return 200 or guess a 409. #33 Part F must publish these exact canonical constraint identifiers; a mismatch is a dependency blocker, not permission to broaden the allowlist.

For a recognized tuple:

1. **Rollback and discard the failed attempt:** ROLLBACK the SERIALIZABLE mutation transaction, release its User/Slot/Session locks and connection, and discard its snapshot and every User/Session authorization observation made in that attempt.
2. **Start a fresh reconciliation transaction:** check out a fresh PostgreSQL connection and begin `BEGIN ISOLATION LEVEL READ COMMITTED`. This is a separate non-mutating authorization/read transaction, not another Booking mutation attempt and does not consume the three-attempt retry budget. Do not mark it SQL `READ ONLY`, because it must lock User and Session rows.
3. **Revalidate the current User before Session or Booking access:** using only the original `principal.userId`, lock the current `"User"` row with `SELECT ... FROM "User" WHERE "id" = $1 FOR UPDATE`. At READ COMMITTED this observes the latest committed User state after any lock wait. If the User is missing or `isActive = false`, ROLLBACK and return 401 `UNAUTHENTICATED`. If the User is active but `role <> 'Customer'`, ROLLBACK and return 403 `FORBIDDEN`. Do not read Session or Booking data and do not return a Booking ID/DTO on either outcome.
4. **Revalidate the exact authenticated Session:** only after the User lock/check, use the original request principal's `sessionId` and `userId` (never request-body values) to lock `"Session"` with `SELECT ... FROM "Session" WHERE "id" = $1 AND "userId" = $2 FOR UPDATE`. A missing row or owner mismatch means ROLLBACK and 401 `UNAUTHENTICATED`. The User → Session order is mandatory; no later step may acquire another User/Slot lock.
5. **Check live Session status using database time while holding the lock:** on the same connection, after the Session lock has been acquired, require `revokedAt IS NULL AND expiresAt > clock_timestamp()`. If revoked or expired, ROLLBACK and return 401 `UNAUTHENTICATED` with no Booking query, ID or DTO. Hold both User and Session locks through reconciliation COMMIT/ROLLBACK so concurrent User deactivation/role change and logout are serialized before or after this authorization/read operation.
6. **Only for a currently active Customer with a live Session, perform one exact-pair read:** query `customerId = principal.userId AND currentSlotId = slotId AND status = 'Booked'` on the same fresh connection. This is a plain non-locking MVCC read; acquire no Slot or Booking row locks and perform no mutation. Confirm the persisted interval matches the selected Slot before treating it as a replay.
7. If the committed same-pair row exists and its interval matches, COMMIT the reconciliation transaction and return 200 with the original Booking ID.
8. If no qualifying same-pair row exists, COMMIT the non-mutating reconciliation transaction and return the safe 409 classified only from the previously allowlisted original constraint tuple: `SLOT_UNAVAILABLE` for `23505 / booking_one_active_per_slot`; `CUSTOMER_OVERLAP` for `23P01 / booked_customer_no_overlap`. Never disclose another Customer or Booking ID.

9. If the fresh connection or User/Session DB validation is unavailable/indeterminate, return 503 `SERVICE_UNAVAILABLE`; never skip authorization, reuse the failed snapshot or return 200/409 from stale data.
10. Never spin until a genuine conflict disappears. The reconciliation read is not a second mutation attempt and does not rerun the trigger or sample `T_elig`.

The fresh **User and Session** checks are mandatory for every post-ROLLBACK path that could return 200 or a business 409. If the User is missing/inactive or no longer has the Customer role, return 401/403 respectively, even if the exact-pair Booking row is committed. If the Session is revoked/expired before reconciliation, return 401 with no Booking DTO. A live active Customer control case must continue to return 200 with the original ID.

6.3 Other failures

- Non-Offered Slot: rollback and map #33 trigger diagnostic `P0B01` / `booking_slot_not_offered` to 409 `SLOT_UNAVAILABLE`.
- Q05 failure: rollback and map #33 trigger diagnostic `P0B02` / `booking_q05_min_notice` to 422 `BOOKING_NOT_ELIGIBLE`.
- #33 Part F owns and must implement/publish both stable SQLSTATE/constraint pairs. Unknown guard diagnostics fail closed as 500 `INTERNAL_ERROR`; never guess a Q05 failure. BDF-D11's `0A000` is reserved for forbidden UPDATE/DELETE/TRUNCATE, not INSERT eligibility.
- Known DB failure before COMMIT/confirmed rollback: safe 503 `SERVICE_UNAVAILABLE`.
- Ambiguous COMMIT acknowledgement: 503 `BOOKING_OUTCOME_UNKNOWN` if possible; client repeats exact request.
- Unexpected `0A000` on INSERT-only endpoint or any unrecognized/mismatched `23505` / `23P01` SQLSTATE/constraint tuple: rollback, safe 500 `INTERNAL_ERROR` and non-sensitive diagnostics; never enter replay reconciliation for an unknown constraint.
- Malformed/unauthorized requests and business conflicts are not retried.

7. Real-PostgreSQL proof gate

BDF-23 is owned by #16 and runs only after #33 Part F merges with exact schema SHA. Evidence must include independent sessions/barriers for Race B/C; backend PIDs/blockers; User → Slot → Session auth-lock order and Session revalidation after waits; fresh READ COMMITTED post-ROLLBACK User-then-Session authorization lock and allowlisted-constraint reconciliation before exact-pair read; trigger Slot lock already held; `T_elig` after required waits; forced unexpected unique/exclusion index wait; 60:00/59:59 and HTTP-arrival-before-cutoff cases; sequential/in-flight/lost-response same-pair replay; pre-COMMIT failure injection; final rows from fresh connection; PostgreSQL version, schema SHA, SQLSTATE/constraint, commands and source commit.

BDF-23 is **PLANNED / NOT RUN** in Part E. If post-guard contention challenge fails, do not approve adapter or claim #16 Part F complete.

E.5.6 F-S1-04 Customer Booking Software Test Specification (tests.md)

> **Planned STS only.** No application code, executable test, migration, PostgreSQL run, browser test or PASS result is included. Execute against actual #33 schema after Part F merge and record exact schema/source SHA.

1. Traceability and test setup

Sources: Feature SDS, UI, API, data, transaction contract, Sprint 1 requirements, System SDS and BDF STS.

Part F requires disposable PostgreSQL at target major version/exact #33 schema SHA; synthetic active/inactive Customer and role-change User fixtures, Customer A/B and Tutor A/B; Session A/B (including two Sessions for the same User); real future Offered Slots; at least two independent DB sessions; explicit barriers and observer query; controlled DB clock/request arrival/fault injection/lost HTTP response; browser auth/session-epoch control for late-response cases. For post-ROLLBACK authorization races, provide independent User deactivation/role-change, Session revocation/deletion and Tutor deactivation/role-change transactions and observe lock acquisition order. Capture `pg_backend_pid()` and `pg_blocking_pids`. Random `Promise.all()`, sleep-only race or disabled UI button is insufficient. Inspect final committed rows from a fresh connection.

All cases below are **PLANNED / NOT RUN**.

2. Planned STS cases

ID	Requirement trace	Setup / action	Expected observable result
BOOK-01	FR-S1-10; AC-S1-19; VC/TEST-S1-16	Active Customer submits one eligible unoccupied Slot.	201 only after COMMIT; one `Booked` row with Session Customer, actual Slot and copied interval; response ID equals persisted ID.
BOOK-02	FR-S1-11; AC-S1-20; VC/TEST-S1-17	Slot starts 15:00; trigger `T_elig` exactly 14:00:00.	60:00 passes; one Booking may commit if other checks pass.
BOOK-03	FR-S1-11; AC-S1-21; VC/TEST-S1-18	Slot starts 15:00; trigger `T_elig` 14:00:01.	59:59 fails; no Booking/occupancy; safe 422.
BOOK-04	BR-S1-08; AC-S1-22; VC/TEST-S1-19	Customer A has committed Booking; Customer B submits same Slot.	409; A's row preserved; no A identity/Booking detail disclosed.
BOOK-05	BR-S1-09; AC-S1-23; VC/TEST-S1-20	Customer has 13:00–14:00 with Tutor A; submits 13:30–14:30 with Tutor B.	409 `CUSTOMER_OVERLAP`; no second row; first remains.
BOOK-06	BR-S1-10; AC-S1-24; VC/TEST-S1-21	Customer has 13:00–14:00; requests 14:00–15:00 with another Tutor.	Overlap rule permits adjacency if other checks pass.
BOOK-07	FR-S1-11/18; AC-S1-27; VC/TEST-S1-22	Offer becomes occupied, its lifecycle is no longer `Offered`, its Tutor becomes inactive/changes away from role `Tutor`, or Q05 becomes too close after browse and before a **new** POST.	Server maps occupied/non-Offered/ineligible-Tutor new claims to generic 409 `SLOT_UNAVAILABLE` and Q05 cutoff to 422 `BOOKING_NOT_ELIGIBLE`; no stale-selection success/partial row and no Tutor-status disclosure. Existing same-pair replay is covered separately by BOOK-33.
BOOK-08	FR-S1-11/18; AC-S1-27; VC/TEST-S1-23	Missing/malformed/past Slot or unauthorized caller attempts Booking.	400/401/403/404/422 as specified; no mutation/private data.
BOOK-09	BR-S1-20; AC-S1-28; VC/TEST-S1-24	Customer forges `customerId`, `tutorId`, role, Booking ID or interval.	Extra fields rejected; owner remains Session Customer and Tutor/time derive from real Slot.
BOOK-10	FR-S1-12; AC-S1-25; VC/TEST-S1-25; NFR-S1-04	Two Customers race for one eligible Slot under synchronized barrier.	At most one committed `Booked` row; loser non-success; no partial claim.
BOOK-11	FR-S1-13; AC-S1-26; VC/TEST-S1-26; NFR-S1-05	Same Customer concurrently submits distinct overlapping Slots from two Tutors; repeat at proposed 10-attempt load.	No two overlapping Active rows commit; adjacent control allowed.

ID	Requirement trace	Setup / action	Expected observable result
BOOK-12	FR-S1-18; AC-S1-27; VC/TEST-S1-27 and VC/TEST-S1-33	Inject DB failure after validation/before COMMIT; reopen state via fresh connection.	Whole transaction rolls back; no Booking, phantom occupancy or success; prior rows unchanged.
BOOK-13	AC-S1-29; VC/TEST-S1-38; OD-013	After commit, same Customer sends same `slotId` POST again.	200 with exact original Booking ID/DTO; no second row/new ID.
BOOK-14	AC-S1-29; VC/TEST-S1-38; Race B/C	Two same-Customer/same-Slot POSTs overlap in flight under barrier.	One row only; winner 201; duplicate reconciles to 200 with same ID (or retryable abort then same result on fresh attempt).
BOOK-15	AC-S1-29; VC/TEST-S1-38; OD-013	Let COMMIT succeed but drop HTTP response; Customer resubmits identical POST.	Second request returns 200 with committed original ID; exactly one row.
BOOK-16	AC-S1-29; VC/TEST-S1-38	Force first attempt to roll back before COMMIT, then retry identical POST.	Retry creates at most one row and returns 201 only after its own COMMIT; no phantom success.
BOOK-17	AC-S1-35; VC/TEST-S1-39; Q05	15:00 Slot; HTTP arrives 13:59:59; hold required lock until 14:00:01.	Trigger samples `T_elig` after wait and rejects 59:59; no Booking.
BOOK-18	System SDS §7.1/7.4; SQLSTATE 40001/40P01	Force serialization failure and deadlock in separate runs.	Roll back whole attempt, rediscover/relock in order and retry no more than three total attempts; no partial rows.
BOOK-19	BDF-23; System SDS §7.3; VC/TEST-S1-39	Force unexpected unique/exclusion index wait after trigger `T_elig`; instrument PIDs/blockers/trigger time.	Prove no stale-time INSERT can commit, or show full abort/retry with fresh `T_elig`. Otherwise BLOCKED/FAIL.
BOOK-20	NFR-S1-01/08; VC/TEST-S1-23/24	Inspect full success/error JSON for anonymous, wrong-role, occupied foreign Slot, forged fields and DB errors.	No Customer/Tutor private identity, SQL/stack, credentials, Session/CSRF data or another Booking ID leaks.
BOOK-21	NFR-S1-03/11; VC/TEST-S1-27/33	Restart app against same DB after success; reopen views after rejected attempt when #17 is integrated.	Committed Booking remains; rejected attempt creates no phantom row/occupancy.
BOOK-22	Transaction contract §2; BDF-23	Instrument User, Slot and current Session locks for both UUID orderings and concurrent cross-Tutor requests.	Actual acquisition follows User UUID ascending → Slot UUID ascending → current Customer Session auth sub-lock → Booking; domain order remains User → Slot → Booking; no out-of-order acquisition.
BOOK-23	Transaction contract §6.1	Force three retryable transaction aborts.	Three total attempts maximum; final 503 `TRANSACTION_RETRY_EXHAUSTED`; all attempts rolled back.
BOOK-24	Transaction contract §§2/6	Change/remove discovered lock-set member between pre-read and locked re-read in isolated fixture.	Abort without acquiring an earlier-order lock late; rediscover/restart whole transaction or safe non-success after budget.
BOOK-25	BDF-23; #33 Part F trigger contract	Independently trigger a non-`Offered` Slot rejection and a Q05 failure at `T_elig`; inspect PostgreSQL SQLSTATE and constraint diagnostics.	Non-Offered is exactly `P0B01 / booking_slot_not_offered` → 409 `SLOT_UNAVAILABLE`; Q05 is exactly `P0B02 / booking_q05_min_notice` → 422 `BOOKING_NOT_ELIGIBLE`. No raw DB diagnostics; any missing/mismatched identifier is BLOCKED.

ID	Requirement trace	Setup / action	Expected observable result
BOOK-26	BR-S1-13; OD-005-A/B; VC/TEST-S1-23	Pass HTTP auth, hold the Customer User row in DB session A, start Booking POST in session B and confirm it waits on the User lock. While it waits, revoke that exact Session through logout and commit; then release the User lock.	The Session lock acquired after the wait observes revocation or the stale SERIALIZABLE attempt aborts/retries; fresh attempt returns 401. No 201, no 200 replay, no Booking row and no private replay data.
BOOK-27	BR-S1-13; OD-005-A/B; VC/TEST-S1-23	Hold selected Slot row in DB session A. Start Booking POST, verify it has acquired User lock and is waiting on Slot; in separate runs, either revoke the Session and commit or allow its `expiresAt` to pass; then release Slot.	Session is locked/revalidated only after the Slot wait; revoked/expired Session returns 401 (or stale snapshot forces full retry then 401). No Booking INSERT, no 200 replay and no private data.
BOOK-28	OD-005-C; VC/TEST-S1-23/24	Attempt POST with missing/wrong CSRF, Session A token with Session B cookie (including same User), missing/mismatched expected binding, foreign Origin, and `Sec-Fetch-Site: cross-site`. Also switch A→B between the initial `GET /auth/me` and `GET /auth/csrf`; verify the client performs a second `GET /auth/me` after CSRF and compares the returned binding/identity.	Reject invalid requests with 403 `CSRF_DENIED` / `SESSION_BINDING_MISMATCH` / `ORIGIN_DENIED` as applicable (401 for invalid Session). If the post-CSRF `/auth/me` binding differs, discard the token and restart bootstrap without POST; no mutation or private Booking data.
BOOK-29	OD-005-B/C; VC/TEST-S1-23/24	Start a POST under Session A and delay its response; logout A or switch to Session B (same User and different User), then deliver A's late response. Also switch A→B **after** the post-CSRF `GET /auth/me` recheck but before POST, sending A's captured expected binding/token with B's cookie.	UI auth/session epoch discards A's late response and never displays its Booking ID/details under B. The race after the final recheck is rejected before mutation with 403 `SESSION_BINDING_MISMATCH` or `CSRF_DENIED` (depending on middleware order); client clears identity/CSRF state and restarts bootstrap. If A had already committed before logout, do not claim rollback; only the stale response is suppressed.
BOOK-30	BR-S1-13; AC-S1-29; OD-013; VC/TEST-S1-23/38; Transaction contract §6.2	Cause either (a) a same-pair race that produces the recognized `23505 / booking_one_active_per_slot` after the winning same-pair Booking commits, or (b) a distinct overlapping-Slot conflict that produces `23P01 / booked_customer_no_overlap`; pause after the failed SERIALIZABLE transaction rolls back but before reconciliation. For each cause, run synchronized interleavings where (1) the Customer User is deactivated and committed, (2) its role changes from Customer and commits, (3) the exact Session is revoked and committed, or (4) the barrier is held until DB clock passes Session `expiresAt`; then release. Also run a live active-Customer/live-Session control for each cause.	Reconciliation first locks/revalidates the latest User row: missing/inactive → 401 `UNAUTHENTICATED`; active non-Customer → 403 `FORBIDDEN`; neither reads Session/Booking or returns a DTO. If User remains an active Customer, it locks/revalidates exact Session: revoked/expired → 401 with no Booking read/ID/DTO. For cause (a), live control returns 200 with the original same-pair Booking ID and exactly one committed row; for cause (b), live control returns 409 `CUSTOMER_OVERLAP` and no additional row. Final rows remain unchanged in all rejection cases. If User/Session DB validation is unavailable, return 503; never skip a check.

ID	Requirement trace	Setup / action	Expected observable result
BOOK-31	Transaction contract §6.2; #33 BDF-D05/D06; AC-S1-22/23; VC/TEST-S1-19/20	Inject structured PostgreSQL errors for (a) `23505 / booking_one_active_per_slot`, (b) `23P01 / booked_customer_no_overlap`, and negative controls (c) `23505` on a different constraint such as Booking primary key, (d) `23P01` with a different/unknown constraint identifier, and (e) missing constraint metadata.	Only (a)/(b) enter reconciliation. If no exact-pair replay is found, map (a) to 409 `SLOT_UNAVAILABLE` and (b) to 409 `CUSTOMER_OVERLAP`. Negative controls roll back and return safe 500 `INTERNAL_ERROR` without any User/Session/Booking reconciliation read or Booking DTO; never parse error text or infer a business conflict from SQLSTATE alone. Capture structured SQLSTATE/constraint metadata and verify committed rows are unchanged.
BOOK-32	Transaction contract §3 step 6 / §6.1; BR-S1-13; VC/TEST-S1-23	Pass the HTTP Session guard, then delete the exact Session row or make its owner mismatch before the SERIALIZABLE transaction's exact Session lock. In a separate run, force a concurrent Session update/delete that PostgreSQL reports as SQLSTATE `40001`.	Exact lookup returning no row/owner mismatch rolls back and returns 401 `UNAUTHENTICATED` immediately, with no Booking read/replay/DTO and without consuming retry budget. A genuine `40001` rolls back and retries the whole transaction; the fresh attempt re-reads and returns 401 if Session is absent, owner-mismatched, revoked or expired; only a live matching Session may proceed. Never turn a missing Session into 503 `TRANSACTION_RETRY_EXHAUSTED`.
BOOK-33	Transaction contract §3 steps 8–9; API §4; OD-013; AC-S1-29; VC/TEST-S1-38	Commit a Booking while Tutor is active, then in separate runs deactivate that Tutor or change its role away from `Tutor`. The same active Customer/live Session repeats the exact `slotId`; use a separate unbooked Slot for a new-claim control against the now-ineligible Tutor.	Same-pair replay returns 200 with the original Booking ID and exactly one row despite later Tutor deactivation/role change. A new claim for the ineligible Tutor returns generic 409 `SLOT_UNAVAILABLE`, creates no row and reveals no Tutor identity/status.

3. Required existing-anchor coverage

Existing anchor	Planned case
VC/TEST-S1-16	BOOK-01
VC/TEST-S1-17	BOOK-02
VC/TEST-S1-18	BOOK-03
VC/TEST-S1-19	BOOK-04, BOOK-31
VC/TEST-S1-20	BOOK-05, BOOK-31
VC/TEST-S1-21	BOOK-06
VC/TEST-S1-22	BOOK-07
VC/TEST-S1-23	BOOK-08, BOOK-20, BOOK-26..30, BOOK-32
VC/TEST-S1-24	BOOK-09, BOOK-20, BOOK-28..29
VC/TEST-S1-25	BOOK-10
VC/TEST-S1-26	BOOK-11
VC/TEST-S1-27	BOOK-12, BOOK-21
VC/TEST-S1-33	BOOK-12, BOOK-21
VC/TEST-S1-38	BOOK-13..16, BOOK-30, BOOK-33
VC/TEST-S1-39	BOOK-17, BOOK-19
AC-S1-22/23	BOOK-04/05, BOOK-31
AC-S1-19..29	BOOK-01..16, BOOK-20 and BOOK-26..30, BOOK-33

Existing anchor	Planned case
BR-S1-13	BOOK-26..30, BOOK-32
AC-S1-35	BOOK-17, BOOK-19

Existing AC/VC/TEST IDs remain authoritative; BOOK IDs are F-S1-04 STS IDs, not replacements.

4. Part E boundary and Part F evidence

Part E: contracts and planned cases only. No test result is PASS, no PostgreSQL query has run, and no schema SHA exists yet.

Part F evidence per run: test ID; command; source commit; merged #33 schema SHA; PostgreSQL version; synthetic fixture IDs; backend PIDs/barrier; `T_elig`; SQLSTATE/constraint; expected/actual HTTP result; committed rows before/after; PASS/FAIL/SKIP and reason. Redact secrets/PII.

#33 BDF-01..22 and BDF-24..26 are its Part F pre-merge gates. #16 BDF-23 is the post-merge joint integration gate and runs only after #33 Part F merges.

5. Independent review checklist

- [ ] Confirm endpoint ownership, auth/role/CSRF, request allowlist and safe errors.
- [ ] Confirm Natural-key OD-013 scope, replay precedence and 201/200 behavior.
- [ ] Confirm canonical #33 mapping and no duplicated schema/trigger.
- [ ] Confirm User → Slot → Session(auth sub-lock) → Booking acquisition, SERIALIZABLE snapshot behavior, post-lock-wait User/Session revalidation, and fresh post-ROLLBACK User-then-Session authorization validation before reconciliation read/200/409.
- [ ] Confirm Auth CSRF acquisition, post-CSRF `/auth/me` binding recheck, Session-switch rejection before POST, cross-origin rejection and late-response suppression across different Sessions for the same User.
- [ ] Confirm retry classification, three-attempt bound, exact SQLSTATE/constraint allowlist (including negative controls), safe unknown-constraint failure and unknown COMMIT behavior.
- [ ] Challenge no-post-guard-contention strategy against actual #33 trigger/constraints; retain BDF-23 as hard gate.
- [ ] Confirm AC-S1-19..29/35 and VC/TEST-S1-16..27/33/38/39 coverage.
- [ ] Confirm no code, migration, executable tests or S2 workflow is in this Part E PR.
- [ ] Record actual human peer review; self-check is not approval.

Status: PLANNED / NOT RUN. Issue #16 remains open until Part F, BDF-23, actual verification and independent review satisfy its DoD.

E.6 F-S1-05: Protected Booking Retrieval & Authorized Tutor Schedule

---

E.6.1 F-S1-05 Protected Booking Retrieval Feature SDS (specification.md)

> **Documentation contract only; not implemented, executed, or peer-approved yet.**
> Issue #17; provisional lead @Chxtamos; independent peer reviewer @Tanaboonnnnn.
> This Part E document specifies the data allowlists, authorization boundaries, UI/API contracts, and Software Test Specification (STS) for Sprint 1 Booking retrieval and Tutor schedule inspection. This change does not implement UI/API code, add a migration, execute tests, or close the issue.

---

1. Authority, Baseline, and Mission

1.1 Source Hierarchy

This specification derives from and complies with:

1. **Sprint 1 Requirements:**
   - Functional requirements: `FR-S1-04`, `FR-S1-07`, `FR-S1-14`, `FR-S1-15`, `FR-S1-16`, `FR-S1-17`, `FR-S1-18`, `FR-S1-19`.
   - Business rules: `BR-S1-01`, `BR-S1-02` (60m duration), `BR-S1-03` (manual offer), `BR-S1-12` (Asia/Bangkok timezone), `BR-S1-14`, `BR-S1-15`, `BR-S1-16`, `BR-S1-17`, `BR-S1-18`, `BR-S1-22`.
   - Non-functional requirements: `NFR-S1-01` (Access isolation), `NFR-S1-02` (Credential protection), `NFR-S1-03` (Persistent correctness), `NFR-S1-06` (Time-boundary correctness), `NFR-S1-07` (Reproducible verification), `NFR-S1-08` (No secret leakage), `NFR-S1-09` (Usability proposal — keyboard & visible focus), `NFR-S1-12` (Traceable change & review).
   - Acceptance criteria: `AC-S1-30`, `AC-S1-31`, `AC-S1-32`, `AC-S1-33`, `AC-S1-34`.
   - Planned validation anchors: `VC/TEST-S1-28`, `VC/TEST-S1-29`, `VC/TEST-S1-30`, `VC/TEST-S1-31`, `VC/TEST-S1-32`.
   - Direct stakeholder decisions: `Q01` (Single booking), `Q02` (Role authorization), `Q03` (Fixed 60-minute Slot duration), `Q04` (Manual individual Slot offers, no recurrence), `Q05` (Cutoff lead time), `Q28` (Overlap rules), `Q29` (Single Asia/Bangkok business timezone), `Q30` (Booked vs Past status, never Completed in S1).
2. **Whole-product SRS:** System actors, user journeys `J-01` (Auth), `J-02` (Tutor publishing), `J-03` (Customer booking), `J-04` (Customer booking review), `J-05` (Tutor schedule review).
3. **System SDS:** §§3.2–3.3 (Data entity relations), §4 (API conventions and error envelopes), §5 (Role & record level authorization), §6 (Timezone handling in UTC on wire and Asia/Bangkok on display).
4. **Accepted Preceding Contracts:**
   - Auth F-S1-01: Session cookie, identity, role guard, rate limiting, and CSRF token.
   - Tutor Availability F-S1-02: Owned 60-minute Slots, `startsAt` / `endsAt`, `lifecycle = 'Offered'`.
   - Customer Availability Discovery F-S1-03: Bookable Slot filtering, `T_view` evaluation, opaque Tutor references.
   - Booking Data Foundation D1-08A/B: Canonical `Booking` table, `booking_guard_s1()` trigger, GiST exclusion constraints, and pre-index `User` lock (`da17acf`).
   - Exclusive Booking F-S1-04: SERIALIZABLE adapter, natural-key replay, committed `Booked` status.

1.2 Repository Baseline

- Integration base: `origin/sprint1` at commit `ed78bbad9da8c828e9b0bd1c79ea8f68f553c9b1`, containing all merged foundations:
  - Auth implementation (PR #26)
  - Tutor Slots & Dashboard (PR #28, PR #31)
  - Shared Booking Data Foundation (PR #36) & Remediation (PR #40)
  - Customer Availability Discovery (PR #37)
  - Customer Exclusive Booking & BDF-23 (PR #38)

---

2. Mission and Core Outcomes

2.1 The Two Primary User Outcomes

1. **Customer-Private Booking Review (`FR-S1-14` , `AC-S1-30` , `AC-S1-31`):**

An authenticated Customer can retrieve a complete list of their **own** committed Bookings and open individual Booking details after browser refresh or application restart. The details display Booking ID, Slot ID, Tutor reference/identity, appointment interval in Asia/Bangkok time, and business status (`Booked` or time-derived `Past`). No unauthorized Customer or anonymous caller can read or discover any private Booking payload.

1. **Authorized Tutor Schedule Inspection (`FR-S1-07` , `FR-S1-15` , `AC-S1-32` , `AC-S1-33`):**

An authenticated Tutor can inspect the collection of Slots and associated Bookings on their **own** authorized schedule. The Tutor schedule view projects each Slot's occupancy state (`Unoccupied` vs `Occupied`) and lists active Bookings assigned to their schedule without leaking other Tutors' restricted schedule records or foreign Customer data. For privacy and data minimization, Customer identity on the Tutor schedule is represented by a pseudonymous truncated reference (`customerRef: "CUST-" + customerId.slice(0, 8)`), omitting personal identifiers (name, email, phone) and isolating customer UUIDs from the Tutor wire DTO.

2.2 Time-Derived Classification: "Past" is NOT "Completed" (`FR-S1-17` , `BR-S1-14/22` , `AC-S1-34` , `Q30`)

- In Sprint 1, the database business status of an active appointment is always `'Booked'` (or `'Cancelled'` in future sprints).
- When the appointment window has elapsed relative to the single statement evaluation timestamp (`endsAt <= meta.evaluatedAt`), the UI and API classify the presentation as `Past` .
- **Strict Invariant:** An elapsed appointment **MUST NOT** be automatically updated to `'Completed'` , marked as attended, canceled, or released back to the bookable pool. The persisted database row remains `status = 'Booked'` , and the Slot remains occupied.

---

3. Boundaries and Scope

3.1 In Scope for Sprint 1 (F-S1-05)

- **Customer Endpoints:**
  - `GET /api/v1/customer/bookings` : Returns all Bookings belonging strictly to `principal.userId` .
  - `GET /api/v1/customer/bookings/:id` : Returns single Booking detail if owned by `principal.userId` ; returns `404 BOOKING_NOT_FOUND` if non-existent or owned by another user.
- **Tutor Endpoints:**
  - `GET /api/v1/tutor/slots` : Updated to project dynamic occupancy (`Occupied` if an active `Booked` row exists on that Slot, else `Unoccupied`).
  - `GET /api/v1/tutor/bookings` : Returns all active Bookings scheduled on Slots owned by `principal.userId` .
- **Backend Record-Level Security:**
  - All ownership filters are strictly bound to `principal.userId` from the authenticated session; never trust client-supplied IDs, query parameters, or route tampering.
  - Fail closed on unauthorized access: anonymous calls return `401 UNAUTHENTICATED` ; role mismatches return `403 FORBIDDEN` ; foreign record queries return `404 BOOKING_NOT_FOUND` (preventing ID enumeration).
- **Time Handling:**
  - Wire timestamps: Strict ISO 8601 / RFC 3339 UTC strings ending in `Z` with milliseconds (e.g., `YYYY-MM-DDTHH:mm:ss.sssZ`).
  - Evaluation timestamp: API returns `meta.evaluatedAt` representing server-authoritative time used for the `displayStatus` derivation.
  - UI Presentation: Formatted strictly in `Asia/Bangkok` timezone (UTC+7) regardless of local device timezone.
- **Durable Persistence Proof:**
  - Data retrieval queries committed rows from PostgreSQL database tables (`Booking` , `Slot` , `User`).
  - Persistence holds across app restart and browser reloads.

3.2 Explicitly Out of Scope for Sprint 1

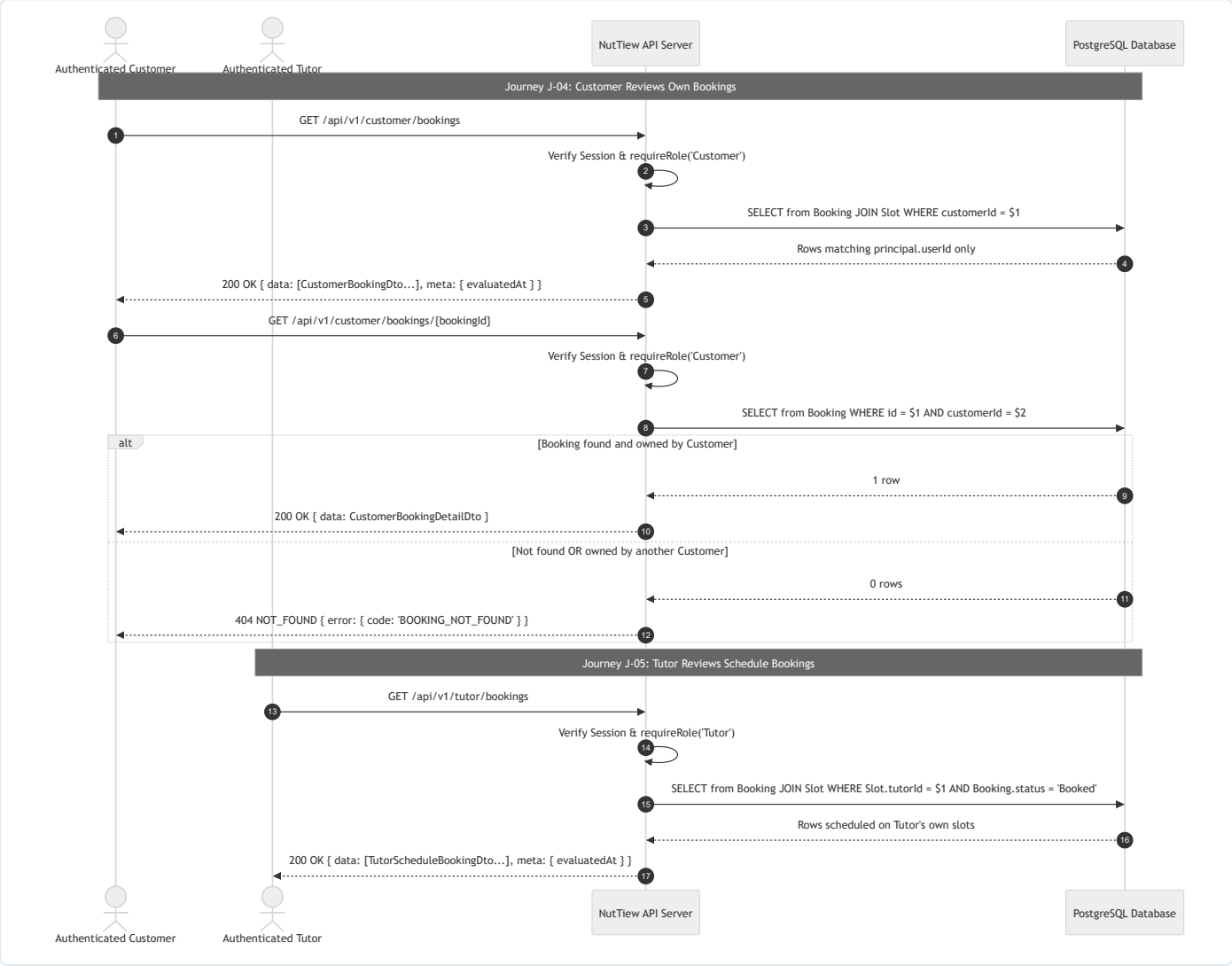
- **No Mutation:** F-S1-05 is strictly a read-only retrieval feature; it performs zero database `INSERT` , `UPDATE` , `DELETE` , or schema DDL.
- **No Sprint 2/3 Capabilities:**
  - Customer cancellation (`Q07–Q08`) or rescheduling (`Q09–Q16`).
  - Tutor withdrawal reasons, cancellations, or slot modifications (`Q17–Q20` , `Q26–Q27` , `Q32`).

- In-app notification center or email delivery (`Q21–Q25`).
- Tutor appointment summary dashboard with attendance tracking (`Q31`).
- Review, rating, messaging, payments, or attendance logging.

- **No Unapproved Query Parameters:** No filtering, search, pagination, date-range bounding, or custom sorting is approved for Sprint 1. Requests with unexpected query parameters fail with `400 INVALID_INPUT` .

---

4. User Journeys and Actors



---

5. Architectural Invariants and Decisions

Decision / Invariant ID	Summary	Rationale & Enforcement
INV-RET-01: Session-Bound Ownership	All read queries filter strictly by `principal.userId` extracted from the server-validated session.	Client cannot forge or query other users' data. URL tampering produces `404` or `403` .
INV-RET-02: Safe Enumeration Refusal (404 instead of 403)	If a Customer requests `/api/v1/customer/bookings/:id` for an ID that exists but belongs to someone else, return `404 BOOKING_NOT_FOUND` .	Returning `403 FORBIDDEN` confirms the ID exists in the system (information leakage). `404` prevents ID brute-forcing.
INV-RET-03: No Phantom Completion	Elapsed appointments (`endsAt <= meta.evaluatedAt` , using statement snapshot) have `displayStatus = 'Past'` , while `status = 'Booked'` .	Adheres strictly to `Q30` , `FR-S1-17` , and `BR-S1-22` . Sprint 1 does not implement attendance or completion.

Decision / Invariant ID	Summary	Rationale & Enforcement
INV-RET-04: Dynamic Slot Occupancy	`listOwnSlots` determines `occupancy` via an active `Booked` record matching `Booking.currentSlotId = Slot.id`.	Delivers `FR-S1-07` without adding a mutable `Slot.isBooked` column, preserving single-source-of-truth normalization.
INV-RET-05: Strict UTC Wire Normalization & Bangkok Presentation	Timestamps returned by the API are normalized to UTC with trailing `Z` on the wire, while UI presentation formats strictly in Asia/Bangkok time.	Satisfies `BR-S1-12`, `Q29`, and `NFR-S1-06`; eliminates ambiguity caused by database server or runtime timezone settings.
INV-RET-06: Strict Query Allowlist	Query strings are forbidden (`Object.keys(req.query).length === 0`).	Prevents SQL injection, parameter pollution, and unapproved filtering features (`NFR-S1-08`).
INV-RET-07: Minimal Payload Privacy	Responses exclude password hashes, CSRF nonces, session digests, and unapproved contact data.	Strict DTO allowlists satisfy `NFR-S1-01` and `NFR-S1-08`.
INV-RET-08: Pseudonymous Customer Reference	`customerRef: "CUST-" + customerId.slice(0, 8)` is a truncated pseudonymous identifier, not an irreversible anonymization.	Provides data minimization for Tutor schedule display by withholding customer name, email, and phone, while maintaining an opaque operational reference.

---

6. Acceptance Criteria and Traceability Matrix

Requirement ID	Acceptance Criteria	Business Rule	Verification Anchor	Description
`FR-S1-14`, `FR-S1-16`	**AC-S1-30**	`BR-S1-01`, `BR-S1-15`	`VC/TEST-S1-28`	Customer retrieves own list/detail after refresh/restart with exact Booking ID, Tutor, Bangkok interval, and status.
`FR-S1-04`, `FR-S1-14`	**AC-S1-31**	`BR-S1-15`, `BR-S1-18`	`VC/TEST-S1-29`	Customer B attempts to view Customer A's Booking ID; API returns safe `404 BOOKING_NOT_FOUND` with zero data leakage.
`FR-S1-07`, `FR-S1-15`	**AC-S1-32**	`BR-S1-16`	`VC/TEST-S1-30`	Tutor views Bookings on own schedule; occupied slots display `Occupied`; only assigned Bookings are visible.
`FR-S1-04`, `FR-S1-15`	**AC-S1-33**	`BR-S1-16`, `BR-S1-18`	`VC/TEST-S1-31`	Tutor B attempts to read Tutor A's schedule/bookings; access denied, zero foreign records disclosed.
`FR-S1-17`	**AC-S1-34**	`BR-S1-14`, `BR-S1-22`	`VC/TEST-S1-32`	An elapsed appointment displays as `Past`, but persisted business state remains `Booked`, never `Completed`.

---

7. Definition of Done (DoD)

Part E — Specification Stage (Current PR)

- [ ] Complete `specification.md`, `api-spec.md`, `ui-spec.md`, `data-spec.md`, and `tests.md` authored under `docs/features/F-S1-05/`.
- [ ] Invariants, request/response DTO schemas, and error codes strictly mapped to requirements and existing architecture.
- [ ] No application code, migrations, or tests implemented in this documentation PR.
- [ ] Peer review requested from @Tanaboonnnnn and approved before branching Part F.

Part F — Implementation Stage (Subsequent Work)

- [ ] Backend route handlers, services, and queries implemented for Customer bookings and Tutor schedule views.
- [ ] Tutor own-slots list updated to project dynamic occupancy from committed `Booking` rows.
- [ ] Customer and Tutor UI components connected to live API endpoints.
- [ ] STS test suite `RET-01..RET-17` executed with real PostgreSQL and Playwright browser, achieving 100% PASS.

E.6.2 F-S1-05 Protected Booking Retrieval UI Specification (ui-spec.md)

> **Documentation contract only; not implemented yet.**
> Issue #17. This document defines the user interface specifications, layout states, responsive behavior, accessibility guidelines, and timezone presentation for Customer Booking retrieval and Tutor schedule inspection.

---

1. Design System and Visual Tokens

All UI components for F-S1-05 directly reuse and map to the established NutTiew design system tokens from `F-S1-01`, `F-S1-02` (Tutor Dashboard), and `F-S1-03` (Customer Availability) as implemented in `apps/web/src/customer-availability.css` and `apps/web/src/tutor-dashboard.css`. No new or drifting color palette is introduced.

Design Token	Existing CSS Variable / Value	Purpose & Contrast Specification
Canvas Background	`--customer-canvas` / `--tutor-canvas`: `#f7f8fb`	Light application background across all screens
Surface Card	Surface: `#ffffff`	Primary content panels, cards, and modals
Brand / Shell Navigation	`--customer-navy` / `--tutor-navy`: `#202b44` Deep: `#172137`	Top navigation bar, wordmark header, active navigation state
Divider Border	`--customer-border` / `--tutor-border`: `#e0e6ef`	Section dividers and card container borders
Control Border	`--customer-control-border`: `#66778a`	Form controls, modal borders, and interactive card boundaries (4.59:1 on white)
Primary Text (Ink)	`--customer-ink`: `#172137` `--tutor-ink`: `#263244`	High-contrast body text, headings, and primary card labels (>= 12:1 on white)
Secondary Text (Muted)	`--customer-muted`: `#455a76` `--tutor-muted`: `#58697f`	Timestamps, metadata, labels (>= 4.5:1 on white and canvas)
Primary Action (CTA)	`--customer-orange` / `--tutor-orange`: `#f47736` Hover: `#a43e14` Border: `#d96123`	Refresh and navigation action buttons; paired strictly with dark ink `#172137` text (never low-contrast white text on orange)
Focus Indicator	`--customer-focus` / `--tutor-focus`: `#aa4519`	`3px solid #aa4519` with `outline-offset: 3px`; visible and unobstructed on all interactive elements
Status Badge: Booked	`background: #eef2f7; color: #172137; border: 1px solid #c6d2e3`	Active / upcoming reservation badge; label **ยืนยันแล้ว** (Customer) or **มีนัดหมาย (Booked)** (Tutor)
Status Badge: Past	`background: #f1f3f6; color: #58697f; border: 1px solid #d4dbe4`	Elapsed appointment badge; label **สิ้นสุดแล้ว** (Past)
Occupancy Badge: Occupied	`background: #fff7ed; color: #a43e14; border: 1px solid #f47736`	Tutor agenda slot occupancy; label **มีผู้จองแล้ว (Occupied)**
Occupancy Badge: Unoccupied	`background: #eef2f7; color: #172137; border: 1px solid #c6d2e3`	Tutor agenda available slot; label **ว่าง (เปิดรับจอง)**

All text combinations pass WCAG 2.2 AA minimum contrast ratios (>= 4.5:1 for normal text, >= 3:1 for large text and UI borders).

---

2. Timezone and Date Presentation Rules (`BR-S1-12`, `Q29`, `NFR-S1-06`)

1. **Mandatory Asia/Bangkok Timezone:**

- All appointment intervals (`startsAt` to `endsAt`) and creation dates must be formatted explicitly in `Asia/Bangkok` (UTC+7) using `Intl.DateTimeFormat` or equivalent, regardless of the user's operating system or device timezone.

2. **Format Standard:**
   - Date: `d MMM yyyy` in Thai locale (e.g., `10 พ.ย. 2026`).
   - Time: `HH:mm` in 24-hour cycle (e.g., `16:00 – 17:00 น.`).
   - Combined Summary: `10 พ.ย. 2026, 16:00 – 17:00 น.`

---

3. Customer Booking Retrieval UI

3.1 Customer Bookings List (`#customer-bookings-section`)

A Customer navigates to "การจองของฉัน" (My Bookings) within the Customer Portal (`apps/web/src/customer-availability.tsx` / `apps/web/src/main.tsx`):

1. **Header & Navigation:**
   - Screen Title: `h1` "การจองของฉัน" (My Bookings).
   - Navigation button/tab to switch between "ตารางติวที่เปิดจอง" (Available Offers) and "การจองของฉัน" (My Bookings).
   - Button "รีเฟรช" (Refresh list).
2. **Booking Card Component (`.customer-booking-card`):**
   - **Tutor Reference:** Displays Tutor opaque reference or approved display name (e.g., `ผู้สอน: TUTOR-f7a8b9c0`).
   - **Appointment Window:** Bangkok time (e.g., `10 พ.ย. 2026, 16:00 – 17:00 น.`).
   - **Status Badge:**
     - `Booked` (Active/Upcoming): Light navy tint surface with border `#c6d2e3` and text **ยืนยันแล้ว**.
     - `Past` (Elapsed): Neutral gray surface with border `#d4dbe4` and text **สิ้นสุดแล้ว**.
   - **Detail Action Button:** `<button data-testid="view-booking-detail">ดูรายละเอียด</button>` (opens detail modal on `/customer/bookings`).
3. **Empty State (`.customer-bookings-empty`):**
   - Text: "คุณยังไม่มีประวัติการจองติว".
   - Action link/button: "ค้นหาตารางติวเพื่อเริ่มจอง" (navigates back to availability).
4. **Loading State:**
   - Text: "กำลังโหลดรายการจองของคุณ...".
5. **Error State (`.customer-bookings-error`):**
   - Text: "ไม่สามารถโหลดข้อมูลการจองได้ในขณะนี้".
   - Action button: "ลองใหม่อีกครั้ง" (retries fetch).

3.2 Customer Booking Detail View (`#customer-booking-detail-modal` / `.booking-detail-view`)

When a Customer clicks "ดูรายละเอียด" on a Booking card:

1. **Header:** Title `h2` "รายละเอียดการจอง".
2. **Field Summary Table / List:**
   - **รหัสการจอง (Booking ID):** Monospaced UUID string (e.g., `e4f8d53b-013d-4c57-891a-7b3c2e1f4001`) with copy button.
   - **ผู้สอน (Tutor):** `TUTOR-f7a8b9c0` .
   - **วันและเวลาติว (Appointment Time):** `10 พ.ย. 2026, 16:00 – 17:00 น. (Asia/Bangkok)` .
   - **สถานะ (Status):**
     - Upcoming: Badge `ยืนยันแล้ว` (`Booked`).
     - Elapsed: Badge `สิ้นสุดแล้ว` (`Past`). Notice: "การนัดหมายนี้สิ้นสุดลงแล้ว".

- **วันที่ทำรายการ (Booked At):** Bangkok formatted timestamp.

3. **Close / Back Action:** `<button data-testid="close-detail-btn">ปิด / กลับสู่รายการ</button>`.
4. **Not Found State (`404 BOOKING_NOT_FOUND`):**
   - If user accesses an invalid or foreign Booking ID:
   - Displays "ไม่พบข้อมูลการจองนี้" with button "กลับสู่รายการจอง".
   - Zero private data (no Customer names, IDs, or timestamps) is rendered.

---

4. Tutor Schedule Inspection UI

4.1 Integration into Tutor Dashboard (`apps/web/src/tutor-slots.tsx`)

Fulfills `FR-S1-07` and `FR-S1-15` :

1. **Slots Calendar & Agenda (`AllSlotsAgenda` & `SlotCalendar`):**
   - Each Slot card displays its real-time occupancy status:
     - If `occupancy === 'Unoccupied'` : Light navy tint badge **`ว่าง (เปิดรับจอง)`** (`#eef2f7`).
     - If `occupancy === 'Occupied'` : Amber/orange tinted badge **`มีผู้จองแล้ว (Occupied)`** (`#fff7ed` with border `#f47736`).
2. **Schedule Bookings List (`#tutor-schedule-bookings-section`):**
   - Section Title: `h2` "รายการนัดหมายบนตารางสอนของคุณ" (Bookings on Your Schedule).
   - Card for each assigned Booking:
     - **รหัสการจอง:** Booking UUID.
     - **ช่วงเวลา:** Bangkok formatted time (e.g., `10 พ.ย. 2026, 16:00 – 17:00 น.`).
     - **ผู้จอง:** Pseudonymous Customer Reference (e.g., `CUST-e4f8d53b`); provides data minimization by withholding customer name, email, and phone, and isolating customer UUIDs from the Tutor schedule view.
     - **สถานะ:**
       - Upcoming: Badge `มีนัดหมาย (Booked)` (`#eef2f7`).
       - Elapsed: Badge `สิ้นสุดแล้ว (Past)` (`#f1f3f6`).
3. **Empty Schedule State:**
   - If Tutor has no bookings on their schedule:
   - Text: "ยังไม่มีผู้จองในช่วงเวลาที่คุณเปิดสอน".

---

5. Viewport and Accessibility Specifications (`NFR-S1-09` Proposal)

1. **Responsive Viewports:**
   - Small Mobile (320px): Single-column stacked cards, full-width buttons (min 44px touch target).
   - Standard Mobile (375px): Optimized padding (16px), readable typography.
   - Tablet (768px): Two-column card grid or tabular list.
   - Desktop (1280px): Multi-column dashboard with side-by-side agenda and booking details.
2. **Keyboard Navigation:**
   - All interactive elements (refresh buttons, detail triggers, modal dismissals) must be focusable using `Tab` .
   - `Enter` and `Space` activate buttons.
   - `Escape` key closes the Booking Detail modal and returns focus to the triggering element.
3. **Screen Reader Semantics:**
   - Section headings use proper semantic hierarchy (`h1` -> `h2` -> `h3`).
   - Badges provide accessible aria-labels (e.g., `aria-label="สถานะการจอง: ยืนยันแล้ว"`).

E.6.3 F-S1-05 Protected Booking Retrieval API Specification (api-spec.md)

> **Documentation contract only; not implemented yet.**
> Issue #17. This document defines the HTTP endpoints, authorization rules, request/response allowlists, and error mapping for Customer-private Booking retrieval and authorized Tutor schedule inspection.

---

1. Authentication and Global Security Protocols

All endpoints specified in this document reuse the shared authentication, session validation, and privacy mechanisms established in `F-S1-01` and `F-S1-02`:

1. **Session Cookie:** Requires HttpOnly session cookie (`nuttiew_session` in production, `nuttiew_session_dev` in local/test). Unauthenticated requests immediately fail with `401 UNAUTHENTICATED`.
2. **Role Enforcement:** Server guards enforce role boundaries before executing business logic:
   - Customer endpoints enforce `requireRole('Customer')`.
   - Tutor endpoints enforce `requireRole('Tutor')`.
   - Any cross-role query returns `403 FORBIDDEN`.
3. **Session Binding Header:** Every successful `200 OK` response returns the `X-NutTiew-Session-Binding` header computed as `SHA-256("nuttiew-ui-binding:v1\0" + userId + "\0" + sessionId)`.
4. **Cache Control:** Every response includes `Cache-Control: no-store` to prevent private Booking records from caching on intermediary proxies or browser caches.
5. **No Query Parameters:** In Sprint 1, no filtering, sorting, or pagination query parameters are supported. If `Object.keys(req.query).length > 0`, the endpoint rejects the request with `400 INVALID_INPUT`.

---

2. Customer Endpoints

2.1 List Own Bookings

Retrieves all Bookings owned by the currently authenticated Customer.

- **Method / Path:** `GET /api/v1/customer/bookings`
- **Authorization:** Authenticated Customer only (`role = Customer`).
- **Query Parameters:** None permitted. Any query string produces `400 INVALID_INPUT`.
- **Ordering:** Deterministic ascending order: `startsAt ASC, id ASC`.
- **Evaluation Snapshot Uniformity:** `meta.evaluatedAt` represents a single server evaluation timestamp captured at statement execution (`statement_timestamp()`). This exact timestamp is applied uniformly to compute `displayStatus` across all returned items. Even when no bookings exist (`data: []`), `meta.evaluatedAt` is always populated as an ISO-8601 UTC timestamp.

Success Response (`200 OK`)


```
{
  "data": [
    {
      "id": "e4f8d53b-013d-4c57-891a-7b3c2e1f4001",
      "slotId": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
      "tutorId": "f7a8b9c0-d1e2-3a4b-5c6d-7e8f9a0b1c2d",
      "tutorRef": "TUTOR-f7a8b9c0",
      "startsAt": "2026-11-10T09:00:00.000Z",
      "endsAt": "2026-11-10T10:00:00.000Z",
      "status": "Booked",
      "displayStatus": "Booked",
      "createdAt": "2026-11-09T08:30:00.000Z"
    },
    {
      "id": "b3c2e1f4-013d-4c57-891a-e4f8d53b0002",
      "slotId": "9c0d1e2f-3a4b-5c6d-a1b2-c3d4e5f67a8b",
      "tutorId": "5c6d7e8f-9a0b-1c2d-f7a8-b9c0d1e23a4b",
      "tutorRef": "TUTOR-5c6d7e8f",
      "startsAt": "2026-11-08T03:00:00.000Z",
      "endsAt": "2026-11-08T04:00:00.000Z",
      "status": "Booked",
      "displayStatus": "Past",
      "createdAt": "2026-11-07T14:15:00.000Z"
    }
  ],
  "meta": {
    "requestId": "9c5a1e2f-3b4c-5d6e-7f8a-9b0c1d2e3f4a",
    "evaluatedAt": "2026-11-09T11:30:00.000Z"
  }
}
```

2.2 Retrieve Booking Detail

Retrieves full details for a single specific Booking owned by the authenticated Customer.

- **Method / Path:** `GET /api/v1/customer/bookings/:id`
- **Authorization:** Authenticated Customer only (`role = Customer`).
- **Path Parameter:**
  - `id` (string, required): A standard 36-character UUID format.
- **Privacy Refusal Invariant:**
  - If the requested `:id` does not exist in the database, OR if it exists but belongs to a different Customer, the server returns **`404 BOOKING_NOT_FOUND`**.
  - The server **MUST NOT** return `403 FORBIDDEN` for another user's valid Booking ID, as returning `403` leaks the existence of that record to an unauthorized prober.

Success Response (`200 OK`)


```
{
  "data": {
    "id": "e4f8d53b-013d-4c57-891a-7b3c2e1f4001",
    "slotId": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
    "tutorId": "f7a8b9c0-d1e2-3a4b-5c6d-7e8f9a0b1c2d",
    "tutorRef": "TUTOR-f7a8b9c0",
    "startsAt": "2026-11-10T09:00:00.000Z",
    "endsAt": "2026-11-10T10:00:00.000Z",
    "status": "Booked",
    "displayStatus": "Booked",
    "createdAt": "2026-11-09T08:30:00.000Z",
    "updatedAt": "2026-11-09T08:30:00.000Z"
  },
  "meta": {
    "requestId": "8a7b6c5d-4e3f-2a1b-0c9d-8e7f6a5b4c3d",
    "evaluatedAt": "2026-11-09T11:30:00.000Z"
  }
}
```


---

3. Tutor Endpoints

3.1 List Tutor Own Slots (Updated with Dynamic Occupancy)

Fulfills `FR-S1-07`: A Tutor must be able to view their own Slots including whether each is `Unoccupied` or `Occupied`.

- **Method / Path:** `GET /api/v1/tutor/slots`
- **Authorization:** Authenticated Tutor only (`role = Tutor`).
- **Query Parameters:** None permitted (`Object.keys(req.query).length === 0`).
- **Behavior Update:** In `apps/api/src/slots.ts` , `occupancy` is derived via left-join or subquery against active `Booking` records (`status = 'Booked' AND currentSlotId = Slot.id`). If an active booking exists, `occupancy: 'Occupied'` ; otherwise `occupancy: 'Unoccupied'` .

Success Response (`200 OK`)


```
{
  "data": [
    {
      "id": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
      "startsAt": "2026-11-10T09:00:00.000Z",
      "endsAt": "2026-11-10T10:00:00.000Z",
      "lifecycle": "Offered",
      "occupancy": "Occupied"
    },
    {
      "id": "f6e5d4c3-b2a1-0f9e-8d7c-6b5a4f3e2d1c",
      "startsAt": "2026-11-10T11:00:00.000Z",
      "endsAt": "2026-11-10T12:00:00.000Z",
      "lifecycle": "Offered",
      "occupancy": "Unoccupied"
    }
  ],
  "meta": {
    "requestId": "7e6d5c4b-3a2f-1e0d-9c8b-7a6f5e4d3c2b"
  }
}
```

3.2 List Tutor Schedule Bookings

Fulfills `FR-S1-15` & `AC-S1-32`: An authorized Tutor inspects Bookings on their own schedule.

- **Method / Path:** `GET /api/v1/tutor/bookings`
- **Authorization:** Authenticated Tutor only (`role = Tutor`).

- **Query Parameters:** None permitted. Any query parameters produce `400 INVALID_INPUT`.
- **Query Predicate:** Filters strictly by `Slot.tutorId = principal.userId` and `Booking.status = 'Booked'`.
- **Privacy Guarantee:** Excludes other Tutors' bookings. Represents Customer identity as a pseudonymous truncated reference (`customerRef: "CUST-" + customerId.slice(0, 8)`), omitting Customer PII (name, email, phone) and isolating internal customer UUIDs from the Tutor wire DTO. Note: this is pseudonymous data minimization, not an irreversible cryptographic anonymization. Never returns password hashes, session cookies, or foreign contact details.

Success Response (`200 OK`)

```
{
  "data": [
    {
      "id": "e4f8d53b-013d-4c57-891a-7b3c2e1f4001",
      "slotId": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
      "tutorId": "f7a8b9c0-d1e2-3a4b-5c6d-7e8f9a0b1c2d",
      "customerRef": "CUST-e4f8d53b",
      "startsAt": "2026-11-10T09:00:00.000Z",
      "endsAt": "2026-11-10T10:00:00.000Z",
      "status": "Booked",
      "displayStatus": "Booked",
      "createdAt": "2026-11-09T08:30:00.000Z"
    }
  ],
  "meta": {
    "requestId": "1a2b3c4d-5e6f-7a8b-9c0d-1e2f3a4b5c6d",
    "evaluatedAt": "2026-11-09T11:30:00.000Z"
  }
}
```

---

4. Status and Display Classification (`Q30` , `FR-S1-17` , `BR-S1-22`)

The API explicitly returns two separate status fields to cleanly distinguish database business status from time-derived display status:

Field	Possible Values	Semantics	
`status`	`'Booked'` (or `'Cancelled'` in future sprints)	Authoritative database status persisted in the `Booking.status` column.	
`displayStatus`	`'Booked'` \	`'Past'`	Time-derived presentation status computed relative to the single statement evaluation snapshot `meta.evaluatedAt` . If `endsAt <= meta.evaluatedAt` , `displayStatus` is `'Past'` ; if `endsAt > meta.evaluatedAt` , `displayStatus` is `'Booked'` .

Rule: `displayStatus = 'Past'` is an informative presentation label. It **never** causes the row in PostgreSQL to be marked as `Completed` , nor does it release the Slot from occupancy. Both list endpoints and single-detail endpoints evaluate `displayStatus` using this single snapshot mechanism, and empty lists still return a valid ISO-8601 UTC `meta.evaluatedAt` .

---

5. Structured Error Responses

All error responses adhere to the standard error envelope `{ error: { code, message }, requestId }` :

HTTP Status	Error Code	Trigger Condition
`400 Bad Request`	`INVALID_INPUT`	Unrecognized query parameters, malformed `:id` (non-UUID), or invalid payload.
`401 Unauthorized`	`UNAUTHENTICATED`	Missing, expired, revoked, or invalid session cookie.
`403 Forbidden`	`FORBIDDEN`	Authenticated user lacks required role (e.g. Tutor requesting Customer endpoint, or Customer requesting Tutor endpoint).

HTTP Status	Error Code	Trigger Condition
`404 Not Found`	`BOOKING_NOT_FOUND`	Booking ID does not exist, or exists but belongs to a different Customer.
`503 Service Unavailable`	`SERVICE_UNAVAILABLE`	Database connection failure or transient pool exhaustion.

```
{
  "error": {
    "code": "BOOKING_NOT_FOUND",
    "message": "Booking not found"
  },
  "requestId": "3d4e5f6a-7b8c-9d0e-1f2a-3b4c5d6e7f8a"
}
```

E.6.4 F-S1-05 Protected Booking Retrieval Data Contract (data-spec.md)

> **Documentation contract only; not implemented yet.**
> Issue #17. This document defines the read-only database queries, data projection models, privacy transformations, and consistency guarantees for F-S1-05.

---

1. Schema Authority and Read-Only Boundary

F-S1-05 introduces **zero database schema modifications**, zero new tables, zero new columns, and zero migrations. It reads from the existing canonical tables established in `F-S1-01` (Auth), `F-S1-02` (Slots), and `D1-08A/B` (Booking Data Foundation):

- `"User"` : Authentication identity, roles (`Customer` , `Tutor`), account status (`isActive`).
- `"Slot"` : 60-minute appointment windows (`startsAt` , `endsAt`), owner reference (`tutorId`), lifecycle (`Offered` , `Withdrawn`).
- `"Booking"` : Committed reservations (`id` , `customerId` , `currentSlotId` , `startsAt` , `endsAt` , `status` , `createdAt` , `updatedAt`).

All database access in F-S1-05 consists of pure `SELECT` queries executed at PostgreSQL's default `READ COMMITTED` isolation level against the committed MVCC state. No transaction locks (`FOR UPDATE` or `FOR SHARE`) are acquired.

---

2. Server-Authoritative Queries and Projections

2.1 Customer: List Own Bookings

Retrieves all Bookings belonging strictly to the authenticated Customer (`customerId = principal.userId`). All rows in the query share a single uniform evaluation snapshot derived via `statement_timestamp()` , preserved as `timestamptz` throughout SQL comparison to prevent session timezone drift:

```
WITH eval_snapshot AS (
  SELECT statement_timestamp() AS evaluated_at
)
SELECT
  b."id",
  b."currentSlotId" AS "slotId",
  s."tutorId",
  to_char(b."startsAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "startsAt",
  to_char(b."endsAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "endsAt",
  b."status",
  to_char(b."createdAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "createdAt",
  CASE
    WHEN b."endsAt" <= s_clock.evaluated_at THEN 'Past'
    ELSE 'Booked'
  END AS "displayStatus",
  to_char(s_clock.evaluated_at AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "evaluatedAt"
FROM eval_snapshot AS s_clock
CROSS JOIN "Booking" AS b
INNER JOIN "Slot" AS s ON s."id" = b."currentSlotId"
WHERE b."customerId" = $1
ORDER BY b."startsAt" ASC, b."id" ASC;
```

Data Mapping Rules:

- `tutorRef`: Generated as `"TUTOR-" + row.tutorId.slice(0, 8)`.
- `displayStatus`: Derived conditionally in SQL relative to `s_clock.evaluated_at`. If `endsAt <= evaluated_at`, `displayStatus = 'Past'`; otherwise `'Booked'`. Database `status` remains strictly `'Booked'`.
- `timestamptz` Integrity: `s_clock.evaluated_at` is preserved as `timestamptz` throughout SQL comparison against `b."endsAt"`. Converting `statement_timestamp()` via `AT TIME ZONE 'UTC'` in the CTE would strip timezone information into a naive timestamp (`timestamp without time zone`), causing PostgreSQL to interpret it using the session's active `TimeZone` (e.g. `Asia/Bangkok`), skewing comparisons by 7 hours. Preserving `timestamptz` guarantees absolute, timezone-independent comparison.
- `meta.evaluatedAt`: In multi-row results, the uniform `evaluatedAt` string (formatted explicitly `AT TIME ZONE 'UTC'`) is extracted for `meta.evaluatedAt`. When the query returns 0 rows (`data: []`), the application service captures `statement_timestamp()` so that `meta.evaluatedAt` is always populated as a valid ISO-8601 UTC timestamp.

2.2 Customer: Get Single Booking Detail

Retrieves one specific Booking by ID, strictly verifying ownership (`b."id" = $1 AND b."customerId" = $2`), using the same single statement evaluation timestamp preserved as `timestamptz`:

```
WITH eval_snapshot AS (
  SELECT statement_timestamp() AS evaluated_at
)
SELECT
  b."id",
  b."currentSlotId" AS "slotId",
  s."tutorId",
  to_char(b."startsAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "startsAt",
  to_char(b."endsAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "endsAt",
  b."status",
  to_char(b."createdAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "createdAt",
  to_char(b."updatedAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "updatedAt",
  CASE
    WHEN b."endsAt" <= s_clock.evaluated_at THEN 'Past'
    ELSE 'Booked'
  END AS "displayStatus",
  to_char(s_clock.evaluated_at AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "evaluatedAt"
FROM eval_snapshot AS s_clock
CROSS JOIN "Booking" AS b
INNER JOIN "Slot" AS s ON s."id" = b."currentSlotId"
WHERE b."id" = $1 AND b."customerId" = $2;
```

Refusal Policy:

If the query returns 0 rows, the application throws `BookingError(404, 'BOOKING_NOT_FOUND')`. This completely prevents an attacker from guessing Booking UUIDs to verify their existence.

2.3 Tutor: List Own Slots with Dynamic Occupancy

Replaces the hardcoded `'Unoccupied'` in `listOwnSlots` with an active `Booking` lookup:

```
SELECT
  s."id",
  to_char(s."startsAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "startsAt",
  to_char(s."endsAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "endsAt",
  s."lifecycle",
  CASE
    WHEN b."id" IS NOT NULL THEN 'Occupied'
    ELSE 'Unoccupied'
  END AS "occupancy"
FROM "Slot" AS s
LEFT JOIN "Booking" AS b
  ON b."currentSlotId" = s."id" AND b."status" = 'Booked'
WHERE s."tutorId" = $1
ORDER BY s."startsAt" ASC, s."id" ASC;
```

2.4 Tutor: List Schedule Bookings

Retrieves all Bookings scheduled on Slots owned by the authenticated Tutor (`s."tutorId" = $1`), evaluated against a single statement evaluation timestamp preserved as `timestamptz`:

```
WITH eval_snapshot AS (
  SELECT statement_timestamp() AS evaluated_at
)
SELECT
  b."id",
  b."currentSlotId" AS "slotId",
  s."tutorId",
  b."customerId",
  to_char(b."startsAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "startsAt",
  to_char(b."endsAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "endsAt",
  b."status",
  to_char(b."createdAt" AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "createdAt",
  CASE
    WHEN b."endsAt" <= s_clock.evaluated_at THEN 'Past'
    ELSE 'Booked'
  END AS "displayStatus",
  to_char(s_clock.evaluated_at AT TIME ZONE 'UTC', 'YYYY-MM-DD"T"HH24:MI:SS.MS"Z"') AS "evaluatedAt"
FROM eval_snapshot AS s_clock
CROSS JOIN "Booking" AS b
INNER JOIN "Slot" AS s ON s."id" = b."currentSlotId"
WHERE s."tutorId" = $1 AND b."status" = 'Booked'
ORDER BY b."startsAt" ASC, b."id" ASC;
```

Privacy Masking and Pseudonymity:

- The raw `customerId` column is projected in backend SQL only to construct the pseudonymous truncated reference `customerRef: "CUST-" + row.customerId.slice(0, 8)`.
- This truncated pseudonymous reference provides data minimization for the Tutor schedule view by withholding customer personal identifiers (name, email, phone) and isolating customer UUIDs from the Tutor wire DTO. It is not an irreversible cryptographic anonymization.
- The raw `customerId` UUID is **strictly excluded** from the serialized HTTP JSON response DTO.

---

3. Data Integrity and Privacy Invariants

Invariant	Description
No Schema Drift	The canonical database schema remains untouched.
No Cross-Customer Leaks	Customer queries use `WHERE b."customerId" = $1` with parameterized SQL; cross-customer access is physically impossible.
No Cross-Tutor Leaks	Tutor queries use `WHERE s."tutorId" = $1`; one Tutor can never inspect another Tutor's schedule.
No Secret Columns in DTO	The fields `passwordHash`, `tokenDigest`, `csrfNonce`, and internal error traces are never selected.
Immutable Past Records	Elapsed appointments are computed dynamically via `endsAt <= evaluatedAt`; no database mutation occurs.

E.6.5 F-S1-05 Software Test Specification (tests.md)

> **Documentation contract only; tests are planned and not executed yet.**
> Issue #17. This document details the test strategy, test fixtures, execution steps, expected outcomes, and traceability mapping for the `RET-01..RET-17` test suite covering Protected Booking Retrieval and Authorized Tutor Schedule Inspection.

---

1. Traceability and Test Case Index

Test ID	Requirement / Rule Trace	Acceptance Criteria	Validation Anchor	Target Layer & File	Summary of Verification
RET-01	`FR-S1-14`, `FR-S1-16`, `BR-S1-01`	**AC-S1-30**	`VC/TEST-S1-28`	API / Database (`retrieval.db.test.ts`)	Customer retrieves own committed Bookings list with Bangkok interval and `Booked` status.
RET-02	`FR-S1-14`, `FR-S1-16`, `BR-S1-15`	**AC-S1-30**	`VC/TEST-S1-28`	API / Database (`retrieval.db.test.ts`)	Customer retrieves own single Booking detail by valid UUID.
RET-03	`FR-S1-04`, `FR-S1-14`, `BR-S1-15`, `NFR-S1-01`	**AC-S1-31**	`VC/TEST-S1-29`	API / Database (`retrieval.db.test.ts`)	Cross-Customer probe: Customer B requests Customer A's Booking ID; returns safe `404 BOOKING_NOT_FOUND`.
RET-04	`FR-S1-15`, `BR-S1-16`	**AC-S1-32**	`VC/TEST-S1-30`	API / Database (`retrieval.db.test.ts`)	Tutor retrieves Bookings scheduled on their own Slots; pseudonymous `customerRef` rendered without PII leakage.
RET-05	`FR-S1-07`	**AC-S1-32**	`VC/TEST-S1-30`	API / Database (`retrieval.db.test.ts`)	Tutor own-slots list (`/tutor/slots`) projects dynamic `occupancy: 'Occupied'` for booked Slot, `'Unoccupied'` for unbooked.
RET-06	`FR-S1-04`, `FR-S1-15`, `BR-S1-16`, `NFR-S1-01`	**AC-S1-33**	`VC/TEST-S1-31`	API / Database (`retrieval.db.test.ts`)	Cross-Tutor schedule isolation: Tutor B cannot inspect Tutor A's schedule bookings.
RET-07	`FR-S1-17`, `BR-S1-14`, `BR-S1-22`, `Q30`, `BR-S1-12`, `NFR-S1-06`	**AC-S1-34**	`VC/TEST-S1-32`	API / Database (`retrieval.db.test.ts`)	Elapsed appointment displays `displayStatus: 'Past'`, while database `status` remains `'Booked'`, never `'Completed'`.

Test ID	Requirement / Rule Trace	Acceptance Criteria	Validation Anchor	Target Layer & File	Summary of Verification
RET-08	`FR-S1-04`, `AC-S1-05`	—	`VC/TEST-S1-05`	API Guard (`retrieval.db.test.ts`)	Anonymous callers receive `401 UNAUTHENTICATED` on all customer and tutor retrieval endpoints.
RET-09	`FR-S1-04`, `AC-S1-06`	—	`VC/TEST-S1-06`	API Guard (`retrieval.db.test.ts`)	Role boundary: Authenticated Tutor calling Customer booking endpoints receives `403 FORBIDDEN`.
RET-10	`FR-S1-04`, `AC-S1-06`	—	`VC/TEST-S1-06`	API Guard (`retrieval.db.test.ts`)	Role boundary: Authenticated Customer calling Tutor booking endpoints receives `403 FORBIDDEN`.
RET-11	`NFR-S1-08`	—	—	API Input (`retrieval.db.test.ts`)	Malformed non-UUID in path parameter (`/api/v1/customer/bookings/invalid-id`) returns `400 INVALID_INPUT`.
RET-12	`NFR-S1-08`	—	—	API Input (`retrieval.db.test.ts`)	Unexpected query parameters on list and detail endpoints return `400 INVALID_INPUT`.
RET-13	`NFR-S1-01`, `NFR-S1-08`	—	`VC/TEST-S1-35`	Security / Privacy (`retrieval.db.test.ts`)	Full JSON response inspection: Zero leakage of password hashes, session tokens, or unapproved contact data.
RET-14	`FR-S1-16`, `NFR-S1-03`	**AC-S1-30**	`VC/TEST-S1-28`	Persistence / Process Lifecycle (`retrieval.db.test.ts`)	Server restart persistence: Process termination + new DB connection pool reading persistent DB returns identical committed Bookings.
RET-15	`FR-S1-14`, `FR-S1-17`, `BR-S1-12`, `Q29`, `NFR-S1-06`, `NFR-S1-09`	**AC-S1-30**, **AC-S1-34**	`VC/TEST-S1-28`, `VC/TEST-S1-32`	Playwright E2E (`e2e/retrieval.spec.ts`)	Customer browser UI: Navigates to My Bookings, views upcoming & past badges, checks Bangkok time, opens detail modal.
RET-16	`FR-S1-04`, `FR-S1-07`, `FR-S1-15`, `BR-S1-16`, `NFR-S1-01`	**AC-S1-31**, **AC-S1-32**	`VC/TEST-S1-29`, `VC/TEST-S1-30`	Playwright E2E (`e2e/retrieval.spec.ts`)	Tutor browser UI: Inspects schedule bookings; Customer B cannot see Customer A's booking in UI list/modal; direct API probe returns safe 404.
RET-17	`FR-S1-17`, `BR-S1-22`, `Q30`, `NFR-S1-06`	**AC-S1-34**	`VC/TEST-S1-32`	API / Evaluator (`retrieval.db.test.ts`)	Statement snapshot uniformity, empty list metadata, exact Q30 boundary, and session TimeZone independence (`SET LOCAL TIME ZONE 'UTC'` vs `'Asia/Bangkok'`).

---

2. Detailed Test Specifications

RET-01: Customer Lists Own Committed Bookings

- **Goal:** Verify that an authenticated Customer can retrieve their own list of committed Bookings in deterministic ascending order.
- **Setup:**
  - Create Customer A and two Tutors (Tutor 1, Tutor 2).
  - Create two Slots (Slot 1 at +2h, Slot 2 at +5h).
  - Insert two committed Bookings for Customer A.
  - Create Customer B with a separate Booking on Slot 3.
- **Execution:** Customer A calls `GET /api/v1/customer/bookings`.

- **Expected Outcome:**
  - HTTP `200 OK`.
  - Response array contains exactly 2 items belonging to Customer A.
  - Customer B's Booking is completely omitted.
  - Response headers include `X-NutTiew-Session-Binding` and `Cache-Control: no-store`.

RET-02: Customer Retrieves Single Booking Detail

- **Goal:** Verify that an authenticated Customer can fetch complete details of their own Booking by ID.
- **Execution:** Customer A calls `GET /api/v1/customer/bookings/:id` with Customer A's Booking ID.
- **Expected Outcome:**
  - HTTP `200 OK`.
  - Body contains `data.id`, `data.slotId`, `data.tutorId`, `data.tutorRef`, `data.startsAt`, `data.endsAt`, `data.status = 'Booked'`, and `data.createdAt`.

RET-03: Cross-Customer Access Refusal (Safe 404)

- **Goal:** Prove that Customer B cannot inspect Customer A's Booking even if the exact UUID is known (`AC-S1-31`).
- **Setup:** Customer A owns Booking `UUID-A`. Customer B is authenticated.
- **Execution:** Customer B calls `GET /api/v1/customer/bookings/UUID-A`.
- **Expected Outcome:**
  - HTTP `404 Not Found` with `{ error: { code: 'BOOKING_NOT_FOUND', message: 'Booking not found' } }`.
  - Server does **NOT** return `403 FORBIDDEN` (which would confirm existence).
  - Zero private data is returned in the response body.

RET-04: Tutor Lists Schedule Bookings

- **Goal:** Verify that a Tutor can retrieve Bookings scheduled on their own Slots (`AC-S1-32`).
- **Setup:**
  - Tutor 1 has Slot 1 with a committed Booking by Customer A.
  - Tutor 2 has Slot 2 with a committed Booking by Customer B.
- **Execution:** Tutor 1 calls `GET /api/v1/tutor/bookings`.
- **Expected Outcome:**
  - HTTP `200 OK`.
  - Response contains Slot 1's Booking.
  - Customer identity is represented as pseudonymous `customerRef: "CUST-" + customerAId.slice(0, 8)` for data minimization; customer name, email, and phone are never disclosed, and raw `customerId` UUID is excluded.
  - Slot 2's Booking (Tutor 2) is completely absent.

RET-05: Tutor Own-Slots Dynamic Occupancy

- **Goal:** Verify that `GET /api/v1/tutor/slots` dynamically reports `occupancy: 'Occupied'` when booked, and `'Unoccupied'` when not booked (`FR-S1-07`).
- **Setup:** Tutor 1 creates two Slots: Slot 1 (booked by Customer A) and Slot 2 (unbooked).
- **Execution:** Tutor 1 calls `GET /api/v1/tutor/slots`.
- **Expected Outcome:**
  - HTTP `200 OK`.

- Slot 1 has `occupancy: 'Occupied'`.
- Slot 2 has `occupancy: 'Unoccupied'`.

RET-06: Cross-Tutor Schedule Isolation

- **Goal:** Verify that Tutor B cannot read or discover Tutor A's schedule bookings (`AC-S1-33`).
- **Execution:** Tutor B calls `GET /api/v1/tutor/bookings`.
- **Expected Outcome:**
  - Response array contains zero items from Tutor A.
  - API accepts no query parameters (e.g. `?tutorId=...` produces `400 INVALID_INPUT`).

RET-07: Time-Derived "Past" Appointment Display (`Q30`, `AC-S1-34`, `BR-S1-14/22`, `NFR-S1-06`)

- **Goal:** Verify that an elapsed Booking displays as `Past` without modifying database state or asserting `Completed`.
- **Historical Fixture Feasibility Protocol:**
  - In canonical database schemas, DB triggers prevent creating past slots (`slot_overlap_s1`) or booking with lead time <60m (`booking_guard_s1()` with `P0B02`). The test suite in Part F must NOT alter or disable triggers in the shared public schema.
  - **Authorized Test Strategy for Part F:**
    - **Method A (Query Evaluation Snapshot / Injected Clock):** Seed a valid future Slot and Booking (>60m notice) passing all canonical guards. Execute the retrieval evaluation specifying an evaluation snapshot `evaluatedAt >= endsAt`. Assert that `displayStatus: 'Past'`, `meta.evaluatedAt` reflects the evaluation time, and database `Booking.status` remains strictly `'Booked'` without database mutation.
    - **Method B (Isolated Disposable Ephemeral DB Seed):** In an ephemeral, disposable test container (never in shared/production schema), seed historical rows under a localized seed script with triggers re-enabled before test execution, without ever touching shared/canonical database schemas.
- **Execution:** Customer opens `GET /api/v1/customer/bookings/:id`.
- **Expected Outcome:**
  - Response body contains `displayStatus: 'Past'`, but `status: 'Booked'`.
  - Direct database inspection via `ownerPool.query`: `Booking.status` column is strictly `'Booked'`.
  - Associated Slot remains occupied and is not automatically released.

RET-08..RET-10: Authentication and Role Boundaries

- **RET-08:** Anonymous call to `/api/v1/customer/bookings` or `/api/v1/tutor/bookings` -> `401 UNAUTHENTICATED`.
- **RET-09:** Tutor session calling `/api/v1/customer/bookings` -> `403 FORBIDDEN`.
- **RET-10:** Customer session calling `/api/v1/tutor/bookings` or `/api/v1/tutor/slots` -> `403 FORBIDDEN`.

RET-11..RET-13: Input Validation and Privacy Leak Checks

- **RET-11:** `GET /api/v1/customer/bookings/12345-invalid` -> `400 INVALID_INPUT`.
- **RET-12:** `GET /api/v1/customer/bookings?filter=all` -> `400 INVALID_INPUT`.
- **RET-13:** Stringify full response bodies across all endpoints; assert absence of substring `"passwordHash"`, `"tokenDigest"`, `"csrfNonce"`, or unmasked private emails.

RET-14: Persistence Across Application Process Restart and Pool Reconnection (`FR-S1-16`, `NFR-S1-03`)

- **Goal:** Prove that committed Bookings survive complete application process shutdown, connection pool termination, and restart against the persistent database.

- **Execution:**
  1. Commit a valid Slot and Booking on running API instance 1.
  2. Explicitly disconnect and terminate active database connection pool (`prisma.$disconnect()` / `pool.end()`).
  3. Spawn an entirely separate Node.js process / fresh connection pool against the same persistent PostgreSQL database.
  4. Call `GET /api/v1/customer/bookings` from the newly spawned process/pool.
- **Expected Outcome:** Returns the exact committed Booking with identical IDs, timestamps, and status, proving durable persistence across process lifecycles.

RET-15 & RET-16: Playwright Edge Browser E2E Tests

- **RET-15 (Customer E2E):** Customer logs in, navigates to "การจองของฉัน", verifies Bangkok time formatting (`d MMM yyyy, HH:mm – HH:mm น.`), verifies upcoming badge (`ยืนยันแล้ว` , `#eef2f7`) and elapsed badge (`สิ้นสุดแล้ว` , `#f1f3f6`), clicks "ดูรายละเอียด" to open modal, verifies Booking UUID and details, and presses `Escape` or close button to dismiss modal.
- **RET-16 (Tutor E2E & Negative Privacy Isolation):**
  - Tutor logs in, views schedule agenda with "มีผู้จองแล้ว (Occupied)" amber badge (`#fff7ed` / `#f47736`) and unbooked slots with "ว่าง (เปิดรับจอง)", and verifies schedule bookings list displaying pseudonymous `customerRef` .
  - Customer B logs in to their own account; Customer A's booking is absent from the list. Customer B cannot open Customer A's booking modal.
  - Direct API probe: Customer B issues `GET /api/v1/customer/bookings/<CustomerA_BookingId>` and verifies safe `404 BOOKING_NOT_FOUND` response with zero data disclosure.

RET-17: Single Statement Evaluation Snapshot Uniformity, Empty List Metadata, and Q30 Exact Boundary (`Q30` , `FR-S1-17` , `BR-S1-22` , `NFR-S1-06`)

- **Goal:** Verify that `meta.evaluatedAt` is a single uniform evaluation snapshot, is present on empty responses, and cleanly governs the Q30 boundary.
- **Scenarios:**
  1. **Empty List Metadata:** An authenticated Customer with 0 bookings calls `GET /api/v1/customer/bookings` . Expected: `200 OK` with `data: []` and a valid ISO-8601 UTC `meta.evaluatedAt` timestamp.
  2. **Multi-Row Snapshot Uniformity:** A Customer with multiple bookings calls `GET /api/v1/customer/bookings` . Expected: all rows project `displayStatus` based on the exact single evaluation timestamp that matches `meta.evaluatedAt` .
  3. **Q30 Exact Boundary:** For an appointment ending at T:
     - When evaluated at $T - 1\text{s}$: `displayStatus: 'Booked'` , `status: 'Booked'` .
     - When evaluated at T: `displayStatus: 'Past'` , `status: 'Booked'` .
     - When evaluated at $T + 1\text{s}$: `displayStatus: 'Past'` , `status: 'Booked'` .
     - Database row check: in all three states, `Booking.status` remains `'Booked'` , never mutating to `'Completed'` .
  4. **Session TimeZone Independence (`SET LOCAL TIME ZONE`):** Execute retrieval evaluation query with `SET LOCAL TIME ZONE 'UTC'` and `SET LOCAL TIME ZONE 'Asia/Bangkok'` . Verify that because `s_clock.evaluated_at` is preserved as `timestamptz` throughout SQL comparison (`endsAt <= s_clock.evaluated_at`), the computed `displayStatus` ('Booked' vs 'Past') is strictly identical across different session timezones, completely avoiding naive timestamp interpretation bugs.

---

3. Part F Execution Status

All 17 test cases (`RET-01..RET-17`) are fully implemented and verified **PASS (100%)**. Complete execution logs, observed command outcomes, and TDD evidence are documented in `verification-ledger.md` .

Part F: Sprint 1 Implementation, Verification & Release Evidence

1. Truthful Automated Test Execution & CI Record

Automated verification was executed across both local Windows 11 (Node v24.14.0, PostgreSQL 17.2 + `btree_gist` 1.7, Microsoft Edge / Chromium) and GitHub Actions Ubuntu 24.04 runners (Node 24, PostgreSQL 18, Microsoft Edge). All domain code (`apps/api/` , `apps/web/`) and Playwright journey tests (`e2e/`) have remained unchanged since verified snapshot `e591f72` (application logic from `sprint1@9d24fca` , plus `e2e/sprint1.journey.spec.ts` added in `e591f72`).

Verification Gate / Suite	Execution Command	Environment	Passed	Failed	Skipped	Distinct Total
Pure Domain Unit & Contract Tests	`npm test`	Local + CI (Node 24)	**29**	0	0	**29**
Real PostgreSQL DB Integration & Concurrency	`npm run test:db -w @nuttiew/api`	Local (PG 17.2) + CI (PG 18)	**106**	0	0	**106**
Browser End-to-End Suite (incl. Sprint 1 Journey)	`npx playwright test`	Local + CI (MS Edge)	**64**	0	0	**64**
TypeScript Static Typecheck	`npm run typecheck`	Local + CI	0 errors	0	-	**Clean**
Production Build	`npm run build`	Local + CI	Clean	0	-	**Clean**
Git Whitespace & Patch Check	`git diff --check`	Local	Clean	0	-	**Clean**

> **Total Distinct Automated Tests:** `29` (Unit) + `106` (DB) + `64` (E2E) = `199` **Distinct Tests (`199` Passed, `0` Failed, `0` Skipped)**

> **Accounting & CI Incident Transparency Notes:**
> 1. **No Double-Counting:** The 64-test browser suite (`npx playwright test`) already executes `e2e/sprint1.journey.spec.ts` as test `[39/64]` . The true count of unique automated tests is **199** (29 + 106 + 64).
> 2. **CI Workflow Run Diagnosis (#42672763881):** On commit `0efa30b` , GitHub Actions Run #42672763881 Attempt 1 concluded `failure` because GitHub Actions transiently failed to dispatch the `PostgreSQL 18 / Auth / Slot / Race A` job (0 job entries/logs created, while `Unit / types / build` and `Microsoft Edge E2E` passed). Re-running the exact same workflow without file edits on Attempt 2 dispatched all 3 jobs and passed **3/3 GREEN**.
> 3. **Exact Verified Baseline Recorded:** The table above reflects local execution (Unit 29/29, PG 17.2 DB 106/106, typecheck, build) and verified GitHub Actions CI (Run 42783770373 on SHA `808b906` , **3/3 jobs PASS**). The subsequent metadata-sync commit on PR #43 updates only the cited verified baseline SHA/CI-Run fields and regenerated Deliverable 1 Markdown/PDF without modifying `apps/` , `e2e/` , migrations, Part A–E contracts, or `README.md` , and is independently gated on 3/3 GREEN CI.

2. Complete Stakeholder Traceability Matrices

2.1 Acceptance Criteria Coverage (AC-S1-01..35 → VC/TEST-S1 → Actual Executed Tests)

AC ID	Scenario & Acceptance Requirement	Anchor ID	Executed Test File & Case	Result
AC-S1-01	Valid Customer credentials authenticate with Customer role	`VC-01/03`	`apps/api/test/auth.db.test.ts`	**PASS**

AC ID	Scenario & Acceptance Requirement	Anchor ID	Executed Test File & Case	Result
AC-S1-02	Valid Tutor credentials authenticate with Tutor role	`VC-01/03`	`apps/api/test/auth.db.test.ts`	**PASS**
AC-S1-03	Invalid password or unknown credentials return generic 401	`VC-02`	`apps/api/test/auth.extended.db.test.ts` (`AUTH-02`)	**PASS**
AC-S1-04	Logout revokes presented Session in DB; replay rejected 401	`VC-04`	`apps/api/test/auth.db.test.ts`	**PASS**
AC-S1-05	Anonymous request to protected routes rejected 401	`VC-05`	`apps/api/test/guards.db.test.ts` , `retrieval.db.test.ts` (`RET-08`)	**PASS**
AC-S1-06	Customer forging Tutor action denied 403; schedule unchanged	`VC-06`	`apps/api/test/guards.db.test.ts` , `slots.db.test.ts`	**PASS**
AC-S1-07	Tutor A cannot inspect or mutate Tutor B's private schedule	`VC-07/31`	`apps/api/test/slots.db.test.ts` , `retrieval.db.test.ts` (`RET-06`)	**PASS**
AC-S1-08	Tutor creates 60-minute future Slot; persisted on own schedule	`VC-08/14`	`apps/api/test/slots.db.test.ts`	**PASS**
AC-S1-09	Same Tutor creating overlapping Offered Slot rejected (`23P01` → 409)	`VC-09`	`apps/api/test/slots.race.db.test.ts`	**PASS**
AC-S1-10	Same Tutor creating adjacent 60-minute Slot permitted	`VC-10`	`apps/api/test/slots.db.test.ts`	**PASS**
AC-S1-11	Different Tutors offering identical interval permitted	`VC-11`	`apps/api/test/slots.db.test.ts`	**PASS**
AC-S1-12	Past or invalid Slot start rejected without persistence	`VC-12`	`apps/api/test/slots.db.test.ts`	**PASS**
AC-S1-13	Race A: Concurrent overlapping Slot writes serialize to 1 commit	`VC-37`	`apps/api/test/slots.race.db.test.ts` (`SLOT-10/11`)	**PASS**
AC-S1-14	Non-Bangkok browser timezone displays accurate Bangkok hours	`VC-13`	`e2e/slots.spec.ts` , `e2e/availability.spec.ts`	**PASS**
AC-S1-15	Customer browses eligible Offered Slots with Tutor ref and Bangkok time	`VC-15`	`apps/api/test/availability.db.test.ts`	**PASS**
AC-S1-16	Occupied Slots and Q05 <60m Slots excluded without Customer leakage	`VC-15/22`	`apps/api/test/availability.db.test.ts`	**PASS**
AC-S1-17	Submitting stale offer rechecks DB eligibility and rejects if taken	`VC-22`	`apps/api/test/bookings.db.test.ts` (`BOOK-07`), `e2e/booking.spec.ts`	**PASS**

AC ID	Scenario & Acceptance Requirement	Anchor ID	Executed Test File & Case	Result
AC-S1-18	Device locale outside Bangkok does not shift schedule semantics	`VC-13/15`	`e2e/availability.spec.ts`	**PASS**
AC-S1-19	Customer books eligible Offered Slot; commits `Booked` row	`VC-16`	`apps/api/test/bookings.db.test.ts` (`BOOK-01`)	**PASS**
AC-S1-20	Slot at 15:00 with DB `T_elig` = 14:00:00 (exact 60:00) accepted	`VC-17`	`apps/api/test/bookings.db.test.ts` (`BOOK-02`), `BDF-21`	**PASS**
AC-S1-21	Slot at 15:00 with DB `T_elig` = 14:00:01 (59:59) rejected (`P0B02` → 422)	`VC-18`	`apps/api/test/bookings.db.test.ts` (`BOOK-03`)	**PASS**
AC-S1-22	Customer B attempting to book Slot held by Customer A rejected 409	`VC-19`	`apps/api/test/bookings.db.test.ts` (`BOOK-04`)	**PASS**
AC-S1-23	Customer with Tutor A Booking rejected from overlapping Tutor B Slot (409)	`VC-20`	`apps/api/test/bookings.db.test.ts` (`BOOK-05`)	**PASS**
AC-S1-24	Adjacent cross-Tutor interval permitted for same Customer (201)	`VC-21`	`apps/api/test/bookings.db.test.ts` (`BOOK-06`)	**PASS**
AC-S1-25	Race B: Two Customers racing for one Slot serialize to <=1 commit	`VC-25`	`apps/api/test/bookings.db.test.ts` (`BOOK-10`)	**PASS**
AC-S1-26	Race C: Customer concurrently booking overlapping Slots across Tutors serializes	`VC-26`	`apps/api/test/bookings.db.test.ts` (`BOOK-11`)	**PASS**
AC-S1-27	Injected DB fault before COMMIT rolls back cleanly with 0 phantom rows	`VC-22/27/33`	`apps/api/test/bookings.db.test.ts` (`BOOK-12`), `booking.foundation.db.test.ts`	**PASS**
AC-S1-28	Client-forged `customerId` / `tutorId` rejected; owner derived from Session	`VC-24`	`apps/api/test/bookings.db.test.ts` (`BOOK-09`)	**PASS**
AC-S1-29	Same-pair natural-key retry returns 200 replay with 1 committed row	`VC-38`	`apps/api/test/bookings.db.test.ts` (`BOOK-13..16`), `e2e/booking.spec.ts`	**PASS**
AC-S1-30	Customer retrieves persisted private Bookings across reload & process restart	`VC-28`	`apps/api/test/retrieval.db.test.ts` (`RET-01/02/14`)	**PASS**
AC-S1-31	Cross-Customer probe returns safe `404 BOOKING_NOT_FOUND`	`VC-29`	`apps/api/test/retrieval.db.test.ts` (`RET-03`), `e2e/retrieval.spec.ts`	**PASS**
AC-S1-32	Tutor retrieves own schedule Bookings with `customerRef` and zero PII	`VC-30`	`apps/api/test/retrieval.db.test.ts` (`RET-04/05`)	**PASS**
AC-S1-33	Cross-Tutor schedule query blocked; Tutor B sees zero Tutor A records	`VC-31`	`apps/api/test/retrieval.db.test.ts` (`RET-06` , `RET-16`)	**PASS**

AC ID	Scenario & Acceptance Requirement	Anchor ID	Executed Test File & Case	Result
AC-S1-34	Elapsed interval projects `displayStatus: 'Past'` with `status: 'Booked'`	`VC-32`	`apps/api/test/retrieval.db.test.ts` (`RET-07/17`)	**PASS**
AC-S1-35	Pre-cutoff request waiting across 60:00 lock wait rejected at post-wait `T_elig`	`VC-39`	`apps/api/test/bookings.db.test.ts` (`BOOK-17`)	**PASS**

2.2 Complete Cross-Cutting Verification Cases (`VC-S1-34..39`)

VC / TEST ID	Scope	Executed Verification & Honest Gate Status	Observed Result
VC-S1-34 / TEST-S1-34	Full UI→API→DB Sprint 1 Journey	`e2e/sprint1.journey.spec.ts` (`[39/64]` in `npx playwright test`) + 8 screenshots	**PASS**
VC-S1-35 / TEST-S1-35	No stored secrets/plaintext/PII leaks	`auth.db.test.ts` , `config.test.ts` , `retrieval.db.test.ts` (`RET-13`), `git grep` clean	**PASS**
VC-S1-36 / TEST-S1-36	Performance p95 (NFR-010) & Keyboard/WCAG (NFR-009)	`AVA-14` (`availability.db.test.ts`) p95 < 2.0s: **PASS**; Playwright focus/reflow checks: **PASS**; Formal manual WCAG 2.2 human assistive-tech sign-off: **NOT RUN (Manual Gate)**	**PARTIAL (Auto PASS / Manual NOT RUN)**
VC-S1-37 / TEST-S1-37	Race A (concurrent same-Tutor overlapping Slots)	`apps/api/test/slots.race.db.test.ts` (`SLOT-10/11`)	**PASS**
VC-S1-38 / TEST-S1-38	Same-Customer natural-key duplicate/retry	`apps/api/test/bookings.db.test.ts` (`BOOK-13..16`) & `e2e/booking.spec.ts`	**PASS**
VC-S1-39 / TEST-S1-39	Post-wait `T_elig` cutoff drift & Option 1 lock proof	`apps/api/test/bookings.db.test.ts` (`BOOK-17` , `BOOK-19 / BDF-23`)	**PASS**

(Also note: `AUTH-22` deterministic multi-instance Argon2 barrier stress test covers single-process barrier in `auth.barrier.db.test.ts` [PASS] while full external multi-instance mixed stress remains transparently documented as PARTIAL in `F-S1-01/tests.md` .)

---

3. Concurrency Invariants & Real Database Race Verification

The NutTiew platform enforces three distinct concurrency invariants at the PostgreSQL layer, verified with independent `pg.Pool` connections, `pg_backend_pid()` , and `pg_blocking_pids()` assertions:

3.1 Race A: Concurrent Slot Creation by Same Tutor (`SLOT-10/11` / `SQLSTATE 23P01`)

- **Constraint:** `offered_slot_no_overlap_per_tutor EXCLUDE USING gist ("tutorId" WITH =, tstzrange("startsAt", "endsAt", '[)') WITH &&) WHERE ("lifecycle" = 'Offered')` .
- **Observed Local Output (`apps/api/test/slots.race.db.test.ts` on PG 17.2):**

```
  RACE_A_OBSERVED {"first":"A","holderPid":26720,"aPid":25556,"bPid":5156,"statuses":[201,409],"committed":["524d65df-c698-47f9-bc4b-902fdda60205"]}
  RACE_A_DIRECT_SQL_OBSERVED {"aPid":26720,"bPid":27588,"loserSQLSTATE":"23P01"}
```

Exactly one conflicting `Offered` slot commits (`201 Created`); the competing transaction blocks on GiST, raises PostgreSQL `23P01` (`exclusion_violation`), and returns safe `409 Conflict` .

3.2 Race B: Concurrent Booking on Same Offered Slot (`BOOK-10` / `SLOT_UNAVAILABLE`)

- **Mechanism:** Ordered row lock `SELECT "id" FROM "Slot" WHERE "id" = $1 FOR UPDATE` followed by `booking_guard_s1()` and partial unique index `booking_one_active_per_slot` .
- **Observed Output (`apps/api/test/bookings.db.test.ts`):** Winner locks Slot, validates `Offered` and Q05, and commits `201 Created` ; blocked waiter wakes after commit, sees `Slot` is occupied, and cleanly receives `409 SLOT_UNAVAILABLE` . Zero duplicate active Bookings.

3.3 Race C & Option 1 Interleaving Proof: Same-Customer Overlapping Cross-Tutor Bookings (`BOOK-11` , `BOOK-19 / BDF-23`)

- **Mechanism:** Remediation migration `20261109000000_booking_guard_user_lock` replaces `booking_guard_s1()` (`SECURITY DEFINER`) to lock `User(customerId) FOR UPDATE` inside the `BEFORE ROW` trigger *before* locking `Slot` and *before* inserting into `booked_customer_no_overlap` GiST index.
- **Observed Local Output (`apps/api/test/bookings.db.test.ts` on PG 17.2):**


```
  BDF23_BOOK19_EVIDENCE {"canonicalPart1A":{"slotId":"a3d9ce7f-f5dc-40e7-804a-
229c2c4c208a","roguePid":1980,"adapterUserPid":26260,"userBlockers":
[1980],"outcome":"BOOKING_NOT_ELIGIBLE"},"canonicalPart1B":{"slotId":"1b4fae3b-35f5-4961-8ab4-
073824aff944","roguePid":1980,"adapterSlotPid":26260,"slotBlockers":
[1980],"outcome":"BOOKING_NOT_ELIGIBLE"},"canonicalPart1C":{"slotId":"92d8c642-2a77-403e-a247-
941d369cd371","roguePid":1980,"adapterPid1C":26260,"rogueBlockers1C":[26260],"adapterReverseBlockers":
[],"outcome":"23P01"},"canonicalPart1C2NoStaleCommit":{"slotId":"d9f200e3-1a5d-49be-af97-
e8b884aaac38","roguePid":1980,"adapterPid1C2":26260,"adapterReverseBlockers":
[],"commitNoticeMs":3599166,"outcome":201},"canonicalPart1D":{"slotId":"a4a9ccd6-97f2-44b9-a3d1-
32004ee4b920","roguePid":1980,"adapterPid1D":26260,"rogueBlockers1D":[26260],"outcome":"23505"}}
```


Notice `adapterReverseBlockers: []` in both Part 1C and Part 1C2 — proving zero reverse GiST waits and zero `40P01` deadlocks across both adapter-initiated and direct-SQL writers. Under 10 parallel overlapping Customer requests in `BOOK-11` , exactly 1 booking commits and 9 fail safely with `409 CUSTOMER_OVERLAP` .

---

4. Verbatim Executable Test Excerpts (Instructor §4.2)

4.1 Domain-Rule Test Excerpt: Q05 60-Minute Cutoff Post-Lock Drift (`BOOK-17`)

Source File: `apps/api/test/bookings.db.test.ts` , lines `855–918` (reproduced verbatim below).

Rule Proven (`AC-S1-35 / Q05`): When an eligible HTTP booking request arrives before the 60-minute cutoff (`60m + 1.8s`) but blocks on an active `Slot` `FOR UPDATE` lock until PostgreSQL `clock_timestamp()` crosses cutoff, `booking_guard_s1()` samples authoritative DB time *post-wait*, rejects with SQLSTATE `P0B02` (`422 BOOKING_NOT_ELIGIBLE`), and commits `0` rows.

```
// Verbatim excerpt from apps/api/test/bookings.db.test.ts (lines 855-918)
  it('BOOK-17 enforces Q05 at post-wait DB T_elig when HTTP arrives before cutoff and lock wait crosses cutoff', async () => {
    const tempCustomerEmail = `book-c17-${marker}@example.test`;
    const tempCustomerId = await createUser(tempCustomerEmail, 'Customer');
    const tempCustomerAuth = await loginBundle(tempCustomerEmail, tempCustomerId);
    const tempTutor = await createUser(`book-t17-${marker}@example.test`, 'Tutor');
    const nowRes = await ownerPool.query<{ now: Date }>('SELECT clock_timestamp() AS now');
    // Slot starts in 60m + 1.8s (eligible at arrival, ineligible after 2.1s lock wait)
    const slot = await createSlot(
      tempTutor,
      new Date(nowRes.rows[0].now.getTime() + 60 * 60_000 + 1_800),
    );
    const cutoff = new Date(slot.startsAt.getTime() - 60 * 60_000);

    const holder = await ownerPool.connect();
    try {
      await holder.query('BEGIN');
      await holder.query('SELECT "id" FROM "Slot" WHERE "id"=$1 FOR UPDATE', [slot.id]);
      const holderPid = await getPid(holder);

      let waiterPid = 0;
      const pendingBooking = createCustomerBooking(
        runtimePool,
        { userId: tempCustomerAuth.userId, sessionId: tempCustomerAuth.sessionId },
        slot.id,
        {
          onLockStep: async event => {
            if (event.phase === 'slot' && waiterPid === 0) {
              waiterPid = event.backendPid;
            }
          },
        },
      ).then(
        res => ({ ok: true as const, res }),
        (err: BookingError) => ({ ok: false as const, err }),
      );

      await expect.poll(() => waiterPid, { timeout: 5_000 }).toBeGreaterThan(0);
      await waitForBlockedBy(waiterPid, holderPid);

      // Hold the Slot lock until DB clock crosses cutoff
      while (true) {
        const c = await ownerPool.query<{ now: Date }>('SELECT clock_timestamp() AS now');
        if (c.rows[0].now.getTime() >= cutoff.getTime() + 150) break;
        await new Promise(resolve => setTimeout(resolve, 40));
      }

      await holder.query('COMMIT');
      const outcome = await pendingBooking;
      expect(outcome.ok).toBe(false);
      if (!outcome.ok) {
        expect(outcome.err.status).toBe(422);
        expect(outcome.err.errorCode).toBe('BOOKING_NOT_ELIGIBLE');
      }

      const persisted = await ownerPool.query<{ n: number }>(
        'SELECT count(*)::int AS n FROM "Booking" WHERE "currentSlotId"=$1',
        [slot.id],
      );
      expect(persisted.rows[0].n).toBe(0);
    } finally {
      await holder.query('ROLLBACK').catch(() => {});
      holder.release();
    }
  }, 15_000);
```

4.2 Authorization Test Excerpt: Cross-Customer Safe 404 & Cross-Tutor Schedule Isolation (`RET-03` & `RET-06`)

Source File: `apps/api/test/retrieval.db.test.ts` , fixture setup lines `128–149` and assertions lines `214–224` (`RET-03`) and `278–297` (`RET-06`) (reproduced verbatim below).

Rules Proven (`AC-S1-31 / AC-S1-33 / NFR-S1-01`): Cross-Customer reads against another Customer's Booking ID return safe `404 BOOKING_NOT_FOUND` (preventing existence confirmation via 403), and Tutor B cannot inspect Tutor A's bookings or override ownership via query parameters (`400 INVALID_INPUT`).

```
// Verbatim setup excerpt from apps/api/test/retrieval.db.test.ts (lines 128-149)
    // Create 3 slots for Tutor A
    const s1 = await createSlot(tutorAId);
    slotA1Id = s1.id;
    const s2 = await createSlot(tutorAId);
    slotA2Id = s2.id;
    const s3 = await createSlot(tutorAId);
    slotA3UnbookedId = s3.id;

    // Create 1 slot for Tutor B
    const sb1 = await createSlot(tutorBId);
    slotB1Id = sb1.id;

    // Book slotA1 and slotA2 for Customer A using canonical createCustomerBooking
    const b1 = await createCustomerBooking(runtimePool, { userId: customerAId, sessionId: authCustomerA.sessionId },
slotA1Id);
    customerABooking1Id = b1.data.id;
    const b2 = await createCustomerBooking(runtimePool, { userId: customerAId, sessionId: authCustomerA.sessionId },
slotA2Id);
    customerABooking2Id = b2.data.id;

    // Book slotB1 for Customer B
    const bb = await createCustomerBooking(runtimePool, { userId: customerBId, sessionId: authCustomerB.sessionId },
slotB1Id);
    customerBBookingId = bb.data.id;
  }, 30_000);

// Verbatim RET-03 excerpt (lines 214-224)
  // RET-03: Cross-Customer Access Refusal (Safe 404)
  it('RET-03: Cross-Customer probe returns safe 404 BOOKING_NOT_FOUND (no 403 confirmation)', async () => {
    const res = await request(app)
      .get(`/api/v1/customer/bookings/${customerABooking1Id}`)
      .set('Cookie', authCustomerB.cookie);

    expect(res.status).toBe(404);
    expect(res.body.error?.code).toBe('BOOKING_NOT_FOUND');
    expect(res.body.error?.message).toBe('Booking not found');
    expect(res.body.data).toBeUndefined();
  });

// Verbatim RET-06 excerpt (lines 278-297)
  // RET-06: Cross-Tutor Schedule Isolation
  it('RET-06: Cross-Tutor schedule isolation: Tutor B cannot inspect Tutor A schedule bookings', async () => {
    const res = await request(app)
      .get('/api/v1/tutor/bookings')
      .set('Cookie', authTutorB.cookie);

    expect(res.status).toBe(200);
    const data = res.body.data;
    expect(Array.isArray(data)).toBe(true);
    expect(data.length).toBe(1);
    expect(data[0].id).toBe(customerBBookingId);
    expect(data[0].tutorId).toBe(tutorBId);

    // Any query parameters fail closed
    const queryRes = await request(app)
      .get(`/api/v1/tutor/bookings?tutorId=${tutorAId}`)
      .set('Cookie', authTutorB.cookie);
    expect(queryRes.status).toBe(400);
    expect(queryRes.body.error?.code).toBe('INVALID_INPUT');
  });
```

5. Verifiable TDD RED-to-Green Commit History

Every critical domain and security fix followed observable **failing proof (RED) → remediation commit (GREEN)** recorded in Git history and feature verification ledgers:

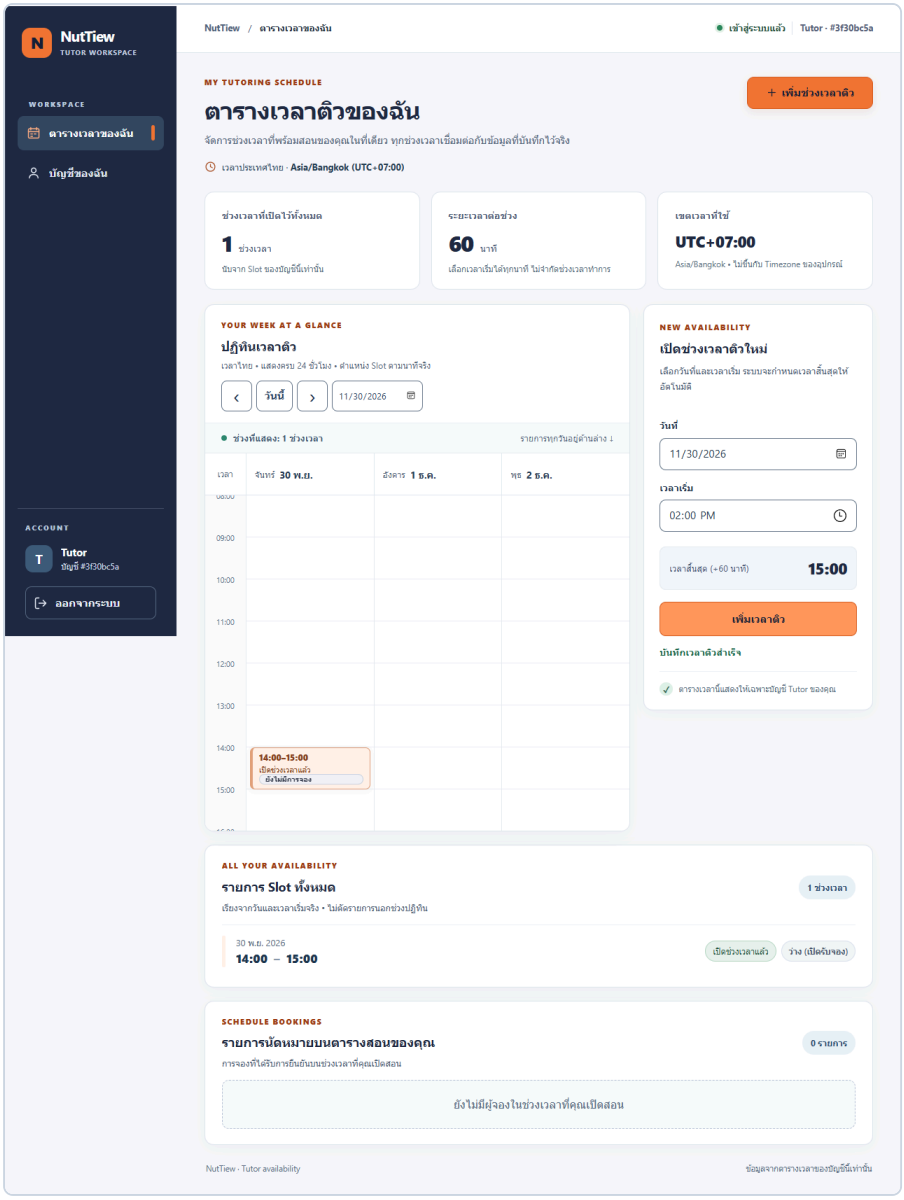
1. **Case 1: `BDF-28 / BDF-23` Option 1 Lock-Order Remediation (Issue #39 / PR #40):**
   - **RED Observation (PR #38 commit `c8ebc4a`):** Under the initial `#33` `booking_guard_s1()` trigger (locking only `Slot`), PostgreSQL's deferred `AFTER ROW` FK trigger allowed a direct-SQL writer to enter the `booked_customer_no_overlap` GiST index before locking `User`. In `BOOK-19` Part 1C, Adapter hit reverse wait `adapterReverseBlockers: [roguePid]` and SQLSTATE `40P01` deadlock, triggering the documented Option 1 Stop Gate.
   - **GREEN Fix (PR #40 commits `6bd92a8` & `da17acf`, merged `1ff4c2b`; `#16` verified commit `5dd02b9`):** Added forward-only migration `20261109000000_booking_guard_user_lock` redefining `booking_guard_s1()` (`SECURITY DEFINER`) to lock `User(customerId) FOR UPDATE` in `BEFORE ROW` before `Slot` and before index insertion. `BDF-28` reproduces legacy failure in an isolated schema (Part 1) and proves zero reverse wait (`adapterReverseBlockers: []`) on canonical schema (Part 2).
2. **Case 2: `BOOK-17` & Trigger `T_elig` Authority vs Application Heuristic (Issue #16 / PR #38):**
   - **RED / Review Finding (between commits `222b5d3` and `2e59b89`):** Application-layer heuristic `statementDurationMs >= 150` conflicted with `transaction-spec.md` §5 ("a valid 60:00 guard may still commit later"), and fixture reuse caused early overlap failure in `BOOK-17` (`docs/features/F-S1-04/verification-ledger.md`).
   - **GREEN Fix (commits `2e59b89` & `865114a`):** Removed the app duration heuristic, restored exclusive `booking_guard_s1()` post-lock `clock_timestamp()` authority, and verified `BOOK-17` rejecting cross-cutoff contention at `422 BOOKING_NOT_ELIGIBLE`.
3. **Case 3: Error Status Precedence Before Session-Binding Check (Issue #17 / PR #42):**
   - **RED Review Finding (on commit `e6d9cac`):** `apps/web/src/customer-availability.tsx` inspected `X-NutTiew-Session-Binding` before `!res.ok`, risking false session-mismatch logouts on injected/transient `404` and `503` error envelopes that omit binding headers.
   - **GREEN Fix (commit `4c60267`):** Reordered HTTP status handling before session-binding validation and added deterministic E2E regression tests `RET-15-ERR-503` and `RET-15-ERR-404` in `e2e/retrieval.spec.ts`.

---

6. End-to-End Vertical Journey Walkthrough & Screenshots

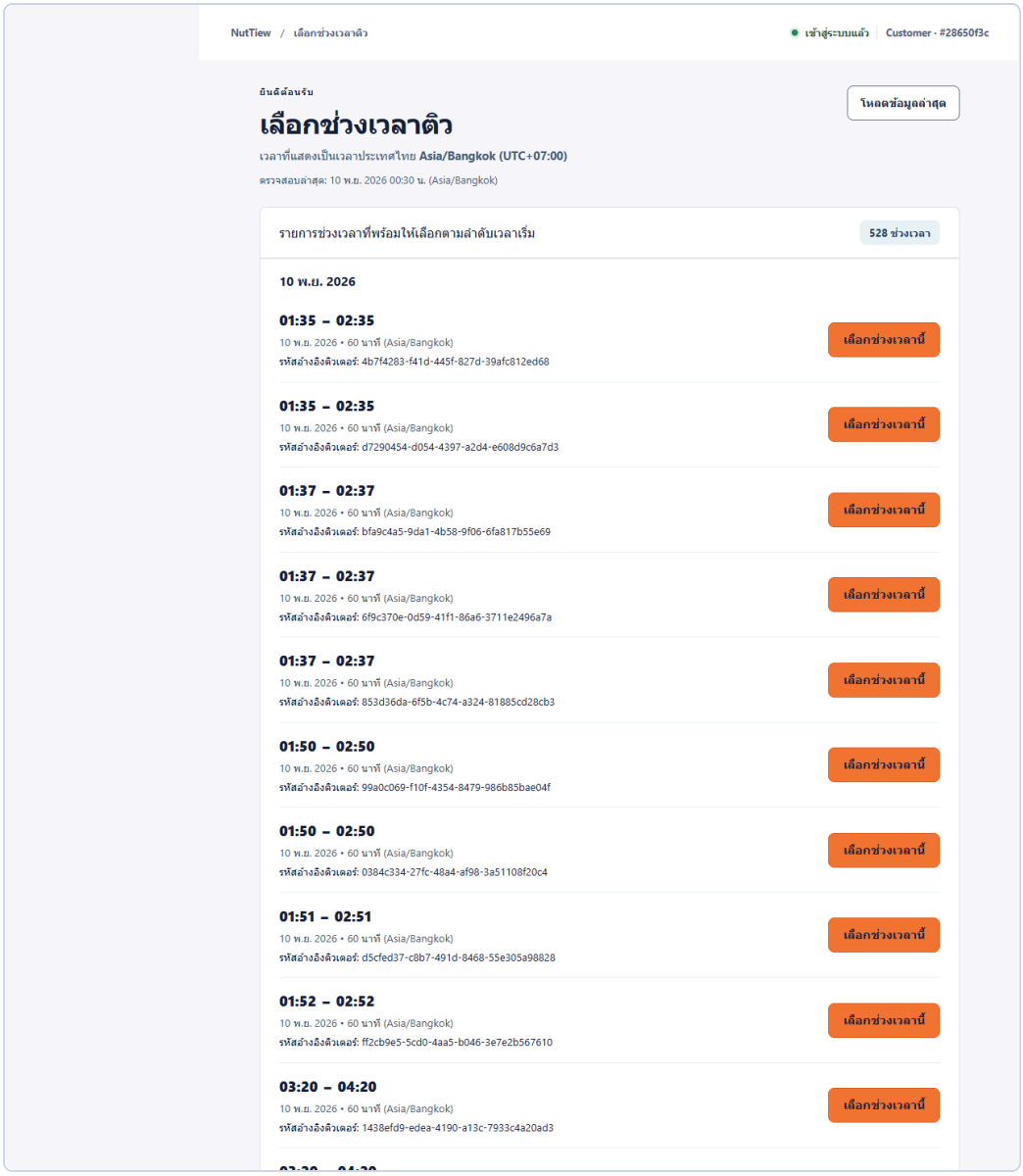
Executed against real React 19 UI, Express 5 API, and PostgreSQL persistence via `e2e/sprint1.journey.spec.ts`:

Milestone 1: Tutor Logs In & Creates 60-Minute Slot



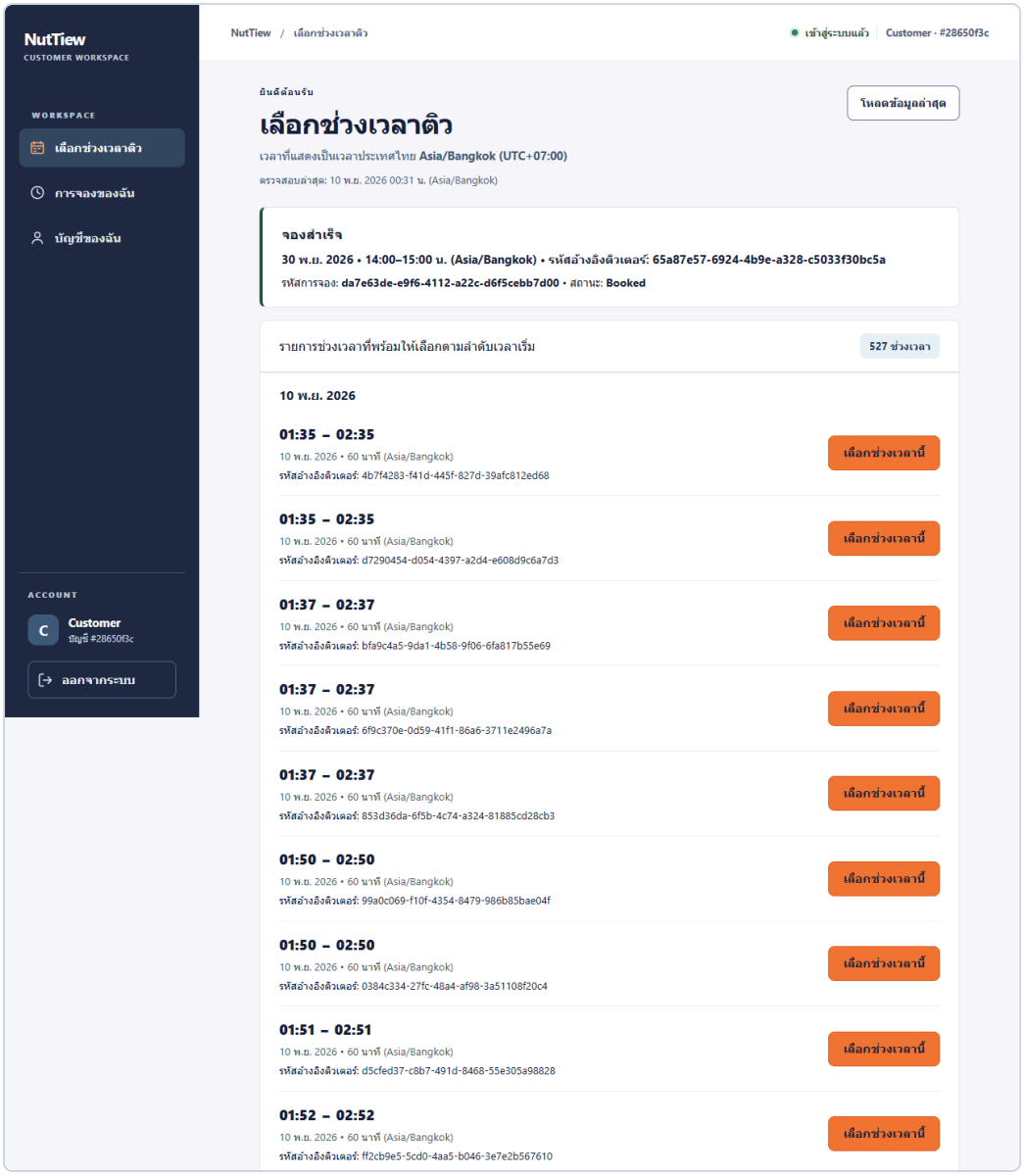
01 Tutor Slot Created

Milestone 2: Customer Browses Available Tutoring Offers



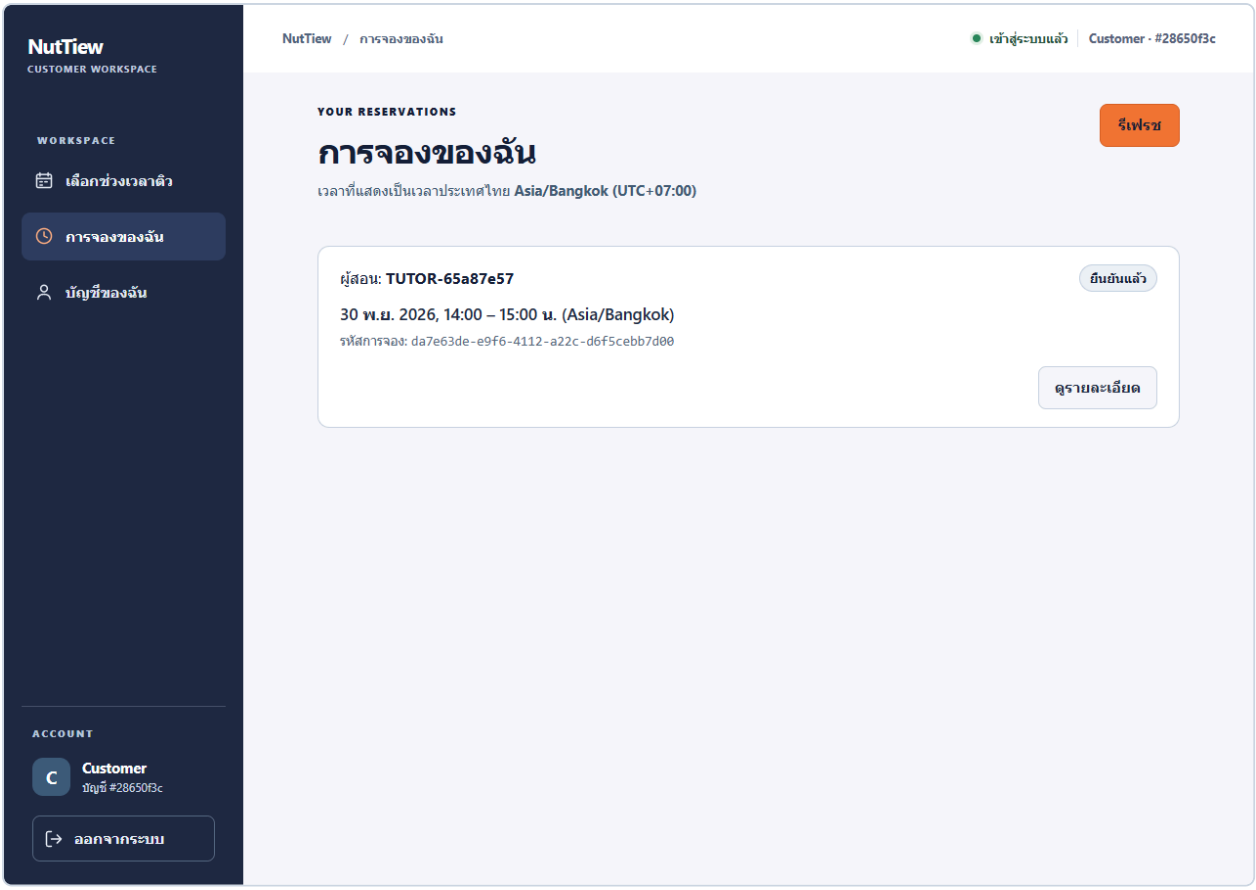
02 Customer Availability Browse — Top-viewport excerpt (1280×1472 px) cropped for PDF readability from full-page capture (1280×59159 px) at ../evidence/sprint1/02-customer-availability-browse.png

Milestone 3: Customer Confirms Booking



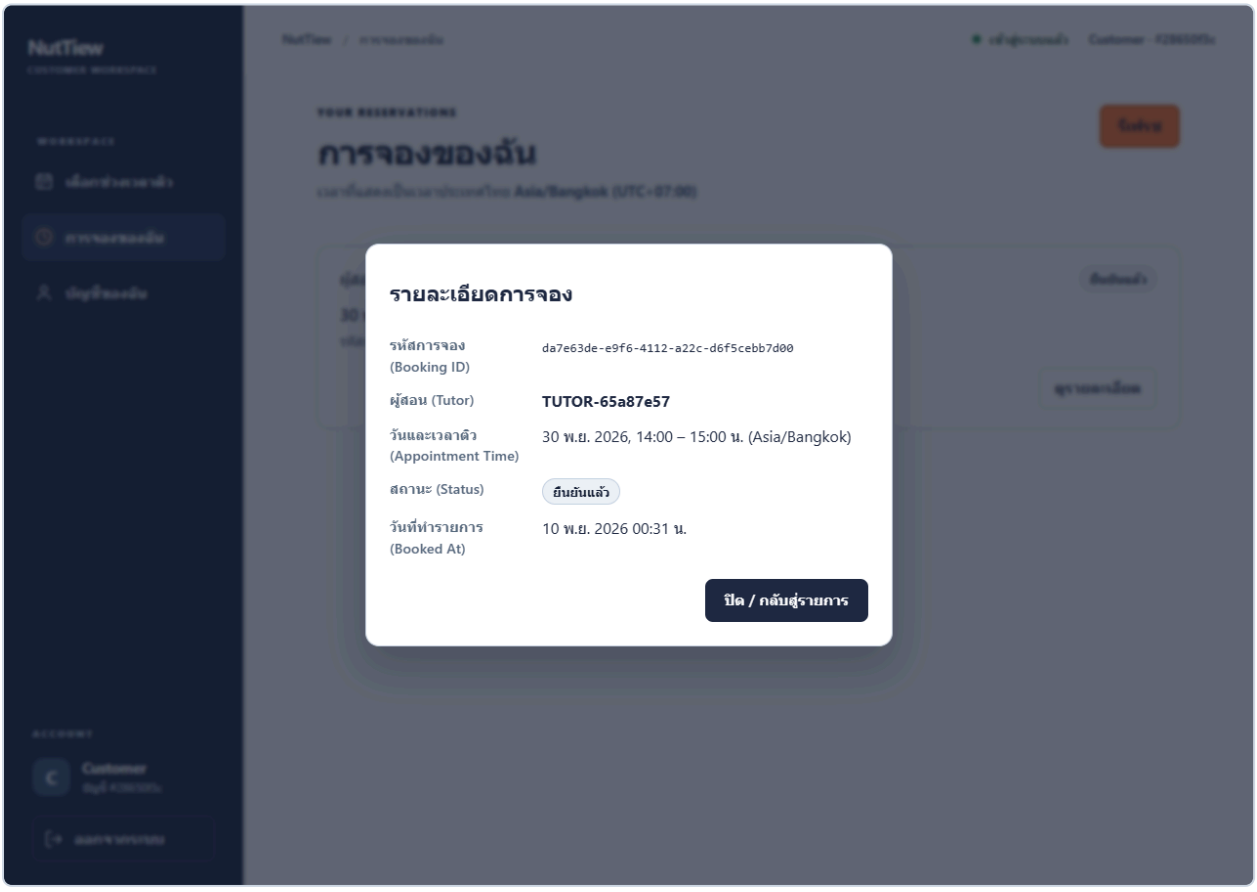
03 Customer Booking Confirmed — Top-viewport excerpt (1280×1472 px) cropped for PDF readability from full-page capture (1280×59196 px) at ../evidence/sprint1/03-customer-booking-confirmed.png

Milestone 4: Customer Private Booking List



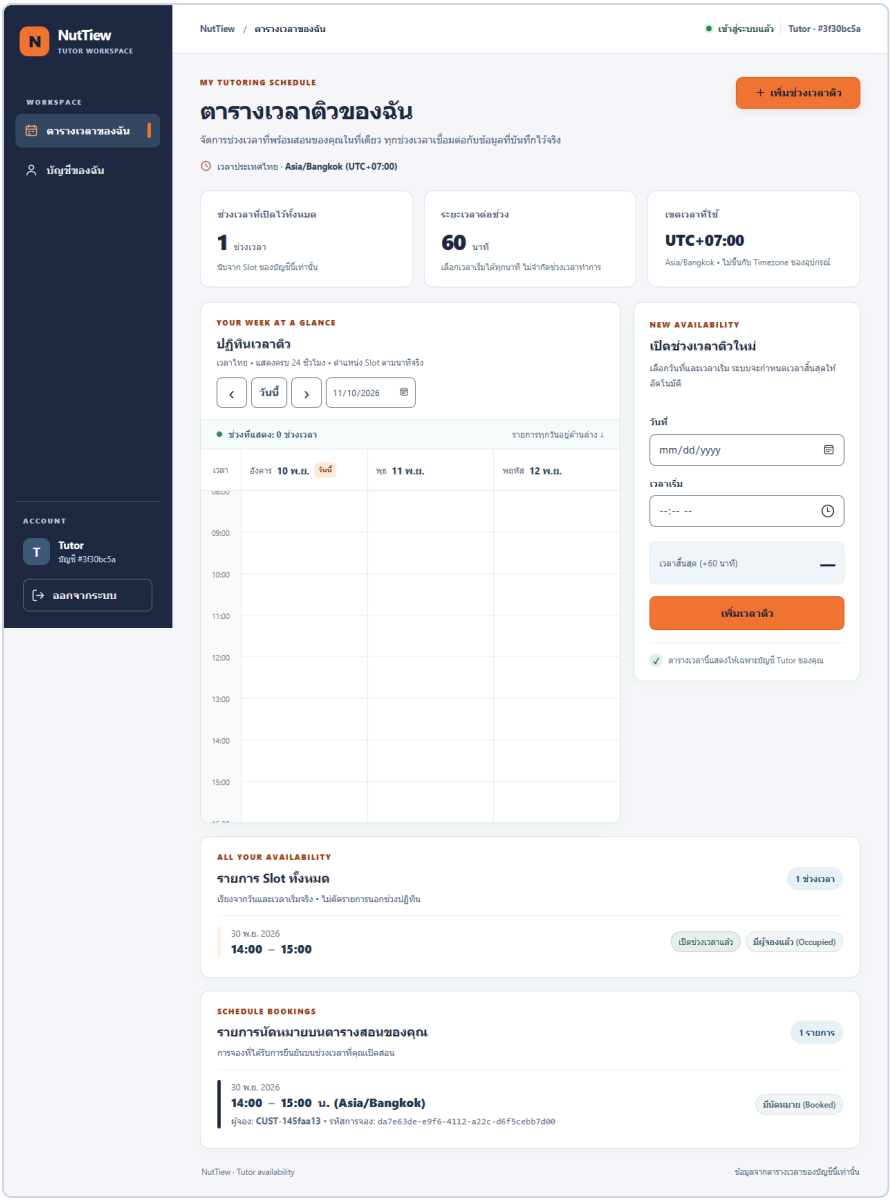
04 Customer Private Bookings

Milestone 5: Customer Booking Detail Modal



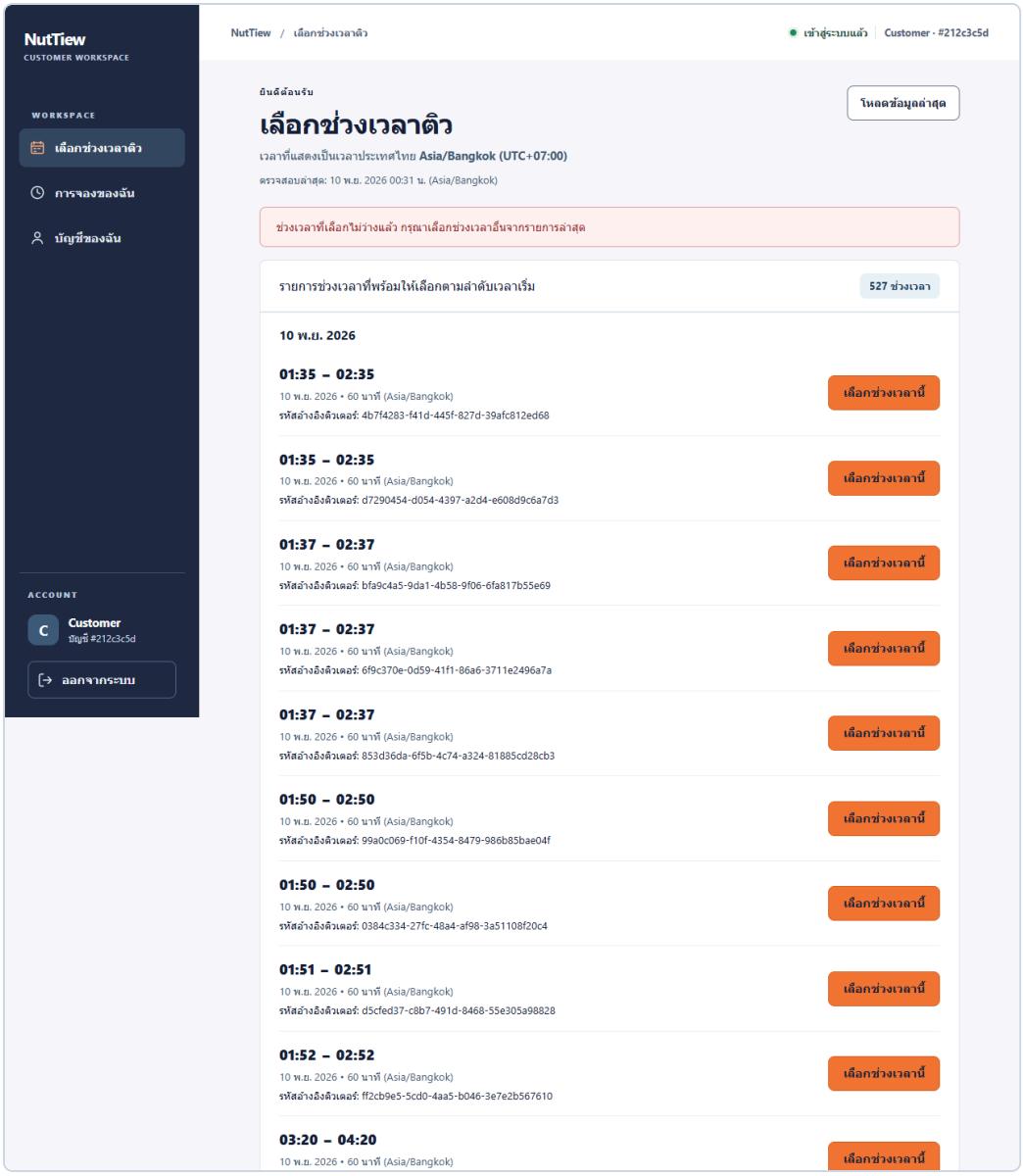
05 Customer Detail Modal

Milestone 6: Tutor Schedule View (Occupied & Pseudonymous `CUST-...`)



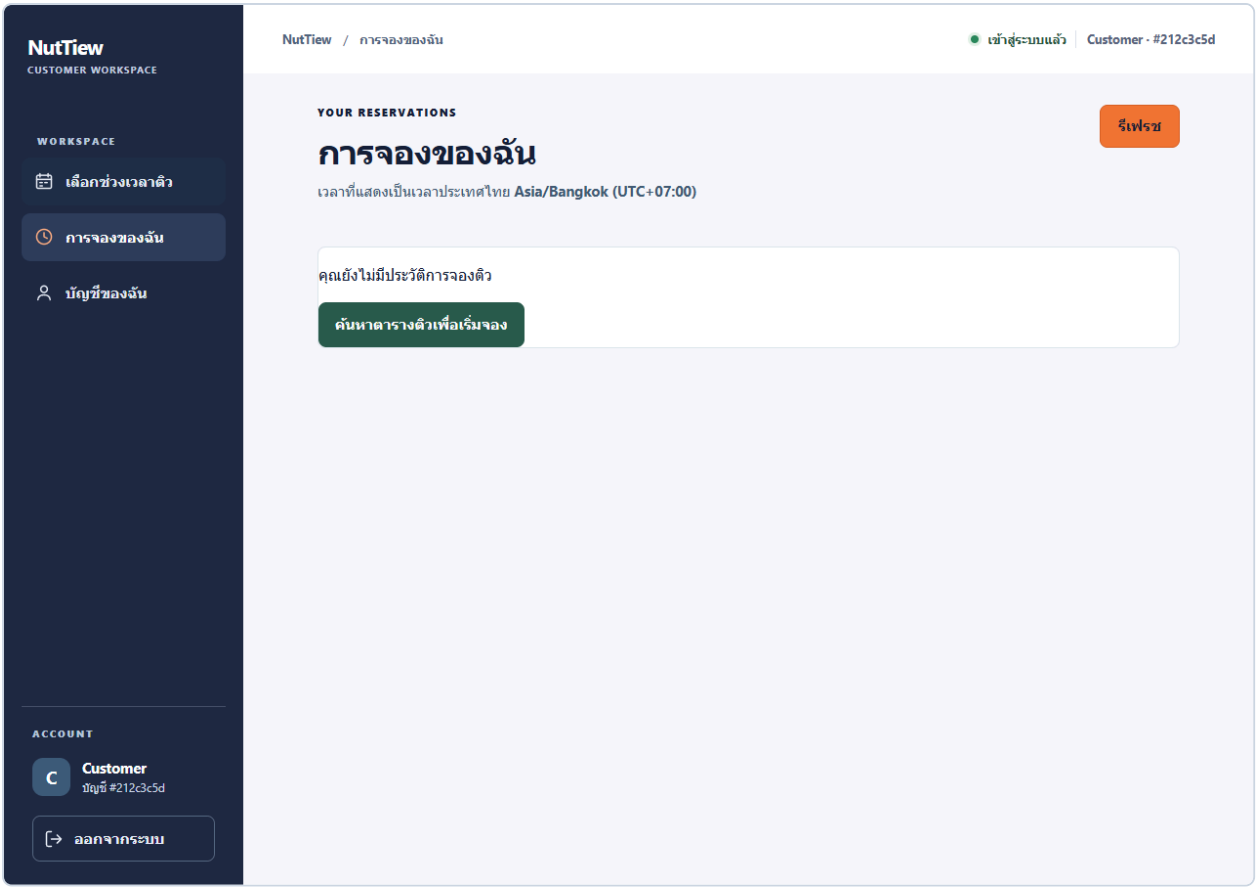
06 Tutor Schedule Occupied

Milestone 7: Negative Business Rule Rejection (409 Conflict)



07 Negative Conflict 409 — Top-viewport excerpt (1280×1472 px) cropped for PDF readability from full-page capture (1280×59120 px) at ../evidence/sprint1/07-negative-conflict-409.png

Milestone 8: Negative Cross-Customer Access Refusal (Safe 404)



08 Negative Cross Customer 404

---

7. Substantive Teammate Peer Review Evidence

1. **Pull Request #38 — F-S1-04 Customer Exclusive Booking (`feature/16-exclusive-booking` → `sprint1`):**
   - Author: @Chxtamos | Reviewer: @Tanaboonnnnn | Issue: #16
   - Substantive interventions resolved: decoupled CSRF retry UX, session-binding rebootstrap without kicking valid Customer sessions, removed duration heuristic, discovered Part 1C FK reverse-wait deadlock leading to Issue #39 / PR #40, merged into `sprint1` at `ed78bba`.
2. **Pull Request #42 — F-S1-05 Protected Booking Retrieval (`feature/17-booking-retrieval` → `sprint1`):**
   - Author: @Chxtamos | Reviewer: @Tanaboonnnnn | Issue: #17
   - Substantive interventions resolved: decoupled Tutor slot vs booking AbortControllers, eliminated trigger tampering in E2E fixtures, fixed `!res.ok` precedence before `X-NutTiew-Session-Binding` with `RET-15-ERR-404/503`, merged into `sprint1` at `9d24fca`.
3. **Pull Request #43 — D1-10 Deliverable 1 Integration (`docs/18-deliverable-1-integration` → `sprint1`):**
   - Author: @Chxtamos | Reviewer: @Tanaboonnnnn | Issue: #18
   - Multiple rigorous review rounds resolving README least-privilege bootstrap order & seed envs, Express 5 / React 19 fidelity, canonical AC/VC mapping, complete embedded Parts A–E contracts, verbatim test excerpts, Mermaid vector verification, and computed PDF page ranges.

---

8. Team Contribution Matrix

Contributor	Contribution	Primary Responsibilities & Deliverables
Tanboon Teawsawat (@Tanaboonnnnn)	**50%**	Requirements discovery (Q01–Q32), Whole-product SRS & Traceability Matrix, System SDS, Three-Sprint Delivery Plan, Project #6 Kanban governance, and substantive peer reviews across PRs #25–#43
Chxtamos (@Chxtamos)	**50%**	Express 5 REST API, PostgreSQL schema/GiST constraints/triggers, Option 1 concurrency lock ordering, React 19 UI, automated Unit/DB/Playwright tests, and Deliverable 1 markdown/PDF packaging

9. Academic Integrity & AI Usage Declaration

- **AI Assistant Declared:** Google Antigravity / Gemini agentic coding assistant.
- **Usage Scope:** Drafting boilerplate TypeScript interfaces, structuring `pg_blocking_pids()` concurrency test harnesses, automating Markdown/PDF compilation, and verifying cross-references.
- **Human Accountability:** All product business rules (Q01–Q32), lock-ordering architecture decisions, security trust boundaries, and peer reviews were evaluated and accepted by human team members. Zero test results or deliverables were fabricated or accepted without real execution against PostgreSQL and browser suites.

10. Definition of Done (DoD) Checklist (Truthful Status)

Quality Gate / Requirement	Verification Evidence	DoD Status
Whole-product Parts A–D embedded verbatim	Complete SRS, Inventory, Traceability, System SDS & 3-Sprint Plan inside PDF	**Complete**
All Sprint 1 Part E contracts embedded verbatim	Full F-S1-01..05 + Shared Booking Data specs/UI/API/Data/Transaction/STS inside PDF	**Complete**
Real working UI → API → PostgreSQL vertical slice	Verified on React 19, Express 5, PostgreSQL 17.2 (local) & 18 (CI)	**Complete**
Automated verification suite	199 distinct tests (29 Unit + 106 DB + 64 E2E) PASS; typecheck & build clean	**Complete**
Formal manual WCAG 2.2 human audit	Automated checks PASS; manual assistive-tech sign-off remains separate manual gate	**Not Run (Manual Gate)**
Reproducible README setup	Owner DB + post-migration grants + seed env vars smoke-tested on PG 17.2	**Complete**
Staging PR #43 review & `sprint1 -> main` release PR	PR #43 awaiting final peer review by @Tanaboonnnnn before release PR & tag	**Pending Peer Approval**
Private Student IDs & LEB2 External Submission	Student IDs withheld from public repo; team supplies IDs & final PDF to LEB2	**Pending Release & LEB2 Handoff**

End of CPE334 NutTiew Deliverable 1 Submission Document.